

Från strategi till autonom exekvering

Teknisk kanonisk manual för strategi, risk, arkitektur och kontrollerad exekvering.

DOKUMENT	Omnira Trading System – Från strategi till autonom exekvering
VERSION	Canonical v1.1
STATUS	Canonical – låst dokumentationsbaslinje
GENERERAD	2026-08-27
OMFATTNING	Kapitel 1–20
SPRÅK	Svenska
KLASSIFICERING	Internt tekniskt dokument

Canonical v1.1. Execution-arkitekturen är sedan 2026-08-28 futures-native och provider-neutral. Det tidigare MetaTrader 5-antagandet var felaktigt för en NQ/MNQ futures-strategi och är korrigerat — se *Canonical Amendments v1.0*, Beslut D. **Tradingstrategin och riskreglerna är oförändrade.** Fjorton medvetet uppskjutna implementation gates kvarstår, klassificerade efter vilken fas de blockerar. Canonical v1.1 innebär *inte* att strategin har bevisad positiv expectancy. Den ska bevisas eller förkastas genom data.

Innehåll

1	Vision	2
2	Tradingfilosofi	18
3	Strategispecifikation	39
4	Riskhantering	69
5	Systemarkitektur	76
6	Marknadsdata	92
7	AI som beslutsstöd	112
8	Exekvering	128
9	Futures Execution Integration	148
10	Backtesting	171
11	Forward testing	194
12	Prop firm-regler	216
13	Tradingjournal	236
14	Prestationsanalys	262
15	Felmoder och failure scenarios	289
16	Säkerhet och kill switches	322
17	Demofas	352
18	Live-deployment	376
19	Kriterier för uppskalning	397
20	Lärdomar och förändringshistorik	418

KAPITEL 1

Vision

Omnira Trading System är visionen om ett trading-system där strategi, AI, risk, exekvering och lärande arbetar tillsammans i en sammanhängande och kontrollerad arkitektur.

Målet är inte att bygga en bot som bara hittar en signal och skickar en order.

Målet är att bygga ett system som kan:

- observera marknaden
- identifiera en definierad strategi
- förklara vad det ser
- kontrollera risk
- kontrollera prop firm-regler
- skapa en tydlig trade proposal
- exekvera endast när rätt behörighet finns
- dokumentera hela processen
- mäta resultat
- lära från varje beslut
- förbättras över tid utan att själv skriva om sina produktionsregler

Den långsiktiga ambitionen är att Omnira ska kunna gå från analysverktyg till kontrollerad autonom trading utan att säkerhetsmodellen behöver byggas om längs vägen. Arkitekturen är därför redan från början uppdelad i separata lager med tydliga ansvarsområden.

Den centrala visionen är:

Omnira Trading ska kunna bli mer intelligent och mer autonom över tid utan att bli mindre kontrollerbart.

Från tradingbot till trading-system

En traditionell tradingbot kan förenklat beskrivas som:

```
Signal  
→ Order
```

Omnira Trading ska medvetet vara mer avancerat än så.

Den canonical pipeline som systemet bygger på är:

Market Data
→ Strategy Engine
→ AI Analysis
→ Risk Engine
→ Prop Firm Rules Engine
→ Trade Proposal
→ Approval / Automation Policy
→ Execution Gateway
→ Execution Runner
→ Futures Execution Provider
→ Broker / Prop Firm
→ Journal & Analytics

Varje steg har ett eget ansvar och inget lager får hoppa över nästa säkerhetslager.

Det innebär bland annat att:

En Strategy Signal inte är en order.

En AI Analysis är inte ett risktillstånd.

Ett Risk PASS är inte execution approval.

Ett Trade Proposal är inte en broker-order.

Execution är en separat privilegierad handling.

Atlas som tradingpartner

Atlas ska vara det intelligenta lagret ovanpå Omnira Trading.

Atlas ska inte vara en fristående AI-trader som fritt tittar på en chart och bestämmer vad som ska köpas eller säljas.

Instället ska Atlas få tillgång till strukturerade objekt från Trading Domain, exempelvis:

- market context
- identified liquidity
- FVG
- Strategy Setup
- Strategy Signal
- Risk Decision
- Prop Decision
- Trade Proposal
- position state
- journal
- performance

Atlas kan därefter hjälpa användaren att förstå vad systemet ser och varför.

AI:n får förklara, sammanfatta, jämföra och analysera. Den får däremot inte kringgå Strategy Engine, Risk Engine eller Prop Firm Rules Engine.

Ett transparent system

Omnira Trading ska inte vara en svart låda.

Användaren ska kunna se:

- vad systemet bevakar
- vilken 4H-thesis som är aktiv
- vilken liquidity som är relevant
- vilka FVG-zoner som används
- om manipulation har inträffat
- vilken 1m-confirmation som saknas eller finns
- setup grade
- planerad entry
- technical stop loss
- target
- R:R
- risk
- prop firm-status
- execution-status
- aktuell position
- break-even-status
- slutresultat

Denna transparens ska visualiseras genom en TradingView-liknande Atlas Market View.

Chart UI är dock endast en visualisering.

Tradinglogiken ska ligga i backendens strukturerade systemstate och inte återskapas av chart-komponenten.

Vad Atlas ser

En av visionens viktigaste delar är att användaren ska kunna se samma tradingstruktur som Strategy Engine använder.

Om systemet exempelvis identifierar:

- selected 10:00 4H-open
- liquidity under price
- en aktiv 15m FVG
- manipulation nedåt
- iFVG + CISD på 1m
- ingen SMT

ska användaren kunna se dessa objekt direkt i Atlas Market View.

Atlas ska sedan kunna förklara:

Manipulationen mot 15m FVG är genomförd. iFVG och CISD har bekräftats på 1m. SMT saknas, vilket ger setup grade A. Technical stop ligger under manipulationens senaste giltiga swing low och första giltiga liquidity target erbjuder minst 2R.

Detta är den typ av interaktion som ska göra Atlas till en faktisk tradingpartner snarare än bara ett generiskt AI-chattfönster.

Den första strategin

Den första canonical strategin i systemet är:

Omnira Liquidity Manipulation – Canonical v1.0

Den första valideringen ska ske på:

- NQ
- MNQ

med ES som comparison instrument för SMT.

Strategin söker efter manipulation från utvalda 4H-candles mot tidigare identifierad liquidity eller Fair Value Gap på 5–15 minuter.

Efter manipulationen söker systemet reversal-confirmation på 1m genom:

- iFVG
- CISD
- eller båda

SMT mellan NQ och ES används som extra confirmation men är inte obligatoriskt för en giltig trade.

Detta är den första strategin.

Arkitekturen ska däremot stödja flera framtida strategier utan att Risk Engine, execution, journal eller övriga kärnlager måste byggas om.

Strategin är en hypotes

Canonical betyder inte att strategin är bevisat lönsam.

Canonical betyder att reglerna är tillräckligt låsta för att implementeras och testas konsekvent.

Strategins grundhypotes är att en manipulation mot relevant liquidity eller FVG, följd av definierad reversal-confirmation, kan ha positiv expectancy mot nästa giltiga liquidity target.

Detta måste bevisas eller förkastas genom data.

Därför är Omnira Trading inte byggt kring antagandet:

Strategin fungerar.

Systemet ska istället fråga:

Kan vi visa att strategin fungerar tillräckligt robust för att förtjäna större authority och större kapitalrisk?

Risk före automation

Omnira Trading ska prioritera:

- säkerhet
- testbarhet
- datakvalitet
- risk
- strategi
- automation
- skalning

Den prioriteringsordningen är en del av systemarkitekturen.

Det innebär att systemet inte får börja med automation och därefter försöka lägga till säkerhet.

Safety måste finnas före execution.

Risk Engine som självständigt veto

Risk Engine ska vara deterministisk och stå över Strategy Engine och Atlas.

En setup kan vara:

A+

och Atlas kan bedöma den som mycket stark.

Men om Risk Engine säger:

DENY

blir slutresultatet:

NO TRADE

Risk får inte mjukas upp av AI-confidence eller subjektiv optimism.

Den första riskbaslinjen

Den första definierade riskbaslinjen innehåller:

Max risk per trade: \$150

Max daily loss: \$450

Max open positions: 1

Max attempts per 4H-thesis: 3

Technical stop loss får inte flyttas för att pressa in en trade i riskbudgeten.

Om minsta handlingsbara position innebär större risk än tillåtet ska trade nekast.

Riskreglerna ska ligga utanför Strategy Engine och kunna versionshanteras separat.

Prop firm som separat kontrollager

Ett av Omnira Tradings viktigaste framtida användningsområden är prop firm-trading.

Därför räcker det inte med intern risk.

Prop firm-regler ska ligga i ett separat:

Prop Firm Rules Engine

Detta lager ska kunna modellera externa regler såsom:

- daily loss
- maximum loss
- trailing drawdown
- static drawdown

- minimum trading days
- consistency
- position limits
- news restrictions
- holding restrictions
- instrument restrictions
- nätverks-/VPS-/VPN-policy där relevant

Varje prop firm-program ska representeras som en separat versionsstyrd profil eftersom olika program kan ha helt olika regler.

Om intern Risk Engine säger ALLOW men Prop Firm Engine säger DENY blir resultatet:

NO TRADE

Fail Closed

En av de viktigaste säkerhetsprinciperna i hela systemet är:

UNKNOWN ≠ SAFE

Om systemet inte vet om en kritisk state är säker ska nya trades blockeras.

Exempel:

- stale market data
- stale account state
- Risk Engine unavailable
- unknown position
- unknown prop state
- fel konto
- runner unavailable
- news state unknown

ska inte tolkas som tillstånd att handla.

Marknadsdata före beslut

Market Data Layer ska vara systemets canonical källa för normaliserad tradingdata.

Strategy Engine ska inte direkt kommunicera med flera providers och försöka tolka olika format.

Flödet ska istället vara:

```
External Data Source
→ Market Data Adapter
→ Validation
→ Normalization
→ Canonical Market Data
→ Strategy Engine
```

Det gör datakvalitet till en explicit safety boundary.

Om kritisk data är fel, stale eller ofullständig får systemet inte skapa exekverbart state.

Samma verklighet i backtest och live

En central vision är att samma Strategy Engine i möjligaste mån ska kunna användas i:

- backtest
- replay
- realtime analysis
- demo
- live

Skillnaden ska huvudsakligen ligga i data- och execution adapters.

Backtest ska alltså inte innehålla en förenklad “snällare” version av strategin.

Detta gör att vi senare kan fråga:

Vad hade Strategy v1.0 faktiskt beslutat här, med endast den information som fanns vid den tidpunkten?

No Look-Ahead

Systemet får aldrig använda framtida information för historiska beslut.

En swing som kräver nästa candle för confirmation får inte existera innan den efterföljande candle faktiskt är tillgänglig.

Samma princip gäller entrykritiska patterns och candle closes.

Detta är avgörande för att backtestresultat ska vara trovärdiga.

Forward testing som verklighetskontroll

Efter historisk testing måste systemet möta tiden framåt.

Forward testing ska bevisa både:

strategin

och:

systemet runt strategin

i realtime.

Den första progressionen är:

```
Analysis Only
→ Shadow Mode
→ Demo Manual Approval
→ Demo Automation
```

Ingen av dessa faser ska hoppas över bara för att backtestet ser starkt ut.

Analysis Only först

Det första realtime-målet är inte att skicka orders.

Målet är att visa att Omnira kan observera marknaden, följa Canonical Strategy v1.0, dokumentera varje beslut och producera reproducerbart beteende utan execution.

Detta är viktigt eftersom vi först måste kunna lita på vad systemet ser innan vi låter det påverka ett konto.

Futures Connectivity (Read Only) först

Samma filosofi gäller providern.

Omnira ska först kunna läsa:

- account
- balance
- equity
- market data
- orders
- positions
- historical orders
- deals

innan orderläggning aktiveras.

Principen är:

Omnira ska först lära sig att observera providern korrekt innan systemet får rätt att påverka providern.

Execution separeras från intelligens

Execution Runner ska vara liten och deterministisk.

Den ska inte tänka.

Den ska:

- verifiera
- exekvera
- rapportera
- reconcila

Den ska endast acceptera strukturerade Execution Intents som redan passerat systemets authority chain.

Strategy Engine ska aldrig direkt kalla providerns orderfunktioner.

Atlas ska aldrig själv formulera en naturlig språkinstruktion till brokern.

Initial executionmiljö

Den första executionmiljön ska vara en separat Execution Runtime kopplad till providern.

Arkitekturen ska vara location-agnostic så att runnern senare kan flyttas till en VPS utan att Strategy Engine, Risk Engine eller Omnira UI behöver byggas om.

Den initiala Windows-miljön är alltså en deploymenttarget.

Den är inte en permanent arkitekturbegränsning.

Broker state är verklig exposure

När execution väl aktiveras ska Omnira skilja mellan:

Expected State

och:

Observed Broker State

Om Omnira tror att inga positions finns men providern visar en position ska systemet inte gissa.

Det ska gå till:

RECONCILIATION_REQUIRED

och blockera ny execution.

Detta är avgörande eftersom brokern är source of truth för faktisk market exposure.

Journalen som systemets minne

Omnira Trading ska inte bara spara vinnare och förlorare.

Systemet ska dokumentera hela beslutskedjan.

Det inkluderar:

- setups
- signals
- AI analysis
- riskbeslut
- prop firm-beslut
- proposals
- approvals
- execution intents
- orders
- fills
- positions
- management events
- exits
- technical incidents
- denied trades
- missed setups

Tradingjournalen ska göra det möjligt att förstå varför systemet fattade ett visst beslut.

Nekade trades är också data

Om Risk Engine nekar en A+-setup är det fortfarande värdefull information.

Systemet ska kunna analysera:

Hur utvecklades setups som vi korrekt nekade?

Detta skapar counterfactual research utan att skriva om det faktiska beslut som togs.

Performance handlar om mer än profit

Omnira Trading ska inte utvärdera systemet enbart med:

Hur mycket pengar tjänade vi?

Analytics ska bland annat mäta:

- expectancy
- Profit Factor
- R
- maximum drawdown
- losing streak
- MFE
- MAE
- session
- setup grade
- attempt
- liquidity type
- SMT
- execution quality
- slippage
- latency
- sample size

Performance ska alltid visas tillsammans med relevant kontext och strategy version.

Edge före skalning

En snygg equity curve är inte tillräcklig.

Systemet ska försöka avgöra om resultatet är:

- robust
- reproducerbart
- stabilt
- out-of-sample-kompatibelt
- realistiskt efter costs
- rimligt över flera market regimes

Backtesting ska därför försöka motbevisa edge snarare än bekräfta den.

Self-Improvement

Den långsiktiga visionen är att Atlas inte bara ska analysera dagens trade.

Systemet ska kunna lära från historiken.

Journal och Analytics ska kunna skapa:

Findings
→ Hypotheses
→ Candidate Versions

Atlas ska exempelvis kunna upptäcka:

Attempt 3 har betydligt lägre expectancy.

och föreslå:

Testa en candidate med max två attempts.

Men detta är endast research.

Productionregeln ändras inte automatiskt.

Learning får vara snabbare än deployment

En central framtida governanceprincip är att Atlas ska kunna lära kontinuerligt utan att production förändras lika snabbt.

En candidate ska kunna gå genom:

Hypothesis
→ Backtest
→ Out-of-Sample
→ Forward Test
→ Review
→ Approval
→ Canonical

innan den får påverka live trading.

Rejected candidates ska också bevaras så att systemet inte glömmer vilka idéer som redan testats.

Atlas ska kunna utveckla kunskap – inte ge sig själv authority

AI får skapa:

- observations
- findings
- hypotheses
- candidateförslag
- explanations

AI får inte själv:

- höja risk
- ändra canonical Strategy
- ändra Prop rules
- aktivera live automation
- kringgå kill switches
- ge sig själv execution permission

Detta är en grundläggande boundary för framtida autonomi.

Autonomi är ett privilegium

Omnira ska inte starta i autonomt läge.

Autonomi ska förtjänas genom dokumenterade gates.

Systemet ska kunna röra sig genom tydliga operation modes, från analysis och read-only till demo, manual live och slutligen Controlled Live Automation.

Högre autonomi innebär inte automatiskt högre risk.

Live är inte slutmålet

När Omnira når live trading börjar en ny valideringsfas.

Den första livefasen ska vara:

Live Read Only

och därefter:

Live Manual Approval

innan Controlled Live Automation övervägs.

Den första live-traden ska behandlas som ett systemtest, inte som starten på maximal kapitalanvändning.

Minimal risk först

Live deployment ska börja med mycket liten praktisk kapitalrisk.

Målet är att bevisa att:

- real fills fungerar
- real SL/TP fungerar
- risk fungerar
- prop compliance fungerar
- journal fungerar
- reconciliation fungerar
- infrastructure är stabil

innan större risk används.

Skalning måste förtjänas

Omnira ska aldrig automatiskt höja risk efter en winning streak eller en bra månad.

Uppskalning ska ske efter dokumenterad evidens.

Kapitaluppskalning och autonomiuppskalning ska dessutom behandlas som två separata dimensioner.

Systemet ska kunna bli mer autonomt utan att öka risk.

Och det ska kunna öka kapital under fortsatt manual approval.

Safety före profit

Ett system kan vara lönsamt och ändå vara tekniskt farligt.

Exempel:

+20R

men:

- duplicate orders
- wrong account
- missing SL

är inte ett godkänt system.

Zero-tolerance safety failures ska kunna blockera vidare scaling oavsett profit.

Kill switches

Systemet ska ha flera nivåer av kill switch:

- Global
- Account
- Strategy
- Instrument
- Runner

Aktiv relevant kill switch ska stoppa ny execution.

Kill switch ska däremot hållas separat från Emergency Close.

Att stoppa nya trades och att tvångsstänga befintlig exposure är två olika actions.

Security by design

Trading-systemet ska byggas enligt least privilege.

Atlas behöver inte broker credentials.

Analytics behöver inte orderbehörighet.

Strategy Engine behöver inte kunna exekvera.

Frontend ska inte kunna tala direkt med providern.

Execution Runner ska endast kunna göra det den explicit auktoriserats att göra.

Ju mindre authority varje komponent har, desto mindre skada kan ett enskilt fel orsaka.

Failure är förväntat

Omnira ska designas utifrån att:

- nätverk kommer falla
- providern kommer disconnecta
- providers kommer ge problem
- processer kommer krascha
- brokers kan neka orders
- data kan vara stale

- AI kan ha fel

Systemets kvalitet avgörs därför inte endast när allt fungerar.

Det avgörs också av hur systemet beter sig när något går fel.

Recovery före återstartad trading

Efter failure ska progressionen vara:

```
FAILURE DETECTED
→ BLOCK
→ RECONNECT / RESTART
→ RESYNC
→ RECONCILE
→ VERIFY
→ READY
```

Inte:

```
FAILURE
→ RESTART
→ TRADE
```

Detta är en central safetyprincip.

Omnira Trading som lärande system

När alla dessa delar kombineras blir Omnira mer än en executionbot.

Systemet får ett långsiktigt minne genom:

- Trading Journal
- Analytics
- Research Registry
- Incident Registry
- Change History

Det ska kunna komma ihåg både:

- vad som fungerade
- vad som inte fungerade
- vad som nekades
- vad som ändrades
- varför det ändrades

Detta är grunden för ansvarsfull self-improvement.

Den långsiktiga målbilden

Den långsiktiga målbilden är ett system där användaren kan öppna Omnira Trading och direkt förstå:

Vad ser Atlas?

Vilken marknadsstruktur bevakas?

Finns en giltig setup?

Vad saknas?

Vilken trade planeras?

Hur stor är risken?

Tillåter prop firm-reglerna trade?

Vad har hänt tidigare i liknande situationer?

Vilka systemhälsoproblem finns?

och när systemet väl är tillräckligt validerat:

Får Omnira exekvera detta automatiskt?

Från information till handling

Visionen kan sammanfattas som:

Observe
→ Understand
→ Validate
→ Control Risk
→ Authorize
→ Execute
→ Measure
→ Learn
→ Improve

Det är den cykel som Omnira Trading System ska bygga.

Vad framgång betyder

Framgång för projektet är inte bara:

Systemet tjänade pengar.

Ett framgångsrikt Omnira Trading System ska också kunna visa att:

- strategin har mätbar edge
- risk är kontrollerad
- prop firm-regler följs
- execution är tillförlitlig
- systemfel upptäcks
- trades är auditerbara
- performance är reproducerbar
- förbättringar testas innan production

- autonomi kan stoppas
- historiken kan förklaras

Detta är en högre standard än för en vanlig tradingbot.

Det är också den standard som behövs om systemet senare ska få större authority över verkligt kapital.

Kapitelstatus

Kapitel: 1 – Vision

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Systemmål: Kontrollerad utveckling från analys till autonom execution

Primär strategi: Omnira Liquidity Manipulation – Canonical v1.0

Primär valideringsmarknad: NQ / MNQ

AI: Beslutsstöd och learning, inte självständig execution authority

Risk: Separat deterministiskt veto

Prop Firm Rules: Separat compliance-veto

Execution: Isolerad och privilegierad

Journal: Hela beslutsprocessen

Self-improvement: Tillåten research, förbjuden direkt production self-modification

Autonomi: Förtjänas genom dokumenterade gates

Live: Inte godkänt utan föregående validation

Omnira Trading System ska byggas för att kunna bli mer autonomt över tid utan att förlora kontroll, transparens eller säkerhet.

KAPITEL 2

Tradingfilosofi

Tradingfilosofin i Omnira Trading System definierar hur systemet ska tänka kring marknaden, risk, osäkerhet, edge, disciplin och besluts kvalitet.

Strategin beskriver vad systemet ska leta efter.

Tradingfilosofin beskriver hur systemet ska förhålla sig till det som händer.

Detta är viktigt eftersom ett robust trading-system inte bara behöver regler för entry och exit.

Det behöver också tydliga principer för hur det ska reagera på:

- vinster
- förluster
- osäkerhet
- drawdown
- missade trades
- starka setups
- dåliga setups
- prop firm-press
- kortsiktiga resultat
- förändrade marknadsförhållanden

Den centrala principen är:

Ett bra tradingbeslut bedöms efter om processen följdes korrekt, inte enbart efter om trade vann eller förlorade.

Marknaden är osäker

Omnira Trading ska aldrig behandla marknaden som deterministisk.

En korrekt setup kan förlora.

En dålig setup kan vinna.

Det betyder att:

Outcome

och:

Decision Quality

inte är samma sak.

Systemet ska därför aldrig lära sig principen:

Vinst = rätt beslut.

eller:

Förlust = fel beslut.

Bra beslut kan förlora

Exempel:

- korrekt session
- korrekt manipulation
- giltig iFVG/CISD
- minimum 2R
- korrekt technical SL
- Risk PASS
- Prop PASS
- korrekt execution

Tradens resultat:

-1R

Det kan fortfarande vara ett:

GOOD DECISION

Förlusten är då en del av strategy variance.

Dåliga beslut kan vinna

Om någon däremot tar en trade:

- utanför session
- under news blackout
- med för hög risk
- utan giltig confirmation

och den råkar ge:

+3R

är beslutet fortfarande dåligt.

Systemet får inte belöna regelbrott bara för att utfallet blev positivt.

Process före resultat

Omnira ska därför utvärdera två saker separat:

Decision Process

och:

Outcome

Det gör det möjligt att få fyra kategorier:

Bra beslut + vinst

Bra process och positivt resultat.

Bra beslut + förlust

Bra process men negativt resultat.

Dåligt beslut + vinst

Fel process men positivt resultat.

Dåligt beslut + förlust

Fel process och negativt resultat.

Den farligaste kategorin är ofta:

Dåligt beslut + vinst

eftersom den kan förstärka fel beteende.

Edge är probabilistisk

Om Strategy v1.0 senare visar positiv expectancy innebär det inte att varje trade har positivt utfall.

Edge ska förstås över en stor mängd trades.

Exempel:

Expectancy = $+0.25R$

betyder inte:

Nästa trade tjänar $+0.25R$.

Det betyder:

Historiskt har systemet i genomsnitt producerat $+0.25R$ per trade över det analyserade samplet.

En enskild trade betyder nästan ingenting

En trade säger väldigt lite om strategy quality.

Även:

- fem trades
- tio trades
- tjugo trades

kan ge missvisande resultat.

Därför ska Omnira alltid väga performance mot:

- sample size
- tidsperiod
- market regime
- session
- setup grade
- execution quality

Ingen jakt på rätt varje gång

Systemets mål är inte att:

Ha rätt på varje trade.

Målet är att:

Följa en process som över många trades har positiv expectancy med acceptabel risk.

Detta är en fundamental skillnad.

Förluster är en kostnad för edge

Om strategy har verklig edge kommer förluster ändå förekomma.

Förlusten ska därför inte automatiskt tolkas som:

- strategy failure
- AI failure
- execution failure

Först måste vi fråga:

Följdes reglerna?

Om ja kan trade helt enkelt vara en normal losing observation.

Trading Loss vs System Failure

Omnira ska alltid skilja mellan:

Trading Loss

och:

Technical/System Failure

Exempel:

Korrekt trade träffar SL:

TRADING LOSS

Fel quantity skickas:

SYSTEM FAILURE

SL saknas hos broker:

EXECUTION FAILURE

Detta är helt olika saker.

Ingen Revenge Trading

Efter en förlust får systemet inte försöka vinna tillbaka pengar.

Re-entry i Strategy v1.0 är tillåten endast därför att strategin uttryckligen tillåter ett nytt giltigt attempt inom samma 4H-thesis.

Re-entry betyder:

Ny giltig setup.

Inte:

Förra traden förlorade, så vi försöker igen större.

Ingen Martingale

Risk ska inte ökas efter förlust.

Om första traden förlorar:

-\$150

ska nästa trade fortfarande följa aktuell RiskProfile.

Det ska inte bli:

-\$300

för att försöka återställa kontot snabbare.

Ingen FOMO

En missad trade är inte automatiskt ett problem.

Om:

- entry redan passerat
- R:R fallit under 2
- proposal expired

ska systemet avstå.

Det ska inte försöka jaga marknaden.

Missad vinst är inte förlust

Omnira ska skilja mellan:

Actual Loss

och:

Missed Opportunity

En setup som gick +5R efter att systemet korrekt nekade den på grund av daily stop är inte en förlust.

Beslutet kan fortfarande ha varit korrekt.

Opportunity Cost ska analyseras – inte ångras

Nekade setups ska sparas för counterfactual analysis.

Atlas kan senare analysera:

Hur hade Risk DENY-setups utvecklats?

Det kan hjälpa framtida research.

Men den informationen får inte användas för att retroaktivt säga att tidigare riskbeslut var fel.

Risk är kostnaden för information

Varje trade riskerar kapital för att testa en hypotes i marknaden.

Därför måste risk vara liten nog att systemet kan överleva när hypotesen är fel flera gånger i rad.

Risk Engine finns inte för att maximera profit per trade.

Den finns för att begränsa skadan när:

- strategy har fel
- market regime förändras
- execution blir sämre
- data är fel
- systemet fungerar dåligt

Kapitalbevarande före aggression

Omnira Trading ska föredra:

survival

framför:

maximum short-term growth

Ett system som överlever kan fortsätta samla data.

Ett system som förstör sitt konto kan inte fortsätta testa sin edge.

Risk ska vara konsekvent

En A+ -setup ska inte automatiskt få mer risk än en B-setup bara för att den ser bättre ut.

Setup grade är strategy information.

RiskProfile styr kapitalrisk.

Om framtida data visar att grade-baserad risk är motiverad ska detta behandlas som en separat candidate riskmodell och testas innan production.

Confidence är inte Position Size

Atlas Confidence får inte översättas direkt till risk.

Exempel:

AI Confidence = 95 %

betyder inte:

Dubbla positionen.

Det betyder endast att AI-lagret uttrycker hög confidence i sin analys.

Hard risk ligger fortfarande i Risk Engine.

Ingen känslostyrd risk

Omnira ska inte använda logik såsom:

Den här setupen känns fantastisk.

eller:

Vi behöver vinna tillbaka dagens losses.

Risk ska vara:

- deterministisk
- versionsstyrd
- reproducerbar

Technical SL ska respekteras

Stop loss ska definieras av strategy structure.

Det ska inte flyttas närmare entry bara för att:

- få lägre risk
- få större quantity
- få bättre R:R

Om technical stop gör minsta position för dyr:

NO TRADE

Det är bättre att missa en trade än att förstöra strategy integrity.

R framför dollar när strategy jämförs

R ska vara det primära normaliserade performanceformatet.

Det gör att vi kan jämföra trades mellan:

- olika account sizes
- olika RiskProfiles
- demo
- live
- backtest

Dollarresultat är fortfarande viktigt.

Men strategy quality ska inte förväxlas med account size.

Money Matters in Risk

För execution och account risk är faktiska dollar fortfarande centrala.

Exempel:

1R

kan vara:

- \$50
- \$100
- \$150

beroende på RiskProfile.

Systemet ska därför alltid förstå både:

R

och:

actual money

Drawdown är normalt – men måste förstås

En strategi med positiv edge kommer kunna gå genom drawdowns.

Frågan är inte:

Kan drawdown undvikas helt?

utan:

Är drawdown inom vad systemet är designat att överleva?

Det ska analyseras genom:

- maximum drawdown
- drawdown duration
- losing streak
- recovery time
- Monte Carlo där relevant

Losing Streak betyder inte automatiskt att edge är borta

Flera förluster i rad kan ligga inom normal historisk distribution.

Systemet ska därför inte förändra strategy efter varje losing streak.

Det ska istället jämföra aktuell performance mot tidigare förväntat behavior.

Men Edge får ifrågasättas

Tradingfilosofin får inte bli ett sätt att försvara strategin oavsett resultat.

Om:

- backtest
- OOS
- forward
- live

över tid visar att expectancy inte längre finns ska systemet kunna säga:

Strategins edge är inte längre tillräckligt stödd.

Ingen strategi är helig.

Marknaden har ingen skyldighet att fortsätta bete sig likadant

Strategy v1.0 är baserad på en hypotes om marknadsbeteende.

Marknaden kan förändras.

Därför ska systemet mäta:

- regime
- volatility
- session behavior
- execution behavior
- rolling expectancy

för att upptäcka om historiska assumptions blir svagare.

Canonical betyder inte evigt

En canonical strategy är den officiellt aktiva versionen.

Det betyder inte att den är permanent.

Om data senare stödjer en förbättring kan:

v1.0

ersättas av:

v1.1

efter korrekt validation.

Ingen ändring efter en dålig vecka

En strategy version ska få tillräckligt med data innan den ändras.

Vi ska undvika:

```
loss
→ rule change
→ loss
→ new rule change
```

Det skulle skapa ett överanpassat system som jagar historiken.

Stability har ett värde

Ett system som hela tiden förändras blir svårt att utvärdera.

Därför är:

NO CHANGE

ett legitimt researchbeslut.

Kontinuerligt lärande betyder inte kontinuerliga productionändringar.

Hypotes före regel

Atlas får upptäcka:

C-grade verkar underpresterar.

Det blir först:

Finding

sedan:

Hypothesis

eventuellt:

Candidate

Inte:

Stäng av C-grade från och med nästa trade.

Data ska få säga emot oss

Om den ursprungliga tron är att SMT förbättrar strategy och datan senare visar att det inte gör det ska systemet acceptera detta.

Research ska inte försöka bevisa att våra tidigare antaganden var rätt.

Målet är att hitta det som faktiskt håller.

Confirmation Bias ska motarbetas

Backtesting och Analytics ska aktivt leta efter:

- svaga perioder
- negativa segments
- dålig OOS-performance
- dålig forward performance
- parameter sensitivity

Ett system som bara visar bästa resultaten är inte ett researchsystem.

Survivorship Bias

Om bara exekverade trades analyseras kan viktiga observations försvinna.

Därför ska systemet även spara:

- denied signals
- invalid setups
- expired proposals
- missed execution
- counterfactual outcomes

Det gör analysen mer komplett.

Overfitting är ett centralt hot

Tradingdata innehåller mycket brus.

Om vi testar tillräckligt många regler kommer något nästan alltid se bra ut historiskt.

Därför ska Omnira motarbeta:

- parameter hunting
- rule mining

- cherry picking
- små sample
- repeated hypothesis testing utan kontroll

Simpelt före komplext

En enklare rule som fungerar robust över flera perioder är ofta mer värdefull än en mycket specifik rule som bara fungerar perfekt på historiken.

Systemet ska inte lägga till complexity utan mätbar anledning.

Complexity måste förtjänas

Varje ny regel har en kostnad.

Den kan:

- minska trade count
- öka implementation complexity
- skapa nya failure modes
- öka overfitting risk

Därför måste nya filters visa att de tillför tillräckligt värde.

Alla datafält behöver inte bli regler

Om Atlas upptäcker att spread är högre på losing trades betyder det inte automatiskt att spread ska bli ett hard filter.

Det kan först vara:

analytics metadata

Sedan:

finding

Sedan:

candidate filter

Trade Frequency är inte mål i sig

Omnira behöver inte ta många trades.

Det behöver ta trades som uppfyller Strategy Specification.

Ett system ska aldrig skapa setups bara för att:

- journalen ser tom ut
- prop challenge behöver fler dagar
- användaren vill se aktivitet

No Trade är ett beslut

NO TRADE

ska betraktas som ett fullt legitimt tradingbeslut.

Det kan bero på:

- ingen setup
- news
- dålig R:R
- risk
- prop rules
- data quality
- system health

Att avstå är en del av strategin.

Prop Firm Target får inte styra strategin

Om ett challenge-account bara saknar lite profit för att nå target får systemet inte börja:

- öka risk
- ta sämre setups
- ta extra trades
- kringgå sessionregler

Profit target är ett objective.

Det är inte strategy logic.

Minsta tradingdagar får inte skapa falska trades

Om ett program kräver ett visst antal trading days ska Omnira följa regeln.

Men det ska inte ta en ogiltig trade enbart för att registrera en tradingdag.

Prop compliance får inte skapa tradingmöjligheter som strategin inte ser.

Prop Rules är ett kontrakt

Ett prop account ska behandlas som ett konto med externa constraints.

Om reglerna säger att något inte är tillåtet:

NO TRADE

även om Strategy och intern Risk annars hade sagt ja.

Challenge Failure ska inte jagas tillbaka

Om en challenge är nära breach ska systemet inte:

ta en sista aggressiv trade.

Istället ska samma Strategy och Risk gälla.

Om kontot inte kan överleva korrekt strategy execution med definierad risk är RiskProfile eller programmet sannolikt felmatchat.

Payout förändrar inte Edge

En kommande payout ska inte påverka hur Strategy Engine identifierar en setup.

Tradingmotorn ska inte börja bete sig annorlunda för att kontot närmar sig en ekonomisk milestone.

Tradingdisciplin ska vara kod

En stor fördel med Omnira är att tradingdisciplin kan representeras i systemet.

Exempel:

- sessions
- risk
- news blackout
- R:R
- attempts
- position limits

ska inte bara vara saker användaren försöker komma ihåg.

De ska vara hard rules.

Människan ska kunna stoppa – inte fuska

Användaren ska alltid kunna:

- stoppa trading
- sänka autonomy
- använda kill switch
- emergency close

Men hard Risk DENY ska inte ha en enkel:

TRADE ANYWAY

-knapp.

Om en regel verkligen ska ändras görs det genom governance och ny version.

Manual Approval är inte Strategy Override

När användaren approve:ar en Trade Proposal betyder det:

Jag tillåter detta redan giltiga beslut att gå till execution.

Det betyder inte:

Jag får göra en ogiltig trade giltig.

Automation förändrar inte strategin

När systemet går från:

Manual Approval

till:

Automation

ska Strategy, Risk och Prop rules vara desamma.

Automation tar bort ett mänskligt klick.

Den ska inte ta bort kontrollagren.

Högre autonomi kräver starkare safety

Ju mer systemet gör själv desto större krav finns på:

- monitoring
- auditing
- failure handling
- idempotency
- reconciliation
- kill switches

Autonomi ska inte betyda mindre kontroll.

Live Risk ska börja lågt

Även om Strategy v1.0 testats med en viss riskbaseline ska första livevalidation kunna använda lägre risk.

Live är ytterligare ett experiment.

Den första frågan är:

Fungerar systemet i den verkliga executionmiljön?

Inte:

Hur snabbt kan vi tjäna pengar?

Scale-Up ska gå långsamt

Risk och autonomy ska öka i steg.

Varje högre nivå skapar större konsekvens om systemet har fel.

Därför ska varje scale tier kräva ny evidens.

Scale-Down ska gå snabbt

Om systemet blir osäkert ska det vara lättare att:

- sänka risk
- stoppa automation
- gå till Read Only

än att höja authority.

Detta är en av systemets viktigaste asymmetrier.

No Automatic Scale-Up

Systemet får inte själv höja risk därför att:

- expectancy är positiv
- kontot är på all-time-high
- Atlas är confidence

- en winning streak pågår

Riskökning kräver explicit governance.

Performance ska ses netto

Gross P/L kan vara intressant för strategy analysis.

Men det är:

Net P/L

efter:

- commissions
- fees
- slippage

som avgör den verkliga ekonomiska performance.

Execution är en del av Edge

En strategy kan ha teoretisk edge men ändå vara olönsam om execution är dålig.

Därför måste Omnira mäta skillnaden mellan:

Theoretical Strategy Result

och:

Actual Execution Result

Detta är viktigt särskilt för en 1m-baserad entrymodell.

Latency ska mätas

Manual approval, network, broker och runner skapar latency.

Latency är inte automatiskt ett problem.

Men den ska mätas.

Om den systematiskt förstör entry eller R:R blir det ett engineeringproblem.

Systemet ska mäta verkligheten, inte önskan

Om backtest säger:

+0.35R expectancy

men forward test säger:

-0.10R

ska vi undersöka skillnaden.

Vi ska inte automatiskt förklara bort den för att backtestresultatet var mer attraktivt.

Out-of-Sample väger tungt

En strategy eller candidate som endast fungerar på development data är svag.

OOS och forward data är mer värdefull evidens för generalisering.

Robusthet före toppresultat

Mellan två candidates ska systemet inte automatiskt välja den som tjänade mest historiskt.

En kandidat med:

- något lägre return
- lägre drawdown
- stabilare performance
- bättre OOS

kan vara mer robust.

Risk-adjusted Edge

Målet är inte:

Maximum Profit

Målet är:

Robust positive expectancy under acceptabel risk

Detta är den ekonomiska kärnan i tradingfilosofin.

Market Regime är kontext

Atlas kan klassificera market regime för analytics och research.

Initialt ska detta inte bli ett hard filter bara för att AI säger:

Marknaden känns choppy.

Strukturerad evidens krävs innan regime blir canonical strategy logic.

UNKNOWN är en legitim state

Systemet ska kunna säga:

UNKNOWN

Det är bättre än att tvinga fram:

TRUE

eller:

FALSE

när data saknas.

Exempel:

ES-feed saknas.

Då ska SMT vara:

UNKNOWN

inte:

FALSE

Detta gäller hela filosofin kring osäkerhet.

Osäkerhet ska exponeras

Atlas ska hellre säga:

Sample size är för liten för en robust slutsats.

än att ge överdrivet självsäkra rekommendationer.

Systemets intelligens ska mätas lika mycket på när det vet att det inte vet.

AI ska vara skeptisk även mot sig själv

Atlas ska kunna presentera:

- supporting factors
- contradicting factors
- uncertainty
- sample limitations

AI:n ska inte vara byggd för att sälja in varje Trade Proposal.

Den ska hjälpa till att granska den.

Atlas ska kunna argumentera för NO TRADE

Ett bra trading-AI ska inte bara vara entusiastiskt när en setup finns.

Det ska också kunna säga:

Strategin är formellt giltig, men Risk Engine nekar eftersom daily headroom inte räcker.

eller:

Ingen trade finns eftersom relevant confirmation saknas.

Detta är lika viktigt som att kunna beskriva en long eller short.

Atlas får inte ersätta Strategy Engine

Visuell AI-analys kan vara värdefull.

Men den ska inte vara canonical detection source för entry-critical patterns.

iFVG och CISD måste definieras deterministiskt innan implementation.

Atlas kan därefter förklara resultatet.

Research ska inkludera misslyckade idéer

Om en candidate visar sig sämre ska den sparas.

Systemet ska inte bara ha en historia över framgångsrika changes.

Det behöver även veta:

Det här testades och fungerade inte.

Detta är centralt för långsiktig learning.

Ingen regel utan versionshistorik

När en materiell strategy- eller riskregel förändras ska systemet kunna svara:

- vad ändrades?
- varför?
- vilken evidens?
- vilken version?
- när aktiverades den?

Tradingfilosofin ska alltså vara evolutionär men spårbar.

Trading som experiment

Varje StrategyVersion kan betraktas som en tydligt definierad hypotes.

Den körs.

Data samlas.

Performance mäts.

Systemet avgör om hypotesen håller.

Detta gör tradingutvecklingen mer lik vetenskaplig experimentation än magkänsla.

Men marknaden är inte ett laboratorium

Marknaden förändras medan vi observerar den.

Resultat är därför aldrig helt slutgiltiga.

Även en stark historical strategy behöver fortsatt monitoring.

Kontinuerlig validering

En strategi som en gång passerat validation ska fortfarande följas live.

Om behavior förändras ska systemet skapa:

- alerts
- findings
- review

inte automatiska otestade regeländringar.

Systemet ska vara kapabelt att sluta handla

En av de viktigaste egenskaperna hos ett autonomt trading-system är att kunna säga:

Jag har inte tillräckligt stöd för att fortsätta.

Det kan gälla:

- data
- strategy
- risk
- prop firm
- execution
- performance

Ett system som alltid hittar ett skäl att fortsätta handla är farligt.

Kapitalet är inte till för att bevisa strategin

Live money ska inte användas för att ta reda på om en oprövad strategy fungerar.

Det är därför progressionen innehåller:

- backtest
- OOS
- forward
- demo

innan Controlled Live.

Prop Challenge är inte Backtest

Att passera en prop challenge är ett bra praktiskt resultat.

Men det är inte i sig bevis på robust strategy edge.

En challenge kan passeras genom variance.

Analytics ska fortfarande bedöma strategy över större data.

Losing Challenge betyder inte automatiskt att Strategy är dålig

På samma sätt kan en challenge misslyckas trots positiv expectancy.

Det kan bero på drawdown distribution eller prop rules.

Vi ska därför skilja:

strategy profitability

från:

prop program suitability

Rätt strategi på fel riskprofil kan vara dålig

En strategy kan ha positiv expectancy men för hög variance för en viss prop firm drawdownmodell.

Då kan problemet vara:

- RiskProfile
- program selection

- account size

inte själva entrylogiken.

Risk of Ruin är viktigare än maximal tillväxt

Ett system med stor potentiell vinst men hög sannolikhet att förlora kontot är inte nödvändigtvis attraktivt.

Omnira ska väga:

- expectancy
- drawdown
- streaks
- breach probability
- survival

tillsammans.

Longevity

Målet är ett system som kan arbeta över:

- månader
- år
- flera strategy versions
- flera market regimes

inte ett system optimerat för nästa vecka.

Tradingfilosofins kärnfrågor

För varje trade ska systemet i princip kunna fråga:

Har vi en giltig setup?

Har vi tillräcklig data?

Är risken tillåten?

Är Prop-reglerna tillåtande?

Är executionmiljön healthy?

Finns rätt authority?

Om svaret på någon hard gate är nej:

NO TRADE

Efter traden

Systemet ska sedan fråga:

Följde vi reglerna?

Hur exekverades traden?

Vad blev resultatet?

Var resultatet normalt relativt historiken?

Finns något att forska vidare på?

Detta sluter lärandecykeln.

Den centrala tradingprincipen

Tradingfilosofin kan sammanfattas så här:

Vi försöker inte kontrollera vad marknaden gör. Vi kontrollerar vilka situationer vi deltar i, hur mycket vi riskerar, hur vi exekverar och hur vi lär från resultatet.

Det är vad Omnira faktiskt kan kontrollera.

Kapitelstatus

Kapitel: 2 – Tradingfilosofi

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Decision quality: Separeras från trade outcome

Edge: Probabilistisk och måste visas över data

Losses: Normal del av strategy variance när processen är korrekt

Revenge Trading: Förbjudet

Martingale: Förbjudet

FOMO execution: Förbjudet

Technical SL manipulation: Förbjudet

Risk: Deterministisk och separat från setup confidence

No Trade: Ett legitimt tradingbeslut

Prop target pressure: Får inte förändra Strategy

Self-improvement: Hypothesis före production rule

Scale-up: Kräver evidens

Scale-down: Ska kunna ske snabbare än scale-up

Tradingmål: Robust positiv expectancy under kontrollerad downside

Omnira Trading ska bedöma kvaliteten på sina beslut genom disciplin, evidens och riskkontroll – inte genom hur attraktiv den senaste vinnaren eller förloraren råkade se ut.

KAPITEL 3

Strategispecifikation

Omnira Trading System behöver en strategi som går att beskriva så exakt att samma beslut kan reproduceras i:

- historisk backtesting
- market replay
- forward testing
- Analysis Only
- demo trading
- manual approval
- Controlled Live Trading
- framtida autonom execution

Den första strategin är:

Omnira Liquidity Manipulation – Canonical v1.0

Strategispecifikationen definierar vad Strategy Engine ska leta efter, hur en setup går från marknadskontext till entry och vilka villkor som måste vara uppfyllda innan en Strategy Signal får skapas.

Strategin är låst som canonical baseline för implementation och validering, men den är ännu inte bevisad lönsam. Canonical innebär här att reglerna är tillräckligt definierade för att kunna implementeras och testas konsekvent, inte att positiv expectancy redan är bevisad.

Den centrala principen är:

Strategy Engine identifierar en tradingmöjlighet. Den bestämmer inte ensam om trade får tas.

Strategins grandidé

Omnira Liquidity Manipulation bygger på en hypotes om hur pris beter sig runt liquidity och inefficiencies.

Strategin söker efter att priset, efter en utvald 4H-open, manipulerar mot tidigare identifierad:

- liquidity
- eller Fair Value Gap

på:

- 5m
- 15m

Efter manipulationen analyseras 1m efter reversal-confirmation genom:

- iFVG
- CISD
- eller båda

SMT mellan NQ och ES används som ytterligare confirmation.

SMT är dock inte ett obligatoriskt entrykrav.

Den grundläggande hypotesen är att en manipulation mot relevant 5–15m liquidity eller FVG, följd av definierad 1m reversal-confirmation, kan skapa positiv expectancy mot nästa giltiga liquidity target.

Hypotes – inte marknadslag

Det är viktigt att skilja mellan:

Strategy Definition

och:

Strategy Edge

Definitionen säger:

Om dessa conditions inträffar skapas en signal.

Edge-frågan är:

Producerar dessa signals faktiskt positiv expectancy över tillräckligt mycket data?

Det senare måste besvaras genom:

- backtest
- out-of-sample
- forward test
- demo
- senare controlled live-data

Strategin får alltså inte betraktas som bevisad enbart för att reglerna nu är tydligt dokumenterade.

Primär marknad

Den första valideringen ska göras på:

Nasdaq-100 Futures – NQ

och:

Micro E-mini Nasdaq-100 Futures – MNQ

NQ och MNQ används som strategi- och executionmarknad beroende på setup och riskprofil.

ES används som comparison instrument för SMT.

Strategin kan i framtiden testas på andra instrument, men inget annat instrument ska betraktas som validerat förrän det genomgått egen backtesting och forward testing.

NQ och MNQ

NQ och MNQ följer samma underliggande Nasdaq-100-marknad men har olika kontraktsstorlek.

Det gör att systemet kan skilja mellan:

Market Analysis

och:

Execution Contract

Exempelvis kan marknadsstrukturen analyseras i Nasdaq-futures medan actual execution sker i MNQ för att position sizing ska passa RiskProfile.

Strategy Engine ska därför arbeta med canonical market/instrument-representationer och inte hårdkoda broker-specifika contracts.

ES

ES används initialt inte som primary execution instrument för Strategy v1.0.

Dess roll är framför allt:

SMT comparison

mot NQ.

SMT analyseras på:

- 1m
- 5m
- 15m

Detta kräver att NQ- och ES-data är tidsmässigt jämförbara.

Timeframe-struktur

Strategin använder tre huvudsakliga analysnivåer.

4H

4H används för:

- övergripande thesis
- selected 4H opens
- tidigare 4H highs/lows
- kontext

5m och 15m

Dessa används för:

- liquidity
- FVG
- manipulation
- market structure

1m

1m används för:

- entry phase
- liquidity events
- iFVG
- CISD
- SMT när relevant
- entry
- break-even
- trade management

Det är alltså inte en strategy där alla beslut tas från samma timeframe.

Top-Down Flow

Strategins grundläggande progression är:

```
4H Context  
→ 5–15m Target / Manipulation  
→ 1m Confirmation  
→ Entry
```

Detta är en viktig del av strategy identity.

1m ska inte användas för att leta fristående setups utan relevant higher-timeframe context.

Canonical Timezone

All strategy- och sessionslogik ska använda:

```
America/New_York
```

Systemet får inte använda en permanent:

UTC-4

-offset.

Anledningen är daylight saving time.

New York local time förändrar relationen till UTC under året, medan strategy windows fortfarande ska motsvara samma lokala tider.

Canonical timestamps kan lagras i UTC, men strategy context ska beräknas i IANA-timezone:

America/New_York

Trading Windows

Strategy v1.0 har endast två tillåtna entry windows.

London

02:00–05:00 America/New_York

New York

10:00–12:00 America/New_York

En ny position måste öppnas inom något av dessa windows.

Selected 4H Opens

Strategin använder endast de 4H-opens som hör till:

02:00

och:

10:00

New York-tid.

Övriga 4H-opens kan finnas i marknadsdatan men ska inte användas som strategy triggers i Canonical v1.0.

Detta är viktigt eftersom strategin inte är:

Trade varje 4H-candle.

Den är byggd runt två särskilt definierade sessionskontexter.

Entry Window vs Position Management

Trading window definierar när en ny position får öppnas.

Det betyder inte automatiskt att en redan öppen trade måste stängas när entry-window tar slut.

För London gäller däremot en obligatorisk window-close break-even vid 05:00: positionen stängs inte, men stop loss flyttas till entry price.

Detta hanteras separat för London och New York senare i strategin.

Higher-Timeframe Context

När relevant selected 4H-open har inträffat ska Strategy Engine identifiera potentiella:

- liquidity targets
- FVG-zoner

på 5–15m.

Detta skapar den potentiella riktningen för manipulationen.

Long Thesis

För en potentiell long ska relevant target context finnas:

- under aktuellt pris

Det kan vara:

- giltig 5–15m liquidity
- eller giltig 5–15m FVG

Systemet väntar därefter på att priset manipulerar nedåt till denna struktur.

Efter den nedåtgående manipulationen söker systemet reversal-confirmation för long.

Short Thesis

För potentiell short gäller motsatsen.

Relevant:

- liquidity
- eller FVG

ska finnas över aktuellt pris.

Systemet väntar därefter på manipulation uppåt.

Efter denna manipulation söker systemet bearish reversal-confirmation.

Giltig Liquidity

Canonical Strategy v1.0 tillåter följande liquiditytyper:

- swing high
- swing low
- equal highs
- equal lows
- previous session high
- previous session low
- previous 4H high
- previous 4H low
- previous day high
- previous day low
- intermediate swing highs
- intermediate swing lows

Dessa ska representeras strukturerat i systemet snarare än endast som visuella chartmarkeringar.

Swing High

En canonical swing high definieras genom tre candles.

Den mittersta candle måste ha:

$\text{High}[i] > \text{High}[i-1]$

och:

$\text{High}[i] > \text{High}[i+1]$

Den måste alltså ha högre high än candle direkt före och direkt efter.

Swing Low

Motsvarande:

$Low[i] < Low[i-1]$

och:

$Low[i] < Low[i+1]$

Den mittersta candle måste ha lägre low än båda sina direkta grannar.

Swing Confirmation

En swing kan inte betraktas som confirmed förrän den efterföljande candle finns.

Detta är särskilt viktigt i:

- live
- replay
- backtest

annars skulle systemet använda information från framtiden.

Intermediate Liquidity

Intermediate liquidity är interna swing highs eller swing lows inom aktuell market structure.

De behöver inte samtidigt vara:

- previous day high/low
- previous session high/low
- previous 4H high/low

De representerar mindre intern liquidity som fortfarande kan vara relevant för strategy flow.

Fair Value Gap

Strategy v1.0 använder en standardiserad:

tre-candle FVG

FVG-zonen definieras av gapet mellan candle 1 och candle 3.

För bullish FVG finns ett gap mellan relevant high från candle 1 och low från candle 3.

Bearish FVG använder motsvarande inverterad struktur.

FVG får identifieras på:

- 5m
- 15m

FVG är strukturerad data

En FVG ska inte endast finnas som en färgad rektangel på chartet.

Systemet ska kunna känna till:

- direction
- timeframe

- upper bound
- lower bound
- formation time
- touch state
- active state

Det gör att samma FVG kan användas av:

- Strategy Engine
- Atlas
- Journal
- Backtest
- Market View

Manipulation

När relevant higher-timeframe target har identifierats väntar systemet på manipulation.

Det finns två alternativa canonical triggers.

Liquidity Manipulation

Manipulation mot liquidity är complete när priset har handlats:

- över relevant liquidity för ett upside sweep
- eller under relevant liquidity för ett downside sweep

Det krävs alltså att nivån faktiskt tas.

FVG Manipulation

Manipulation mot FVG är complete när priset touchat relevant 5–15m FVG-zone.

Liquidity och FVG är alternativa triggers

Strategin kräver inte:

Liquidity sweep + FVG touch

samtidigt.

Canonical regel är:

Liquidity manipulation OR FVG manipulation

kan föra setupen vidare till 1m Entry Phase.

Övergång till 1m

När manipulationen är complete övergår setupen till:

1m Entry Phase

Systemet börjar då leta efter reversal-confirmation.

1m Liquidity Sweep

En 1m liquidity sweep är:

preferred

men inte obligatorisk.

Detta är viktigt.

Strategin kräver inte att alla valid trades först måste sweepa ytterligare 1m liquidity.

Det som är obligatoriskt är istället confirmation genom:

- iFVG
- eller CISD

iFVG och CISD

iFVG och CISD är kärnan i Strategy v1.0:s entry confirmation.

En trade kräver minst en av dem.

Canonical Strategy Specification låser deras roll, men den säger samtidigt att deras exakta programmeringsdefinitioner ska dokumenteras som separata deterministiska detection rules innan Strategy Engine implementeras.

Atlas får alltså inte själv titta på chartet och avgöra vad som “ser ut som” iFVG eller CISD.

Samma detection logic måste senare användas i:

- backtest
- replay
- analysis
- demo
- live execution.

Varför detta är viktigt

Om iFVG definieras på ett sätt i backtest men en människa eller AI använder en annan definition live får vi i praktiken två olika strategier.

Det skulle göra performance-resultaten svåra att lita på.

Strategispecifikationen definierar därför konceptet.

Den separata detection specificationen ska definiera exakt machine behavior.

iFVG

iFVG är en giltig standalone confirmation i Canonical v1.0.

Om iFVG uppstår utan CISD kan setupen fortfarande vara tradable.

Den får då Grade B.

CISD

CISD kan också fungera som standalone confirmation.

Om CISD uppstår utan iFVG kan setupen vara tradable.

Den får Grade C.

iFVG + CISD

När båda finns får setupen starkare canonical grade.

Det ger:

Grade A

innan eventuell SMT-upgrade.

SMT

SMT jämför:

NQ

mot:

ES

på:

- 1m
- 5m
- 15m

Canonical definition på övergripande nivå är att ett instrument tar relevant liquidity medan det andra inte gör det.

SMT fungerar som:

extra confirmation

inte som en egen trade trigger.

SMT kan inte skapa en trade

Om systemet ser SMT men saknar:

- iFVG
- CISD

får ingen Strategy Signal skapas.

SMT får aldrig användas för att skapa en trade ensam.

SMT och Grade

SMT kan endast uppgradera:

A

till:

A+

Det innebär att SMT inte höjer:

$B \rightarrow A$

eller:

$C \rightarrow B$

Canonical setup grade är explicit kopplad till confirmationkombinationen.

SMT = UNKNOWN

Om ES-data saknas eller inte kan verifieras ska systemet inte behandla detta som:

SMT = FALSE

Det korrekta tillståndet ska vara:

SMT = UNKNOWN

En setup kan fortfarande vara giltig eftersom SMT inte är obligatorisk.

Men systemet får då inte felaktigt ge A+.

Setup Grades

Canonical v1.0 använder fyra grades.

A+

iFVG + CISD + SMT

A

iFVG + CISD

B

iFVG only

C

CISD only

Alla fyra är giltiga i canonical baseline.

Minimum Grade

Minimum allowed grade ska vara konfigurerbar så att framtida research kan testa exempelvis:

- A+ only
- A+ och A
- A+ till B
- alla grades

Men canonical grundkonfiguration för Strategy v1.0 accepterar:

A+, A, B och C

Det är viktigt att framtida analytics inte i efterhand låtsas att bara A/A+ alltid var strategy.

Grade är data

Setup grade ska alltid journalföras.

Det gör att systemet senare kan analysera:

- expectancy A+
- expectancy A
- expectancy B
- expectancy C

utan att skriva om historiska trades.

Entry

När samtliga tidigare strategy requirements passerat sker entry på 1m.

Entry sker:

direkt på close av confirmation candle

Det gäller både:

- iFVG
- CISD

Ingen obligatorisk Retest

Canonical v1.0 kräver inte:

- retrace
- retest
- limit-order tillbaka till pattern

efter confirmation.

Entry planeras direkt efter confirmation close.

Detta är särskilt viktigt för backtest- och live-paritet.

Confirmation Close måste vara verklig

Systemet får inte känna till candle close innan candle faktiskt stängt.

I backtest måste alltså event chronology respekteras.

Annars uppstår look-ahead.

Entry Price och verklig Fill

Strategy Engine definierar strategy entry.

Actual execution kan senare ske något annorlunda på grund av:

- bid/ask
- latency
- slippage

Därför ska systemet skilja mellan:

Strategy Entry

och:

Actual Fill

Execution Gateway ska dessutom kunna neka trade om price movement gör att strategy rules inte längre håller.

Stop Loss

Stop loss är strategy-defined.

Long

SL placeras:

under manipulationens senaste giltiga swing low

Short

SL placeras:

över manipulationens senaste giltiga swing high

Technical Stop

Detta är:

Technical Stop Loss

Den kommer från market structure.

Risk Engine får använda den för position sizing.

Risk Engine får inte ändra den för att få en trade att passa.

Stop Integrity

Om minsta handlingsbara position med korrekt technical SL överskrider RiskProfile:

TRADE DENIED

Vi flyttar alltså inte stoppen närmare.

Vi avstår istället traden.

Take Profit

TP ska sättas mot:

första giltiga liquidity target som ger minst 2.0R

Det innebär att systemet först identifierar liquidity targets i tradens riktning.

Targets som ligger närmare än 2R får inte användas som slutligt TP i Canonical v1.0.

Första giltiga target

Om flera liquidity targets erbjuder minst 2R används:

den första giltiga

Vi hoppar inte automatiskt över en närmare giltig target för att försöka få exempelvis 5R istället för 2.5R.

Minimum Risk/Reward

En setup måste erbjuda minst:

1:2

vid entry.

Om inget giltigt liquidity target erbjuder minst:

2.0R

är setupen invalid.

R:R är strategy validity

Minimum 2R är alltså inte bara analytics.

Det är en faktisk entry gate.

Exempel:

1.93R

→ INVALID

2.00R

→ kan passera denna regel.

Break-Even

Strategy v1.0 använder break-even-management.

Break-even-triggern är explicit definierad.

Long

Efter entry identifieras närmaste bekräftade 1m swing high.

När priset tar denna nivå:

SL → Entry Price

Short

Närmaste bekräftade 1m swing low efter entry används.

När priset tar nivån:

SL → Entry Price

Denna regel är canonical.

Window-close Break-Even — London

Utöver den swing-baserade triggern ovan har London-sessionen en andra, tidsbaserad break-even-trigger.

Om positionen fortfarande är öppen exakt när London entry-window stänger 05:00 America/New_York:

SL → Entry Price

Detta sker även om den swing-baserade triggern ännu inte har inträffat.

Har swing-triggern redan flyttat SL till entry price gör window-close-triggern ingenting.

Confirmed Swing gäller även BE

BE-trigger använder samma swingdefinition som tidigare.

Nivån måste alltså vara confirmed.

Systemet får inte använda en framtida swing som ännu inte existerade vid beslutstidpunkten.

Ingen fortsatt Trailing

Efter break-even:

SL = Entry

och ligger normalt kvar där.

Strategy v1.0 använder ingen kontinuerlig trailing stop.

Partial Profits

Strategy v1.0 använder:

inga partial profits

Positionen reduceras alltså inte stegvis vid olika targets.

Nästan träffat TP

Om priset nästan når TP och sedan vänder gör strategin inte någon discretionary exit.

TP ändras inte.

Systemet tar inte partial profit.

Positionen fortsätter enligt sin definierade management mot:

- TP
- BE
- news exit
- time exit
- annan explicit systemexit

Max en öppen position

Strategy v1.0 tillåter endast:

1 öppen position

åt gången.

Ingen ny Strategy v1.0-position får öppnas medan en befintlig strategy position fortfarande är öppen.

Risk Engine och broker reconciliation förstärker denna regel.

Motsatt Setup

Om en valid opposite setup uppstår medan en position är öppen ska den:

ignoreras för execution

Systemet ska inte automatiskt:

- hedge
- reverse
- close current trade
- öppna opposite position

Den observerade setupen kan däremot journalföras för research.

Re-entry

Re-entry får endast ske efter:

förlorande trade

Efter en loss får ett nytt attempt ske när en ny fullständigt giltig setup uppstår inom samma 4H-thesis.

Ingen normal strategy cooldown krävs.

Max tre Attempts

Canonical gräns:

max 3 attempts per 4H thesis

Det betyder:

Attempt 1

→ loss

Attempt 2 kan bli möjlig.

→ loss

Attempt 3 kan bli möjlig.

Efter tre attempts är thesisen färdig för execution.

Risk Engine kan stoppa trading tidigare.

Re-entry efter Winner

Efter en vinnande trade:

ingen ytterligare re-entry på samma thesis

Re-entry efter Break-Even

Samma gäller efter BE.

Break-even avslutar fortsatt re-entry för denna thesis.

Re-entry är inte Revenge Trading

Ett nytt attempt kräver:

en helt ny valid setup

Det räcker alltså inte att föregående trade stoppades.

Risk Baseline

Canonical strategy-dokumentet refererar till den initiala manuella riskbaslinjen:

Max risk/trade: \$150

Max daily drawdown: \$450

Max attempts/4H thesis: 3

Riskvärdena är dock separata RiskProfile-parametrar och ska inte hårdkodas inne i Strategy Engine.

Strategy och Risk separeras

Strategy Engine svarar:

Är setupen giltig?

Risk Engine svarar:

Får kontot ta den?

En setup kan därför vara:

STRATEGY_PASS

men:

RISK_DENY

Det är ett korrekt systemresultat.

News Policy

Strategy v1.0 har en egen explicit news policy.

Den gäller relevant:

high-impact USD-news

och inkluderar bland annat:

- FOMC
- CPI
- NFP

New Entries runt News

Inga nya entries tillåts från:

T-1 timme

till:

T+4 timmar

kring relevant high-impact USD-event.

Existing Position före News

Om en position redan är öppen ska den stängas:

T-15 minuter

före relevant high-impact USD-news.

Detta är en explicit canonical exitregel.

Atlas får inte välja att hålla positionen genom eventet därför att den “ser stark ut”.

News-regler ska vara strukturerade

News policy får inte bero på att Atlas läser en rubrik och subjektivt bedömer:

Den här nyheten verkar inte så viktig.

Systemet ska använda strukturerade NewsEvents och definierad impact classification.

Unknown News State

I execution-enabled mode ska okänd critical news state inte tolkas som:

SAFE

Systemets generella fail-closed-princip ska gälla.

London Trade Management

London entry window är:

02:00–05:00 America/New_York

En trade måste öppnas inom detta window.

Om positionen fortfarande är öppen exakt 05:00 America/New_York gäller obligatorisk window-close break-even:

SL → Entry Price

Detta gäller även om den swing-baserade break-even-triggern ännu inte har inträffat.

Efter denna action:

SL = Entry Price

Positionen får därefter fortsätta.

London trades har:

ingen explicit fyratimmarsgräns

De fortsätter tills en annan explicit canonical exitregel inträffar, exempelvis:

- TP
- SL
- BE
- relevant news exit
- emergency- eller safety-exit
- annan explicit canonical exitregel

Window-close break-even är deterministisk och tidsstyrd. Den får inte hoppas över av AI, UI eller operatör.

New York Trade Management

New York entry window är:

10:00–12:00 America/New_York

En position får fortsätta efter 12:00 om den redan öppnats korrekt.

Men för New York trades gäller:

Max trade duration = 4 timmar från entry

förutsatt att relevant news inte kräver tidigare exit.

Time Exit

Om en New York trade fortfarande är öppen fyra timmar efter entry ska den stängas enligt strategy time-exit.

Detta ska journalföras separat från:

- TP
- SL
- BE
- news exit

Strategy Invalidation före Entry

En setup ska betraktas som invalid om något av följande gäller:

- entry utanför trading window
- required confirmation saknas
- R:R under 2.0
- relevant news blackout aktiv
- annan position redan öppen
- max attempts nått

Utöver detta kan Risk Engine och Prop Firm Rules Engine neka en annars giltig setup.

Invalid betyder inte Risk Denied

Det är viktigt att skilja:

STRATEGY_INVALID

från:

RISK_DENIED

Exempel:

R:R = 1.7

är Strategy Invalid.

Men:

R:R = 2.4 Risk required = \$170 Max risk = \$150

kan vara:

Strategy PASS

Risk DENY.

Dessa resultat måste journalföras separat.

Strategy State Machine

Strategy v1.0 ska kunna representeras genom en deterministisk state machine.

Den canonical ordningen är:

WAIT_FOR_4H_OPEN

↓

IDENTIFY_5_15M_TARGETS

↓

WAIT_FOR_MANIPULATION

↓

MANIPULATION_CONFIRMED

↓

ENTER_1M_CONFIRMATION_PHASE

↓

WAIT_FOR_IFVG_OR_CISD

↓

EVALUATE_SMT

↓

GRADE_SETUP

↓

CALCULATE_ENTRY

↓

CALCULATE_TECHNICAL_SL

↓

FIND_FIRST_VALID_TARGET_GE_2R

↓

STRATEGY_VALIDATION

↓

STRATEGY_SIGNAL

Varför State Machine behövs

State machine gör strategin:

- reproducerbar
- testbar
- auditerbar
- visualiserbar

Atlas Market View kan exempelvis visa:

```
WAIT_FOR_MANIPULATION
```

och förklara:

Relevant 15m FVG finns under price men har ännu inte touchats.

Eller:

```
WAIT_FOR_IFVG_OR_CISD
```

och förklara:

Manipulationen är complete men 1m confirmation saknas fortfarande.

Strategy Signal

När samtliga strategy requirements är uppfylla skapas:

```
STRATEGY_SIGNAL
```

Signal kan bland annat innehålla:

- strategy version
- thesis
- instrument
- direction
- grade
- entry
- technical SL
- target
- R:R
- relevant confirmation
- timestamps

Strategy Signal är inte en Trade

Detta är en av systemets viktigaste boundaries.

Strategy Engine slutar sin authority vid:

```
Strategy Signal
```

Efter detta börjar kontrollkedjan.

Authority Chain

Efter signal går processen:

```
Strategy Engine
→ AI Analysis
→ Risk Engine
→ Prop Firm Rules Engine
→ Trade Proposal
→ Approval / Automation Policy
→ Execution
→ Futures Execution Provider
```

Strategy Engine får inte skicka orders direkt.

AI:s roll i strategin

Atlas får:

- förklara setup
- sammanfatta structure
- beskriva manipulation
- visa confirmation
- förklara grade
- jämföra historik
- uttrycka osäkerhet

Atlas får inte:

- skapa egna entryregler
- flytta technical SL
- kringgå news
- kringgå Risk
- kringgå Prop
- exekvera själv

Atlas Market View

Strategin ska vara visuellt transparent.

Atlas Market View ska kunna visa:

- NQ/MNQ chart
- selected 4H opens
- 5m/15m liquidity
- FVG
- swing highs/lows
- intermediate liquidity
- manipulation
- 1m structure
- iFVG
- CISD

- SMT
- proposed entry
- SL
- TP
- R:R
- grade
- strategy state
- Risk status
- Prop status
- proposal
- position
- BE state
- final result

UI:t ska visa strategy state.

Det ska inte skapa strategy state.

Journal

Varje relevant setup ska kunna journalföras även om ingen trade tas.

Detta ska bland annat göra det möjligt att analysera:

- rejected setups
- invalid setups
- grades
- attempts
- liquidity types
- SMT
- execution
- performance

Strategispecifikationen kräver därför betydligt mer historik än en vanlig lista över closed trades.

Backtesting

Backtesting ska testa strategin exakt som definierad.

Vi får inte i efterhand förändra regler och fortfarande kalla resultatet:

Strategy v1.0

En materiell förändring kräver ny version.

Strategy Versioning

Ändringar av exempelvis:

- entry
- confirmation
- SL
- TP

- grades
- sessions
- news policy
- BE
- re-entry
- duration
- filters

ska versionshanteras.

Candidate Strategy

Exempel:

Canonical:

v1.0

Atlas upptäcker senare att C-grade verkar svag.

Vi testar:

v1.1-candidate

som endast tillåter B eller högre.

Detta är en ny candidate.

Historiska v1.0-trades behåller sin ursprungliga classification.

Performance per komponent

Systemet ska kunna mäta strategyresultat efter:

- A+
- A
- B
- C
- session
- long/short
- SMT
- confirmation type
- target type
- timeframe
- market regime
- attempt
- fees/slippage

På så sätt kan strategin utvecklas utifrån data istället för magkänsla.

Rejected Trade Analytics

Även trades som inte genomförs ska kunna analyseras.

Exempel:

STRATEGY_PASS
RISK_DENY

eller:

STRATEGY_FAIL
RR_BELOW_2

Detta gör att Atlas senare kan göra counterfactual research utan att historiska beslut ändras.

Strategy Baseline Safety Principle

Strategin bygger på fyra separata begrepp:

Strategy Signal

= observation att strategy conditions är uppfyllda.

Trade Proposal

= strukturerad plan som kan presenteras.

Risk/Prop Approval

= tillstånd från kontrollagren.

Execution

= faktisk extern handling.

Dessa får aldrig behandlas som samma sak.

Vad Canonical v1.0 låser

Canonical v1.0 låser bland annat:

- NQ/MNQ som primär validering
- ES för SMT
- selected 02:00 och 10:00 4H opens
- London 02:00–05:00
- New York 10:00–12:00
- 5m/15m liquidity/FVG context
- manipulation via liquidity sweep eller FVG touch
- 1m confirmation
- iFVG eller CISD minimum
- SMT som extra confirmation
- A+/A/B/C grades
- direct confirmation-close entry
- technical structure SL
- första liquidity target $\geq 2R$
- minimum 2R

- canonical BE trigger
- inga partials
- ingen continuous trailing
- en position
- max tre attempts efter losses
- ingen re-entry efter winner/BE
- news blackout T-1h → T+4h
- existing position news exit T-15m
- London position management
- New York max fyra timmar

Detta är Strategy v1.0.

Vad som fortfarande måste bli machine-readable före implementation

Canonical Strategy Specification definierar uttryckligen att de exakta programmeringsdefinitionerna av:

- iFVG
- CISD

ska dokumenteras som separata deterministic detection rules innan Strategy Engine implementeras.

Detta innebär inte att strategy direction eller tradingprincipen är olåst.

Det betyder att den tekniska detektorn måste definieras exakt nog för kod.

Vi ska inte fylla detta gap genom att låta Claude, Atlas eller utvecklaren improvisera under implementation.

Samma Definition Everywhere

När detection specificationen väl är låst ska samma definition användas i:

Historical Backtest

= samma.

Replay

= samma.

Forward

= samma.

Demo

= samma.

Live

= samma.

Detta är nödvändigt för att Strategy v1.0 verkligen ska vara samma strategy i alla environments.

Den fullständiga Strategy Flow

Canonical Strategy v1.0 kan sammanfattas som:

Selected 4H Open

↓

Identifiera 5–15m Liquidity / FVG

↓

Vänta på Manipulation

↓

Liquidity Sweep eller FVG Touch

↓

1m Entry Phase

↓

Optional 1m Liquidity

↓

iFVG / CISD

↓

SMT om tillgänglig

↓

Setup Grade

↓

Entry på Confirmation Close

↓

Technical SL bakom Manipulation Swing

↓

Första Liquidity Target $\geq 2R$

↓

Strategy Signal

↓

Risk Engine

↓

Prop Firm Rules Engine

↓

Trade Proposal

↓

Approval / Automation

↓

Execution

↓

Journal & Analytics

Strategins filosofi i praktiken

Strategin försöker alltså inte förutsäga varje rörelse i Nasdaq.

Den väntar på en specifik process:

Context

→ Manipulation

→ Confirmation

→ Defined Risk

→ Defined Target

Om processen inte uppstår:

NO TRADE

Det är ett korrekt strategy outcome.

Kapitelstatus

Kapitel: 3 – Strategispecifikation

Bok: Omnira Trading System – Från strategi till autonom exekvering

Strategi: Omnira Liquidity Manipulation – Canonical v1.0

Status: Bokbaseline dokumenterad

Primär marknad: NQ / MNQ

SMT comparison: ES

Timezone: America/New_York

London entry: 02:00–05:00

New York entry: 10:00–12:00

Selected 4H opens: 02:00 och 10:00

HTF context: 5m / 15m Liquidity och FVG

Manipulation: Liquidity sweep OR FVG touch

1m liquidity sweep: Preferred, inte obligatorisk

Mandatory confirmation: iFVG OR CISD

Grades: A+, A, B, C

Entry: Confirmation candle close

Technical SL: Bakom senaste giltiga manipulation swing

TP: Första giltiga liquidity target $\geq 2R$

Minimum R:R: 2.0R

Partial profits: Nej

Trailing: Nej, utöver canonical BE

Break-even: När närmaste confirmed 1m swing efter entry tas, samt obligatoriskt vid London window-close 05:00

Max positioner: 1

Re-entry: Endast efter loss

Max attempts: 3 per 4H thesis

Re-entry efter winner/BE: Nej

News entry blackout: T-1h → T+4h

Existing position news exit: T-15m

London max duration: Ingen explicit 4h-limit

New York max duration: 4 timmar

iFVG/CISD programming definitions: Separat deterministic detection specification krävs före Strategy Engine implementation

Strategy Signal: Inte execution authority

Omnira Liquidity Manipulation v1.0 är därmed den definierade tradingbaseline som resten av Omnira Trading System ska implementera, testa, försöka motbevisa och – endast om evidensen håller – senare få rätt att exekvera.

KAPITEL 4

Riskhantering

Omnira Trading System är byggt utifrån principen att en tradingstrategi aldrig får stå över riskkontrollen.

Strategin kan identifiera en setup. Atlas kan analysera den. Systemet kan bedöma att sannolikheten ser god ut. Men inget av detta innebär att trade får tas.

Risk Engine har alltid sista ordet.

Detta är en central säkerhetsprincip i hela Omnira Trading System.

Risk Engine står över AI och strategi

Risk Engine är deterministisk och regelstyrd.

Den ska inte försöka förutsäga marknaden och den ska inte påverkas av AI-confidence, känslor, winning streaks eller tidigare förluster.

Om Strategy Engine exempelvis identifierar en A+-setup och Atlas bedömer marknadskontexten som mycket stark, men Risk Engine upptäcker att dagens tillåtna risk redan är förbrukad, ska resultatet alltid vara:

TRADE DENIED

Ingen AI-modell får kunna kringgå detta beslut.

Initial riskmodell

Den första versionen av Omnira Trading använder följande interna riskbaseline:

- max risk per trade: \$150
- max daily drawdown: \$450
- max öppna positioner: 1
- max tre attempts per 4H-thesis
- ingen extra losing-streak-regel
- ingen separat max trades-per-day
- inget spread-filter
- ingen separat intern total drawdown-gräns
- ingen separat margin utilization-gräns

Dessa regler är inte avsedda att vara universella för alla framtida konton.

Riskparametrarna ska vara konfigurerbara och versionsstyrda.

Risk per trade

Den tekniska stop lossen bestäms av strategin.

Risk Engine får därefter använda:

- entry
- stop loss
- tick size
- tick value
- contract specification
- minsta handlingsbara position

för att beräkna hur stor position som är tillåten.

Risk Engine får aldrig flytta den tekniska stop lossen närmare entry enbart för att få positionen att passa riskbudgeten.

Om den minsta möjliga positionen riskerar mer än \$150 ska trade nekast.

Principen är:

Anpassa positionen efter risken. Anpassa aldrig den tekniska risken efter önskad position.

Daily drawdown

Den interna Omnira-regeln är:

max \$450 realiserad förlust per tradingdag.

Detta innebär att den interna daily loss-beräkningen baseras på realiserade förluster.

Floating P/L används inte som den interna Omnira daily-loss-mätaren.

Detta ska inte förväxlas med regler hos en prop firm.

En prop firm kan exempelvis använda equity-baserad drawdown och därmed inkludera öppna förluster.

Prop Firm Rules Engine hanterar dessa regler separat.

En trade måste därför kunna passera både:

Internal Risk Engine

och:

Prop Firm Rules Engine

Om något av systemen nekar trade får den inte exekveras.

Reserved risk

Att dagsmätaren är realiserad betyder inte att en ny trade får ta hela den återstående budgeten i anspråk utan kontroll.

Innan en ny trade tillåts ska dess möjliga initiala risk reserveras mot återstående dagsbudget:

`realized_daily_loss + reserved_risk_for_new_trade <= daily_loss_limit`

Exempel:

```

realized_daily_loss = $300
daily_loss_limit    = $450
daily_remaining     = $150

ny trade risk = $128 → passerar denna regel
ny trade risk = $151 → DENY

```

Reserved risk är en **pre-entry-kontroll**. Den ändrar inte definitionen av realiserad daily loss, och den skriver inte in något i dagsmätaren. Den avgör bara om en ny trade får öppnas.

Reservationen frisläpps när traden stängs. Det faktiska utfallet, vinst eller förlust, bokförs då mot den realiserade mätaren.

Daily reset

Den interna daily risk-budgeten återställs vid:

```
00:00 America/New_York
```

Detta är den canonical tradingdagen för den interna riskmodellen.

Systemet får inte använda serverns lokala timezone som implicit daily-reset.

Daily stop

När:

```
realized_daily_loss >= $450
```

ska följande gälla:

- inga nya trades tillåts
- risk state blir **BLOCKED** / **DAILY_STOP**
- execution av nya intents blockeras
- state är persistent över restart
- trading förblir blockerad till nästa canonical reset
- ett audit event skapas

Risk Engine ska visa kontot som blockerat för nya entries fram till nästa daily reset.

En daily stop ska inte kunna återställas genom att ladda om UI:t.

Detta är en explicit riskregel och inte en frivillig rekommendation.

Ingen intern floating force-close

Den interna dagsregeln stänger **inte** en redan öppen position.

Eftersom Omniras interna mätare endast räknar realiserade förluster, och strategin tillåter högst en öppen position, kan den interna gränsen inte brytas av en position som fortfarande är öppen. Att införa en intern tvångsstängning baserad på floating P/L skulle motsäga realized-only-modellen.

En redan öppen position hanteras därför vidare genom sina övriga giltiga regler:

- teknisk stop loss
- take profit

- canonical break-even
- London window-close break-even 05:00
- news exit
- New York time exit
- emergency- och safety-kontroller
- Prop Firm Rules

Om ett aktivt PropFirmProfile använder equity-, floating- eller trailingbaserade gränser och kräver en tidigare skyddsåtgärd gäller den striktare prop-regeln. Intern risk och prop firm-risk hålls fortsatt separata.

Att stoppa nya trades och att tvångsstänga befintlig exponering är två olika actions.

Max tre attempts per 4H-thesis

Omnira Liquidity Manipulation tillåter maximalt tre attempts på samma 4H-thesis.

Detta betyder inte att tre trades alltid är tillåtna.

Varje attempt måste fortfarande passera Risk Engine.

Exempel:

Attempt 1 förlorar \$150.

Attempt 2 förlorar \$150.

Dagens realiserade förlust är då \$300.

Om nästa giltiga setup kräver mer än återstående \$150 i tillåten daily risk ska trade nekas, även om det fortfarande är det tredje möjliga attemptet.

Attempt-limit och risk-budget är två separata regler.

Båda måste passera.

Re-entry är inte martingale

Re-entry efter en förlust innebär inte att risk ska höjas.

Varje nytt attempt använder samma riskregler.

En tidigare förlust får aldrig automatiskt motivera större position size på nästa trade.

Omnira Trading ska inte använda martingale som standardprincip.

En öppen position åt gången

Den första strategiversionen tillåter endast en öppen position åt gången.

Detta förenklar:

- riskberäkning
- exposure
- reconciliation
- journaling
- prop firm-kontroll

Om providern visar en position som Omnira inte känner till ska systemet behandla detta som en säkerhetsincident.

Ny trading ska blockeras tills positionen har identifierats och systemets state är reconciled.

Hard limits och mänsklig override

Ett Risk DENY ska inte ha en vanlig:

Trade anyway

-knapp.

Om en riskregel ska förändras ska själva Risk Profile ändras genom en kontrollerad och versionsstyrd process.

Detta säkerställer att Risk Engine inte reduceras till en rekommendation som kan ignoreras när användaren känner starkt för en trade.

Fail closed

När Risk Engine inte kan avgöra om en trade är säker ska standardresultatet vara:

DENY

Exempel:

- account data är gammal
- market data är gammal
- instrumentmetadata saknas
- tick value är okänd
- position state är okänd
- riskkonfigurationen är felaktig
- kill switch är aktiv
- execution runner är offline
- broker connection är osäker

Systemet får aldrig tolka avsaknad av information som att trading är tillåten.

Kill switch

Omnira Trading ska stödja flera nivåer av kill switch:

- global
- konto
- strategi
- instrument
- execution runner

Aktiv kill switch blockerar ny execution.

Kill switch ska vara synlig i Omnira Trading UI.

Risk i Atlas Market View

Risk Engine ska vara visuellt tydligt inne i Omnira.

Användaren ska kunna se exempelvis:

- risk per trade
- risk i procent
- quantity
- daily loss used
- daily loss remaining
- attempts used
- max attempts
- öppna positioner
- kill switch-status
- Risk Engine-resultat

En setup kan därför visuellt visa:

Strategy Engine: PASS

Risk Engine: DENY

Det ska direkt gå att förstå varför systemet inte får ta traden.

Prop firm-risk är separat

Omnira ska inte hårdkoda en enda prop firms regler som systemets globala riskmodell.

Varje prop firm och challenge ska kunna ha en egen versionsstyrd regelprofil.

Exempel på regler som Prop Firm Rules Engine senare måste kunna hantera är:

- daily loss
- maximum loss
- static drawdown
- trailing drawdown
- equity-based drawdown
- reset-tider
- challenge-regler
- news restrictions
- holding restrictions
- consistency rules

Den striktaste tillåtna gränsen mellan intern risk och prop firm-risk ska vinna.

Risk och self-improvement

Atlas ska kunna analysera Risk Engines historiska beslut.

Det innebär bland annat att systemet senare kan mäta:

- hur många trades som nekats
- varför trades nekats

- hur nekade trades senare utvecklades
- hur olika riskprofiler hade påverkat performance
- hur risk påverkat drawdown och expectancy

Atlas får därefter skapa hypoteser och föreslå förbättringar.

Atlas får inte själv ändra riskprofilen live.

Ett förbättringsförslag måste behandlas som en ny kandidatversion som testas och godkänns innan den kan bli canonical.

Den centrala riskprincipen

Risk Engine finns inte för att maximera vinsten.

Risk Engine finns för att begränsa skadan när något annat i systemet har fel.

Strategin kan ha fel.

Atlas kan ha fel.

Market data kan ha fel.

Execution kan misslyckas.

Marknaden kan bete sig på ett sätt som aldrig tidigare observerats.

Därför ska systemet byggas utifrån antagandet att fel förr eller senare kommer att inträffa.

Risk Engine ska begränsa konsekvenserna när de gör det.

Kapitelstatus

Kapitel: 4 – Riskhantering

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Revision: 2026-08-27 – reserved risk och daily stop gjorda explicita, intern floating force-close borttagen

Max risk per trade: \$150

Intern daily loss: \$450, realized only

Daily reset: 00:00 America/New_York

Reserved risk: Pre-entry-kontroll, ändrar inte realiserad mätare

Intern floating force-close: Finns inte

Max öppna positioner: 1

Max attempts per 4H-thesis: 3

Human override av hard risk DENY: Finns inte

Prop firm-risk: Separat lager, striktaste gräns vinner

KAPITEL 5

Systemarkitektur

Omnira Trading System ska inte byggas som en enda bot som både analyserar marknaden, bestämmer risk och skickar order.

Systemet ska istället bestå av flera tydligt separerade lager med olika ansvar och olika auktoritet.

Den grundläggande arkitekturen är:

```
Market Data
→ Strategy Engine
→ AI Analysis
→ Risk Engine
→ Prop Firm Rules Engine
→ Trade Proposal
→ Approval / Automation Policy
→ Execution Gateway
→ Execution Runtime
→ Futures Execution Provider
→ Broker / Prop Firm
→ Journal & Analytics
```

Denna separation är en av systemets viktigaste säkerhetsprinciper.

Ingen enskild komponent ska ha större auktoritet än den behöver.

Trading som egen domän i Omnira

Trading ska vara en tydligt isolerad domän inom Omnira.

Tradingdomänen ska äga sina egna:

- konton
- brokers
- instrument
- strategier
- strategy versions
- market data
- setups
- trade proposals
- riskprofiler
- prop firm-profiler
- orders
- positions
- trades
- journal
- analytics
- approvals

- execution state
- audit logs

Atlas ska kunna läsa och arbeta med denna information genom tydliga Trading APIs och events.

Tradinglogik ska inte spridas genom andra generella delar av Omnira.

Det ska göra systemet lättare att testa, säkra, versionshantera och senare bygga ut med flera strategier eller flera konton.

Market Data Layer

Market Data Layer är systemets första tekniska lager.

Dess uppgift är att:

- läsa market data
- normalisera data
- kontrollera timestamps
- kontrollera instrument
- kontrollera timeframes
- upptäcka stale eller saknad data
- leverera kontrollerad data vidare till Strategy Engine

För den första strategin behövs framför allt:

- NQ eller MNQ
- ES
- 1m
- 5m
- 15m
- 4H

Data ska i möjligaste mån kunna komma från flera framtida providers utan att Strategy Engine måste byggas om.

Market Data Layer ska därför fungera som en adapter mellan externa datakällor och Omniras interna format.

Datakvalitet före analys

Strategy Engine får inte arbeta vidare som om allt är normalt om kritisk data saknas eller är osäker.

Exempel på problem är:

- bars saknas
- candles kommer i fel ordning
- timestamp är fel
- market data är för gammal
- fel instrument används
- fel timeframe används
- duplicate bars
- data source är disconnected

Vid kritiska dataproblem ska systemet som standard:

FAIL CLOSED

Det innebär att analys kan markeras som osäker, men ingen exekverbar trade proposal får skapas.

Strategy Engine

Strategy Engine ansvarar för själva strategin.

Den första strategin är:

Omnira Liquidity Manipulation – Canonical v1.0

Strategy Engine ska bland annat:

- identifiera relevant 4H-open
- identifiera liquidity
- identifiera FVG
- vänta på manipulation
- upptäcka iFVG
- upptäcka CISD
- utvärdera SMT
- grade setup
- beräkna entry
- definiera technical stop loss
- hitta target
- beräkna initial R:R
- skapa Strategy Signal

Strategy Engine ska vara deterministisk.

Om exakt samma strategy version får exakt samma market data och configuration ska samma resultat produceras.

Strategy Signal är inte en order

Detta är en central princip.

När Strategy Engine producerar:

STRATEGY_SIGNAL

betyder det:

Strategins regler är uppfyllda.

Det betyder inte:

Skicka en order.

Strategy Signal går vidare till nästa lager för ytterligare granskning.

Detta skiljer systemet från enklare tradingbots där signal och execution ofta är samma steg.

Strategy Plugin Model

Omnira ska inte byggas specifikt runt endast en strategi.

Den första strategin ska fungera som Strategy #1, men systemarkitekturen ska stödja flera framtida strategier.

Exempel:

Strategy A – Omnira Liquidity Manipulation

Strategy B – framtida trendstrategi

Strategy C – framtida mean reversion

Alla strategier ska implementera samma principiella kontrakt.

Exempel:

- required data
- detect setup
- evaluate setup
- calculate entry
- calculate invalidation
- calculate targets
- grade setup
- explain signal

Detta gör att Risk Engine, execution, journal, analytics och Atlas Market View inte behöver byggas om när en ny strategi introduceras.

Strategy Versioning

Varje strategi ska versionshanteras.

Exempel:

Omnira Liquidity Manipulation v1.0

och senare:

Omnira Liquidity Manipulation v1.1

Om en regel ändras ska tidigare performance fortfarande höra till den version som faktiskt användes.

En ändring av exempelvis:

- entry
- SL
- TP
- minimum R:R
- news policy
- setup grade
- break-even

- session

får inte i efterhand blandas ihop med resultat från en tidigare strategy version.

AI Analysis

AI Analysis ligger efter Strategy Engine.

Atlas ska alltså inte primärt sitta och titta på rå chart och fritt bestämma om en trade ska tas.

Strategy Engine ska först skapa en strukturerad signal.

Därefter kan Atlas analysera:

- setupen
- market context
- setup grade
- market regime
- historisk performance
- liknande setups
- eventuella osäkerheter

AI ska vara ett beslutsstöd.

AI får inte själv skapa hard risk limits eller kringgå deterministic rules.

Atlas roll

Atlas ska fungera som det intelligenta lagret ovanpå de deterministiska komponenterna.

Atlas ska kunna:

- förklara varför en setup finns
- beskriva vad Strategy Engine väntar på
- sammanfatta marknadsförhållanden
- jämföra mot historiska setups
- visa riskstatus
- visa prop firm-headroom
- identifiera osäkerheter
- skapa ett begripligt trade proposal
- analysera historiska resultat
- hjälpa till med self-improvement

Atlas ska däremot inte kunna säga:

Jag är 95 % säker, därför ignorerar vi daily stop.

Hard limits gäller oavsett AI-bedömning.

Risk Engine

Risk Engine är ett separat deterministiskt säkerhetslager.

Det ansvarar bland annat för:

- max risk per trade
- daily loss

- max öppna positioner
- max attempts
- position sizing
- minimum contract risk
- kill switches
- account health
- market data health
- execution health

Risk Engine kan returnera:

ALLOW

eller:

DENY

Risk Engine har veto.

Prop Firm Rules Engine

Prop Firm Rules Engine är separat från intern risk.

Anledningen är att en prop firm kan använda andra regler än Omniras egna.

Exempel:

- equity-based daily drawdown
- static maximum loss
- trailing drawdown
- minimum trading days
- consistency rules
- news restrictions
- weekend holding
- instrument restrictions
- account-specific limits

Varje prop firm-profil ska vara versionsstyrd.

Ingen generell prop firm-regel ska hårdkodas globalt i systemet.

Den striktaste regeln vinner

En trade måste passera både:

Internal Risk Engine

och:

Prop Firm Rules Engine

Om intern risk säger:

ALLOW

men prop firm säger:

DENY

blir slutresultatet:

DENY

Samma gäller åt andra hållet.

När två risklager har olika praktiska gränser ska den striktaste tillåtna gränsen vinna.

Trade Proposal

När en strategy signal har analyserats och de relevanta kontrollagren passerats kan Omnira skapa ett Trade Proposal.

Trade Proposal ska kunna innehålla:

- instrument
- direction
- strategy
- strategy version
- setup grade
- entry
- stop loss
- take profit
- R:R
- quantity
- risk i dollar
- risk i procent
- Atlas analys
- Risk Engine-resultat
- Prop Firm-resultat
- proposal expiry
- status

Trade Proposal är fortfarande inte en order.

Approval och automation modes

Omnira Trading ska kunna arbeta i flera olika operation modes.

Analysis Only

Systemet analyserar marknaden.

Ingen broker execution är tillåten.

Read Only

Systemet läser konto- och market data från providern.

Orderläggning är tekniskt avstängd.

Demo Manual Approval

Omnira skapar Trade Proposal.

Användaren måste godkänna.

Demo Automation

Systemet får automatiskt exekvera godkända strategisignaler på demo.

Live Manual Approval

Live trade kräver explicit mänsklig approval.

Controlled Live Automation

Tillåts endast efter senare definierade performance- och safety-gates.

Systemet får aldrig automatiskt höja sin egen autonomy level.

Execution Gateway

Execution Gateway fungerar som säkerhetsgränsen mellan Omnira och providermiljön.

Gateway ska verifiera:

- proposal ID
- approval
- expiry
- kill switch
- account
- duplicate execution
- current risk state
- current prop firm state

Först därefter får ett Execution Intent skapas.

Execution Intent

Execution Intent är den strukturerade instruktionen till execution-runnern.

Det kan innehålla:

- execution ID
- proposal ID
- account
- instrument
- direction
- quantity
- order type

- SL
- TP
- expiry
- approval reference

Execution Intent ska vara idempotent.

Om samma request skickas två gånger på grund av nätverksproblem får detta inte kunna skapa två trades.

Execution Runtime

Den första executionmiljön ska vara en dedikerad Windows-dator.

Den stationära datorn kan användas som 24/7-rigg i den första fasen.

Runnern ansvarar för:

- kommunikation med Omnira
- kommunikation med providern
- read-only account sync
- framtida order execution
- broker response
- position updates
- heartbeat
- reconciliation

Runnern ska vara så enkel och deterministisk som möjligt.

Den ska inte innehålla självständig strategi-AI.

Varför execution separeras från Omnira

Att lägga providern execution i en separat runner ger flera fördelar.

Omnira kan fortsätta vara ansvarig för:

- logik
- risk
- UI
- Atlas
- approvals
- historik
- analytics

medan Windows-miljön ansvarar för själva providerintegrationen.

Det gör det senare möjligt att flytta execution från stationär dator till VPS utan att hela Omnira behöver byggas om.

Initial 24/7-rigg

Den första Windows-riggen ska minst ha:

- sleep avstängt

- stabil internetanslutning
- automatisk startup
- provideranslutning autostart
- Execution Runner autostart
- korrekt systemtid
- health monitoring
- loggning
- diskövervakning
- kill switch
- reconciliation efter restart

Om riggen förlorar kontakt med Omnira ska standardläget vara:

inga nya trades

Befintliga broker-native SL och TP ska fortsätta skydda redan öppna positioner.

VPN och nätverk

Executionmiljön ska vara flexibel.

Den ska kunna stödja exempelvis:

- direct connection
- approved VPN
- static VPN
- VPS

Men systemet får inte anta att VPN alltid är tillåtet.

Broker- och prop firm-regler ska kunna avgöra vilken network policy som är godkänd för ett specifikt konto.

Execution Provider Adapter

providerintegrationen ska delas upp i två logiska delar.

Read Adapter

Ansvarar för att läsa:

- account info
- balance
- equity
- quotes
- bars
- ticks
- orders
- positions
- historical trades

Execution Adapter

Ansvarar senare för:

- order pre-check
- order submission
- modification
- close
- broker response

Read-only ska implementeras och verifieras innan Execution Adapter aktiveras.

Pre-Execution Revalidation

Även om ett Trade Proposal redan har godkänts ska systemet kontrollera läget igen direkt före execution.

Exempel på saker som kan ha förändrats:

- pris
- R:R
- spread
- daily risk
- open position
- news state
- kill switch
- prop firm headroom
- broker health

Om trade conditions inte längre är giltiga ska execution stoppas.

Fail Closed

Alla kritiska systemfel ska leda till:

ingen ny trade

Det gäller exempelvis:

- Risk Engine offline
- Runner offline
- stale data
- broker disconnected
- invalid account
- reconciliation failure
- unknown position
- expired proposal
- auth failure

Trading ska aldrig fortsätta på antagandet att allt förmodligen är okej.

Reconciliation

Omnira och providern måste kunna återställa gemensam state efter exempelvis:

- restart
- nätverksfel
- runner crash
- Omnira outage

Vid startup ska systemet kontrollera:

- open positions
- pending orders
- account state
- senaste execution
- senaste known Omnira state

Om states inte matchar ska ny trading blockeras tills discrepancy är löst.

Unknown Position

Om providern visar en öppen position som Omnira inte känner igen ska den markeras som:

UNKNOWN_POSITION

Ny execution ska blockeras.

Systemet ska inte automatiskt anta att positionen tillhör Omnira.

Atlas Market View

Omnira Trading ska innehålla en central TradingView-liknande marknadsvy.

Detta är den visuella representationen av vad Atlas och Trading System ser.

Vyn ska kunna visa:

- candles
- 4H-open
- liquidity
- FVG
- manipulation
- iFVG
- CISD
- SMT
- entry
- stop loss
- take profit
- setup grade
- R:R
- strategy state
- riskstatus
- prop firm-status

- trade proposal
- approval
- open position
- break-even
- trade result

Målet är att användaren ska kunna öppna Omnira Trading och snabbt förstå:

Vad ser Atlas?

Vad väntar Strategy Engine på?

Finns en setup?

Var planeras entry?

Var ligger SL och TP?

Vad säger Risk Engine?

Vad säger Prop Firm Engine?

Varför tas eller nekas trade?

UI är inte source of truth

TradingView-liknande UI ska endast visualisera systemets state.

Chart-komponenten får inte själv implementera strategi-regler.

Source of truth ska ligga i backend.

Om användaren stänger UI:t ska Strategy Engine fortfarande ha exakt samma state.

Realtime Events

Tradingdomänen ska kunna publicera events såsom:

- SETUP_CREATED
- MANIPULATION_CONFIRMED
- ENTRY_CONFIRMATION_DETECTED
- STRATEGY_SIGNAL_CREATED
- RISK_DENIED
- PROP_DENIED
- PROPOSAL_CREATED
- PROPOSAL_APPROVED
- EXECUTION_REQUESTED
- ORDER_FILLED
- POSITION_UPDATED
- TRADE_CLOSED
- KILL_SWITCH_ACTIVATED

Atlas Market View kan använda dessa events för realtime-uppdatering.

Journal och Analytics

Journalen ska inte endast lagra trade-resultatet.

Systemet ska kunna återskapa hela beslutsprocessen.

Det innebär att journalen bland annat ska veta:

- vilken strategy version användes
- vilken market data fanns
- vilken setup identifierades
- vad Atlas analyserade
- vad Risk Engine beslutade
- vad Prop Firm Engine beslutade
- vem som godkände
- vad Execution Runner skickade
- vad providern svarade
- vilket fill som erhöles
- varför traden stängdes
- slutligt resultat

Replayability

En historisk trade eller setup ska i möjligaste mån kunna replayas genom samma Strategy Engine som används live.

Systemet ska kunna fråga:

Vad hade Strategy v1.0 sett vid denna tidpunkt, med endast information som fanns tillgänglig då?

Detta är viktigt för att motverka look-ahead bias.

Backtest och Live ska dela logik

Backtestmotorn ska använda samma Strategy Engine-kontrakt som live-systemet.

Det ska inte finnas en separat förenklad strategi som endast används i backtest.

Skillnader ska främst ligga i:

- data adapter
- execution adapter

Detta gör testresultaten mer relevanta för verklig drift.

Atlas Trading Learning & Improvement Layer

Omnira Trading ska innehålla ett separat lager för kontinuerligt lärande och förbättring.

Detta lager ska analysera:

- alla setups
- vinnande trades
- förlorande trades
- break-even trades
- nekade trades
- setups som aldrig nådde entry
- setup grades

- sessions
- SMT
- market regimes
- MFE
- MAE
- spread
- slippage
- execution quality
- risk decisions
- prop firm decisions

Målet är att Atlas över tid ska bygga bättre förståelse för vilka kombinationer som faktiskt har positiv expectancy.

Self-Improvement är inte Self-Modification

Atlas får:

- upptäcka mönster
- jämföra performance
- hitta svaga market regimes
- identifiera starka setupkombinationer
- skapa hypoteser
- föreslå parameterförändringar
- föreslå ny strategy version
- föreslå ny Risk Profile-version

Atlas får inte själv ändra canonical produktionregler.

Förbättringsflödet ska vara:

```
Observe
→ Measure
→ Detect Pattern
→ Create Hypothesis
→ Candidate Version
→ Backtest
→ Out-of-Sample Test
→ Forward Test
→ Review / Approval
→ Ny Canonical Version
```

På detta sätt kan systemet bli smartare utan att samtidigt bli okontrollerbart.

Arkitekturens långsiktiga mål

Den långsiktiga visionen är inte bara en bot som tar trades.

Målet är ett trading-system där:

- Strategy Engine identifierar edge
- Atlas förstår och förklarar kontext

- Risk Engine begränsar skada
- Prop Firm Engine skyddar externa regler
- Execution Runner utför exakt godkända instruktioner
- Journalen dokumenterar allt
- Analytics mäter verklig performance
- Learning Layer hittar förbättringar
- användaren kan se hela processen visuellt

Autonomi är den sista delen av denna kedja, inte den första.

Den centrala arkitekturprincipen

Omnira Trading ska följa principen:

Ingen komponent ska ha större auktoritet än den behöver.

Strategy Engine får identifiera trades.

Atlas får analysera.

Risk Engine får skydda kapital.

Prop Firm Engine får skydda externa regler.

Approval Policy får bestämma behörighet.

Execution Runner får exekvera explicit godkända instruktioner.

Journalen ska dokumentera vad som hände.

Learning Layer får lära sig av resultatet.

Ingen av dessa roller ska i hemlighet ta över de andras ansvar.

KAPITEL 6

Marknadsdata

Marknadsdata är den faktiska verklighet som Omnira Trading System bygger alla sina beslut på.

Strategin kan vara korrekt definierad.

Risk Engine kan vara perfekt implementerad.

Atlas kan ha tillgång till avancerade analysmodeller.

Men om marknadsdatan är fel, fördröjd, ofullständig eller tidsmässigt inkonsekvent kan hela systemets beslutsprocess ändå bli fel.

Därför ska datakvalitet behandlas som en säkerhetsfråga och inte bara som ett tekniskt integrationsproblem.

Den grundläggande principen är:

Om systemet inte kan lita på datan får systemet inte lita på beslutet som bygger på datan.

Marknadsdata som source of truth

Market Data Layer ska vara Omniras canonical källa för normaliserad tradingdata.

Strategy Engine ska inte själv kommunicera direkt med olika externa providers och försöka tolka deras format.

Istället ska flödet vara:

```
External Data Source
→ Market Data Adapter
→ Validation
→ Normalization
→ Canonical Market Data
→ Strategy Engine
```

Detta ger en tydlig gräns mellan externa datakällor och intern strategi-logik.

Varför ett separat Market Data Layer behövs

Olika providers kan representera samma information på olika sätt.

Skillnader kan exempelvis finnas i:

- symbolnamn
- timestamps
- timezone
- candle boundaries
- volume
- spread
- tick format
- contract naming

- session definitions
- historical availability

Strategy Engine ska inte behöva känna till dessa skillnader.

Den ska exempelvis kunna begära:

MNQ

1m

och få samma interna datamodell oavsett vilken godkänd provider som levererar datan.

Primära instrument

Den första valideringen av Omnira Liquidity Manipulation ska ske på:

- NQ
- MNQ

ES används som comparison instrument för SMT.

Det betyder att systemet initialt behöver tillförlitlig data för:

NQ / MNQ

och:

ES

Strategins första edge ska inte antas fungera på andra instrument innan dessa testats separat.

NQ och MNQ

NQ och MNQ följer samma underliggande Nasdaq-100-marknad men är separata futureskontrakt med olika kontraktstorlek.

Det innebär att systemet ska kunna skilja mellan:

- marknadsanalys
- instrument
- execution contract

En analys kan exempelvis göras utifrån Nasdaq-strukturen samtidigt som execution sker i MNQ för att kunna använda mindre position size.

Instrumentens metadata måste därför lagras separat från strategy logic.

ES som SMT-instrument

ES används inte primärt som executioninstrument i Strategy v1.0.

ES används för SMT-confirmation mot NQ.

Systemet behöver därför kunna synkronisera relevanta marknadsstrukturer mellan:

- NQ
- ES

på:

- 1m
- 5m
- 15m

SMT får endast användas när båda instrumentens data är tillräckligt färsk och tidsmässigt jämförbar.

Om ES-data saknas får systemet inte låtsas att:

SMT = false

Det korrekta tillståndet är istället exempelvis:

SMT = UNKNOWN

Eftersom SMT inte är obligatoriskt för entry kan setupen fortfarande existera om övriga strategiregler tillåter det.

Men systemet får inte felaktigt ge A+ grade när comparison data saknas.

Timeframes

Omnira Liquidity Manipulation använder:

- 4H
- 15m
- 5m
- 1m

Varje timeframe har ett specifikt ansvar.

4H

Används för:

- relevanta 4H-opens
- övergripande thesis
- tidigare 4H highs/lows

15m

Används för:

- liquidity
- FVG
- manipulation
- struktur

5m

Används för samma huvudsakliga syften som 15m men med högre detaljnivå.

1m

Används för:

- liquidity

- iFVG
- CISD
- entry
- break-even
- exakt trade management

Timeframe consistency

Systemet ska känna till vilken 1m-data som bygger en 5m-, 15m- eller 4H-candle när aggregation används.

Candle boundaries måste vara reproducerbara.

En 15m-candle får inte skapas på ett sätt i backtest och ett annat sätt live.

Det skulle kunna skapa olika:

- FVG
- swing points
- liquidity levels
- manipulation events

och därmed olika trades.

Canonical timezone

Alla canonical timestamps ska lagras i:

UTC

Strategiregler ska därefter utvärderas mot:

America/New_York

Detta innebär att systemet tydligt skiljer mellan:

storage time

och:

trading context time

Sessionslogik får inte byggas utifrån en permanent UTC-offset eftersom New York använder daylight saving time.

Trading date

Varje relevant event ska kunna kopplas till en trading date.

Systemet behöver exempelvis kunna avgöra att:

2026-08-26 02:15 America/New_York

hör till rätt trading session och daily risk period.

Detta är viktigt för:

- London session
- New York session
- daily reset
- journal
- analytics
- news
- prop firm-regler

Selected 4H Opens

Strategy v1.0 använder endast de relevanta 4H-opens som hör till:

02:00 New York

och:

10:00 New York

Market Data Layer måste därför kunna leverera korrekt 4H-context kring dessa tider.

Övriga 4H-opens får fortfarande finnas i market data, men Strategy v1.0 ska ignorera dem som strategy triggers.

Det är Strategy Engines ansvar.

OHLC-data

Varje bar ska minst kunna innehålla:

- instrument
- timeframe
- open time
- close time
- open
- high
- low
- close
- volume när relevant
- source
- completion state

Systemet ska kunna skilja mellan:

open candle

och:

confirmed closed candle

Detta är kritiskt för exempelvis swing-definitioner där en efterföljande candle måste finnas innan swing point är bekräftad.

Candle finality

Strategy Engine får inte använda framtida information.

Om ett pattern kräver candle close får det inte betraktas som bekräftat innan candle faktiskt stängt.

Detta gäller särskilt:

- swing highs/lows
- iFVG
- CISD
- entry confirmation

Backtestmotorn måste följa samma regel.

Annars uppstår look-ahead bias.

Tick Data

Tick data är inte nödvändigt för alla delar av Strategy v1.0.

Men systemarkitekturen ska stödja tick data eftersom den senare är viktig för:

- precise execution simulation
- bid/ask
- spread
- slippage
- order timing
- fill quality
- intrabar sequencing

Tick data kan därför bli betydligt viktigare i execution- och backtestingfaserna än i första analys-MVP:n.

Bid, Ask och Last

Systemet ska där källan stödjer det kunna skilja mellan:

- bid
- ask
- last traded price

Detta är viktigt eftersom chart price och faktisk execution price inte alltid är samma sak.

En long kan exempelvis exekveras mot ask medan chartdata huvudsakligen visar last eller bid beroende på provider.

Dessa skillnader ska senare inkluderas i realistiska executionmodeller.

Spread

Strategy v1.0 använder inget hard spread-filter.

Det innebär inte att spread ska ignoreras.

Systemet ska fortfarande mäta och journalföra spread vid:

- setup

- signal
- proposal
- execution
- fill

Detta gör det möjligt för Atlas Trading Learning & Improvement Layer att senare analysera om hög spread påverkar:

- win rate
- expectancy
- slippage
- execution quality

Om data senare visar ett robust samband kan Atlas föreslå ett framtida spread-filter som kandidatregel.

Atlas får inte lägga till filtret automatiskt.

Volume

Volume ska lagras när källan tillhandahåller det.

Det är inte ett canonical entrykrav för Strategy v1.0.

Det kan däremot bli användbart för:

- forskning
- market regime classification
- future strategies
- self-improvement

Data som inte används i strategin idag kan fortfarande ha forskningsvärde senare.

Instrument Metadata

Varje instrument ska ha metadata såsom:

- canonical symbol
- broker symbol
- tick size
- tick value
- contract size
- minimum quantity
- quantity step
- exchange
- currency
- trading hours

Risk Engine ska använda denna metadata för korrekt position sizing.

Strategy Engine ska inte hårdkoda exempelvis MNQ:s tick value.

Futures Contract Mapping

Futures använder kontraktsmånader.

Därför behöver Omnira kunna skilja mellan:

canonical market

och:

active contract

Exempel:

MNQ

kan representera den canonical marknaden medan den faktiska broker-symbolen representerar ett specifikt futureskontrakt.

Market Data Layer och Execution Layer måste veta vilket kontrakt som är aktivt.

Strategin ska inte innehålla hårdkodade kontraktsnamn.

Contract Rollover

Futureskontrakt byts över tid.

Systemet behöver därför senare hantera rollover på ett kontrollerat sätt.

Detta påverkar:

- historical data
- chart continuity
- prices
- backtesting
- execution symbols

Rollover får inte ske genom att ett gammalt kontrakt plötsligt börjar representera ett nytt utan metadata.

Continuous Futures Data

För lång historisk analys kan continuous futures-data vara användbar.

Men continuous series kan innehålla justeringar som inte motsvarar faktiska handlingsbara priser.

Därför ska systemet skilja mellan:

research/continuous data

och:

actual contract execution data

En backtestmodell måste veta vilken typ av data som används.

Historical Data

Backtesting kräver historisk market data.

Historiska datasets ska versionshanteras eller åtminstone identifieras tydligt.

Ett BacktestRun ska kunna referera till:

- dataset
- provider
- date range
- instruments
- timeframes
- data version
- quality status

Om data senare korrigeras ska det gå att förstå vilken version ett tidigare resultat byggde på.

Realtime Data

Live analysis kräver realtime eller tillräckligt nära realtime-data.

Systemet ska kunna mäta:

```
data_age
```

Skillnaden mellan:

- aktuell systemtid
- senaste market timestamp

Om data blir för gammal ska systemet markera:

```
STALE_DATA
```

och blockera execution där färsk data krävs.

Data freshness

Olika datatyper kan ha olika freshness requirements.

Exempelvis kan:

- 1m entry data
- account state
- 4H historical context

behöva olika thresholds.

Freshness ska därför vara explicit konfigurerad.

Det ska inte finnas en generell regel som behandlar alla datatyper identiskt.

Missing Data

Om bars saknas mitt i en serie ska systemet kunna upptäcka detta.

Exempel:

10:01

10:02

10:04

saknar:

10:03

Strategy Engine får inte tyst anta att serien är komplett.

Systemet ska kunna markera:

INCOMPLETE_SERIES

Duplicate Data

Duplicerade bars eller ticks ska identifieras.

Det ska inte vara möjligt att oavsiktligt skapa två separata strategy events från samma market event på grund av duplicate ingestion.

Canonical identifiers och timestamps ska användas för deduplicering.

Out-of-Order Data

Realtime events kan tekniskt anlända i fel ordning.

Market Data Layer ska kunna upptäcka detta.

Strategy Engine ska inte basera state transitions på en tidsserie som den tror är kronologisk om datan inte är det.

Corrected Data

En provider kan i vissa fall korrigera historisk data.

Systemet ska kunna skilja mellan:

- original observation
- corrected data

Historiska systembeslut ska inte skrivas om bara för att en provider senare korrigerar sin feed.

Audit trail ska visa vilken data systemet faktiskt såg när beslutet fattades.

Data Quality States

Market data ska kunna klassificeras som exempelvis:

VALID

STALE

INCOMPLETE

DUPLICATE

OUT_OF_ORDER**SUSPECT****CORRECTED**

Strategy Engine och Risk Engine ska kunna läsa denna state.

Data Quality Gate

Före en exekverbar Strategy Signal ska kritisk data passera Data Quality Gate.

Exempel:

NQ 1m = VALID

NQ 5m = VALID

NQ 15m = VALID

NQ 4H = VALID

ES comparison = VALID

Om kritisk NQ-data är invalid:

BLOCK

Om endast optional SMT-data är unavailable ska systemet istället kunna fortsätta utan SMT, men tydligt registrera att confirmation saknades på grund av data availability.

Data Provenance

Varje viktig datapunkt ska kunna spåras tillbaka till sin källa.

Systemet bör kunna svara på:

Vilken provider gav denna candle?

När mottogs den?

Var den korrigerad?

Vilken adapter normaliserade den?

Vilken data version användes i backtestet?

Detta är särskilt viktigt när olika providers ger små skillnader.

Multi-Source Data

Arkitekturen ska stödja flera providers i framtiden.

Men systemet får inte blanda data från olika feeds godtyckligt mitt i en setup.

Det ska finnas regler för:

- primary source
- fallback source
- source transition

- validation

Om source byts ska detta vara ett explicit systemevent.

Data Source Failure

Om primary market-data source faller bort ska systemet inte automatiskt anta att en fallback är identisk.

Fallback kan tillåtas först när systemet verifierat:

- symbol mapping
- timestamp alignment
- data freshness
- price consistency

I execution-enabled mode ska tveksam data leda till:

NO NEW TRADE

Account Data är också tradingdata

Market Data Layer och Account Data Layer är tekniskt separata concerns, men båda behövs för ett tradingbeslut.

Från providern behöver Omnira exempelvis kunna läsa:

- balance
- equity
- margin
- free margin
- positions
- orders
- historical deals

Risk Engine ska inte använda gammal account state samtidigt som Strategy Engine använder färsk market data.

News Data

Ekonomiska nyheter är också en kritisk datakälla för Strategy v1.0.

Systemet behöver kunna representera:

- event
- currency
- impact
- scheduled time
- event type
- status

För Strategy v1.0 gäller:

No new entry: T-1h till T+4h

och:

Existing position exit: T-15m

Därför är felaktig news timestamp en faktisk risk.

News Data Failure

Om systemet inte kan avgöra om ett relevant high-impact USD-event finns ska execution-enabled modes inte anta att kalendern är tom.

Resultatet ska vara:

NEWS_STATE_UNKNOWN

och:

ingen ny trade

tills state är verifierad.

Analysis Mode kan fortsätta med tydlig varning.

Market Snapshot

Vid viktiga strategy events ska systemet kunna skapa en Market Snapshot.

Exempel:

- 4H thesis start
- manipulation detected
- entry confirmation
- Strategy Signal
- Trade Proposal
- execution
- exit

Snapshot kan innehålla:

- candles
- liquidity
- FVG
- SMT
- current price
- spread
- session
- news state
- strategy state

Detta gör senare analys och debugging betydligt starkare.

Chart Snapshot

Omnira kan dessutom skapa en visuell chart snapshot.

Den visuella bilden är användbar för:

- mänsklig review
- journal
- Atlas explanations
- jämförelser

Men bilden är inte source of truth.

Den strukturerade market-data representationen ska vara source of truth.

Atlas Market View

Atlas Market View ska rendera samma marknadsobjekt som Strategy Engine arbetar med.

När Strategy Engine identifierar:

LiquidityLevel

ska UI:t kunna visa exakt samma nivå.

När Strategy Engine identifierar:

FVGZone

ska UI:t visa exakt samma zon.

Det ska alltså inte finnas:

Atlas liquidity

och:

UI liquidity

som två olika tolkningar.

Det ska finnas en canonical representation.

Vad Atlas ska kunna se

Atlas ska kunna konsumera strukturerad data såsom:

- CandleSeries
- SwingPoint
- LiquidityLevel
- FVGZone
- SMTObservation
- ManipulationEvent
- StrategyState
- MarketSnapshot

Atlas kan därefter förklara denna information för användaren.

AI:n ska inte behöva visuellt gissa var liquidity ligger om Strategy Engine redan identifierat den deterministiskt.

Historisk och live data ska tala samma språk

En av de viktigaste arkitekturprinciperna är att:

backtest

och:

live

ska använda samma canonical market-data format.

Historiska candles kan komma från en dataset adapter.

Live candles kan komma från providern.

Men Strategy Engine ska få samma interna objekt.

Detta reducerar risken att strategin fungerar annorlunda mellan testing och production.

Replay

Market data ska stödja replay.

Replay innebär att historisk data spelas fram kronologiskt som om den vore live.

Strategy Engine får endast se information som hade varit tillgänglig vid den aktuella tidpunkten.

Replay ska kunna användas för:

- strategy validation
- debugging
- demonstration
- Atlas Market View
- regression tests

Look-Ahead Bias

Systemet ska aktivt designas för att undvika look-ahead bias.

Exempel på förbjudet beteende är:

- använda candle high innan candle stängt
- veta att en swing point blir bekräftad innan nästa candle existerar
- använda framtida FVG-status
- använda framtida news outcome
- välja liquidity target baserat på vad priset senare gjorde

Backtestresultat som innehåller look-ahead bias är inte tillförlitliga.

Survivorship och Data Selection Bias

När systemet senare testas på flera marknader ska datasets väljas på ett sätt som inte enbart inkluderar marknader eller perioder där strategin råkar fungera.

Atlas Learning Layer ska inte få välja endast positiva perioder och kalla det robust edge.

Dataurval ska dokumenteras.

Data och Self-Improvement

Atlas Trading Learning & Improvement Layer ska kunna analysera market data tillsammans med trade-resultat.

Det kan exempelvis identifiera samband mellan:

- volatility
- session
- setup grade
- liquidity type
- FVG type
- SMT
- spread
- entry timing
- MFE
- MAE
- outcome

Detta kan skapa hypoteser såsom:

A-setups efter 15m FVG-touch fungerar bättre än efter intermediate liquidity under vissa market regimes.

Detta är en hypotes.

Den får inte automatiskt bli en ny tradingregel.

Feature Extraction

I framtiden kan systemet skapa strukturerade features från market data.

Exempel:

- volatility state
- distance from 4H open
- FVG size
- manipulation depth
- time since session open
- number of liquidity sweeps
- SMT state
- stop distance
- target distance

Features kan användas för:

- analytics
- market regime classification
- research

- AI analysis

Features ska vara reproducerbara där de används för forskning.

Raw Data och Derived Data

Systemet ska skilja mellan:

Raw Data

och:

Derived Data

Raw Data kan exempelvis vara:

- candle
- tick
- bid
- ask

Derived Data kan vara:

- swing
- liquidity
- FVG
- SMT
- market regime
- volatility state

Det ska gå att förstå hur derived data skapats från raw data.

Versionering av Detection Logic

Om algoritmen som identifierar exempelvis FVG eller swing points ändras ska versionen dokumenteras.

Annars kan två backtests på samma candles generera olika setups utan att det syns varför.

Detection logic är därför en del av systemets reproducerbarhet.

Datamängd och lagring

Högupplöst market data kan bli mycket omfattande.

Systemet ska därför senare skilja mellan:

- hot realtime data
- operational history
- research datasets
- archived tick data

Men kostnadsoptimering får inte förstöra möjligheten att reproducera viktiga tradingbeslut.

Retention

Trade-relaterade Market Snapshots ska bevaras långsiktigt.

Full tick history kan senare få en separat retention policy beroende på lagringskostnad och backtestbehov.

Data som krävs för audit av en live-trade ska prioriteras för bevarande.

Databas är inte chart

Systemet ska inte behandla chart-rendering som datalagring.

TradingView-liknande UI är en presentation.

Den canonical representationen ska ligga i strukturerad data.

Det innebär att Omnira senare kan:

- byta chart library
- skapa mobil vy
- generera journalbilder
- bygga replay UI

utan att ändra strategy logic.

Testning av Market Data Layer

Market Data Layer ska ha automatiserade tester för exempelvis:

- timezone conversion
- daylight saving transitions
- missing candles
- duplicate candles
- out-of-order bars
- wrong timeframe
- wrong instrument
- stale data
- candle close boundaries
- futures rollover
- provider disconnect
- source switch

Time handling ska betraktas som säkerhetskritisk kod.

Regression Testing

När Market Data Layer ändras ska systemet kunna köra tidigare kända testserier igen.

Exempel:

En given historisk dataserie ska producera samma:

- candles
- swing points
- FVG

- liquidity
- strategy events

så länge ingen relevant algorithm version medvetet ändrats.

Observability

Systemet ska kunna mäta Market Data Layers tekniska hälsa.

Exempel på metrics:

- latest data timestamp
- ingestion latency
- missing bars
- duplicate events
- reconnects
- source failures
- validation failures
- clock drift
- backlog

Atlas och UI ska kunna visa när marknadsdatan inte är healthy.

Market Data och Execution

Det är viktigt att skilja:

analysis data

från:

execution reality

Strategy Engine kan identifiera en entry på en candle close.

Execution sker därefter mot broker price.

Därför måste systemet senare registrera:

- strategy expected entry
- actual requested entry
- broker fill
- slippage

Skillnaden ska journalföras.

Execution Quality

Om en strategi ser lönsam ut före execution costs men förlorar efter:

- spread
- commission
- slippage

har systemet inte bevisat live edge.

Datamodellen måste därför göra det möjligt att mäta execution quality separat från strategy quality.

Data Principle för Omnira Trading

Den viktigaste dataprincipen är:

Omnira ska aldrig behöva gissa vilken data ett historiskt beslut baserades på.

För varje viktigt beslut ska systemet i efterhand kunna rekonstruera:

- vilken data som fanns
- från vilken källa
- vid vilken tid
- med vilken quality state
- vilken strategy version som använde den

Det är grunden för:

- robust backtesting
- säker live trading
- debugging
- audit
- Atlas self-improvement
- framtida autonomi

Kapitelstatus

Kapitel: 6 – Marknadsdata

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Primära instrument: NQ / MNQ

SMT comparison: ES

Canonical storage timezone: UTC

Strategy timezone: America/New_York

Timeframes: 1m, 5m, 15m, 4H

Market Data implementation: Ej påbörjad

Historisk dataset: Ej vald

Realtime provider: Ej slutligt vald

Execution data: Ej implementerad

Marknadsdata ska valideras och normaliseras innan den får påverka ett exekverbart tradingbeslut.

KAPITEL 7

AI som beslutsstöd

AI i Omnira Trading System ska inte fungera som en fristående trader som fritt försöker förutsäga marknaden.

Atlas ska istället fungera som ett intelligent beslutsstöd ovanpå ett deterministiskt trading-system.

Det innebär att Strategy Engine först identifierar vad strategin ser.

Risk Engine avgör vad som är tillåtet.

Prop Firm Rules Engine avgör vad externa regler tillåter.

Atlas använder därefter strukturerad information för att:

- förstå
- förklara
- jämföra
- analysera
- sammanfatta
- upptäcka mönster
- hjälpa användaren att fatta informerade beslut

Den centrala principen är:

AI får förbättra förståelsen av ett beslut, men får inte ersätta de regler som skyddar systemet.

Atlas som intelligent lager

Atlas ska ligga ovanpå Trading Domains strukturerade data.

Atlas ska inte behöva gissa hela marknadsstrukturen utifrån en skärmbild.

Istället ska Atlas kunna få tillgång till objekt såsom:

- StrategySetup
- StrategySignal
- MarketContext
- LiquidityLevel
- FVGZone
- SMTObservation
- RiskDecision
- PropDecision
- TradeProposal
- Position
- TradeJournalEntry
- PerformanceRecord

Detta gör att AI:n kan resonera kring samma data som resten av systemet använder.

Deterministisk logik före AI

Strategy Engine ska först avgöra exempelvis:

- relevant 4H-open
- liquidity
- FVG
- manipulation
- iFVG
- CISD
- SMT
- setup grade
- entry
- stop loss
- target
- R:R

Atlas ska därefter kunna förklara dessa signaler.

AI ska alltså inte få skapa en egen parallell strategi i bakgrunden.

Om Strategy Engine säger:

```
SETUP_INVALID
```

ska Atlas inte själv omklassificera den till en giltig trade.

Vad Atlas får göra

Atlas får exempelvis:

- förklara varför setupen identifierades
- beskriva vad marknaden har gjort hittills
- sammanfatta den aktuella 4H-thesisen
- presentera relevanta liquiditynivåer
- förklara vilken FVG som används
- beskriva manipulationen
- visa varför entry confirmation är giltig
- beskriva SMT
- förklara setup grade
- jämföra mot historiskt liknande setups
- beskriva rådande market regime
- lyfta osäkerheter
- presentera riskbeslutet
- presentera prop firm-beslutet
- skapa en begriplig trade thesis
- hjälpa till med journal och efteranalys

Vad Atlas aldrig får göra

Atlas får inte:

- kringgå Strategy Engine
- kringgå Risk Engine
- kringgå Prop Firm Rules Engine
- ändra technical SL för att få bättre position size
- höja risk baserat på confidence
- aktivera live automation själv
- kringgå kill switch
- ignorera news-regler
- skapa en order utan korrekt approval
- ändra canonical strategi-regler live
- ändra riskprofil live
- ändra prop firm-regler live

AI Confidence

Om Atlas producerar ett confidence-värde ska det behandlas som analysmetadata.

Exempel:

AI confidence = 91%

är inte samma sak som:

Trade probability = 91%

och är inte samma sak som:

Risk permission = ALLOW

Confidence får aldrig användas för att mjuka upp hard limits.

Exempel:

Strategy Engine: PASS

Atlas Confidence: 96%

Risk Engine: DENY

Slutresultat:

NO TRADE

Explainability

En av Atlas viktigaste roller är att göra trading-systemet begripligt.

Användaren ska inte bara se:

LONG

utan exempelvis:

Den valda 10:00 4H-candlen har manipulerat ned i en aktiv 15m FVG. På 1m har ett iFVG och CISD bekräftats. SMT mot ES saknas, vilket ger setup grade A istället för A+. Entry planeras på confirmation close. Technical SL ligger under manipulationens senaste swing low och första liquidity target ger 2.6R.

Detta är betydligt mer användbart än en svart låda.

Atlas Market View

AI-lagret ska vara nära integrerat med Atlas Market View.

När användaren öppnar Trading-projektet ska Atlas kunna förklara det som visas på chartet.

Exempel:

Aktuell status:

WAITING_FOR_MANIPULATION

Atlas kan då förklara:

Jag bevakar liquidity under aktuellt pris. Ingen giltig manipulation har ännu genomförts och därför letar jag inte efter entry på 1m.

Senare:

WAITING_FOR_CONFIRMATION

Atlas kan säga:

15m FVG har touchats. Manipulationen är därför giltig. Jag väntar nu på iFVG eller CISD på 1m innan någon trade proposal kan skapas.

Planned Entries

Atlas Market View ska kunna visa vad systemet planerar.

Om en setup är nära färdig ska användaren kunna se:

- direction
- potential entry
- technical stop
- target
- current R:R
- setup grade
- missing confirmation
- risk estimate
- state

Det ska tydligt framgå om nivåerna är:

- observation
- preliminary
- confirmed

- proposed
- approved
- executed

AI ska inte hitta på saknade data

Om SMT-data saknas ska Atlas inte säga:

Ingen SMT finns.

Det korrekta svaret kan vara:

SMT kan inte utvärderas eftersom ES-data saknas.

På samma sätt ska Atlas skilja mellan:

FALSE

och:

UNKNOWN

Detta är viktigt för hela systemets integritet.

Market Regime Classification

Atlas kan senare hjälpa till att klassificera market regime.

Exempel på möjliga kategorier:

- trending
- ranging
- high volatility
- low volatility
- expansion
- compression
- news-driven
- abnormal conditions

Market regime är initialt ett analysfält.

Det ska inte automatiskt bli ett hard strategy filter.

Först när historisk data visar robust effekt kan en regime-regel föreslås som kandidatversion.

Historisk jämförelse

Atlas ska kunna jämföra en aktuell setup mot historiskt liknande observationer.

Exempel:

De senaste 214 A-setups under New York-sessionen hade en expectancy på +0.31R.

eller:

CISD-only setups efter intermediate liquidity har historiskt haft lägre expectancy än setups efter 15m FVG-touch.

Sådana uppgifter ska alltid baseras på faktisk journal- och analyticsdata.

Atlas får inte hitta på historiska statistikvärden.

Sample Size

Atlas ska alltid ta hänsyn till sample size.

Exempel:

5 trades

är inte samma evidensnivå som:

500 trades

AI ska därför inte presentera små dataset som robust edge.

Systemet ska senare kunna använda miniminivåer för när analytics får betecknas som:

- insufficient
- preliminary
- moderate evidence
- strong evidence

Exakta thresholds definieras senare.

Performance Context

När Atlas presenterar performance ska den helst visa mer än win rate.

Exempel:

- trade count
- win rate
- expectancy
- average R
- profit factor
- max drawdown
- losing streak
- sample size
- period
- strategy version

En strategi med hög win rate kan fortfarande ha dålig expectancy.

AI Analysis Object

AI Analysis ska sparas som ett eget dataobjekt.

Det gör att systemet i efterhand kan jämföra:

- vad Strategy Engine såg
- vad AI:n så
- vad Risk Engine gjorde
- vad marknaden sedan gjorde

Detta blir viktigt för att kunna utvärdera om AI-lagret faktiskt tillför värde.

AI-versionering

AI Analysis ska kunna kopplas till:

- model
- model version
- prompt version
- analysis policy version

Om Atlas beteende förändras ska vi kunna se vilken version som producerade tidigare analyser.

Detta är särskilt viktigt om AI senare används som en del av beslutsstödet i live trading.

AI får inte vara enda beslutsgrund

Om en trade endast existerar därför att en språkmodell säger:

Detta ser bra ut.

har systemet lämnat den arkitektur som definierats för Omnira Trading.

Alla exekverbara trades ska först ha en strukturerad Strategy Signal.

AI kan lägga till kontext.

AI får inte ensam skapa själva tradingmöjligheten.

Osäkerhet

Atlas ska uppmuntras att uttrycka osäkerhet.

Exempel:

Setupen uppfyller de formella reglerna, men historisk data för denna specifika regime är begränsad.

Det är bättre än att AI:n överdriver precision.

Osäkerhet ska vara en del av analysen, inte något som döljs.

Contradicting Factors

AI Analysis bör kunna innehålla:

- supporting factors
- contradicting factors

Exempel:

Supporting:

- iFVG + CISD
- clean manipulation
- historical session edge

Contradicting:

- ingen SMT

- ovanligt hög volatility
- få historiska jämförelser

Detta gör analysen mer balanserad.

Trade Proposal Explanation

Atlas ska kunna generera en standardiserad sammanfattning för varje Trade Proposal.

Exempel:

Direction: LONG Strategy: Omnira Liquidity Manipulation v1.0 Setup Grade: A Entry: 24,120.25 SL: 24,105.50 TP: 24,151.00 R:R: 2.08R Risk: \$147.50

Strategy: PASS Risk: PASS Prop Firm: PASS

Atlas Thesis: 15m FVG har touchats under den valda 4H-open. Pris har därefter producerat iFVG och CISD på 1m. SMT saknas. Technical SL ligger under manipulationens senaste swing low. Första giltiga liquidity target ger mer än 2R.

Denial Explanation

Atlas ska också kunna förklara när en trade nekas.

Exempel:

Strategin är giltig men Risk Engine nekar trade eftersom endast \$95 av dagens riskbudget återstår och minsta MNQ-position med aktuell technical SL kräver \$136 risk.

Detta är bättre än ett generiskt:

ERROR

AI och News

Atlas kan förklara news context men hard news-regler ska komma från strukturerad kalenderdata.

Atlas får inte läsa en rubrik och själv avgöra att:

Den här nyheten verkar nog inte så viktig.

Canonical NewsEvent och policy avgör hard restrictions.

AI och Prop Firm-regler

Atlas kan hjälpa användaren att förstå prop firm-regler.

Exempel:

Denna trade hade passerat intern risk men nekas eftersom prop firm-kontot endast har \$72 equity headroom kvar före maximum-loss-gränsen.

Själva beräkningen ska komma från Prop Firm Rules Engine.

AI och Execution

Atlas ska kunna följa execution state.

Exempel:


```
PROPOSAL_APPROVED
→
EXECUTION_DISPATCHED
→
ORDER_ACKNOWLEDGED
→
FILLED
```

Atlas kan då presentera för användaren vad som händer.

AI ska inte själv kommunicera direkt med broker utan executionkontrollerna.

AI och Journal

Efter en trade ska Atlas kunna skapa en strukturerad efteranalys.

Exempel:

- vilken thesis användes
- vilken setup grade
- vilken entry confirmation
- om SMT fanns
- hur MFE/MAE utvecklades
- om break-even aktiverades
- varför trade stängdes
- slutligt R
- eventuella execution issues

Journalen ska fortfarande innehålla rå strukturerad data som source of truth.

AI-texten är ett analyslager ovanpå den.

Post-Trade Review

Atlas ska kunna genomföra konsekvent post-trade review.

Den ska inte bara analysera förlorare.

Vinnare ska också granskas.

En vinnande trade kan ha varit dåligt exekverad.

En förlorande trade kan ha varit perfekt enligt reglerna.

Systemet ska därför skilja mellan:

Decision Quality

och:

Outcome

Decision Quality vs Outcome

Detta är en viktig princip.

En trade som:

- följde Strategy v1.0
- passerade risk
- exekverades korrekt
- och sedan förlorade

är inte automatiskt ett dåligt systembeslut.

På samma sätt är en regelöverträdelse som råkar vinna inte ett bra beslut.

Atlas ska lära sig denna skillnad.

Self-Improvement

Atlas Trading Learning & Improvement Layer ska använda historisk data från hela beslutsprocessen.

Det innebär att Atlas ska kunna lära från:

- alla setups
- alla signals
- nekade trades
- exekverade trades
- winners
- losers
- break-even
- sessions
- liquidity types
- FVG types
- setup grades
- SMT
- market regimes
- execution quality
- risk decisions
- prop decisions

Målet är att skapa bättre kunskap över tid.

Wisdom Layer

Self-improvement ska inte bara samla statistik.

Atlas ska kunna bygga en högre nivå av sammanställd kunskap, eller wisdom.

Exempel:

A+ setups har generellt stark expectancy, men förbättringen jämfört med A är liten under London och betydligt större under New York.

eller:

Re-entry attempt 3 har historiskt negativ expectancy även om attempt 1–2 är positiva.

Sådan kunskap ska kunna sparas som strukturerade findings med:

- evidence
- sample size

- period
- strategy version
- confidence
- status

Hypothesis Generation

Atlas får skapa hypoteser.

Exempel:

Candidate hypothesis: blockera C-grade under low-volatility regime.

Hypotesen ska inte aktiveras live.

Den ska gå vidare till Research & Validation.

Candidate Versions

Ett förbättringsförslag som ändrar materiella regler ska skapa en candidate version.

Exempel:

Omnira Liquidity Manipulation v1.1-candidate

Den ska testas separat mot:

Canonical v1.0

Det ska gå att jämföra:

- expectancy
- drawdown
- profit factor
- stability
- sample size
- different regimes
- out-of-sample

Anti-Overfitting

Atlas ska inte optimera strategin genom att välja regler som råkar passa historiken perfekt.

Förbättringsförslag ska därför testas på:

- training period
- holdout data
- out-of-sample data
- forward test

Parametrar ska även kunna stress-testas.

En regel som endast fungerar på exakt ett parameter-värde är mindre robust än en regel som fungerar över ett rimligt område.

Self-Improvement Governance

Flödet ska vara:

```

Observe
→ Measure
→ Learn
→ Hypothesis
→ Candidate Version
→ Backtest
→ Out-of-Sample
→ Forward Test
→ Review
→ Approval
→ Canonical Version
  
```

Atlas får inte hoppa över dessa steg när ändringen påverkar production trading.

Canonical Knowledge

Atlas ska alltid kunna skilja mellan:

Canonical Rule

och:

Research Finding

och:

Candidate Hypothesis

Exempel:

Canonical:

Minimum R:R = 2.0

Research Finding:

Trades mellan 2.0R–2.2R har hittills haft lägre expectancy

Candidate Hypothesis:

Testa minimum R:R = 2.3

Detta förhindrar att analysdata långsamt förvandlas till regler utan ett explicit beslut.

Learning from Rejected Trades

Nekade trades är viktiga.

Om Risk Engine nekar en setup ska systemet fortfarande kunna följa vad marknaden gjorde efteråt.

Atlas kan då analysera:

Hur hade trade gått om den hade tagits?

Detta får användas för forskning.

Det får inte användas för att i efterhand säga att Risk Engine hade fel.

Risk Engine bedöms på riskkontroll, inte på om en enskild nekad trade senare blev vinnare.

Learning from Missed Setups

Systemet ska även kunna studera setups som:

- nästan blev giltiga
- invalidated
- expired
- saknade confirmation

Detta kan hjälpa till att förstå vilka delar av strategy funnel som producerar edge.

Counterfactual Analysis

Atlas ska senare kunna utföra frågor som:

Vad hade hänt om vi endast tradat A och A+?

Vad hade hänt utan SMT?

Vad hade hänt om max attempts varit 2?

Vad hade hänt med annan news blackout?

Counterfactual analysis ska göras på forskningens kopia av reglerna.

Canonical live-history får aldrig skrivas om.

AI Performance Evaluation

AI-lagret självt ska också mätas.

Vi ska kunna fråga:

Gav Atlas analysis faktiskt någon information som förbättrade beslutsförståelsen?

Identifierade Atlas risks eller regimes korrekt?

Var confidence kalibrerad?

AI får inte automatiskt betraktas som värdefull bara för att AI används.

Confidence Calibration

Om Atlas använder confidence ska vi senare kunna mäta om confidence korrelerar med faktiska observationer.

Exempel:

Om setups med:

90–100% confidence

inte presterar bättre än:

60–70%

är confidence-värdet sannolikt inte särskilt användbart.

Då ska systemet ändra hur det används eller helt sluta exponera det.

AI Fallback

Om AI-tjänsten är unavailable ska det deterministiska trading-systemet fortfarande kunna fungera i de modes där policy tillåter det.

Strategy Engine och Risk Engine ska alltså inte vara tekniskt beroende av att en språkmodell svarar.

Beroende på operation mode kan AI outage:

- visa varning
- stoppa Trade Proposal
- eller låta deterministic pipeline fortsätta

Den slutliga policyn definieras senare.

AI får inte bli en single point of failure för broker-native protection.

Human Oversight

Atlas ska göra det enklare för människan att granska systemet.

Särskilt under:

- Analysis Mode
- Demo Manual Approval
- Live Manual Approval

ska användaren kunna se hela decision chain innan approval.

Det inkluderar:

- vad strategin såg
- vad Atlas analyserade
- vad riskmotorn sa
- vad prop firm-motorn sa
- vad systemet planerar att göra

Ingen dold reasoning krävs

Omnira behöver inte spara eller visa en språkmodells privata interna resonemang.

Det viktiga är att systemet sparar:

- strukturerade inputs
- beslut
- relevanta factors
- sammanfattad motivering
- outputs
- versionsmetadata

Explainability ska komma från systemets explicit dokumenterade data och regler.

Atlas som tradingpartner

Målet är att Atlas över tid ska fungera mindre som en enkel chatbot och mer som en intelligent tradingpartner.

Atlas ska kunna säga:

Den här setupen är giltig enligt Strategy v1.0. Den är Grade B eftersom endast iFVG finns. Liknande B-setups under denna session har hittills lägre expectancy än A-setups. Risk Engine tillåter positionen, men vi har bara 33 % av dagens riskbudget kvar.

Det är en mycket mer värdefull användning av AI än:

Jag tror Nasdaq går upp.

Långsiktig vision

På lång sikt ska Atlas kunna kombinera:

- realtidsdata
- strategy state
- risk
- prop firm-regler
- historik
- analytics
- market regime
- execution quality
- learning findings

och presentera en sammanhängande bild av trading-systemet.

Men den grundläggande maktfördelningen ska förbli densamma.

AI blir smartare.

Riskgränserna blir inte frivilliga.

Kapitelstatus

Kapitel: 7 – AI som beslutsstöd

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Atlas roll: Beslutsstöd och explainability

Strategy authority: Deterministisk

Risk authority: Absolut veto

Self-improvement: Tillåtet

Self-modification i produktion: Förbjudet

Candidate strategy generation: Tillåtet

Canonical activation: Kräver testning och approval

AI implementation: Ej påbörjad

Atlas ska bli bättre på att förstå och förklara trading över tid utan att få obegränsad rätt att förändra systemet den styr.

KAPITEL 8

Exekvering

Exekvering är den del av Omnira Trading System där ett analyserat och godkänt tradingbeslut förvandlas till en faktisk order hos broker eller prop firm.

Det är också den punkt där ett tekniskt fel kan få direkt ekonomisk konsekvens.

Därför ska exekveringslagret vara:

- enkelt
- deterministiskt
- auditerbart
- idempotent
- återhämtningsbart
- isolerat från strategi-AI

Den centrala principen är:

Execution Runner ska inte tänka. Den ska verifiera och exekvera exakt det den har fått tillstånd att exekvera.

Från Trade Proposal till order

En trade ska inte gå direkt från Strategy Signal till broker.

Den ska först passera hela kontrollkedjan:

```
Strategy Signal
→ AI Analysis
→ Risk Engine
→ Prop Firm Rules Engine
→ Trade Proposal
→ Approval / Automation Policy
→ Execution Gateway
→ Execution Intent
→ Execution Runtime
→ Futures Execution Provider
→ Broker / Prop Firm
```

Varje steg har ett separat ansvar.

Trade Proposal är inte execution

Trade Proposal beskriver vad systemet vill göra.

Det kan exempelvis innehålla:

- instrument
- direction
- entry
- stop loss

- take profit
- position size
- risk
- R:R
- setup grade
- strategy version
- Risk Engine-resultat
- Prop Firm-resultat

Men ett Trade Proposal får inte i sig själv skapa en order.

Det måste först få rätt execution authority.

Approval

Under tidiga systemfaser ska användaren själv godkänna Trade Proposal.

Ett approval ska vara explicit.

Det ska inte räcka att:

- öppna en proposal
- klicka någonstans i chartet
- lämna UI:t öppet

Approval ska vara en tydlig handling.

Exempel:

APPROVE TRADE

Approval ska lagras som ett separat objekt.

Approval ska kunna expire

Ett tradingbeslut är tidskänsligt.

Ett Trade Proposal som var korrekt vid:

10:21:02

kan vara helt irrelevant några minuter senare.

Approval ska därför ha:

- created_at
- decided_at
- expires_at

När approval är expired krävs ny evaluation.

Automation Policy

När Omnira senare går från manual approval till demo automation ska approval istället kunna komma från en explicit automation policy.

Det betyder inte att kontrollstegen försvinner.

Det betyder endast att:

Human Approval

ersätts av:

Approved Automation Policy

Strategy Engine, Risk Engine och Prop Firm Engine måste fortfarande passera.

Execution Gateway

Execution Gateway är den säkerhetsmässiga gränsen mellan Omnira och executionmiljön.

Gateway ska kontrollera:

- att proposal finns
- att proposal är aktuell
- att rätt account används
- att proposal har giltigt approval
- att proposal inte expired
- att kill switch inte är aktiv
- att samma proposal inte redan exekverats
- att rätt environment används

Först därefter får ett Execution Intent skapas.

Execution Intent

Execution Intent ska vara en fullständigt strukturerad instruktion.

Exempel på data:

- execution_id
- proposal_id
- account_id
- instrument
- side
- quantity
- order type
- expected entry
- allowed deviation
- stop loss
- take profit
- expiry
- approval reference
- strategy version
- risk decision reference
- prop decision reference

Execution Runner ska inte behöva tolka naturligt språk.

Ingen AI-prompt till providern

Execution ska aldrig fungera genom instruktioner såsom:

Köp ungefär två MNQ här och sätt stoppen under senaste low.

Detta är för tvetydigt.

Runnern ska istället få strukturerade värden:

```
symbol = MNQ
side = BUY
quantity = 1
```

SL = 24105.50

TP = 24151.00

```
execution_id = ...
```

Exekveringslagret ska vara maskinprecist.

Idempotency

Idempotency är ett av de viktigaste säkerhetskraven i execution.

Anta att Omnira skickar ett orderrequest.

Runnern skickar ordern till providern.

Sedan försvinner nätverket innan Omnira får svar.

Om Omnira bara försöker igen utan skydd kan resultatet bli:

två positioner

Det får inte vara möjligt.

Varje execution ska därför använda ett unikt:

```
idempotency_key
```

eller:

```
execution_id
```

Samma execution får endast kunna utföras en gång.

Duplicate Protection

Execution Runner ska kunna svara:

```
ALREADY_PROCESSED
```

om samma execution intent skickas igen.

Detta ska gälla även efter processrestart när det är tekniskt möjligt.

Idempotency-state måste därför vara persistent eller kunna rekonstrueras.

Pre-Execution Revalidation

Ett tidigare godkänt Trade Proposal betyder inte att ordern fortfarande är säker vid submission.

Direkt före orderläggning ska systemet kontrollera igen:

- current price
- technical SL
- R:R
- quantity
- actual account state
- daily risk
- open positions
- attempt count
- news state
- prop firm-state
- kill switches
- broker connection
- runner health

Om något relevant förändrats ska execution stoppas.

Price Movement

Anta att setupen skapades med:

Entry = 24,120

SL = 24,100

TP = 24,165

och godkändes med ett giltigt R:R.

Om priset redan står på:

24,140

när execution ska ske kan samma plan vara helt förändrad.

Systemet får då inte skicka order bara för att proposal tidigare var godkänd.

Slippage Guard

Execution ska ha stöd för maximal tillåten deviation.

Om actual entry innebär att:

- risken ökar över tillåten risk
- R:R faller under strategy minimum
- entry ligger utanför tillåten range

ska ordern inte skickas eller godkännas.

Exakta toleranser ska kalibreras senare.

Market Order och framtida Order Types

Den första implementationen behöver inte stödja alla ordertyper.

Arkitekturen ska dock kunna hantera exempelvis:

- market
- limit
- stop

Strategy v1.0 använder entry direkt på confirmation close, vilket gör market execution till en naturlig första kandidat.

Den faktiska implementationen ska valideras mot brokerbeteende och slippage.

Position Size Integrity

Quantity ska komma från Risk Engine.

Runnern får inte:

- avrunda upp
- öka size
- ändra size för att matcha en brokerpreferens
- välja ett större kontrakt

Om broker kräver annan quantity än godkänd ska execution nekas.

Instrument Integrity

Runnern ska verifiera att:

- instrument finns
- symbol mapping är korrekt
- contract är rätt
- symbol är tradeable
- tick size är korrekt
- quantity step är korrekt

Fel futureskontrakt får inte exekveras bara för att symbolnamnen liknar varandra.

Account Integrity

Execution Intent ska vara kopplat till ett specifikt konto.

Runnern ska kontrollera att det providerkonto som faktiskt är aktivt motsvarar:

```
account_id
```

i execution intent.

Om fel konto är inloggat:

```
NO EXECUTION
```

Detta är en kritisk safety gate.

Environment Integrity

Demo och live ska hållas isär.

Ett execution intent för:

```
environment = demo
```

får inte exekveras på:

live

och tvärtom.

Miljön ska vara explicit verifierad.

Futures Execution Provider

Providern är den externa executionmiljön. Ingen specifik provider är vald (GATE-15).

Omnira ska inte låta Strategy Engine kommunicera direkt med providern.

Kommunikationen ska gå genom execution-lagret.

Det skapar ett säkerhetsmässigt mellanrum där alla kontroller kan göras innan order submission.

Execution Runtime

Den första runnern ska köras på en dedikerad Windows-miljö.

Initialt kan detta vara användarens stationära dator.

Runnersns ansvar är begränsat till:

- kommunikation med Omnira
- kommunikation med providern
- read operations
- execution requests
- order status
- position status
- heartbeat
- reconciliation

Runnern ska inte innehålla fri AI.

24/7 Drift

Execution Runner ska kunna köras kontinuerligt.

Den initiala workstation-miljön behöver därför minst:

- sleep avstängt
- stabil nätverksanslutning
- auto-start
- provideranslutning auto-start
- runner auto-start
- korrekt systemtid

- health monitoring
- log retention
- diskövervakning

Om datorn startar om ska systemet inte omedelbart börja handla.

Reconciliation måste först genomföras.

Heartbeat

Runnern ska regelbundet rapportera att den lever.

Heartbeat kan innehålla:

- runner status
- providern connection
- broker connection
- active account
- environment
- latest sync
- runner version
- execution enabled

Om heartbeat försvinner ska Omnira markera runnern som unavailable.

Ingen ny execution ska skickas.

Broker Connection

Runnern ska skilja mellan:

- providerprocess running
- terminal connected
- broker connected
- account tradeable

Att providerklienten är öppet betyder inte automatiskt att execution är möjlig.

Order Pre-Check

Innan faktisk submission ska runnern eller relevant adapter använda de kontroller som broker/providern erbjuder när detta är möjligt.

Pre-check kan upptäcka exempelvis:

- invalid volume
- invalid stops
- insufficient margin
- closed market
- symbol restrictions

Pre-check ersätter inte Omniras Risk Engine.

Det är ytterligare defense-in-depth.

Broker-Native Stop Loss

Kritisk stop loss ska så långt som möjligt skickas till broker tillsammans med eller omedelbart kopplat till positionen.

SL ska inte endast finnas som:

Omnira remembers to close later

Om Omnira eller internet försvinner ska broker-native SL fortfarande kunna skydda positionen.

Broker-Native Take Profit

Samma princip gäller TP där executionmodellen tillåter det.

Broker-native TP minskar systemets beroende av att Omnira hela tiden är online för normal exit.

Break-Even Modification

När canonical break-even-trigger inträffar ska Omnira kunna skicka en position modification:

```
SL → entry price
```

Även denna förändring ska:

- identifieras
- auditeras
- verifieras
- få broker response

Systemet får inte bara ändra UI-markeringen utan att bekräfta att brokerpositionen faktiskt uppdaterats.

Stop Modification Failure

Om systemet försöker flytta SL till break-even men brokern nekar ändringen ska detta behandlas som ett executionproblem.

Atlas och riskmonitorering ska kunna visa:

```
BREAK_EVEN_MODIFICATION_FAILED
```

Systemet ska inte anta att positionen är skyddad när den inte är det.

News Exit

För Strategy v1.0 ska en öppen position stängas:

T - 15 minuter

före relevant high-impact USD-news.

Execution Layer ska kunna exekvera denna stängning.

Detta är en systemexit och ska loggas som:

```
NEWS_EXIT
```

inte som strategy TP eller SL.

Time Exit

New York-positioner får ligga öppna maximalt:

4 timmar från entry

om ingen tidigare exitregel har aktiverats.

Execution Layer ska kunna genomföra denna exit och logga:

```
TIME_EXIT
```

London-positioner

London-positioner har ingen generell fyratimmarsgräns i Strategy v1.0.

De kan fortsätta utanför entry window så länge andra exitregler inte triggas.

Execution Layer ska därför skilja mellan:

- entry window
- position management window

Att entry-fönstret stänger innebär inte automatiskt att positionen ska stängas.

Closing Orders

När en position ska stängas ska systemet veta varför.

Exempel:

- SL
- TP
- BE
- news exit
- time exit
- emergency close
- manual close
- prop rule emergency
- reconciliation action

Exit reason ska alltid journalföras.

Partial Fills

Även om första MNQ-execution ofta är liten ska datamodellen stödja partial fills.

En order kan tekniskt få:

- full fill
- partial fill
- multiple fills
- rejection

Position state ska baseras på faktiska fills.

Fill Data

Varje fill ska kunna lagra:

- requested price
- fill price
- quantity
- timestamp
- commission
- fees
- slippage
- broker deal ID

Detta behövs för att analysera execution quality.

Strategy Price vs Fill Price

Systemet ska skilja mellan:

Strategy Entry

och:

Actual Fill

Skillnaden ska mätas.

Exempel:

Strategy entry = 24,120.25

Actual fill = 24,121.00

Slippage:

+0.75 points

Detta påverkar faktisk risk och R:R.

Execution Quality

Execution Layer ska senare kunna analyseras separat från strategy performance.

En strategi kan ha positiv theoretical expectancy men dålig faktisk performance om execution costs är för höga.

Därför ska Omnira mäta:

- average slippage
- commissions
- rejection rate
- fill latency
- missed trades
- modification failures
- reconnects

Latency

Latency ska mätas på flera nivåer när möjligt.

Exempel:

```
signal_time
→ proposal_time
→ approval_time
→ execution_request_time
→ runner_receive_time
→ broker_submit_time
→ fill_time
```

Detta gör att systemet kan identifiera var fördröjningar uppstår.

Manual Approval Latency

Under manual approval kan användaren själv vara den största latency-källan.

Det är inte ett tekniskt fel.

Men systemet ska fortfarande kunna mäta hur lång tid approval tog.

Det hjälper senare att jämföra manual mode med automation.

Proposal Expiry

En proposal ska ha en begränsad livslängd.

När den expirerar ska status bli:

```
EXPIRED
```

En expired proposal får inte återanvändas.

Ny strategy evaluation krävs.

Retry Policy

Retries ska vara extremt försiktiga i execution.

Network request får ibland retryas.

Broker-order får inte blint retryas om systemet inte vet om den första submissionen lyckades.

Detta är ytterligare en anledning till:

- idempotency
- broker order reconciliation
- execution state machine

Unknown Execution State

Om Omnira inte vet om en order skickades eller fylldes ska state vara:

```
UNKNOWN_EXECUTION_STATE
```

Systemet får inte skicka en ny order för att “vara säker”.

Trading på kontot ska pausas tills reconciliation visar verkligt broker state.

Reconciliation

Reconciliation innebär att Omniras förväntade state jämförs med providerns verkliga state.

Systemet ska exempelvis jämföra:

- expected orders
- actual orders
- expected positions
- actual positions
- quantities
- SL
- TP
- account
- latest fills

Discrepancies ska loggas.

Startup Reconciliation

Efter runner restart ska följande ske:

```
START
→ CONNECT providern
→ VERIFY ACCOUNT
→ READ ORDERS
→ READ POSITIONS
→ READ RECENT DEALS
→ COMPARE WITH OMNIRA
→ RECONCILED
```

Först efter:

```
RECONCILED
```

får ny execution tillåtas.

Unknown Position

Om providern har en position som Omnira inte känner igen ska den markeras:

```
UNKNOWN_POSITION
```

Ny trading ska blockeras.

Användaren kan exempelvis ha öppnat positionen manuellt.

Omnira får inte automatiskt stänga en okänd position utan explicit emergency policy.

Manual Trades

Systemet ska kunna skilja:

OMNIRA_GENERATED

från:

MANUAL_EXTERNAL

En manuellt öppnad trade ska inte räknas som performance för Omnira Liquidity Manipulation. Men positionen måste fortfarande påverka account risk.

Orphan Orders

Samma princip gäller orders som finns hos broker men saknas i Omnira state. De ska identifieras och reconcileras innan ny execution.

Kill Switch

Execution Runner ska respektera kill switches.

Exempel:

- global kill
- account kill
- strategy kill
- instrument kill
- runner kill

Aktiv relevant switch betyder:

NO NEW EXECUTION

Kill Switch och öppna positioner

Att stoppa nya trades och att stänga befintliga trades är två olika funktioner.

En vanlig kill switch ska minst stoppa:

NEW EXECUTION

Emergency close ska hanteras separat.

Detta förhindrar att en safety switch oavsiktligt gör något destruktivt med en redan skyddad position.

Emergency Close

Systemet kan senare stödja en särskild emergency close.

Den kan exempelvis användas vid:

- broker anomaly
- corrupted strategy state
- severe risk incident
- user emergency action

Emergency close ska vara:

- explicit
- auditerad
- tydligt markerad
- separerad från normal strategy exit

Network Failure

Om nätverket mellan Omnira och runnern går ner:

- inga nya trades
- inga nya re-entries
- inga nya discretionary modifications

Redan broker-native SL/TP ska fortsätta skydda positionen.

Det är en central anledning till att critical protection ska ligga hos broker där möjligt.

Omnira Outage

Samma princip gäller om Omnira-processen går ner.

Execution Runner ska inte börja skapa egna trades.

Den ska som default sluta acceptera nya execution instructions tills kontrollsystemet är tillbaka.

Runner Outage

Om runnern är offline ska Omnira tydligt visa:

EXECUTION UNAVAILABLE

Strategy Engine kan eventuellt fortsätta analysera beroende på operation mode.

Men inga executable proposals får behandlas som om execution finns tillgänglig.

Provider Outage

Om providerprocessen eller broker connection försvinner ska execution blockeras.

Systemet ska kunna skilja detta från att hela runnern är offline.

Logging

Varje execution event ska loggas.

Exempel:

- intent created
- intent dispatched
- runner received
- precheck started
- precheck passed
- order submitted
- broker acknowledged
- fill received
- SL confirmed

- TP confirmed
- modification requested
- modification confirmed
- close requested
- close filled
- reconciliation performed

Immutable Execution History

Historiska execution events ska inte redigeras i efterhand.

Om exempelvis en tidigare broker response tolkats fel och senare korrigeras ska systemet skapa ett nytt correction event.

Audit trail ska visa båda.

Execution Correlation ID

Hela executionkedjan ska kunna följas via ett gemensamt correlation ID.

Det ska gå att gå från:

Trade

bakåt till:

Fill

```
→ Order
→ ExecutionIntent
→ TradeProposal
→ RiskDecision
→ StrategySignal
→ StrategySetup
```

Detta är kritiskt för debugging och audit.

Security

Execution Runner är en privilegierad komponent.

Den ska därför använda:

- autentiserad kommunikation
- krypterad transport
- minimal behörighet
- secret storage
- explicit account allowlist
- environment restrictions

providercredentials får inte exponeras till frontend eller AI-promptar.

Signerade Execution Intents

På sikt bör execution intents kunna autentiseras eller signeras så att runnern kan verifiera att request verkligen kommer från rätt Omnira-miljö.

Runnern ska inte acceptera godtyckliga lokala network requests som kan skapa orders.

Account Allowlist

Runnern bör explicit veta vilka konton den får arbeta med.

Om ett okänt account identifieras ska execution blockeras.

Instrument Allowlist

Samma princip kan användas för instrument.

Under första valideringen kan execution exempelvis begränsas till:

- MNQ

även om providerkontot erbjuder hundratals andra symbols.

Environment Banner

Omnira Trading UI ska tydligt visa:

ANALYSIS

DEMO

eller:

LIVE

Live-miljö ska vara visuellt omöjlig att förväxla med demo.

Detta är en human-factors-säkerhetsregel.

Manual Approval UI

Trade Proposal ska visa hela planen före approval.

Minst:

- account
- environment
- instrument
- direction
- entry
- SL
- TP
- quantity
- risk
- R:R
- strategy
- setup grade
- Risk Engine
- Prop Firm Engine

- proposal expiry

Användaren ska förstå exakt vad som kommer skickas.

Atlas och Execution

Atlas ska kunna förklara execution state.

Exempel:

Trade Proposal är godkänd. Pre-execution revalidation pågår.

eller:

Execution stoppades eftersom priset har flyttat sig och R:R nu är 1.74, under strategins minimum 2.0.

eller:

Ordern fylldes 0.5 points över planerad entry. Actual initial risk är fortfarande inom tillåten gräns.

Atlas beskriver execution.

Atlas utför inte broker-logiken själv.

Execution och Self-Improvement

Execution-data ska användas av Atlas Trading Learning & Improvement Layer.

Systemet ska exempelvis kunna lära:

- vilka sessions som har mest slippage
- vilka entrytyper som är känsligast för latency
- om manual approval försämrar fill quality
- om vissa market regimes ger större execution costs
- om vissa broker-tider ger fler rejections

Detta kan leda till förbättringsförslag.

Det får inte automatiskt förändra live execution policy.

Execution Candidate Improvements

Atlas kan exempelvis föreslå:

Candidate execution policy: sätt max allowed slippage till X under normal volatility.

Eller:

Limit execution gav bättre fills än market execution i historisk simulering.

Sådana förändringar ska behandlas som versionsstyrda kandidater och testas separat.

Demo före Live

Execution Layer ska först bevisa stabilitet på demo.

Vi ska kunna verifiera:

- inga duplicate orders
- korrekt quantity
- rätt account

- rätt symbol
- korrekt SL/TP
- korrekt break-even
- korrekt news exit
- korrekt time exit
- korrekt reconciliation
- korrekt restart recovery
- korrekt kill switch

Först därefter får live manual approval övervägas.

Failure Injection

Execution ska aktivt testas mot fel.

Exempel:

- network disconnect
- providerklient restart
- broker rejection
- duplicate request
- stale proposal
- wrong account
- wrong environment
- partial fill
- SL modification failure
- unknown position
- runner crash efter submission

Systemet ska visa säkert beteende i varje scenario.

Success betyder inte bara fill

En lyckad execution är inte bara:

ORDER FILLED

Den ska också innebära att:

- rätt account användes
- rätt instrument användes
- rätt quantity användes
- risk låg inom gräns
- SL är korrekt
- TP är korrekt
- journalen är uppdaterad
- state är reconciled

Exekveringens huvudprincip

Den viktigaste principen i execution-lagret är:

Gör exakt det som har godkänts, högst en gång, på rätt konto, med rätt risk, och bevisa efteråt vad som faktiskt hände.

Detta är viktigare än att ordern skickas några millisekunder snabbare under de första utvecklingsfaserna.

Robusthet kommer före hastighet.

Kapitelstatus

Kapitel: 8 – Exekvering

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Initial executionmiljö: Windows workstation

Framtida miljö: VPS-kompatibel

Initial platform: providern

Execution authority: Separat från Strategy Engine och AI

Idempotency: Obligatoriskt

Pre-execution revalidation: Obligatoriskt

Broker-native SL/TP: Prioriterat

Reconciliation: Obligatoriskt

Execution implementation: Ej påbörjad

Live execution: Förbjuden tills senare gates är uppfyllda

Execution Layer ska betraktas som en säkerhetskritisk del av Omnira Trading System.

KAPITEL 9

Futures Execution Integration

Omnira Trading System handlar NQ och MNQ. Det är futures, och integrationen ska därför vara futures-native.

Ingen specifik execution provider är vald. Kapitlet definierar den provider-neutrala integrationsarkitekturen: vad Omnira kräver av en extern execution-miljö, och var gränsen mellan Omnira och den miljön går. En eller flera **Execution Provider Adapters** kan senare implementeras mot den gränsen.

***Rättelse 2026-08-28.** Tidigare version av detta kapitel antog MetaTrader 5 som execution-plattform. Det var ett felaktigt implementation-specifikt antagande för en futures-strategi. Se Canonical Amendments v1.0, Beslut D. Tradingstrategin och riskreglerna är oförändrade.*

Integrationen ska byggas stegvis.

Den första fasen är strikt:

READ ONLY

Ingen orderläggning ska vara tekniskt möjlig innan read-only-flödet är verifierat.

Det innebär att systemet först måste kunna läsa och synkronisera:

- konto
- balance
- equity
- instruments
- market data
- active orders
- open positions
- historical orders
- historical deals

innan trading requests får aktiveras.

Den centrala principen är:

Omnira ska först lära sig att observera providern korrekt innan systemet får rätt att påverka den.

Ansvarsfördelningen mot providern

En Futures Execution Provider är bryggan mellan Omnira Trading System och det faktiska handelskontot hos broker, FCM eller prop firm.

Providern ansvarar för den externa tradingmiljön:

- brokeranslutning
- account state
- symboler

- priser
- orders
- positions
- fills
- trade history

Omnira ansvarar istället för:

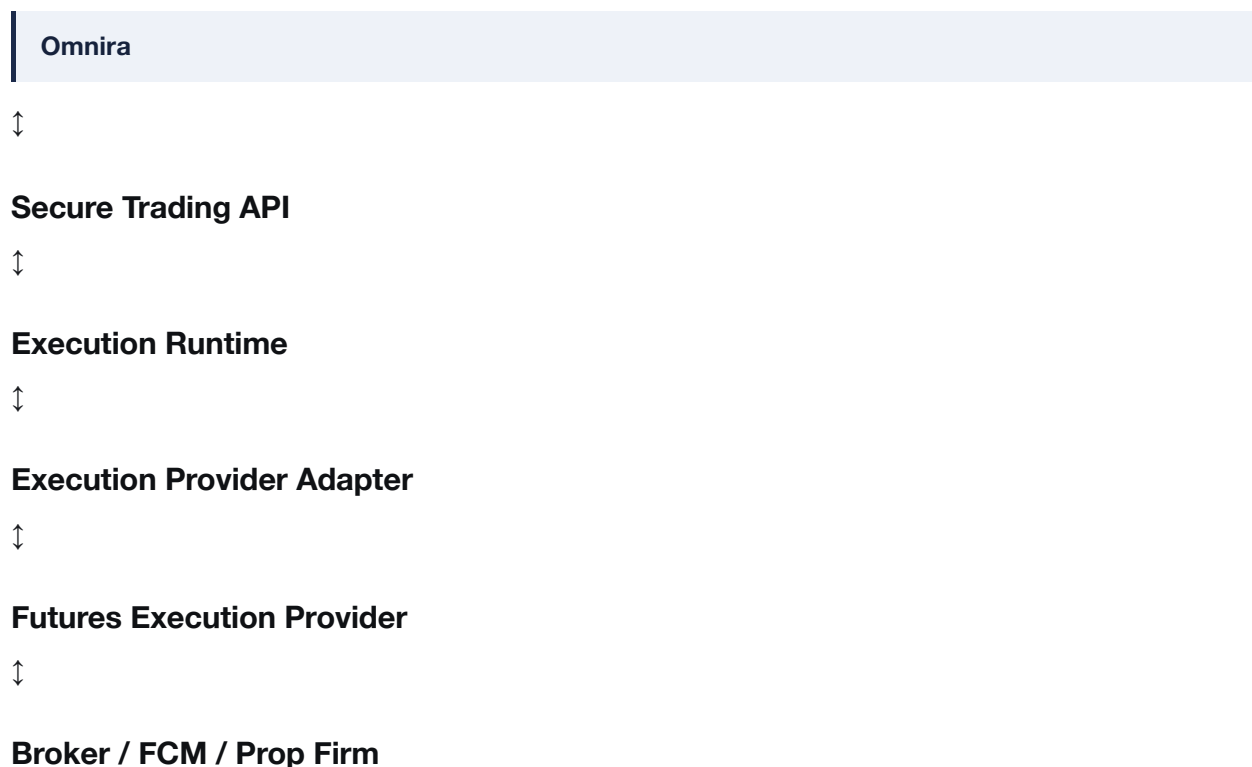
- strategi
- AI-analys
- risk
- prop firm-regler
- approvals
- journal
- analytics
- UI
- governance

Providern ska alltså inte bli Omniras intelligenslager.

Execution Runtime — deployment-beroende

Om integrationen kräver en separat process eller host beror på vald provider och driftsmodell. Det är **inte** en universell arkitekturinvariant, och ingen executionplattform är låst. Se Systemarkitektur v0.2 §2.1 och §18, samt GATE-17.

Flödet när ett separat runtime **behövs**:



När providern kan nås direkt och säkert från Omnira faller runtime-ledet bort och Execution Gateway talar direkt med adaptern. Adaptergränsen finns i båda fallen.

Adaptern är den enda komponent som får känna till en specifik providers API.

Varför providern inte kopplas direkt till frontend

Ingen tradingfunktion ska kommunicera:

Web UI → Provider

direkt.

Det skulle göra behörigheter, auditing och riskkontroll betydligt svagare.

Alla provideroperationer ska gå genom serverkontrollerade tradinglager.

Frontend ska endast:

- visualisera state
- visa proposal
- ta emot explicit approval
- visa execution-resultat

Provider API-integration

Ingen provider och inget SDK är valt. Se GATE-15 och GATE-16.

Oavsett provider ska integrationen exponera separata kapabiliteter för exempelvis:

- connection
- account information
- symbol information
- bars
- ticks
- active orders
- open positions
- historical orders
- historical deals

och **separat** funktionalitet för trading requests.

Den separationen är ett krav på providern, inte en tillfällighet: read-kapabilitet och execution-kapabilitet ska kunna aktiveras och behörighetsstyras var för sig. En provider som inte tillåter den uppdelningen är svårare att göra säker, och det ska vägas in i providervalet.

Connection Lifecycle

Adaptern ska hantera provideranslutningen som ett explicit state.

Exempel:

```
DISCONNECTED
→
INITIALIZING
→
CONNECTED
→
ACCOUNT_VERIFIED
→
SYNCHRONIZING
→
READY
```

Runnern får inte behandlas som execution-ready bara för att Pythonbiblioteket lyckats ansluta till terminalen.

Terminal Verification

Efter anslutning ska runnern kontrollera:

- provider endpoint tillgänglig
- terminal version
- connection state
- trading permission state
- active account
- server
- environment
- terminal path när relevant

Fel terminal eller fel account ska stoppa processen.

Account Information

Read-only-integrationen ska kunna läsa aktuell account information.

Minst:

- external account ID
- broker/server
- account currency
- balance
- equity
- margin
- free margin
- leverage
- trading permissions

Account state ska normaliseras till Omniras interna TradingAccount och AccountSnapshot-modeller.

Account Identity

Kontot som providern faktiskt rapporterar måste matcha det konto Omnira förväntar sig.

Exempel:


```
expected_account = PROP_DEMO_01
```

men terminalen rapporterar:

```
PERSONAL_LIVE_01
```

Resultat:

```
ACCOUNT_MISMATCH
```

och:

```
NO EXECUTION
```

Detta gäller även om båda kontona tekniskt är giltiga.

Read-Only Mode

Read-only är ett explicit operation mode.

I detta läge får runnern läsa:

- account info
- terminal info
- symbols
- quotes
- bars
- ticks
- orders
- positions
- history

Men all funktionalitet som kan skapa eller ändra en trade ska blockeras på Omnira-nivå.

Det ska inte bara vara en UI-regel.

Execution Capability Flag

Runnern ska ha explicit capability state.

Exempel:

```
READ_ONLY
```

eller:

```
EXECUTION_ENABLED
```

Default ska vara:

```
READ_ONLY
```

Execution får endast aktiveras genom senare godkänd deployment configuration.

Defense in Depth

Orderläggning ska blockeras på flera nivåer under read-only-fasen.

Exempel:

- Omnira operation mode
- Execution Gateway
- runner capability
- account environment
- allowlists

Det ska alltså inte räcka att någon av misstag anropar en orderfunktion.

Symbol Discovery

Providern erbjuder ett stort antal symbols beroende på broker.

Omnira ska kunna läsa symbolinformationen och skapa mappings mellan:

Canonical Instrument

och:

Broker Symbol

Exempel:

MNQ

kan behöva mappas till ett brokerspecifikt kontraktsnamn.

Denna mapping får inte gissas vid execution.

Symbol Metadata

För varje relevant symbol behöver systemet minst känna till:

- broker symbol
- tick size
- contract size
- volume minimum
- volume maximum
- volume step
- trading status
- currency
- execution-related metadata

Risk Engine ska använda normaliserad instrumentmetadata.

Futures Contract Resolution

NQ och MNQ är futures och kontraktet förändras över tid.

Execution Provider Adapter måste därför kunna identifiera vilket providersymbolnamn som representerar rätt aktivt futureskontrakt.

Strategy Engine ska arbeta med canonical instrument.

Execution Adapter ansvarar för broker mapping.

Market Data via providern

Adaptern ska kunna läsa bars och ticks från providern när providern används som market-data source.

Bars ska kunna begäras för relevanta timeframes:

- 1m
- 5m
- 15m
- 4H

Providerdata ska därefter gå genom Market Data Layer innan Strategy Engine använder den.

Providerdata är inte automatiskt trusted data

Att data kommer från providern betyder inte att validation kan hoppas över.

Omnira ska fortfarande kontrollera:

- timestamps
- freshness
- missing bars
- duplicates
- ordering
- symbol
- timeframe

Providern är datakälla.

Omniras Market Data Layer är validation boundary.

Tick Data

Integrationen ska kunna läsa tickdata där det behövs.

Tickdata kan senare användas för:

- current bid
- ask
- last
- spread
- execution timing
- slippage analysis
- detailed backtesting

Första read-only-MVP:n behöver inte lagra obegränsad tick history.

Current Price

När systemet visar aktuellt pris ska källan vara explicit.

Exempel:

- bid
- ask
- last

Atlas Market View får inte presentera ett enda anonymt "price" om skillnaden är viktig för aktuell analys eller execution.

Active Orders

Read Adapter ska kunna läsa aktiva orders.

Dessa ska synkroniseras till Omniras interna state.

Om providern rapporterar en order som inte finns i Omnira ska systemet kunna markera:

```
UNKNOWN_ORDER
```

Open Positions

Read Adapter ska kunna läsa samtliga relevanta öppna positions.

Minst:

- broker position ID
- symbol
- direction
- quantity
- entry
- SL
- TP
- current P/L
- open time

Providerns state är source of truth för vad brokern faktiskt håller öppet.

Expected State och Observed State

Omnira ska skilja mellan:

```
Expected State
```

och:

```
Observed Broker State
```

Exempel:

Omnira expected:

0 positions

Provider observed:

1 MNQ long

Resultat:

RECONCILIATION_REQUIRED

Ny execution blockeras.

Historical Orders

Integrationen ska kunna läsa orderhistorik.

Detta behövs för:

- audit
- restart recovery
- reconciliation
- execution debugging

Order history är inte samma sak som trade performance.

Historical Deals

Providerns deal history representerar faktiska execution events.

Den ska användas för att rekonstruera:

- fills
- opens
- closes
- partial fills
- commissions
- broker deal IDs

Omnira ska kunna länka deals tillbaka till sina egna ExecutionIntent och Trade-objekt när positionen skapats av Omnira.

Orders, Deals och Positions är olika saker

Systemet måste hålla dessa begrepp separata.

En order är en instruktion.

Ett deal/fill är en faktisk execution.

En position är det resulterande öppna marknadsexponeringen.

Dessa får inte lagras som samma generiska objekt.

Historical Import

När Fas 2 – Futures Connectivity (Read Only) aktiveras kan Omnira importera tidigare tradinghistorik.

Denna historik ska märkas exempelvis:

IMPORTED_HISTORICAL

och inte automatiskt betraktas som trades från Strategy v1.0.

Detta gör det möjligt att använda gammal data utan att förorena systemets validerade strategy performance.

Manual Trades

Om en användare tar en trade manuellt hos providern ska Omnira kunna upptäcka detta.

Origin:

```
MANUAL_EXTERNAL
```

Dessa trades kan journalföras och påverka account risk men får inte räknas som automatiskt Strategy v1.0-resultat.

Unknown Origin

Om systemet inte säkert kan avgöra positionens ursprung ska den märkas:

```
UNKNOWN
```

Unknown position ska vara en blockerande state tills reconciliation genomförs.

Sync Loop

Runnern ska kontinuerligt synkronisera relevant broker state.

Exempel:

```
Account  
→ Orders  
→ Positions  
→ Recent Deals  
→ Health
```

Synkintervall ska anpassas efter datatyp.

Execution-critical state behöver högre freshness än exempelvis lång historik.

Snapshot och Event

Omnira ska använda både:

```
Snapshots
```

och:

```
Events
```

Snapshot beskriver aktuellt tillstånd.

Event beskriver vad som hände.

Exempel:

Snapshot:

position currently open

Event:

position filled at 10:24:03

Båda behövs för robust reconciliation.

Heartbeat

Runnern ska skicka heartbeat till Omnira.

Heartbeat ska inte enbart säga:

I'm alive

utan även kunna inkludera:

- provider connected
- broker connected
- active account
- environment
- read capability
- execution capability
- reconciliation state
- latest successful sync

Runner State

Exempel på runner states:

```
OFFLINE
STARTING
PROVIDER_DISCONNECTED
ACCOUNT_MISMATCH
SYNCING
RECONCILING
READ_ONLY_READY
EXECUTION_READY
BLOCKED
```

Atlas Market View ska kunna visa relevant state.

Startup

Vid start ska runnern inte gå direkt till ready.

Flödet ska exempelvis vara:

```
START
→ INITIALIZE PROVIDER
→ VERIFY TERMINAL
→ VERIFY ACCOUNT
→ SYNC ACCOUNT
→ SYNC ORDERS
→ SYNC POSITIONS
→ SYNC RECENT HISTORY
→ RECONCILE
→ READY
```

Restart Recovery

Runnern ska kunna startas om utan att Omnira förlorar kontroll över account state.

Efter restart ska historical deals och current positions kunna användas för att återuppbygga execution state.

Ny execution blockeras tills detta är klart.

Last Error

Providerintegrationens tekniska fel ska översättas till strukturerade Omnira error states.

Råa felkoder får lagras för debugging.

Atlas och UI ska däremot kunna visa begriplig status.

Exempel:

```
PROVIDER_CONNECTION_FAILED
SYMBOL_NOT_FOUND
ACCOUNT_MISMATCH
ORDER_REJECTED
```

Read Adapter

Read Adapter ska vara separat från Execution Adapter.

Read Adapter ansvarar för:

- connection
- terminal metadata
- account data
- symbols
- market data
- orders
- positions
- history

Read Adapter ska kunna köras även när Execution Adapter är avstängd.

Execution Adapter

Execution Adapter introduceras först i en senare fas.

Den ansvarar för:

- broker pre-check
- order request construction
- order submission
- order result
- position modification
- position close

Execution Adapter ska aldrig innehålla strategy logic.

Order Pre-Check

Innan en trading request skickas kan providerns pre-check-funktionalitet, där den finns, användas för att kontrollera om requesten är tekniskt acceptabel.

Det kan bland annat ge information om huruvida requesten kan utföras med aktuell account state.

Detta är defense-in-depth.

Ett lyckat broker pre-check betyder inte att Omniras Strategy-, Risk- eller Prop Firm-regler automatiskt är uppfyllda.

Order Submission

Trading requests ska senare skickas genom den separata execution-funktionen.

Denna capability ska betraktas som privilegierad.

I Analysis och Read Only ska denna väg vara blockerad.

När den senare aktiveras får den endast nås genom:

```
Approved ExecutionIntent  
→ Pre-Execution Revalidation  
→ Execution Adapter
```

Ingen direkt order submission från Strategy Engine

Det ska arkitektoniskt vara omöjligt för Strategy Engine att göra:

```
submitOrder()
```

Strategy Engine känner endast till:

```
STRATEGY_SIGNAL
```

Detta minskar blast radius om strategy-kod innehåller ett fel.

Broker Pre-Check är inte Risk Engine

Det är viktigt att skilja dessa.

Providern kan exempelvis godkänna att kontot tekniskt har tillräcklig margin för en order.

Men Omnira kan ändå säga:

DENY

på grund av:

- \$150 risk limit
- daily stop
- position limit
- prop firm drawdown
- news
- kill switch

Broker permission är inte Omnira permission.

Market Order Request

När Strategy v1.0 senare exekveras ska execution request innehålla exakta parametrar.

Exempel:

- action
- symbol
- quantity
- order direction
- execution parameters
- stop loss
- take profit
- unique identifier
- comment/correlation metadata där möjligt

Exakt request-format ska definieras under implementation och verifieras på demo.

Magic / Strategy Identifier

När providern stödjer identifierande metadata ska Omnira använda en konsekvent identifierare för sina orders.

Det ska göra det lättare att skilja:

- Omnira-generated
- manual trades
- andra automatiska system

Identifieraren ska inte ensam användas som security boundary.

Broker Response

Alla broker responses ska registreras.

Resultat ska kunna skilja mellan exempelvis:

- request accepted
- filled
- partially filled

- rejected
- invalid stops
- invalid volume
- market unavailable
- insufficient resources
- unknown failure

Rå providerresponse ska bevaras där det är säkert och relevant.

Submission är inte Fill

Att providern accepterar en request betyder inte automatiskt att den slutliga executionen blev exakt som planerat.

Systemet måste kontrollera actual resulting state.

Detta innebär att Omnira efter submission ska läsa:

- resulting orders
- resulting deals
- resulting position

Actual Fill

Efter execution ska Omnira registrera:

- actual fill price
- actual quantity
- broker deal ID
- fill timestamp
- slippage
- commissions
- fees

Strategins theoretical entry ska inte skrivas över.

Båda ska bevaras.

Stop Loss Verification

Efter att positionen öppnats ska systemet verifiera att actual broker position har korrekt SL.

Det ska inte räcka att requesten innehöll rätt värde.

Om actual SL saknas eller är fel:

```
CRITICAL_EXECUTION_INCIDENT
```

Take Profit Verification

Samma kontroll ska genomföras för TP när canonical trade-plan kräver broker-native TP.

Position Modification

När Strategy v1.0 triggas break-even ska Execution Adapter senare modifiera positionen.

För long:

SL flyttas till entry efter canonical 1m swing-high-trigger.

För short:

SL flyttas till entry efter canonical 1m swing-low-trigger.

Providerresultatet måste verifieras.

Closing Position

Systemet ska senare kunna stänga en position genom Execution Adapter vid exempelvis:

- news exit
- time exit
- emergency close

Resultatet ska verifieras genom broker state.

News Exit

Strategy v1.0 kräver:

Existing Position Exit = T-15m

inför relevant high-impact USD-news.

Runnern måste därför ha tillgång till aktuell execution instruction i tid.

Om Omnira är unavailable ska systemets framtida outage-policy avgöra hur en sådan aktiv managementregel hanteras.

Detta ska testas före live.

Native Protection vs Active Management

SL och TP bör där möjligt ligga broker-native.

Break-even, time exit och news exit kräver däremot active management events.

Det innebär att systemets riskmodell måste förstå skillnaden mellan:

Protection that survives Omnira outage

och:

Management that requires active runtime

Connection Loss

Om runtime/adaptern tappat provideranslutningen ska den:

- markera connection unhealthy
- stoppa nya execution requests
- försöka återansluta enligt kontrollerad policy
- genomföra reconciliation efter reconnect

Den får inte fortsätta anta att gamla position states är aktuella.

Reconnect

Efter reconnect:

```
READ ACCOUNT
→ READ POSITIONS
→ READ ORDERS
→ READ RECENT DEALS
→ RECONCILE
→ READY
```

Execution får inte återaktiveras innan reconciliation passerat.

Broker Server och VPN

Runnern ska kunna köras via en godkänd network policy.

Detta kan exempelvis vara:

- direct
- approved VPN
- VPS

Men nätverkskonfigurationen ska inte ligga i Strategy Engine.

Prop Firm Profile och deployment policy ska styra vad som är tillåtet.

Latency Monitoring

Runnern ska kunna mäta relevant latency.

Exempel:

- Omnira → Runner
- Runtime → Provider
- request → broker response
- signal → actual fill

Detta hjälper senare vid VPS-beslut.

Stationär dator som första rigg

Deployment target är inte låst och följer av providervalet. Se GATE-17.

Miljön ska användas för:

- provideranslutning
- runtime om ett sådant krävs
- demo
- read-only verification
- system health tests

Den ska inte betraktas som permanent infrastruktur.

Arkitekturen ska göra VPS-migration enkel senare.

VPS Migration

När systemet behöver högre tillgänglighet eller lägre latency ska runnern kunna flyttas.

Omnira ska inte behöva bygga om:

- Strategy Engine
- Risk Engine
- Prop Firm Engine
- Atlas Market View
- journal
- analytics

Endast execution host och deployment config ska förändras.

Credentials

Providercredentials ska behandlas som secrets.

De ska:

- inte lagras i frontend
- inte skickas till AI
- inte skrivas i vanliga logs
- inte ligga i Git
- inte finnas i strategy config

Runnern får endast tillgång till de credentials den behöver.

Separate Demo och Live Credentials

Demo och live ska använda separata credential references.

En deployment får inte automatiskt välja live credentials när demo credentials saknas.

Fail closed gäller.

Account Allowlist

Runnern ska bara få arbeta med explicit godkända accounts.

Om providern ansluter till ett konto utanför allowlist:

BLOCKED

Symbol Allowlist

Under första implementationen bör tillåtna execution-symboler begränsas.

Exempel:

MNQ

Det minskar risken för att fel symbol exekveras.

Logging och Audit

Execution Provider Adapter ska logga systemevents som:

- initialized
- connected
- account verified
- sync completed
- position discovered
- order discovered
- deal imported
- reconnect
- execution requested
- execution result
- modification result
- reconciliation result

Secrets ska aldrig hamna i log payload.

Data Precision

Providerdata måste normaliseras med korrekt instrumentprecision.

Priser, volume och quantity ska respektera broker-symbolens faktiska steg.

Omnira får inte anta att alla instruments använder samma decimalprecision.

Error Recovery

Olika typer av fel ska behandlas olika.

Exempel:

Transient:

- temporary network failure

kan retryas kontrollerat.

State-uncertain:

- connection lost efter order submission

kräver reconciliation före retry.

Permanent config error:

- wrong symbol
- wrong account

ska inte retryas automatiskt.

No Blind Retry

Trading request får aldrig skickas om blint.

Om systemet är osäkert på om den första requesten exekverades ska det först fråga providern om verkligt state.

Detta är kritiskt för duplicate protection.

Testing Strategy

Providerintegrationen ska testas stegvis.

Testnivå 1 – Offline Contract Tests

Testa adapters mot mockade responses.

Testnivå 2 – Local Provider Read Only

Verifiera riktig provideranslutning.

Testnivå 3 – Demo Account Sync

Verifiera account, symbols, orders, positions och history.

Testnivå 4 – Demo Execution

Först efter read-only sign-off.

Testnivå 5 – Failure Testing

Testa disconnects, restarts och reconciliation.

Testnivå 6 – Live Manual Approval

Endast efter senare gates.

Read-Only Acceptance Criteria

Fas 2 ska inte betraktas som färdig förrän systemet konsekvent kan:

- connecta till providern
- verifiera rätt account
- läsa account state
- läsa relevanta symbols
- läsa NQ/MNQ/ES-data
- läsa open orders
- läsa open positions
- läsa history
- identifiera manuella/okända positions
- reconnecta
- reconcila efter restart
- rapportera health till Omnira

utan att kunna skicka en live-order.

Execution Acceptance Criteria

När Execution Adapter senare aktiveras på demo ska systemet bland annat verifiera:

- rätt account
- rätt symbol
- rätt quantity
- exakt en order
- correct SL
- correct TP
- broker response captured
- fill captured
- position reconciled
- break-even modification
- news exit
- time exit
- duplicate protection
- restart recovery

Atlas Market View och providern

Atlas Market View ska kunna visa både:

Omnira State

och:

Broker State

Exempel:

Planned entry:

24,120.25

Actual fill:

24,120.75

Planned SL:

24,105.50

Broker SL:

24,105.50

Status:

POSITION VERIFIED

Det gör execution transparent.

Driftindikatorer i UI

Trading-projektet ska kunna visa:

- Runner Online
- Provider Connected
- Broker Connected
- Account Verified
- Data Fresh
- Reconciled
- Read Only / Execution Enabled
- Demo / Live
- Last Heartbeat

Användaren ska inte behöva öppna terminalen för att veta om integrationskedjan är healthy.

Atlas och providerfel

Atlas ska kunna översätta strukturerade tekniska fel.

Exempel:

Execution är blockerad eftersom runtime rapporterar att providern är ansluten men fel account är aktivt.

eller:

Positionen öppnades korrekt men broker verifierar inte planerad stop loss. Kontot har därför satts i execution-blocked state.

AI:n ska inte försöka dölja tekniska fel med optimistiskt språk.

Providerdata och Self-Improvement

Providerdata blir även en viktig källa för Atlas Trading Learning & Improvement Layer.

Systemet kan över tid analysera:

- slippage
- fill latency
- broker rejections
- commissions
- execution time
- missed execution
- position modification success

Detta gör det möjligt att förbättra execution utan att blanda ihop execution quality med strategy quality.

Strategy Quality vs Execution Quality

Om en trade hade:

Theoretical +2R

men actual fill och execution costs resulterar i betydligt sämre performance måste systemet kunna identifiera orsaken.

Det kan vara:

- strategy problem
- latency problem
- slippage
- broker costs
- implementation problem

Execution Provider Adapter ska ge tillräcklig data för att göra denna separation.

Ingen live-aktivering genom koddeploy

Att Execution Adapter finns i kodbasen betyder inte att live execution automatiskt är tillåten.

Live authority ska vara separat konfiguration och governance.

Detta minskar risken att en ny deployment av misstag börjar handla riktiga pengar.

Säker standard

Den initiala standarden för alla nya providerintegrationsmiljöer ska vara:

READ_ONLY

En ny runner, nytt account eller ny deployment måste explicit kvalificeras innan execution får aktiveras.

Kapitelstatus

Kapitel: 9 – Futures Execution Integration

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Revision: 2026-08-28 – kapitlet omskrivet provider-neutralt (Beslut D). Filnamn och rubrik ändrade från MetaTrader 5-integration. Struktur och resonemang bevarade.

Initial integration: Ingen provider vald (GATE-15). Adapterkontrakt ej låst (GATE-16)

Initial host: Ej låst. Runtime är deployment-beroende (GATE-17)

Initial capability: READ ONLY

Account sync: Planerad

Market data sync: Planerad

Positions/history sync: Planerad

Order pre-check: Framtida execution defense-in-depth
Order submission: Förbjuden under Read Only

Demo execution: Ej aktiverad

Live execution: Förbjuden

Host migration: Arkitekturen ska stödja den

Omnira ska först bevisa att systemet korrekt kan observera, synkronisera och reconcila den externa providern innan det får rätt att skicka en enda tradingorder.

KAPITEL 10

Backtesting

Backtesting är den första riktiga prövningen av om Omnira Liquidity Manipulation faktiskt har en mätbar edge.

Strategin är nu formellt definierad.

Riskreglerna är definierade.

Arkitekturen är definierad.

Men inget av detta bevisar att strategin tjänar pengar.

Backtesting ska därför inte användas för att bekräfta att strategin fungerar.

Backtesting ska användas för att försöka hitta när, varför och hur strategin inte fungerar.

Den centrala principen är:

En backtest ska försöka motbevisa edge, inte bekräfta en önskad slutsats.

Vad backtesting ska svara på

Backtesting ska bland annat kunna svara på:

- har strategin positiv expectancy?
- hur stabil är performance över tid?
- fungerar den i både London och New York?
- fungerar A+, A, B och C lika bra?
- ger SMT faktiskt mervärde?
- fungerar 5m och 15m FVG lika bra?
- hur påverkar re-entry resultatet?
- hur många trades behövs innan resultaten börjar stabiliseras?
- vilka market regimes fungerar bäst?
- vilka market regimes fungerar sämst?
- hur stor drawdown kan strategin skapa?
- hur långa losing streaks förekommer?
- hur påverkar commissions och slippage?
- fungerar strategin fortfarande när reglerna testas out-of-sample?

Strategy Specification är source of truth

Backtestmotorn får inte skapa en förenklad variant av strategin.

Den ska testa:

Omnira Liquidity Manipulation – Canonical v1.0

med samma regler som senare används live.

Det betyder bland annat:

- samma sessions
- samma 4H-opens
- samma liquidity definitions
- samma FVG-definition
- samma manipulation rule
- samma iFVG/CISD detection
- samma SMT-logik
- samma setup grades
- samma entry
- samma stop loss
- samma target
- samma minimum 2R
- samma break-even
- samma re-entry
- samma news policy
- samma riskmodell där relevant

Om en regel behöver ändras för backtestet ska detta skapa en ny candidate strategy version.

Samma Strategy Engine i Backtest och Live

Om möjligt ska backtest använda samma Strategy Engine som live.

Skillnaden ska främst vara:

Live:

Realtime Market Data Adapter

Backtest:

Historical Data Adapter

Strategy Engine ska inte veta om datan kommer från historik eller live.

Detta minskar risken för att strategin fungerar annorlunda i test än i produktion.

Event-Driven Backtesting

Backtestmotorn bör byggas event-driven.

Historiska marknadsdata ska spelas fram kronologiskt.

Exempel:

09:59 candle

→ process

10:00 candle

→ process

10:01 candle

→ process

Strategy Engine får endast se data som hade varit tillgänglig vid den aktuella tidpunkten.

Detta gör simulationen mer lik live trading.

Look-Ahead Bias

Look-ahead bias är ett av de största hoten mot ett trovärdigt backtest.

Det uppstår när systemet använder information från framtiden för att fatta ett historiskt beslut.

Exempel:

En swing high kräver en efterföljande candle för att bli bekräftad.

Backtestet får därför inte behandla swing high som känt innan nästa candle faktiskt har existerat.

Samma princip gäller:

- candle close
- FVG confirmation
- CISD
- iFVG
- SMT
- liquidity sweeps
- market regime
- targets

Om framtida data smyger in blir resultatet artificiellt bra.

Candle Close Integrity

Om entry enligt Strategy v1.0 sker på confirmation candle close måste backtestet vänta tills candle är stängd.

Det får inte anta att close-priset var känt under candle.

Entry kan först simuleras efter att confirmation är bekräftad.

Intrabar Ambiguity

OHLC-data visar:

- open
- high
- low
- close

men inte alltid exakt ordning mellan high och low.

Det kan skapa problem.

Exempel:

Under samma 1m-candle kan både:

- stop loss
- take profit

ha träffats.

Med endast OHLC går det inte alltid att veta vilken som träffades först.

Backtestmotorn får då inte automatiskt välja det mest lönsamma resultatet.

Conservative Intrabar Policy

När intrabar-sekvens är okänd ska systemet använda:

- tickdata
- lägre timeframe
- eller en konservativ executionregel

Det ska dokumenteras vilket alternativ som används.

Om vi inte kan avgöra om TP eller SL träffades först ska testet hellre underskatta än överskatta performance.

Tick Data för Execution Precision

Första strategy research kan genomföras med bars där reglerna tillåter det.

Men när vi ska validera:

- entries
- stop loss
- TP
- break-even
- slippage

behöver tickdata eller tillräckligt detaljerad intrabar-data användas när bar-data inte räcker.

Detta blir särskilt viktigt på 1m.

Historical Dataset

Varje backtest ska veta exakt vilket dataset som användes.

Minst:

- provider
- instrument
- contract
- date range
- timeframes
- timezone treatment
- data version
- quality status
- continuous eller actual contract data

Backtest-resultat utan identifierbar datasetversion är svåra att reproducera.

Futures Rollover

NQ och MNQ är futures.

Historiska tester måste därför hantera contract rollover.

Systemet ska skilja mellan:

- actual contract data
- continuous futures data

Continuous data är användbar för research men kan innehålla price adjustments som aldrig varit verkligt handlingsbara.

Execution-simulering ska därför vara extra försiktig när continuous data används.

Contract-by-Contract Validation

När möjligt bör kritisk strategy validation även göras på faktiska kontrakt.

Detta gör att vi kan kontrollera:

- verkliga prices
- gaps
- liquidity
- contract transitions
- execution economics

Rolloverperioder ska kunna analyseras separat.

In-Sample och Out-of-Sample

Data ska delas upp.

Exempel:

In-Sample

används för:

- utveckling
- debugging
- initial research

Out-of-Sample

hålls undan.

Den får inte användas för att justera strategy rules.

Först när candidate strategy är låst testas den på out-of-sample-data.

Holdout Data

Holdout-data ska fungera som ett verkligt prov.

Om vi tittar på holdout-resultatet och därefter ändrar strategin baserat på det har datasetet inte längre samma värde som holdout.

Det måste då behandlas som development data och nytt holdout behövs.

Walk-Forward Testing

Utöver enkel train/test-split kan systemet senare använda walk-forward testing.

Exempel:

- utveckla på period A
- testa på period B
- rulla fram
- utveckla på B
- testa på C

Detta visar bättre hur strategin hade fungerat när tiden faktiskt går framåt.

Ingen Parameter Hunting

Systemet ska inte testa tusentals kombinationer tills en fantastisk equity curve hittas.

Exempel:

- 1.85R
- 1.86R
- 1.87R
- 1.88R
- 1.89R

och sedan välja exakt den bästa.

Detta ökar risken för overfitting.

Parametrar ska ha tradingmässig motivering och robusthet över ett rimligt intervall.

Robust Parameter Region

En parameter är mer trovärdig om närliggande värden också fungerar.

Exempel:

Om:

minimum $RR = 2.0$

fungerar bra,

och:

1.9

samt:

2.1

också ger rimliga resultat,

är modellen mer robust än om endast exakt:

2.03

fungerar.

Strategy v1.0 ska testas först

Vi ska först testa strategy baseline som den faktiskt är definierad.

Inte optimera direkt.

Initial fråga:

Har Canonical v1.0 edge över huvud taget?

Först därefter börjar candidate research.

Sample Size

En liten mängd trades kan ge missvisande resultat.

Exempel:

20 trades

kan se fantastiska ut av slump.

Vi ska därför alltid visa sample size tillsammans med performance.

Ingen universal miniminivå är ännu låst, men statistik med små sample ska tydligt märkas som preliminär.

Trade Count per Segment

När performance delas upp på:

- session
- setup grade
- SMT
- market regime

minskar sample size snabbt.

Atlas ska därför inte säga:

Grade A+ under London är bäst

om slutsatsen endast bygger på exempelvis fem trades.

Core Metrics

Backtest ska minst mäta:

- trade count
- win rate
- loss rate
- break-even rate
- expectancy
- average R
- median R
- gross P/L
- net P/L
- profit factor

- maximum drawdown
- average drawdown
- longest losing streak
- longest winning streak
- MFE
- MAE

Expectancy

Expectancy är en av de viktigaste metrics.

I R kan den konceptuellt uttryckas som:

Expectancy = average R per trade

Exempel:

Om strategin över många trades har:

+0.24R

i genomsnitt per trade,

har den positiv historisk expectancy.

Det är mer informativt än win rate ensam.

Win Rate är inte Edge

En strategi kan ha:

35% win rate

och ändå vara mycket lönsam om vinnarna är tillräckligt stora.

En annan kan ha:

80% win rate

och ändå vara dålig om förlusterna är stora.

Atlas ska därför aldrig presentera win rate ensam som bevis på kvalitet.

Profit Factor

Profit factor ska mätas som relationen mellan:

- gross profits
- gross losses

Det ger ytterligare information om hur mycket vinst strategin producerar relativt förlust.

Den ska alltid tolkas tillsammans med:

- sample size
- drawdown
- expectancy

Drawdown

Backtestet ska mäta maximum drawdown.

Det ska också kunna visa:

- duration
- recovery time
- depth
- frequency

En strategi med god avkastning men extrem drawdown kan vara opraktisk för prop firm-konton.

Losing Streak

Losing streak är särskilt viktig för den aktuella riskmodellen.

Med:

\$150 risk/trade

och:

\$450 daily stop

är sekvenser av förluster direkt relevanta.

Systemet ska därför mäta:

- max consecutive losses
- frequency of 2-loss streaks
- frequency of 3-loss streaks
- recovery after streaks

R-Based Performance

Alla trades ska kunna analyseras i R.

Det gör resultatet mer jämförbart oberoende av account size.

Exempel:

+2R

-1R

0R

Detta ska finnas parallellt med dollarresultat.

MFE

Maximum Favorable Excursion visar hur långt traden gick i positiv riktning innan exit.

Det kan hjälpa Atlas analysera:

- om targets är för korta
- om break-even sker för tidigt
- om vinnare ofta lämnar mycket potential

MFE är researchdata.

Det ska inte automatiskt ändra target-regler.

MAE

Maximum Adverse Excursion visar hur långt priset gick mot positionen innan exit.

Det kan användas för att förstå:

- stop placement
- setup quality
- entry timing

Även detta är researchdata.

Performance per Session

London och New York ska analyseras separat.

Exempel:

London

- trade count
- expectancy
- drawdown
- win rate

New York

samma metrics.

Om sessionerna fungerar olika ska detta synliggöras.

Performance per Setup Grade

A+, A, B och C ska analyseras separat.

Det är viktigt eftersom alla grades är tillåtna i Canonical v1.0.

Vi vill senare kunna avgöra om exempelvis C faktiskt tillför positiv expectancy.

SMT Analysis

Systemet ska mäta:

- A med SMT
- A utan SMT
- performance per SMT direction
- sample size

Detta gör att vi kan avgöra om SMT verkligen förbättrar resultat.

Liquidity Type Analysis

Performance ska kunna delas upp efter manipulation target.

Exempel:

- previous day high/low

- session liquidity
- previous 4H
- equal highs/lows
- intermediate liquidity
- FVG

Detta kan avslöja vilka setupfamiljer som faktiskt driver edge.

FVG Timeframe

5m och 15m FVG ska kunna analyseras separat.

Om performance skiljer sig kraftigt ska detta bli research finding.

Entry Confirmation Analysis

Systemet ska kunna jämföra:

- iFVG only
- CISD only
- iFVG + CISD
- iFVG + CISD + SMT

Detta följer direkt av setup grades.

Attempt Analysis

Re-entry attempts ska journalföras separat.

Exempel:

- attempt 1
- attempt 2
- attempt 3

Atlas ska kunna mäta expectancy för varje attempt.

Det kan senare visa om attempt 3 faktiskt är värdefull eller bara ökar drawdown.

Break-Even Analysis

Backtest ska mäta effekten av canonical break-even-regeln.

Vi ska kunna se:

- hur många trades gick till BE
- hur många BE-trades hade senare nått TP
- hur mycket drawdown BE minskade
- hur mycket profit BE eventuellt kostade
- fördelningen mellan swing-baserad BE och London window-close BE
- hur London window-close BE påverkade utfallet separat

Canonical regeln ska inte ändras i samma test.

Detta blir input till candidate research.

News Filter Analysis

Canonical v1.0 använder:

No entry T-1h → T+4h

och:

Existing trade close T-15m

Backtest ska följa dessa regler.

Separat research kan senare jämföra andra blackout windows.

Trading Costs

Backtesting ska inkludera realistiska kostnader.

Minst:

- commissions
- fees
- spread där relevant
- slippage

Resultat ska visas:

gross

och:

net

Slippage

Slippage ska inte alltid antas vara exakt noll.

Systemet ska senare kunna testa flera executionmodeller.

Exempel:

- optimistic
- expected
- conservative

En strategi som endast fungerar vid perfekt fills är inte robust.

Commission Model

Commission ska motsvara den account/broker setup som testet försöker simulera.

Om exakta historiska avgifter saknas ska modellen dokumenteras.

Atlas ska inte presentera net performance utan att kunna säga vilken cost model som användes.

Execution Latency

När strategin exekverar på 1m kan latency påverka resultat.

Backtest ska därför senare kunna simulera:

- instant execution
- liten latency
- större manual approval latency

Detta kan bli särskilt viktigt när vi jämför manual approval med demo automation.

Manual Approval Simulation

Under tidig livefas krävs human approval.

Backtest/replay bör därför kunna analysera:

Hur mycket påverkar en realistisk approval-delay strategy performance?

Det kan hjälpa oss förstå om strategin är för snabb för manual live approval.

Prop Firm Simulation

Backtesting ska senare kunna köras med Prop Firm Rules Engine aktiv.

Då kan systemet mäta:

- challenge survival
- rule breaches
- drawdown headroom
- probability of passing challenge
- number of trading days
- daily loss interactions

Strategy profitability och prop firm suitability är inte exakt samma sak.

Internal Risk Simulation

Backtest ska kunna köras med:

\$150 risk/trade

\$450 daily stop

max 3 attempts/thesis

max 1 open position

Det gör simulationen mer realistisk för den första deploymenten.

Strategy-Only vs Full-System Test

Systemet ska kunna visa två nivåer.

Strategy-Only

Hur signalerna hade presterat utan account risk constraints.

Full-System

Hur faktiskt tillåtna trades hade presterat med:

- Risk Engine
- Prop Firm Engine
- news
- execution costs
- position limits

Båda är värdefulla men svarar på olika frågor.

Rejected Trade Simulation

Nekade trades ska kunna följas counterfactually.

Exempel:

Risk Engine:

DENY

Backtest research kan ändå registrera:

Den hypotetiska trade hade blivit +2R.

Det betyder inte att riskbeslutet var fel.

Det betyder att datan kan användas för forskning.

Survivorship Bias

När fler marknader senare testas får vi inte bara välja marknader som fortfarande är populära eller som vi redan vet fungerade bra.

Data selection ska dokumenteras.

Regime Bias

Vi får heller inte endast testa exempelvis starka trendår om strategin senare ska köras i alla marknadsförhållanden.

Data ska täcka olika typer av perioder.

Market Regime Analysis

När regime classification finns ska backtest kunna analysera exempelvis:

- trend
- range
- high volatility
- low volatility
- expansion
- compression

Detta kan bli viktigt för Atlas self-improvement.

Bull och Bear Perioder

Strategin ska testas under både:

- stigande marknadsmiljö
- fallande marknadsmiljö
- sidledes perioder

Long och short ska också analyseras separat.

Long vs Short

Performance ska segmenteras på:

- long
- short

Det kan avslöja asymmetri.

En strategi behöver inte ha samma edge i båda riktningar.

Seasonal Analysis

När datasetet är tillräckligt stort kan Atlas analysera:

- månad
- kvartal
- dag i veckan

Men seasonal findings ska betraktas försiktigt.

Ju fler segment vi testar, desto större risk att något ser bra ut av slump.

Multiple Testing Problem

Om Atlas testar hundratals möjliga samband kommer några av dem statistiskt se fantastiska ut av slump.

Learning Layer ska därför registrera hur många hypoteser som testats och vara konservativ med slutsatser.

Detta är viktigt för self-improvement.

Candidate Strategy Testing

När Atlas senare föreslår:

Strategy v1.1-candidate

ska den testas mot v1.0.

Jämförelsen ska använda samma:

- dataset
- costs
- execution model
- risk model

så långt det är möjligt.

Baseline Comparison

Canonical v1.0 ska fungera som baseline.

En candidate ska inte godkännas bara för att den tjänar pengar.

Den ska visa att den sannolikt är bättre än baseline på relevanta mått.

Förbättring betyder mer än Profit

En candidate kan ha högre total profit men vara sämre genom:

- högre drawdown
- längre losing streak
- sämre stability
- mindre sample
- extrem parameter sensitivity

Därför ska Atlas använda en bredare performancebild.

Out-of-Sample Gate

En candidate som ser bättre ut in-sample men sämre out-of-sample ska normalt inte uppgraderas.

Out-of-sample-resultatet är en central gate.

Forward-Test Gate

Backtest räcker inte för production activation.

En candidate måste senare gå genom forward testing.

Detta behandlas i nästa kapitel.

Monte Carlo Analysis

När det finns tillräckligt med trades kan Monte Carlo-liknande simuleringar användas för att förstå variation i resultat.

Exempel:

Samma historiska trades kan ordnas i olika sekvenser för att uppskatta möjliga:

- drawdowns
- losing streaks
- equity paths

Det hjälper till att förstå path risk.

Bootstrap och Robustness

Systemet kan senare använda statistiska resampling-metoder för att uppskatta hur känsliga resultaten är för enskilda trades.

Om all lönsamhet kommer från två extrema vinnare ska detta synas.

Sensitivity Testing

Viktiga parametrar ska stress-testas.

Exempel:

- slippage
- commission
- latency
- break-even behavior
- data quality

Om strategin kollapsar vid mycket små realistiska förändringar är edge:n svagare än equity curve antyder.

Missing Trade Simulation

Systemet ska kunna simulera att en del trades missas.

Detta kan hända live på grund av:

- network outage
- approval latency
- broker rejection
- runner downtime

En robust strategi bör inte vara helt beroende av att exakt varje historisk vinnare fångas.

Failure Costs

Backtest/replay kan senare inkludera sällsynta tekniska fel.

Exempel:

- större slippage
- missed break-even
- failed entry
- delayed close

Målet är inte att förutsäga alla tekniska problem.

Målet är att förstå hur känsligt systemet är.

Reproducibility

Varje BacktestRun ska kunna reproduceras.

Det ska minst registrera:

- strategy version
- strategy configuration
- dataset
- date range
- instrument
- code version
- detection version
- execution model
- cost model

- risk profile
- prop profile när relevant
- random seed där simulation använder randomness

Backtest ID

Varje körning ska få ett unikt ID.

Exempel:

BT-2026-00124

Det ska göra det möjligt att referera till exakt test i:

- journal
- Atlas findings
- candidate proposals
- boken
- development logs

Immutable Results

Ett färdigt backtestresultat ska inte redigeras för att passa en ny strategi.

Ny strategi:

→ nytt BacktestRun

Ändrad cost model:

→ nytt BacktestRun

Ändrat dataset:

→ nytt BacktestRun

Historiska testresultat ska bevaras.

Atlas Backtest Analysis

Atlas ska kunna analysera ett färdigt test.

Exempel:

Canonical v1.0 producerade positiv expectancy totalt, men nästan hela fördelen kom från London A/A+ setups. C-grade hade negativ expectancy i båda sessionerna.

Atlas ska alltid stödja slutsatsen med faktisk testdata.

Atlas ska leta efter Problem

Atlas ska inte endast sammanfatta det positiva.

Den ska aktivt leta efter:

- koncentrerad edge
- svaga perioder

- drawdown clusters
- parameter sensitivity
- dåliga setup grades
- dåliga regimes
- execution sensitivity
- data anomalies

AI-lagret ska fungera som kritisk reviewer.

Backtest Report

Varje större backtest ska kunna generera en standardiserad rapport.

Rapporten ska exempelvis innehålla:

Test Configuration

- strategy
- version
- dataset
- period
- costs
- risk model

Performance

- trade count
- expectancy
- profit factor
- drawdown
- win rate

Segmentation

- session
- grade
- direction
- SMT
- liquidity type
- market regime

Robustness

- out-of-sample
- sensitivity
- execution assumptions

Conclusion

- PASS

- FAIL
- INCONCLUSIVE

PASS betyder inte Live

Ett lyckat backtest innebär inte att live trading är godkänd.

Ett backtest PASS betyder endast att strategin har klarat den definierade historiska testgaten.

Nästa steg är forward testing.

FAIL

Om Canonical v1.0 visar negativ expectancy ska vi inte försöka dölja resultatet genom att genast optimera bort alla förlorare.

Resultatet ska dokumenteras.

Vi kan därefter forska fram en candidate v1.1.

Men v1.0-resultatet ska finnas kvar.

INCONCLUSIVE

Ett test kan också vara:

INCONCLUSIVE

Exempel:

- för få trades
- dålig datakvalitet
- otillräcklig period
- osäker execution simulation

Det är bättre att säga att vi inte vet än än att låtsas ha bevis.

Backtesting och Self-Improvement

Backtestmotorn är den första stora experimentmiljön för Atlas Learning Layer.

Atlas ska kunna:

- skapa hypothesis
- definiera candidate
- köra eller begära test
- jämföra resultat
- mäta robustness
- dokumentera finding

Men production-regler får inte ändras direkt.

Research Registry

Varje self-improvement hypothesis bör senare kunna lagras med:

- hypothesis ID

- reason
- supporting data
- candidate change
- test plan
- test results
- conclusion
- status

Exempel:

HYP-0041

Testa om Grade C bör blockeras under New York-sessionen.

Det gör systemets lärande auditerbart.

No Cherry Picking

Atlas får inte kassera testresultat bara för att de inte stödjer hypotesen.

Negativa resultat är också kunskap.

Wisdom från Backtesting

Över tid ska Atlas kunna bygga findings såsom:

Re-entry attempt 3 försämrar expectancy men minskar inte antalet vinnande dagar tillräckligt för att motivera extra drawdown.

eller:

SMT förbättrar A-setups tydligt under New York men har liten effekt under London.

Dessa findings är:

Research Knowledge

inte automatiskt:

Canonical Rules

Första Backtestmålet

Den första stora backtesten ska inte försöka optimera strategin.

Dess mål är att besvara:

Har Omnira Liquidity Manipulation – Canonical v1.0 positiv historisk expectancy på NQ/MNQ när reglerna implementeras exakt och realistiska kostnader inkluderas?

Detta är den första riktiga edge-gaten.

Minimikrav före godkänt Backtestprogram

Innan vi litar på testresultat ska följande vara verifierat:

- strategy rules implemented
- no look-ahead

- timezones correct
- session boundaries correct
- swing detection correct
- FVG detection correct
- iFVG/CISD detection correct
- SMT logic correct
- entry timing correct
- SL/TP correct
- break-even correct
- news policy correct
- re-entry correct
- realistic costs
- reproducible data

Golden Trades

Några manuellt verifierade historiska trades bör användas som golden test cases.

För varje sådan trade vet vi ungefär vad strategin borde identifiera.

Backtestmotorn ska kunna reproducera:

- context
- manipulation
- entry
- SL
- TP
- outcome

Golden trades används för implementation verification.

De ska inte användas som statistiskt bevis på edge.

Negative Golden Cases

Vi behöver också exempel där systemet inte ska ta trade.

Exempel:

- under 2R
- news blackout
- fel session
- saknad confirmation
- max attempts reached
- position redan öppen

Dessa är minst lika viktiga som vinnande exempel.

Regression Suite

När Strategy Engine ändras ska samma golden dataset köras igen.

Om tidigare canonical behavior förändras utan medveten versionsändring ska testet falla.

Backtesting som Safety Gate

Backtesting är inte endast research.

Det är också en säkerhetsgate.

Systemet ska bevisa att implementationen:

- gör vad dokumentationen säger
- inte använder framtida data
- inte skapar oväntade positions
- respekterar riskregler

innan demo automation får övervägas.

Kapitelstatus

Kapitel: 10 – Backtesting

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Primär strategy: Omnira Liquidity Manipulation – Canonical v1.0

Primär marknad: NQ / MNQ

Comparison instrument: ES

Backtestmotor: Ej implementerad

Historical dataset: Ej slutligt vald

Golden test cases: Ska samlas

Out-of-sample: Obligatoriskt

Forward test: Krävs efter backtest

Look-ahead bias: Förbjudet

Realistic costs: Obligatoriskt

Backtest PASS: Ger inte live-godkännande

Ett snyggt historiskt resultat är inte målet.

Målet är att skapa tillräckligt stark evidens för att vi ska våga fortsätta försöka motbevisa strategin i nästa testfas.

KAPITEL 11

Forward testing

Forward testing är steget där Omnira Trading System lämnar den rena historiska simulationen och börjar möta verklig marknadsdata i realtid.

Detta är en avgörande fas.

En strategi kan se stark ut i backtest men ändå misslyckas när den möter:

- verklig latency
- verkliga spreads
- verkliga brokerförhållanden
- realtidsdata
- providersynkronisering
- marknadsförändringar
- execution delays
- tekniska avbrott
- faktisk operationell komplexitet

Forward testing ska därför inte ses som en formalitet efter ett lyckat backtest.

Det är en ny och separat bevisfas.

Den centrala principen är:

Forward testing ska bevisa att det system vi faktiskt byggt fungerar i verklig drift, inte bara att strategin såg bra ut historiskt.

Vad forward testing ska validera

Forward testing ska validera både:

Strategin

och:

Systemet runt strategin

Det innebär att vi ska testa:

- realtime Market Data
- Strategy Engine
- Atlas Analysis
- Risk Engine
- Prop Firm Rules Engine
- Trade Proposal
- Approval flow
- Execution Runtime

- provider synchronization
- Journal
- Analytics
- Reconciliation
- Kill switches
- Failure handling

En strategi kan vara bra samtidigt som systemet runt den är dåligt.

Båda måste fungera.

Forward Test är inte Live Trading

Forward testing ska initialt ske utan verklig kapitalrisk.

De första miljöerna är:

- Analysis Only
- Shadow Mode
- Demo / Paper Trading

Live trading kommer senare.

Analysis Only

I Analysis Only får systemet:

- läsa realtime market data
- identifiera setups
- skapa Strategy Signals
- köra AI Analysis
- köra Risk Engine
- köra Prop Firm Rules Engine
- skapa hypotetiska Trade Proposals
- journalföra allt

Ingen order skickas.

Detta är den säkraste första realtime-fasen.

Shadow Mode

Shadow Mode innebär att systemet beter sig som om det skulle handla men inte exekverar.

Systemet ska registrera:

- planned entry
- planned quantity
- planned SL
- planned TP
- expected execution time
- expected costs
- expected result

Därefter följer systemet marknaden och beräknar hur traden hade utvecklats.

Shadow Mode är särskilt värdefullt innan demo execution eftersom det testar:

- realtime strategy behavior
- timing
- state transitions
- logging
- UI
- analysis

utan brokerexecution.

Demo Trading

Efter att Shadow Mode fungerar stabilt får systemet gå vidare till demo.

I demo kan Omnira:

- skapa riktiga orders i providerns demo
- läsa actual fills
- mäta latency
- mäta slippage
- testa SL/TP
- testa break-even
- testa news exit
- testa time exit
- testa reconciliation
- testa restart recovery

Ingen riktig kapitalrisk ska finnas.

Forward Test Modes

En lämplig progression är:

Mode 1 – Analysis Only

→

Mode 2 – Shadow Mode

→

Mode 3 – Demo Manual Approval

→

Mode 4 – Demo Automation

Varje mode ska ha egna acceptance criteria.

Ingen fas ska hoppas över bara för att tidigare backtest såg bra ut.

Realtime Market Data

Forward test ska använda samma typ av realtime-data som senare production.

Detta gör att vi kan upptäcka skillnader mellan:

- historical data
- providerdata
- broker timestamps
- current contracts
- session boundaries

Strategy Engine ska få canonical data genom Market Data Layer.

Data Freshness

Forward testing ska aktivt mäta:

- senaste market timestamp
- ingestion latency
- missing bars
- duplicates
- reconnects
- stale events

Om realtime data inte är healthy ska systemet inte producera executable state.

Time Handling

Forward testing är en viktig kontroll av timezone-logiken.

Systemet ska verifiera att:

- 02:00 New York öppnar rätt trading window
- 05:00 stänger rätt London-entry window och utlöser window-close break-even
- 10:00 öppnar New York-window
- 12:00 stänger New York-entry window
- daylight saving transitions fungerar
- daily risk reset sker rätt

Dessa regler ska testas i faktisk realtime-drift.

Realtime Strategy State

Atlas Market View ska visa Strategy Engine state live.

Exempel:

```
WAIT_FOR_4H_OPEN
→
IDENTIFY_TARGETS
→
WAIT_FOR_MANIPULATION
→
MANIPULATION_CONFIRMED
→
WAIT_FOR_CONFIRMATION
→
STRATEGY_SIGNAL
```

Detta gör det möjligt att manuellt kontrollera att systemet verkligen följer strategin.

Visual Verification

Under tidig forward testing ska användaren kunna jämföra:

- vad Atlas Market View visar
- vad en erfaren trader ser på TradingView eller annan referenschart

Detta är särskilt viktigt för:

- liquidity
- FVG
- swing points
- manipulation
- iFVG
- CISD
- SMT

Avvikelser ska dokumenteras.

Golden Realtime Cases

Precis som i backtesting ska vissa realtime setups markeras som särskilt värdefulla exempel.

När systemet identifierar en tydlig drömsetup kan den sparas som:

```
FORWARD_GOLDEN_CASE
```

Den kan senare användas som regressionstest.

Negative Realtime Cases

Vi ska även spara situationer där systemet korrekt avstår.

Exempel:

- news blackout
- R:R under 2
- saknad confirmation
- fel session
- max attempts nådd

- position redan öppen
- risk denied

Dessa är viktiga bevis på att systemet respekterar reglerna.

Forward Test Journal

Varje realtime setup ska journalföras med samma struktur som senare live.

Det innebär minst:

- setup
- strategy version
- session
- market context
- liquidity
- FVG
- manipulation
- confirmations
- setup grade
- planned entry
- planned SL
- planned TP
- risk
- AI analysis
- RiskDecision
- PropDecision
- result

Forward test-data ska inte behandlas som tillfälliga utvecklingslogs.

Det är researchdata.

Shadow Fill Model

I Shadow Mode måste systemet använda en tydlig executionmodell.

Det ska inte bara anta:

```
fill = strategy entry
```

Vi kan simulera:

- current bid/ask
- commission
- slippage
- latency

Detta gör Shadow Mode mer realistiskt.

Demo Fill Data

I Demo Mode ska faktisk providern fill användas.

Systemet ska jämföra:

Planned Entry

mot:

Actual Demo Fill

Skillnaden journalförs som execution slippage.

Manual Approval Forward Test

Demo Manual Approval ska testa hela människan-i-loop-processen.

Användaren ska se Trade Proposal och aktivt godkänna.

Systemet ska mäta:

- signal time
- proposal time
- approval time
- execution time
- fill time

Detta gör det möjligt att mäta human latency.

Human Latency

Manual approval kan påverka en snabb 1m-strategi.

Det måste mätas istället för att antas vara acceptabelt.

Exempel:

Om median approval tar:

37 sekunder

och detta regelbundet försämrar R:R eller entry quality kan manual approval visa sig vara olämplig för vissa setups.

Det är ett faktiskt systemfynd.

Approval Expiry

Trade Proposal ska expire om marknaden förändrats för mycket eller om proposal blivit för gammal.

Forward testing ska hjälpa till att kalibrera en realistisk expiry-policy.

Den ska inte väljas enbart teoretiskt.

Demo Automation

När manual demo execution fungerar stabilt kan systemet aktivera Demo Automation.

I detta läge behövs ingen explicit human approval per trade.

Alla andra lager måste fortfarande passera.

Det innebär:

Strategy PASS**Risk PASS****Prop Firm PASS**

Automation Policy PASS

→

Execution

Demo Automation är en stor gate

Demo Automation är första gången hela tradingkedjan får arbeta självständigt.

Det ska därför kräva att systemet redan bevisat:

- korrekt strategy detection
- korrekt risk
- korrekt account sync
- korrekt providern execution
- duplicate protection
- reconciliation
- kill switch
- logging
- position management

Ingen Live direkt från Shadow Mode

Systemet får inte gå:

Shadow Mode

→

Live

bara för att strategin ser bra ut.

Demo execution behövs för att testa den verkliga executionkedjan.

Realtime Risk Engine

Risk Engine ska köras på riktig realtime account state under demo.

Det innebär att vi ska verifiera:

- \$150 risk
- \$450 daily stop
- 1 position max
- 3 attempts max
- minimum contract risk
- daily reset

- active position state

Riskregler ska inte endast testas i unit tests.

De ska även verifieras i verklig demo-drift.

Daily Stop Test

Vi ska aktivt testa daily stop.

På demo kan systemet medvetet sättas upp så att threshold nås under kontrollerade former.

Vi ska verifiera att:

- nya trades blockeras
- status visas korrekt
- risk state blir BLOCKED / DAILY_STOP
- audit event skapas
- en redan öppen position tvångsstängs **inte** av den interna dagsregeln
- öppen position fortsätter hanteras av SL, TP, BE, window-close BE, news- och time-exit
- reserved risk nekar en ny trade vars risk överstiger återstående dagsbudget
- reset sker korrekt
- state överlever restart

Prop Firm Simulation

Även innan ett verkligt prop firm-konto används kan forward test använda en virtuell Prop Firm Profile.

Exempel:

50k challenge simulation

Det gör att systemet kan utvärdera:

- max loss
- daily limits
- challenge survival
- rule headroom

utan att riskera en riktig challenge.

News Forward Test

News policy måste verifieras mot riktig kalenderdata i realtid.

Systemet ska kunna demonstrera att:

- ny trade blockeras T-1h
- blackout kvarstår till T+4h
- befintlig position stängs T-15m
- calendar state är korrekt
- timezone är korrekt

Detta är en viktig production gate.

News Provider Failure

Forward testing ska även inkludera scenario där news-data försvinner.

Expected behavior:

NEWS_STATE_UNKNOWN

och:

NO NEW EXECUTION

Systemet ska visa detta tydligt.

Break-Even Testing

Canonical break-even-regeln ska testas på demo.

Vi ska verifiera att:

- rätt 1m swing identifieras
- trigger sker vid rätt tid
- SL modification skickas
- providern bekräftar ändringen
- journal uppdateras
- Atlas Market View visar rätt state

Break-Even Failure

Vi ska även simulera eller fånga fall där broker nekar modification.

Systemet ska då:

- inte anta att SL är flyttad
- visa incident
- uppdatera actual broker state
- blockera felaktiga assumptions

Time Exit Testing

New York-positioner ska testas mot:

max 4 timmar

Vi ska verifiera att systemet stänger korrekt och journalför:

TIME_EXIT

London Position Testing

London-trades kan fortsätta längre.

Forward test ska kontrollera att systemet inte felaktigt stänger dem bara för att entry-window har stängt.

Forward test ska dessutom verifiera window-close break-even:

- en London-position som fortfarande är öppen 05:00 får SL flyttad till entry price
- detta sker även när den swing-baserade BE-triggern ännu inte har inträffat
- har swing-triggern redan flyttat SL till entry price sker ingen ytterligare förändring
- positionen fortsätter efter actionen
- ingen fyratimmarsgräns tillämpas på London-trades
- actionen journalförs separat från swing-baserad BE

Re-entry Testing

Max tre attempts per thesis ska testas live/demo.

Systemet måste hålla korrekt attempt count även efter:

- restart
- reconnect
- loss
- re-entry

Attempt count får inte återställas av processrestart.

Opposite Setup Testing

När position är öppen ska motsatt setup ignoreras enligt Strategy v1.0.

Forward testing ska bekräfta detta.

Systemet ska gärna journalföra den observerade motsatta setupen för research utan att exekvera den.

Reconciliation Testing

Demo är rätt plats att utsätta reconciliation för riktiga fel.

Exempel:

- starta om runner
- starta om provideranslutningen
- bryt nätverk
- reconnect
- skapa manual demo position

Systemet ska återställa korrekt state innan ny trading.

Manual Position Test

En manuell demo-position kan öppnas direkt i providern.

Omnira ska då upptäcka:

```
MANUAL_EXTERNAL
```

eller:

```
UNKNOWN_POSITION
```

och följa definierad riskpolicy.

Detta är ett viktigt säkerhetstest.

Duplicate Execution Test

Systemet ska aktivt försöka skicka samma execution intent flera gånger i testmiljö.

Resultat ska vara:

exakt en order

Detta är en obligatorisk gate före live.

Network Failure Test

Forward testing ska inkludera:

- Omnira disconnect
- runner disconnect
- provider disconnect
- broker disconnect

Vi ska dokumentera vad som händer med:

- nya trades
- öppna positions
- reconciliation
- journal

Restart Test

Runnern ska kunna startas om medan en demo-position är öppen.

Efter restart ska systemet:

- identifiera position
- verifiera SL/TP
- återställa state
- återuppta övervakning

utan duplicate execution.

Outage med Broker-Native Protection

Testerna ska demonstrera värdet av broker-native SL/TP.

Om Omnira försvinner ska dessa protections fortfarande ligga hos broker.

Active management kan däremot pausas.

Detta ska dokumenteras som residual risk.

Residual Risk

Ingen architecture eliminerar all risk.

Forward testing ska identifiera kvarvarande risker.

Exempel:

- gap genom SL
- broker outage
- internet outage nära news exit
- provider malfunction
- execution slippage
- unavailable calendar
- corrupted provider data

Dessa ska dokumenteras.

Technical Incidents

Forward Test ska skilja mellan:

Trading Loss

och:

Technical Incident

Exempel:

En korrekt exekverad -1R trade är en trading loss.

En trade med fel quantity är ett technical incident.

Dessa får inte blandas ihop.

Incident Log

Technical incidents ska registreras med:

- type
- severity
- timestamp
- affected component
- affected trade
- root cause
- resolution
- recurrence prevention

Systemet måste visa att incidentfrekvens minskar innan live.

Performance Metrics

Forward testing ska använda samma grundmetrics som backtest:

- trade count
- win rate
- expectancy
- average R
- profit factor
- drawdown

- losing streak
- MFE
- MAE

Men dessutom tekniska metrics:

- execution latency
- slippage
- rejection rate
- runner uptime
- data outages
- reconciliation errors

Backtest vs Forward Test

Atlas ska kunna jämföra:

Backtest

och:

Forward Test

Exempel:

Backtest expectancy = +0.31R

Forward expectancy = +0.28R

Det kan vara rimligt.

Om forward istället visar:

-0.19R

måste skillnaden undersökas.

Performance Degradation

En viss försämring från backtest till forward test är förväntad.

Orsaker kan vara:

- costs
- slippage
- latency
- changing market conditions
- data differences

Systemet ska försöka förklara degradation.

Forward Test ska inte optimeras medan det kör

När en forward test-version startas ska strategy och risk configuration låsas.

Om vi ändrar regler mitt i testet är det inte längre samma test.

En materiell ändring ska starta:

New ForwardTestRun

ForwardTestRun

Varje forward test ska ha:

- test ID
- strategy version
- config version
- risk profile
- prop profile
- environment
- start
- end
- mode
- account
- runner version
- code version

Detta gör testet reproducerbart och auditerbart.

Minimum Duration

Vi ska inte godkänna forward test efter några få bra dagar.

Exakt minimum ska beslutas senare baserat på:

- trade frequency
- sample size
- market regimes
- strategy behavior

Både tid och antal trades är relevanta.

Sample Size

Forward test ska tydligt visa sample size.

Om systemet bara fått:

17 trades

ska Atlas inte kalla resultatet robust.

Regime Coverage

Testperioden ska helst innehålla flera olika marknadsförhållanden.

Exempel:

- trend
- range
- high volatility

- low volatility

Detta kan kräva längre testtid än vi först hoppas.

Session Coverage

Både London och New York ska få tillräcklig forward-testdata innan full strategy approval.

Annars kan en session vara relativt oprövad.

Grade Coverage

A+, A, B och C ska följas separat även i forward test.

Vi ska se om backtestens relation mellan grades kvarstår i realtime.

Attempt Coverage

Attempt 1, 2 och 3 ska följas separat.

Om attempt 3 beter sig helt annorlunda forward än historiskt ska det analyseras.

Execution Cost Validation

Demo och senare controlled live ska användas för att verifiera om cost-modellen från backtest var realistisk.

Exempel:

expected slippage

mot:

observed slippage

Skillnaden används för att förbättra framtida simulation.

Self-Improvement under Forward Test

Atlas Learning Layer får samla och analysera data under forward test.

Den får skapa:

- findings
- hypotheses
- candidate versions

Men den aktiva testversionen ska förbli låst.

Detta hindrar att testet förändras medan det mäts.

Candidate Discovery

Atlas kan exempelvis upptäcka:

B-grade har hittills stark forward expectancy medan C-grade fortsätter underprestera.

Detta är ett finding.

Det betyder inte att C stängs av mitt i testet.

Test Integrity

För att forward test ska ha värde måste vi kunna säga:

Den här versionen kördes oförändrad under testperioden.

Detta är lika viktigt som performance.

Shadow vs Demo Comparison

När både shadow och demo finns ska vi kunna jämföra:

- theoretical fill
- demo fill
- outcome difference

Detta hjälper oss kalibrera simulation.

Manual vs Automated Demo

Systemet ska senare kunna jämföra:

Manual Approval Demo

mot:

Automated Demo

Skillnader kan finnas i:

- latency
- slippage
- missed trades
- performance

Detta blir viktig evidens inför automation.

Forward Testing av Atlas

AI-lagret ska också utvärderas.

Vi ska mäta:

- kvalitet på explanations
- consistency
- hallucination rate
- unknown handling
- market regime classification
- historical comparison correctness

Atlas ska inte gå till live decision support bara för att Strategy Engine fungerar.

AI Hallucination Incident

Om Atlas påstår att:

SMT confirmed

när strukturerad data säger:

SMT = UNKNOWN

är detta ett AI quality incident.

Det ska loggas.

UI source of truth ska fortfarande visa canonical state.

Forward Testing av Prop Firm Engine

Virtual eller demo Prop Firm Profile ska användas för att verifiera:

- headroom
- resets
- drawdown calculations
- denials
- UI explanation

Det är särskilt viktigt innan en betald challenge används.

Challenge Simulation

När Prop Firm Engine är klar ska historiska eller forward trades kunna köras genom en challenge simulation.

Vi ska kunna uppskatta:

- pass rate
- fail reason
- days to target
- drawdown breaches
- risk efficiency

Go/No-Go Gate

Varje forward-testfas ska avslutas med ett explicit beslut:

PASS

FAIL

eller:

INCONCLUSIVE

PASS

Forward test PASS betyder att den aktuella fasens definierade kriterier är uppfyllda.

Det betyder inte automatiskt att nästa fas är live.

FAIL

FAIL ska dokumenteras.

Systemet ska inte försöka dölja tekniska eller tradingmässiga problem.

Efter fix krävs ny testversion där det är relevant.

INCONCLUSIVE

Exempel:

- för få trades
- för kort tid
- för homogen market regime
- data outage
- stora systemförändringar under testet

Då behöver testet fortsätta eller startas om.

Demo Automation Gate

Innan Demo Automation får aktiveras ska minst följande vara verifierat:

- Analysis Mode stable
- Shadow Mode stable
- Futures Connectivity (Read Only) stable
- strategy detection verified
- Risk Engine verified
- Prop Firm Engine verified
- manual demo execution stable
- duplicate protection
- reconciliation
- restart recovery
- SL/TP handling
- break-even
- news exit
- time exit
- kill switch

Controlled Live är senare

Även ett starkt Demo Automation-resultat ska följas av en separat Controlled Live-process.

Live innebär nya risker:

- riktiga pengar
- verklig psykologi
- prop firm enforcement
- live broker behavior

Det behandlas i ett senare kapitel.

Dokumenterad Evidens

Varje promotion mellan modes ska stödjas av dokumenterad evidens.

Inte:

Det känns stabilt nu.

Utan:

- test IDs
- trade count
- uptime
- incident count
- performance
- failure tests
- acceptance criteria

Atlas Forward Test Dashboard

Omnira ska senare kunna visa en central forward-test dashboard.

Exempel:

Strategy: v1.0 Mode: Demo Manual Approval Days: 34 Trades: 87 Expectancy: +0.24R Profit Factor: 1.41 Max DD: -6.2R Execution incidents: 1 Duplicate orders: 0 Reconciliation failures: 0

Status: RUNNING

Det gör utvecklingen transparent.

Forward Test Timeline

Systemet ska kunna visa progressionen:

```
Analysis
→ Shadow
→ Demo Manual
→ Demo Auto
```

med:

- startdatum
- slutdatum
- version
- outcome
- sign-off

Forward Test och boken

Större resultat och lärdomar ska dokumenteras i projektets bok.

Boken ska därför fungera även som utvecklingsjournal.

Exempel:

Strategy v1.0 klarade Shadow Mode men manual approval producerade för stor entry latency. Demo Automation introducerades därför som nästa experiment efter separat safety validation.

På så sätt bevaras varför arkitekturval gjordes.

Första Forward Test-målet

Det första forward-testmålet är inte att tjäna pengar.

Målet är att visa:

Omnira kan observera marknaden i realtid, följa Canonical Strategy v1.0 konsekvent, dokumentera varje beslut och reproducera förväntat beteende utan att skicka en order.

Detta är Analysis Only / Shadow Gate.

Andra Forward Test-målet

Nästa mål är:

Omnira kan genomföra samma beslut genom ett providerns demo-konto med korrekt risk, execution och reconciliation.

Detta är Demo Manual Gate.

Tredje Forward Test-målet

Därefter:

Omnira kan själv genomföra samma pipeline på demo utan human approval och utan att bryta strategy-, risk-, prop-, execution- eller safety-regler.

Detta är Demo Automation Gate.

Forward Testing Principle

Den viktigaste principen är:

Forward testing ska mäta skillnaden mellan hur vi tror att systemet fungerar och hur det faktiskt fungerar när tiden går framåt.

Backtest ger historisk evidens.

Forward test ger operationell evidens.

Båda krävs.

Kapitelstatus

Kapitel: 11 – Forward testing

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Första mode: Analysis Only

Andra mode: Shadow Mode

Tredje mode: Demo Manual Approval

Fjärde mode: Demo Automation

Live: Ej tillåtet

Strategy: Canonical v1.0

ForwardTestRun: Planerad

Acceptance criteria: Ska konkretiseras inför implementation

Self-improvement: Aktiv forskning tillåten

Self-modification under test: Förbjuden

Forward testing ska bevisa både strategy behavior och teknisk drift innan Omnira får närma sig verklig kapitalrisk.

KAPITEL 12

Prop firm-regler

Prop firm-läget är en central del av Omnira Trading System eftersom ett prop firm-konto inte bara kräver en lönsam strategi.

Systemet måste också förstå och följa externa regler som kan skilja sig kraftigt mellan:

- olika prop firms
- olika challenge-typer
- funded-konton
- olika account sizes
- olika programversioner
- olika payout-modeller

En strategi kan vara lönsam och ändå misslyckas med en challenge om systemet bryter mot:

- drawdown
- daily loss
- consistency
- max position size
- minimum trading days
- news restrictions
- holding restrictions
- andra programregler

Den centrala principen är:

Prop Firm Rules Engine ska behandla varje konto som ett separat regelstyrkt kontrakt och aldrig anta att två prop firm-program använder samma regler.

Prop Firm Rules Engine

Prop firm-regler ska hanteras i ett separat lager:

```
Strategy Engine
→ Risk Engine
→ Prop Firm Rules Engine
→ Trade Proposal
```

Prop Firm Rules Engine ska inte ersätta Omniras interna Risk Engine.

Båda lagren måste passera.

Varför intern risk och prop firm-risk separeras

Omniras interna Risk Engine definierar hur mycket risk vi själva accepterar.

Exempel på den första interna baslinjen:

- max \$150 risk per trade

- max \$450 realiserad daily loss
- max en öppen position
- max tre attempts per 4H-thesis

En prop firm kan däremot använda helt andra beräkningsmetoder.

Det kan exempelvis finnas:

- equity-based daily drawdown
- trailing maximum loss
- static maximum loss
- end-of-day trailing loss
- consistency requirements

Därför får dessa två regeltyper inte blandas ihop.

Den striktaste regeln vinner

Anta:

Internal Risk Engine

tillåter:

\$150

i ny risk.

Men Prop Firm Rules Engine beräknar att account headroom endast tillåter:

\$90

Resultatet får då aldrig bli \$150.

Systemet ska använda den striktaste praktiskt tillåtna gränsen.

Om minsta handlingsbara kontrakt riskerar mer än \$90:

TRADE DENIED

Regelprofiler

Varje prop firm-program ska representeras genom en versionsstyrd:

PropFirmProfile

Profilen ska kunna innehålla:

- provider
- program
- account size
- account type
- rule version
- effective date
- daily loss
- maximum loss

- drawdown calculation
- reset policy
- profit target
- minimum days
- consistency rules
- position limits
- instrument restrictions
- news restrictions
- holding restrictions
- payout rules
- network/VPN/VPS policy

Inga globala hårdkodade regler

Omnira får inte innehålla logik såsom:

```
all_prop_firms_daily_loss = 5%
```

Det vore fel.

Regler förändras dessutom över tid.

Samma firma kan samtidigt ha flera olika program med olika modeller.

Exempel på verklig regelvariation

Aktuella prop firm-program visar varför regelmotorn måste vara generell.

FTMO:s 2-Step-program använder exempelvis en Maximum Daily Loss där equity inklusive öppna positioners P/L, swaps och commissions påverkar beräkningen, medan deras Maximum Loss i 2-Step är statisk. FTMO:s 1-Step använder däremot en annan daily-loss-procent och en end-of-day trailing Maximum Loss-modell.

Topsteps aktuella Trading Combine använder istället en Maximum Loss Limit som är trailing och baseras på end-of-day balance-utveckling. För ett \$50K Trading Combine är Maximum Loss Limit initialt \$2,000 under startbalansen.

Detta är exakt varför Omnira inte ska ha en enda generell "drawdown"-variabel.

Rule Types

Prop Firm Rules Engine ska minst kunna modellera följande typer av regler.

Daily Loss

Daily loss kan skilja sig mellan program.

Det kan baseras på:

- balance
- equity
- realized P/L
- unrealized P/L

- commissions
- swaps
- start-of-day balance
- initial capital

Profilen måste därför definiera en exakt:

```
calculation_method
```

Daily Reset

Reset-tid ska vara explicit.

Exempelvis använder FTMO aktuell CE(S)T-midnatt för vissa daily-loss-beräkningar.

Omnira får inte anta att prop firmens tradingdag använder:

America/New_York

bara för att vår strategi gör det.

Maximum Loss

Maximum Loss kan vara:

- static
- trailing
- end-of-day trailing
- intraday trailing
- equity based
- balance based

Dessa är fundamentalt olika regler.

Static Drawdown

Static drawdown innebär konceptuellt att en fast lägstanivå definieras från account start.

Exempel:

Initial Balance = \$50,000

Maximum Loss = \$2,000

Hard Floor = \$48,000

Om regeln verkligen är static ändras floor inte när kontot går upp.

Trailing Drawdown

Trailing drawdown följer account performance enligt firmans definierade metod.

Det innebär att risken kan bli striktare när kontot tjänar pengar.

Systemet måste därför kontinuerligt beräkna aktuell:

```
drawdown_floor
```

inte enbart läsa startvärdet.

End-of-Day Trailing

I en end-of-day trailing-modell justeras gränsen vid definierade dagsgränser snarare än efter varje tick.

Topsteps aktuella Maximum Loss Limit är ett exempel där gränsen stiger när end-of-day balance växer och aldrig flyttas ned igen.

Detta kräver separat state över dagar.

Equity-Based Rules

En equity-baserad regel innebär att floating P/L kan orsaka regelbrott innan positionen är stängd.

Det är särskilt viktigt eftersom Omniras interna första daily-riskmodell använder realiserade förluster.

Prop Firm Engine måste därför kunna blockera eller framtvinga riskåtgärder tidigare än Internal Risk Engine.

Balance-Based Rules

Andra regler kan vara balance-baserade.

Omnira måste veta skillnaden.

balance

och:

equity

får aldrig behandlas som samma fält.

Profit Target

Challenge-profiler kan ha profit target.

Systemet ska kunna lagra:

- target amount
- target percentage
- calculation basis
- completion requirements

Profit target är ett objective.

Det får inte användas som argument för att öka risk.

Minimum Trading Days

Vissa program kräver ett minsta antal tradingdagar.

FTMO:s aktuella 2-Step har exempelvis ett krav på minst fyra tradingdagar i challenge- och verification-faserna.

Omnira ska kunna följa:

- trading days completed
- required days

- qualification state

Systemet ska inte ta onödiga trades bara för att uppfylla ett dagkrav om de inte är giltiga enligt strategin.

Consistency Rules

Vissa prop firms använder consistency-regler.

Topsteps aktuella Trading Combine använder exempelvis ett consistency target kopplat till hur stor andel av det relevanta profitmålet som kommer från den bästa tradingdagen.

Andra program kan ha helt andra modeller.

Consistency måste därför representeras som formel, inte bara boolean.

Payout Consistency

Consistency kan även finnas först i funded/payout-fasen.

Topsteps nuvarande Express Funded Account Consistency använder exempelvis en 40%-modell där den största vinnande dagen jämförs med total net profit.

Det visar att samma provider till och med kan ha flera consistency-modeller.

Maximum Position Size

Prop firms kan begränsa hur många contracts som får hållas samtidigt.

Topsteps Trading Combine använder exempelvis account-size-specifika contract limits och en definierad micro-to-mini ratio.

Omnira måste därför kunna kontrollera:

- requested quantity
- existing quantity
- micro/mini equivalence
- account-level maximum

före execution.

Scaling Plans

Vissa funded-program kan förändra tillåten position size baserat på performance.

Detta innebär att max position size inte alltid är ett statiskt accountfält.

Profilen ska kunna representera:

- tier
- threshold
- active days
- current allowed quantity

Dynamic Risk Programs

Prop firm-regler kan även förändras med performance.

Topstep beskriver exempelvis ett aktuellt Dynamic Live Risk Expansion-program där Daily Loss Limit och maximum position size kan förändras när vissa performance- och active-day-villkor uppfylls.

Omnira måste därför kunna modellera stateful regler.

Stateful Rules

En stateful regel beror på historik.

Exempel:

- highest EOD balance
- largest winning day
- days completed
- current scaling tier
- previous payout
- current payout cycle

Prop Firm Engine kan därför inte endast evaluera en trade isolerat.

Den behöver korrekt account history.

News Restrictions

Prop firms kan ha egna regler kring news.

Dessa kan skilja sig mellan:

- challenge
- funded
- account type

Omnira Liquidity Manipulation har redan sin egen striktare strategy news-policy.

Prop Firm Engine måste ändå kontrollera firmans separata regel.

Om någon regel blockerar:

NO TRADE

Holding Restrictions

Program kan reglera:

- overnight holding
- weekend holding
- news holding

Om en strategy position riskerar att gå in i en förbjuden period måste systemet känna till detta före entry.

Instrument Restrictions

PropFirmProfile ska kunna definiera:

- allowed instruments
- prohibited instruments

- contract limits
- asset classes
- market hours

Ett brokerinstrument som tekniskt går att handla behöver inte vara tillåtet enligt firmans regler.

Trading Hours

Prop firmens definierade trading hours kan skilja sig från Strategy Engines entry windows.

Båda måste respekteras.

Strategy:

10:00–12:00 NY

kan vara giltig.

Men om firmans konto är blockerat eller marknaden enligt programmet måste vara flat tidigare:

Prop Firm Engine kan neka.

Flat-by-Time Rules

Systemet ska kunna stödja regler såsom:

all positions must be closed by X

Detta är separat från strategy time exit.

Prop firmens tidigare deadline vinner.

Network Policy

Vissa firms kan ha policies kring:

- VPN
- VPS
- geographic location
- IP changes

Därför ska executionmiljön ha en konfigurerbar:

```
network_policy
```

Detta är särskilt relevant när den initiala Windows-riggen senare flyttas till VPS.

Rule Source

Varje viktig prop firm-regel ska ha en källa.

Exempel:

- official rules page
- terms
- account dashboard
- provider documentation

Profilen bör lagra:


```
source_reference
```

och:

```
verified_at
```

Regelverifiering

Prop firm-regler förändras.

Därför ska en gammal regelprofil inte antas vara korrekt för alltid.

Systemet ska kunna markera:

```
RULESET_REVIEW_REQUIRED
```

om reglerna inte verifierats inom lämplig tid eller om programmet ändrats.

Versioning

Exempel:

Topstep Trading Combine Rules v2026-07

senare:

v2026-10

Historiska trades ska fortsätta referera till den regelversion som gällde när de togs.

Effective Dates

Varje profil ska kunna ha:

- effective_from
- effective_until

Detta gör att historisk prop compliance kan rekonstrueras korrekt.

Account Binding

Varje TradingAccount ska vara kopplat till exakt relevant PropFirmProfile.

Det ska inte räcka att account metadata säger:

```
provider = X
```

Eftersom samma provider kan ha många program.

Challenge vs Funded

Challenge och funded ska behandlas som olika account states eller profiler.

Reglerna kan förändras efter att challenge passerats.

Omnira får inte automatiskt kopiera challenge-regler till funded-kontot.

Account Lifecycle

Ett prop account kan konceptuellt gå genom:

```
CHALLENGE
→
VERIFICATION
→
FUNDED
→
PAYOUT_CYCLE
```

eller andra provider-specifika states.

Profilen ska kunna representera denna lifecycle.

PropDecision

Varje exekverbar setup ska skapa ett separat:

PropDecision

Det ska innehålla:

- account
- ruleset version
- evaluation timestamp
- result
- failed rules
- warnings
- current headroom
- relevant state

Resultat:

ALLOW

eller:

DENY

Rule-by-Rule Result

Omnira ska kunna visa:

```
Daily Loss → PASS
Maximum Loss → PASS
Position Limit → PASS
Consistency → PASS
Trading Hours → PASS
News → FAIL
```

Final:

DENY

Det gör beslutet auditerbart.

Headroom

Prop Firm Engine ska beräkna hur mycket utrymme som återstår.

Exempel:

Maximum Loss Floor: \$48,000 Current Equity: \$48,620 Headroom: \$620

Atlas kan sedan förklara:

Strategin är giltig, men aktuell technical SL med 1 MNQ skulle använda för stor del av återstående prop firm-headroom.

Safety Buffer

Omnira bör senare kunna använda en intern safety buffer ovanför firmans absoluta breachnivå.

Exempel:

Prop rule:

breach at \$48,000

Omnira operational floor:

\$48,200

Detta kan skydda mot:

- slippage
- fees
- latency
- sudden price moves

Exakt buffer ska vara konfigurerbar och testad.

Det ska inte hittas på av AI.

Hard Rule vs Objective

Prop Firm Engine ska skilja mellan:

Hard Rule

och:

Objective

Hard Rule kan innebära att account failar.

Objective kan exempelvis vara:

- profit target
- minimum days

Ett objective behöver inte blockera en trade på samma sätt som en breachregel.

Rule Severity

Varje regel ska kunna klassificeras som exempelvis:

- hard_breach
- execution_block
- objective
- warning
- informational

Det gör motorn mer generell.

Challenge Optimization

Atlas får senare analysera hur strategin fungerar inom en challenge.

Exempel:

- estimated pass probability
- expected days to target
- drawdown usage
- rule pressure

Men challenge optimization får inte innebära att Strategy Engine börjar ta trades som annars är ogiltiga.

Ingen Forced Trading

Om minsta trading days krävs får Atlas inte skapa en falsk setup för att fylla en dag.

Om ingen giltig trade finns:

NO TRADE

Challenge tar hellre längre tid än att systemet bryter strategy integrity.

Ingen Risk Chasing

När profit target nästan är nått får systemet inte automatiskt höja risk.

Samma RiskProfile gäller tills den explicit ändras.

Ingen Recovery Gambling

Om account närmar sig maximum-loss-gränsen ska systemet inte höja risk för att försöka rädda challenge.

Det är exakt när riskdisciplinen är viktigast.

Prop Firm Kill Switch

Om Prop Firm Engine upptäcker att account inte längre kan handlas säkert ska account kunna sättas:

PROP_BLOCKED

Alla nya trades stoppas.

Breach Detection

Om en prop firm-regel redan har brutits ska systemet inte fortsätta trading.

State ska vara:

```
RULE_BREACH
```

eller motsvarande.

Atlas ska tydligt visa vilken regel som bröts.

Near-Breach State

Systemet ska också kunna visa:

```
NEAR_LIMIT
```

innan faktisk breach.

Detta kan ge användaren tydlig riskinformation utan att ändra canonical strategy.

Reset Handling

När prop firmens daily rule reset sker måste Prop Firm Engine beräkna om state korrekt.

Reset får inte ske utifrån Omniras interna daily-reset om firmans regel använder en annan timezone.

Day Boundary Testing

Prop Firm Engine måste testas hårt kring:

- sekunden före reset
- exakt reset
- sekunden efter reset

Time-zone bugs kan annars orsaka challenge failure.

Fees och Commissions

När prop firmens drawdown definition inkluderar commissions eller andra costs måste dessa inkluderas.

Omnira får inte använda gross P/L när regeln faktiskt baseras på net/equity.

Floating P/L

Om regeln inkluderar unrealized P/L måste Risk/Prop Engine följa detta kontinuerligt.

Det räcker då inte att kontrollera reglerna endast vid entry.

Continuous Monitoring

Vissa prop-regler måste utvärderas medan positionen är öppen.

Flödet är därför inte bara:

Pre-Trade Prop Check

utan även:

Open Position Monitoring

Forced Protective Action

Om en öppen position närmar sig en hård prop firm breach kan framtida riskpolicy tillåta skyddsåtgärd.

Exempel:

- emergency close

Detta måste definieras i policy innan live.

Atlas får inte spontant bestämma sådan exit.

Internal Risk vs Prop Breach

En situation kan exempelvis vara:

Internal daily loss:

\$150 used of \$450

men:

Prop maximum-loss headroom:

\$80

Internal Risk Engine kanske fortfarande skulle säga ALLOW.

Prop Firm Engine säger:

DENY

Final:

DENY

Challenge Simulation

Prop Firm Engine ska kunna användas i:

- backtest
- forward test
- demo
- live

Detta gör att samma strategy history kan simuleras mot flera prop firm-program.

Prop Firm Backtesting

Vi ska kunna fråga:

Hur hade Strategy v1.0 klarat ett 50K-program med dessa exakta regler?

Systemet ska kunna mäta:

- pass/fail

- days to target
- breach reason
- maximum headroom usage
- consistency status
- drawdown path

Program Comparison

När flera prop firms senare övervägs kan Atlas jämföra deras strukturella fit mot strategin.

Exempel:

Strategy v1.0 passar bättre med Program A än Program B eftersom dess normala intraday drawdown ofta ligger nära Program B:s trailing limit.

Detta kan bli värdefullt beslutsstöd.

Jämförelse får inte baseras på marknadsföring

Atlas ska utgå från:

- faktiska regler
- fees
- payout conditions
- historical strategy behavior

inte endast account size eller annonserat profit target.

Prop Firm Analytics

Systemet ska kunna mäta:

- challenge attempts
- pass rate
- breach rate
- average days to pass
- most common breach reason
- risk used
- payout eligibility
- performance per program

Learning Layer

Atlas Trading Learning & Improvement Layer ska även lära från prop firm-resultat.

Exempel:

Strategin är lönsam totalt men 38 % av simulerade challenges misslyckas på trailing drawdown efter re-entry attempt 3.

Det kan skapa en research hypothesis.

Det får inte automatiskt förändra strategin.

Candidate Prop Profiles

Atlas kan senare föreslå:

- annan internal safety buffer
- annan account risk profile
- annan challenge mode

Men prop firmens faktiska hard rules får aldrig ändras av Omnira.

Compliance Knowledge

Atlas ska kunna bygga strukturerad kunskap om:

- provider
- program
- rule versions
- common failure modes
- strategy fit

Denna knowledge ska alltid kunna spåras till regelkälla och aktuell version.

Rule Freshness

Eftersom regler förändras ska Atlas inte använda flera år gammal prop firm-information som om den vore aktuell.

Varje aktiv live-profil ska ha en definierad freshness-policy.

Manual Verification före Live

Innan ett riktigt prop account används ska användaren kunna granska:

- provider
- program
- account size
- exact rules
- source links
- verified date

och explicit godkänna profilen.

Prop Firm Profile Lock

Efter att en live-session börjat ska profileversionen inte ändras tyst.

Om firman ändrar regler krävs:

- new profile version
- impact review
- testing
- activation

Atlas Market View

Trading UI ska kunna visa prop firm-state bredvid Risk Engine.

Exempel:

Internal Risk

PASS

Prop Firm

PASS

Maximum Loss Headroom

\$1,284

Consistency

36% / 50%

Position Limit

1 / 5

Status

SAFE

Denial Example

Atlas ska kunna förklara:

Setup A+ är giltig och Internal Risk Engine tillåter \$150 risk. Prop Firm Rules Engine nekar däremot trade eftersom en full SL skulle kunna föra equity under aktuell trailing maximum-loss-gräns.

Detta gör externa regler begripliga.

Regelbrott är inte Trading Loss

Om en challenge failar för att systemet bryter en prop firm-regel är detta inte en vanlig strategy loss.

Det är:

Compliance Failure

och ska behandlas som en system-/policyincident.

Prop Firm Incident Log

Incidenter ska kunna inkludera:

- rule violated
- ruleset version

- account state
- trigger event
- root cause
- prevention action

Målet ska vara:

zero preventable prop rule breaches

före Controlled Live Automation.

Testing

Prop Firm Rules Engine ska ha boundary tests för:

- exact daily limit
- one tick/cent över limit
- trailing updates
- EOD reset
- equity movement
- commissions
- open positions
- position limits
- consistency thresholds
- minimum days
- news boundaries
- trading time boundaries

Golden Rule Cases

För varje stöddad PropFirmProfile bör vi skapa officiella exempel från firmans dokumentation och verifiera att Omnira producerar samma resultat.

Detta blir golden compliance tests.

Unknown Rule State

Om en aktiv regel inte kan utvärderas på grund av:

- missing account data
- stale firm configuration
- uncertain calculation
- unknown reset state

ska resultatet vara:

PROP_STATE_UNKNOWN

I execution-enabled mode:

DENY

Prop Firm Mode

När systemet kör ett prop-konto ska Prop Firm Mode vara tydligt synligt.

Exempel:

PROP MODE – ACTIVE

Provider: Topstep Program: Trading Combine Profile: Version X Account: 50K Environment: Simulation / Funded enligt konto

Det ska vara svårt att glömma vilket regelverk systemet arbetar under.

Första Prop Firm-profil

Den första faktiska PropFirmProfile ska väljas när vi vet vilken challenge eller provider som ska användas först.

Vi ska då inte kopiera generella exempel från denna bok.

Vi ska läsa firmans aktuella officiella regler och skapa en separat versionsstyrd profile specification.

Varför vi väntar med exakt profil

Detta kapitel definierar motorn.

Det definierar inte ännu vår första firm-specifika implementation.

Det är avsiktligt.

Prop firms kan ändra:

- program
- regler
- account sizes
- payout conditions
- limits

Därför ska firm-specifika värden inte byggas in i systemarkitekturens kärna.

Prop Firm Constitutional Principle

Den viktigaste principen är:

Omnira ska förstå prop firmens regler bättre än den behöver förstå dess marknadsföring.

En challenge är ett regelstyrt system.

Omnira ska kunna räkna reglerna exakt, följa dem kontinuerligt och säga nej till en trade innan den riskerar kontot.

Kapitelstatus

Kapitel: 12 – Prop firm-regler

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Prop Firm Rules Engine: Specificerad på arkitekturnivå

Firm-specifik första profil: Ej vald

Internal Risk Engine: Separat

Rule versioning: Obligatoriskt

Rule source tracking: Obligatoriskt

Unknown prop state: Fail closed

Challenge simulation: Planerad

Live prop execution: Förbjuden tills separat profile verification och senare safety gates är godkända

Prop Firm Rules Engine ska göra det möjligt för Omnira Trading att anpassa samma trading-system till flera challenge- och funded-program utan att den underliggande strategin eller riskarkitekturen byggs om.

KAPITEL 13

Tradingjournal

Tradingjournalen är minnet i Omnira Trading System.

Den ska inte bara svara på:

Vann eller förlorade trade?

Den ska kunna svara på:

- vad såg Strategy Engine?
- vilken marknadskontext fanns?
- varför blev setupen giltig?
- vilken strategy version användes?
- vad analyserade Atlas?
- vad beslutade Risk Engine?
- vad beslutade Prop Firm Rules Engine?
- blev trade godkänd?
- hur exekverades den?
- vad gjorde marknaden efteråt?
- hur bra var själva beslutet?
- vad kan systemet lära sig?

Den centrala principen är:

Omnira ska journalföra hela beslutsprocessen, inte bara resultatet.

Journalen som systemets historiska minne

Varje viktig tradinghändelse ska lämna ett spår.

Det gäller inte bara exekverade trades.

Omnira ska också journalföra:

- setups som aldrig blev färdiga
- setups som blev invalid
- Strategy Signals
- Trade Proposals
- nekade trades
- expired proposals
- break-even trades
- manual trades
- technical incidents
- compliance failures
- missed executions

Detta gör journalen användbar både för audit och self-improvement.

Varför bara vinnare och förlorare inte räcker

Om systemet endast sparar exekverade trades förlorar vi stora delar av informationen.

Exempel:

En A+-setup kan identifieras men nekas på grund av daily stop.

Om den inte journalförs kan Atlas aldrig senare analysera:

Hur såg de nekade A+-setuperna ut?

eller:

Hur hade de utvecklats counterfactually?

Det betyder att även ett korrekt DENY är värdefull data.

Journal Entity

Tradingjournalen ska bygga på strukturerade dataobjekt.

Ett journalobjekt kan exempelvis länka till:

- TradingAccount
- StrategyVersion
- MarketContext
- StrategySetup
- StrategySignal
- AIAnalysis
- RiskDecision
- PropDecision
- TradeProposal
- Approval
- ExecutionIntent
- Order
- Fill
- Position
- Trade
- MarketSnapshot
- ChartSnapshot

Journalen blir därmed en sammanhängande vy över flera tradingobjekt.

Correlation ID

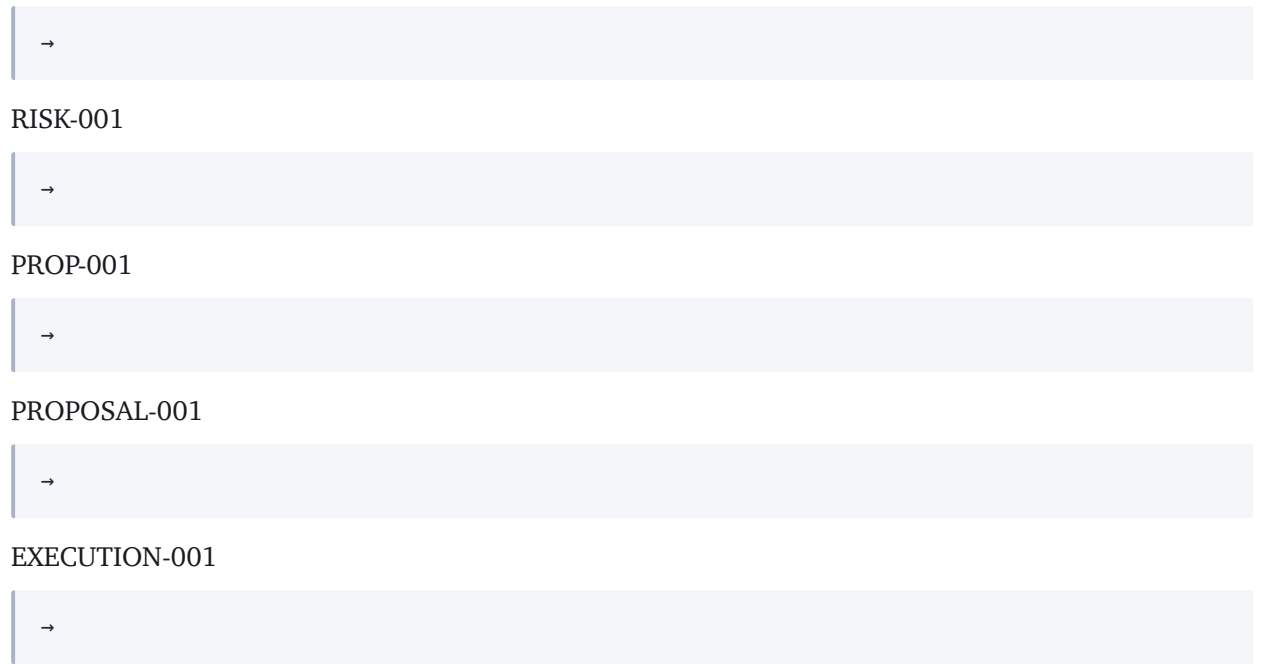
Hela beslutsflödet ska kunna följas via ett gemensamt correlation ID.

Exempel:

SETUP-001

→

SIGNAL-001



→

RISK-001

→

PROP-001

→

PROPOSAL-001

→

EXECUTION-001

→

TRADE-001

Det ska vara möjligt att från slutresultatet gå hela vägen tillbaka till den första marknadsobservationen.

Journal för varje Setup

Varje identifierad setup ska få ett eget ID.

Exempel:

SETUP-2026-000124

Setupen ska kunna finnas även om någon trade aldrig tas.

Setup Data

Journalen ska minst kunna registrera:

- strategy
- strategy version
- account
- instrument
- direction
- selected 4H-open
- session
- setup start
- setup end
- current state
- setup grade
- final status

Market Context

Vid setupens start ska relevant market context sparas.

Exempel:

- current price
- 4H-open
- session
- volatility
- relevant liquidity
- FVG
- market regime
- news state

Detta gör senare jämförelser möjliga.

Liquidity Journal

Om Strategy Engine identifierar liquidity ska journalen veta vilken typ det var.

Exempel:

- swing high
- swing low
- equal highs
- equal lows
- previous session high
- previous session low
- previous 4H high
- previous 4H low
- previous day high
- previous day low
- intermediate liquidity

Det ska inte bara stå:

liquidity present

FVG Journal

För varje relevant FVG ska systemet kunna spara:

- timeframe
- direction
- upper boundary
- lower boundary
- creation time
- touch time
- status

Detta gör det möjligt att analysera om vissa FVG-typer fungerar bättre än andra.

Manipulation Event

När manipulationen blir complete ska ett explicit event journalföras.

Exempel:

```
MANIPULATION_CONFIRMED
```

med:

- timestamp
- liquidity/FVG source
- direction
- price
- relevant context

Det ska gå att se exakt varför systemet gick vidare till 1m entryfas.

1m Entry Context

När systemet går ner på 1m ska följande kunna sparas:

- relevant 1m liquidity
- sweep status
- iFVG state
- CISD state
- SMT state
- confirmation candle
- confirmation time

Detta är viktigt eftersom entrylogiken är central för Strategy v1.0.

Setup Grade

Journalen ska spara exakt grade.

Exempel:

A+

A

B

C

och vad som skapade graden.

Exempel:

iFVG = true

CISD = true

SMT = false

```
→
```

Grade A

På detta sätt kan Atlas senare mäta performance per komponent.

Strategy Signal

När Strategy Engine producerar signal ska journalen spara:

- signal time
- direction
- entry
- SL
- target
- R:R
- grade
- strategy version
- rule evaluation

Strategy Signal är den rena strategidelen innan AI och risk.

AI Analysis

Atlas analys ska sparas separat.

Det kan innehålla:

- summarized thesis
- supporting factors
- contradicting factors
- market regime assessment
- historical comparison
- uncertainty
- AI confidence där sådan används
- model version
- prompt/policy version

AI-texten ska inte ersätta strukturerade strategiobjekt.

Risk Decision

Risk Engine-resultatet ska journalföras.

Minst:

- PASS/DENY
- risk per trade
- quantity
- daily loss used
- daily loss remaining
- open positions
- attempts used
- rule-by-rule result
- RiskProfile version

Risk Denial

Om Risk Engine säger DENY ska setupen fortfarande följas.

Exempel:

DENY_REASON = DAILY_STOP

eller:

DENY_REASON = MINIMUM_POSITION_EXCEEDS_RISK

Detta ger värdefull rejected-trade-data.

Prop Decision

Prop Firm Rules Engine ska journalföras separat.

Det ska kunna visa:

- ruleset version
- result
- failed rule
- headroom
- current account state

En prop denial ska inte blandas ihop med intern risk denial.

Trade Proposal

Om setupen går vidare ska Trade Proposal sparas.

Exempel:

- proposed entry
- proposed SL
- proposed TP
- quantity
- risk
- R:R
- expiry
- proposal state

Proposal Status

Proposal kan få state såsom:

- CREATED
- PENDING_APPROVAL
- APPROVED
- DENIED
- EXPIRED
- CANCELLED
- EXECUTED

State transitions ska journalföras.

Human Approval

När human approval används ska journalen spara:

- decision
- user
- timestamp
- proposal version
- approval latency

Det ska gå att mäta hur approval påverkar execution quality.

Automation Approval

I Demo Automation eller senare Controlled Live Automation ska journalen istället spara:

- automation policy
- policy version
- decision
- timestamp

Det ska aldrig stå bara:

approved automatically

utan vilken policy som gav behörighet.

Execution Intent

När execution startar ska journalen länka till:

- execution ID
- proposal
- account
- symbol
- quantity
- requested SL
- requested TP
- execution mode
- timestamp

Order

Orderobjektet ska lagra broker request/result.

Exempel:

- broker order ID
- requested quantity
- requested price
- order type
- status
- rejection reason

Fill

Actual fill ska journalföras separat.

Minst:

- fill price
- fill quantity
- timestamp
- commission
- fees
- slippage
- broker deal ID

Planned Entry vs Actual Fill

Båda ska bevaras.

Exempel:

Planned Entry

24,120.25

Actual Fill

24,121.00

Slippage

+0.75

Detta ska inte skrivas över till ett enda entryfält.

Position Journal

När positionen är öppen ska systemet journalföra position management.

Exempel:

- initial SL
- initial TP
- current SL
- current TP
- break-even state
- current unrealized P/L
- MFE
- MAE

Management Events

Varje viktig positionändring ska bli ett event.

Exempel:

```

BREAK_EVEN_TRIGGERED
SL_MODIFICATION_SENT
SL_MODIFICATION_CONFIRMED
NEWS_EXIT_TRIGGERED
TIME_EXIT_TRIGGERED
POSITION_CLOSED

```

Break-Even

Canonical break-even-regeln ska journalföras exakt.

För long:

- nearest confirmed 1m swing high efter entry
- price takes swing
- BE trigger
- SL moved to entry

För short:

motsvarande swing low.

Journalen ska veta både trigger och broker confirmation.

Break-Even Trigger Type

Strategy v1.0 har två giltiga break-even-triggers. Journalen ska skilja dem åt, annars går det inte att mäta dem separat i efterhand.

```
be_trigger_type = SWING | WINDOW_CLOSE
```

- SWING — närmaste bekräftade 1m swing efter entry togs
- WINDOW_CLOSE — London-positionen var fortfarande öppen 05:00 America/New_York

Båda ger samma action, SL → entry price, men de har olika orsak och olika performancekaraktär. Utan denna separation kan analysen inte avgöra hur mycket window-close-regeln faktiskt kostar eller sparar.

Break-Even Failure

Om systemet skickar BE modification men broker nekar ska det journalföras som technical incident.

Tradens data ska visa att intended state och actual state skiljde sig.

MFE

Maximum Favorable Excursion ska sparas.

MFE visar den största positiva rörelsen traden nådde innan exit.

Det ska normalt sparas i både:

- points/ticks
- R

där möjligt.

MAE

Maximum Adverse Excursion ska också sparas.

Detta gör det möjligt att analysera entry quality och stop behavior.

Exit

När positionen stängs ska journalen innehålla:

- exit time
- exit price
- quantity
- fees
- reason
- final P/L
- final R

Exit Reason

Exit reason ska vara strukturerad.

Exempel:

- TAKE_PROFIT
- STOP_LOSS
- BREAK_EVEN
- NEWS_EXIT
- TIME_EXIT
- EMERGENCY_CLOSE
- MANUAL_CLOSE
- PROP_PROTECTION
- UNKNOWN

Final R

Trade performance ska alltid kunna uttryckas i R.

Exempel:

+2.18R

-1.00R

0.00R

Det gör jämförelser lättare mellan accounts och riskprofiler.

Dollar Result

Parallellt ska actual dollar-resultat sparas.

Det ska baseras på:

- fills

- commissions
- fees

inte enbart theoretical price movement.

Gross och Net

Journalen ska kunna skilja:

Gross Result

från:

Net Result

Detta gör execution costs synliga.

Winner, Loser och Break-Even

Trade outcome ska kunna klassificeras:

- WIN
- LOSS
- BREAK_EVEN

Men denna enkla klassifikation ska aldrig ersätta final R.

Decision Quality

Efter trade ska systemet kunna bedöma decision quality separat från outcome.

Exempel:

En -1R trade kan ha:

DECISION_QUALITY = VALID

om alla regler följdes.

En +2R trade som togs i strid med policy kan ha:

DECISION_QUALITY = INVALID

Resultat och besluts kvalitet är olika.

Rule Adherence

Journalen ska kunna svara:

Följde systemet strategy v1.0 exakt?

Exempel:

- correct session
- correct manipulation
- correct confirmation
- correct grade
- correct entry

- correct SL
- correct TP
- correct news handling
- correct attempts

Technical Quality

Separat ska systemet mäta technical execution quality.

Exempel:

- expected fill
- actual fill
- latency
- slippage
- modification success
- broker errors

Detta gör det möjligt att skilja Strategy Problem från Execution Problem.

Manual Notes

Användaren ska kunna lägga till manuella anteckningar.

Dessa ska ligga som ett eget lager och inte skriva över systemdata.

Exempel:

Tyckte setupen visuellt såg ovanligt stökig ut.

Det kan senare bli researchinput.

Atlas Post-Trade Review

Efter trade close ska Atlas kunna skapa en standardiserad review.

Exempel:

Setup

Grade A, New York.

Execution

Fill 0.5 points över planned entry.

Management

Break-even aktiverades efter närmaste 1m swing high.

Outcome

+2.04R net.

Decision Quality

Alla canonical rules följdes.

Observation

Liknande setups ska jämföras mot tidigare 15m FVG-manipulationer.

Atlas ska inte skriva om historiken

Om Atlas senare gör en ny analys ska originalanalysen bevaras.

Ny analys:

```
→ nytt analysis object.
```

Historisk text ska inte ersättas.

Det gör det möjligt att se hur Atlas analysförmåga förändras över tid.

Chart Snapshot

Vid viktiga tradingstadier ska systemet kunna skapa chart snapshots.

Exempel:

- setup detected
- manipulation confirmed
- entry signal
- execution
- exit

Snapshot ska visa samma canonical objects som Atlas Market View.

Screenshot är inte Source of Truth

Chart snapshot är mänskligt användbart.

Men analys ska primärt bygga på strukturerad data.

En PNG-bild ska inte vara enda beviset på vad systemet såg.

Market Snapshot

Market Snapshot ska kunna innehålla maskinläsbar state.

Exempel:

- candles
- swings
- liquidity
- FVG
- SMT
- session
- current price
- news

- StrategyState

Detta gör replay möjligt.

Reconstructible Trade

Målet är att en viktig trade ska kunna rekonstrueras.

Vi ska kunna fråga:

Visa exakt vad systemet visste två sekunder innan Strategy Signal skapades.

Detta kräver både tidsstämplad data och versionsinformation.

Rejected Trades

Alla nekade Trade Proposals ska journalföras.

Denial reason kan exempelvis vara:

- RISK_DENY
- PROP_DENY
- NEWS
- RR_TOO_LOW
- MAX_ATTEMPTS
- OPEN_POSITION
- STALE_DATA
- EXECUTION_UNAVAILABLE

Follow Rejected Trades

När möjligt ska systemet fortsätta följa marknaden efter en denied setup.

Detta skapar counterfactual research.

Exempel:

Actual Decision = DENY

Hypothetical Outcome = +2R

Det betyder inte att beslutet var fel.

Missed Setups

Journalen ska också kunna registrera setups som aldrig blev trade på grund av:

- no confirmation
- setup expired
- invalidation
- session ended
- target no longer $\geq 2R$

Detta är viktigt för strategy funnel analysis.

Strategy Funnel

Omnira ska kunna mäta:

4H Contexts

- Manipulations
- 1m Confirmation
- Strategy Signals
- Risk Pass
- Prop Pass
- Proposals
- Executions
- Winners

Detta visar var trades filtreras bort.

Attempt Tracking

Varje thesis ska veta:

- attempt 1
- attempt 2
- attempt 3

Efter loss kan nästa attempt länkas till samma thesis.

Efter winner eller BE ska inga nya attempts tillåtas enligt Strategy v1.0.

Journalen måste göra denna relation synlig.

Thesis ID

Varje selected 4H-open kan skapa ett:

THESIS_ID

Exempel:

NQ-2026-08-27-1000-LONG

Samtliga attempts kan sedan länkas till samma thesis.

Session Analytics

Journalen ska göra det möjligt att analysera:

- London
- New York

separat.

Det gäller både setups och trades.

Grade Analytics

A+, A, B och C ska kunna jämföras.

Journalen måste därför spara grade även för denied/missed setups.

Annars får self-improvement ett snedvridet dataset.

Direction Analytics

Long och short ska kunna jämföras.

Liquidity Analytics

Systemet ska kunna analysera performance per liquiditytyp.

Exempel:

- previous day
- previous session
- 4H
- equal highs/lows
- intermediate

FVG Analytics

Performance per:

- 5m FVG
- 15m FVG

ska kunna mätas.

SMT Analytics

Journalen ska skilja mellan:

- SMT true
- SMT false
- SMT unknown

Detta är viktigt.

UNKNOWN får inte lagras som FALSE.

Market Regime

När regime classification finns ska den lagras med setupen.

Exempel:

```
HIGH_VOLATILITY
```

Det gör senare segmentanalys möjlig.

News Context

Journalen ska lagra:

- nearest relevant event
- time to event
- blackout state
- news provider
- policy version

Detta behövs för att bevisa att newsregler följdes.

Risk History

Daily risk state ska journalföras över tid.

Exempel:

09:00 = \$0 loss

10:30 = -\$150

11:15 = -\$300

12:04 = -\$450 → DAILY_STOP

Det gör riskincidenter rekonstruerbara.

Prop Firm History

Samma gäller prop firm headroom.

Exempel:

- equity
- maximum-loss floor
- daily loss
- consistency
- position limits

Systemet ska kunna visa varför en trade var tillåten eller förbjuden vid exakt den tidpunkten.

Imported Historical Trades

Trades som importeras från providern men inte skapats av Omnira ska markeras tydligt.

Exempel:

ORIGIN = IMPORTED

eller:

MANUAL_EXTERNAL

De får inte automatiskt räknas som Strategy v1.0-resultat.

Demo och Live separeras

Journaldata ska alltid innehålla environment.

Exempel:

- BACKTEST
- SHADOW
- DEMO
- LIVE

Performance från dessa miljöer ska inte blandas omedvetet.

Strategy Version separeras

Trade från:

v1.0

och:

v1.1

ska analyseras separat.

En strategy update får inte retroaktivt ändra gamla trade labels.

Risk Version separeras

Även RiskProfile version ska lagras.

Det gör att Atlas kan analysera hur riskförändringar påverkade resultat.

Prop Profile Version separeras

Samma gäller PropFirmProfile.

Data Version

Viktiga researchresultat ska kunna kopplas till data/provider version.

Detta stärker reproducerbarheten.

Code Version

När möjligt ska journalen även kunna referera till relevant code/build version.

Det gör debugging enklare.

Incident Journal

Technical incidents ska finnas i samma övergripande auditmiljö.

Exempel:

- runner disconnect
- duplicate prevention triggered
- SL modification failed
- data outage
- wrong account
- reconciliation mismatch

Trading Loss vs System Incident

Journalen ska tydligt skilja:

Market Loss

från:

System Failure

Detta är avgörande för kvalitetsanalys.

Compliance Incident

Prop firm-regelbrott ska klassificeras separat.

Det ska betraktas som:

COMPLIANCE_INCIDENT

inte bara dålig trade performance.

Immutable Journal

Historiska journalhändelser ska i princip vara append-only.

Om något behöver korrigeras ska ett correction event skapas.

Originalen ska finnas kvar.

Correction

Exempel:

Felaktig tidigare label:

Grade B

korrigeras senare till:

Grade A

Journalen ska visa:

- original value
- corrected value
- reason
- who/what corrected
- timestamp

Detta är viktigare än att historiken ser "ren" ut.

Audit Trail

För varje live-trade ska systemet kunna skapa en komplett audit trail.

Det ska kunna användas för:

- debugging
- governance
- prop firm review
- performance analysis
- strategy research

Searchable Journal

Atlas ska kunna söka i journalen strukturerat.

Exempel:

Visa alla Grade A New York long setups efter 15m FVG där SMT saknades.

eller:

Visa alla trades där attempt 2 gav bättre outcome än attempt 1.

Detta kräver strukturerade fields, inte bara fritext.

Journal UI

Omnira Trading ska ha en tydlig journalvy.

Användaren ska kunna filtrera efter:

- date
- instrument
- session
- grade
- direction
- strategy version
- outcome
- environment
- denial reason

Trade Detail View

En trade ska kunna öppnas som en timeline.

Exempel:

10:00:00

4H thesis started

10:14:00

15m FVG touched

10:21:00

Manipulation confirmed

10:24:00

iFVG + CISD

10:24:01

Strategy Signal

10:24:02

Risk PASS

10:24:02

Prop PASS

10:24:03

Proposal created

10:24:08

Approved

10:24:09

Execution sent

10:24:09

Filled

10:41:00

Break-even activated

11:06:00

Take Profit

Detta gör hela systemet transparent.

Atlas Journal Summary

Atlas ska kunna sammanfatta dagen.

Exempel:

Idag identifierades 7 setups. Tre nådde Strategy Signal. En nekades på daily risk och två exekverades. Resultatet blev +1.05R netto. Båda exekverade trades följde canonical rules utan technical incidents.

Weekly Review

Atlas ska senare kunna skapa strukturerad veckoreview.

Exempel:

- total setups
- executed trades
- denied trades
- expectancy
- grades
- sessions
- largest drawdown
- incidents
- emerging findings

Monthly Review

Månadsreview kan dessutom inkludera:

- statistical stability
- strategy drift
- regime changes
- execution degradation
- research hypotheses

Learning Dataset

Journalen blir den viktigaste källan till Atlas Trading Learning & Improvement Layer.

Learning Layer ska inte behöva läsa ostrukturerade screenshots och försöka förstå allt från början.

Journalen ska redan ha gjort beslutet maskinläsbart.

Lär från alla beslut

Atlas ska lära från:

- taken
- denied
- expired
- missed
- invalidated

Detta minskar selection bias.

Counterfactual Dataset

Denied och missed setups kan skapa ett separat counterfactual dataset.

Det ska hållas tydligt separerat från actual realized performance.

Research Finding

Atlas kan exempelvis skapa finding:

Grade C producerade negativ expectancy i 312 historiska och 67 forward setups.

Finding ska innehålla:

- source sample
- strategy version
- date range
- methodology
- result
- confidence

Hypothesis

Finding kan därefter skapa:

HYPOTHESIS

Exempel:

Testa candidate där Grade C inte är execution-eligible.

Den går vidare till separat validation.

Journalen ändrar inte strategin

Journal och analytics beskriver vad som hänt.

De ändrar inte productionregler.

Detta bevarar separationen mellan:

Observation

och:

Authority

Datakvalitet i Journalen

Journalposten ska kunna markera om data quality vid beslutet var:

- VALID
- SUSPECT
- DEGRADED

En trade från dålig data ska inte utan vidare räknas tillsammans med normal performance.

Missing Journal Data

Om kritiska fields saknas i live mode ska detta kunna flaggas.

Ett system som tjänar pengar men inte kan förklara sina trades är inte redo för högre autonomi.

Journal Completeness

Omnira ska senare kunna mäta:

journal completeness score

inte som trading signal utan som operational metric.

Målet för live should vara mycket nära fullständig auditability.

Privacy och Secrets

Journalen får inte innehålla:

- account passwords
- API secrets
- privata credentials

Account IDs kan pseudonymiseras i exporter där det behövs.

Export

Journaldata ska senare kunna exporteras för:

- research
- backup
- human review
- external analysis

Export ska behålla versionsmetadata.

Backup

Tradingjournalen är affärskritisk data.

Den ska därför ingå i backup- och recovery-plan.

En förlorad journal innebär inte bara förlorad historik.

Det innebär förlorad tränings- och auditdata för hela systemet.

Retention

Live trade records, audit events och relevanta snapshots ska bevaras långsiktigt.

Stora råa tickdatasets kan ha separat retentionpolicy.

Journalens roll i autonomi

Ju mer autonomt systemet blir, desto viktigare blir journalen.

När människan inte längre godkänner varje trade måste det ändå i efterhand gå att förstå:

- varför beslutet skapades
- vilka regler som gällde
- om safety gates passerades
- vad som exekverades
- vad resultatet blev

Autonomi får inte minska transparensen.

Den ska öka kraven på den.

Den centrala journalprincipen

Omnira ska följa:

If it influenced a trading decision, it should be observable later.

Det betyder inte att varje teknisk byte måste sparas för alltid.

Men varje meningsfullt beslut, state transition och säkerhetskontroll ska kunna rekonstrueras.

Kapitelstatus

Kapitel: 13 – Tradingjournal

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Journalmodell: Strukturerad och eventbaserad

Exekverade trades: Journalförs

Nekade trades: Journalförs

Missade/invalidated setups: Journalförs

MFE/MAE: Obligatoriskt för analys där data stödjer det

Decision Quality: Separat från Outcome

Strategy/Risk/Prop versions: Bevaras

Snapshots: Strukturerade + visuella

Audit trail: Obligatorisk

Self-improvement source: Tradingjournalen är primär datakälla

Tradingjournalen ska göra varje viktig tradingprocess till ett analyserbart, sökbart och reproducerbart historiskt objekt.

KAPITEL 14

Prestationsanalys

Prestationsanalys är lagret där Omnira Trading System går från enskilda trades till faktisk kunskap om hur strategin beter sig över tid.

Tradingjournalen beskriver vad som hände.

Prestationsanalysen försöker besvara:

- hur bra är strategin?
- hur stabil är performance?
- var kommer edge ifrån?
- när fungerar strategin sämre?
- hur mycket risk krävs för avkastningen?
- vilka setup grades bidrar?
- vilka attempts förstör eller förbättrar resultatet?
- påverkar session, liquiditytyp eller SMT utfallet?
- håller edge över flera marknadsregimer?
- är resultatet statistiskt trovärdigt eller kan det vara brus?

Den centrala principen är:

Performance ska bedömas som ett system av flera mått, inte genom en enda snygg siffra.

Prestationsanalysens roll

Analytics Layer ska ligga ovanpå Trading Journal och standardiserad performance-data.

Flödet är:

```
Trading Events
→ Journal
→ Performance Records
→ Analytics
→ Atlas Interpretation
→ Research Findings
→ Candidate Hypotheses
```

Analytics ska kunna användas i:

- backtest
- forward test
- demo
- controlled live
- prop firm simulation

Samma grundläggande metrics ska användas genom hela systemet.

Resultat måste alltid ha kontext

Ett värde såsom:

Profit Factor = 1.62

är inte tillräckligt i sig.

Systemet måste också kunna visa:

- strategy version
- environment
- period
- number of trades
- instrument
- session
- cost model
- risk profile
- prop profile
- data quality

Performance utan kontext är lätt att misstolka.

Strategy Version

All analytics ska grupperas efter strategy version.

Resultat från:

Omnira Liquidity Manipulation v1.0

får inte automatiskt blandas med:

v1.1

En materiell strategyändring skapar ny statistikserie.

Environment

Performance ska också separeras efter environment.

Exempel:

- BACKTEST
- SHADOW
- DEMO
- LIVE

Dessa får jämföras.

De får inte blandas ihop som om de representerar samma typ av evidens.

Gross och Net Performance

Systemet ska kunna visa:

Gross performance

före execution costs.

och:

Net performance

efter:

- commissions
- fees
- slippage

Net performance är den viktigaste ekonomiska bilden.

Gross performance är fortfarande användbar för att skilja strategy quality från execution quality.

R som primär jämförelseenhet

R ska vara ett centralt performanceformat.

Exempel:

+2R

-1R

OR

Det gör det möjligt att jämföra trades mellan:

- olika account sizes
- olika position sizes
- demo
- live
- backtest

Dollarresultat ska finnas parallellt.

Trade Count

Varje performancevärde ska visas tillsammans med trade count.

Exempel:

Expectancy = +0.48R

baserat på:

12 trades

är betydligt svagare evidens än samma expectancy efter:

800 trades.

Atlas ska ta hänsyn till detta när resultat presenteras.

Setup Count

Trade count är inte alltid tillräckligt.

Systemet ska också kunna visa antal:

- detected setups
- Strategy Signals
- risk denials
- prop denials
- expired proposals
- actual executions

Detta gör det möjligt att analysera hela strategy funnel.

Win Rate

Win rate definierar andelen vinnande trades.

Exempel:

Wins / Total Trades

Men win rate ska aldrig användas ensam för att bedöma edge.

En låg win rate kan kombineras med stora vinnare.

En hög win rate kan kombineras med stora förluster.

Loss Rate

Loss rate ska också visas.

Break-even ska kunna behandlas som separat outcome.

Det gör att:

Win Rate + Loss Rate

inte nödvändigtvis behöver vara exakt 100 % om BE klassificeras separat.

Break-Even Rate

Canonical Strategy v1.0 använder break-even management.

Därför är:

BE Rate

en viktig metric.

Vi vill veta:

- hur ofta BE aktiveras
- hur ofta BE-positioner sedan hade nått TP
- hur mycket risk BE reducerar
- hur mycket potentiell avkastning BE eventuellt kostar

Expectancy

Expectancy är ett av de viktigaste måtten i systemet.

En praktisk R-baserad definition är:

Average realized R per trade

Exempel:

Efter 500 trades:

Expectancy = +0.27R

Det betyder historiskt att varje trade i genomsnitt producerat +0.27 gånger initial risk.

Expectancy per Setup

Systemet ska också kunna mäta expectancy per setupkategori.

Exempel:

A+

+0.46R

A

+0.31R

B

+0.08R

C

-0.14R

Detta kan bli mycket viktigt för framtida candidate versions.

Expected Dollar Value

Om risk per trade är konstant kan expectancy också omvandlas till förväntat dollarvärde.

Exempel:

Expectancy = +0.25R

och:

1R = \$150

ger historiskt:

+\$37.50 per trade

före eventuella andra faktorer.

Detta ska alltid ses som historisk statistik, inte garanti.

Average Winner

Systemet ska mäta:

Average Winning R

Exempel:

+2.18R

Det gör det lättare att förstå relationen mellan win rate och expectancy.

Average Loser

På samma sätt:

Average Losing R

Canonical SL gör att många fulla losses kan ligga kring:

-1R

men actual losses kan avvika på grund av:

- slippage
- manual close
- emergency exit
- fees

Payoff Ratio

Systemet kan mäta:

Average Winner / Average Loser

Det ger information om strategy payoff structure.

Median R

Average kan påverkas kraftigt av extrema trades.

Median R ska därför också finnas.

Skillnaden mellan median och average kan avslöja att resultatet drivs av ett litet antal extrema observations.

Profit Factor

Profit Factor ska beräknas som:

Gross Profit / Gross Loss

Exempel:

$PF = 1.50$

betyder historiskt att \$1.50 i gross profit producerades för varje \$1 i gross loss.

Profit Factor ska bedömas tillsammans med sample size och drawdown.

Gross Profit

Total vinst från vinnande trades före relevanta nettokostnader där definitionen kräver det.

Gross Loss

Total absolut förlust från losing trades.

Net Profit

Net Profit ska inkludera faktiska trading costs där de finns.

För live är detta den ekonomiskt relevanta bilden.

Maximum Drawdown

Maximum Drawdown är en av systemets viktigaste riskmetrics.

Den visar största peak-to-trough-fallet i performance curve.

Den ska kunna uttryckas i:

- dollar
- R
- procent där lämpligt

Drawdown Duration

Djup är inte allt.

Systemet ska även mäta hur länge drawdownperioder varar.

En strategi som återhämtar sig från -8R på två dagar beter sig annorlunda än en som behöver tre månader.

Recovery Time

Atlas ska kunna analysera hur lång tid strategin historiskt behöver för att återhämta drawdowns.

Detta är särskilt relevant för prop firm-challenges och användarens riskupplevelse.

Average Drawdown

Maximum Drawdown visar värsta historiska observationen.

Average Drawdown kan ge en bättre bild av typiskt beteende.

Underwater Curve

Analytics ska senare kunna visa hur långt equity ligger under tidigare peak över tid.

Det gör drawdown duration visuellt tydlig.

Consecutive Losses

Systemet ska mäta längsta losing streak.

Exempel:

Max Consecutive Losses = 8

Detta är viktigt även om den dagliga Risk Engine stoppar efter tre fulla \$150-förluster samma dag.

Förluster kan fortsätta över flera dagar.

Consecutive Wins

Längsta winning streak ska också mätas.

Det ska dock inte användas för att motivera automatisk riskökning.

Streak Distribution

Vi vill inte bara känna till den värsta streaken.

Analytics ska kunna visa frekvensen av:

- 2 losses i rad
- 3 losses
- 4 losses
- 5+

Detta ger bättre riskförståelse.

Daily Performance

Systemet ska mäta performance per tradingdag.

Exempel:

- average daily R
- best day
- worst day
- winning days
- losing days
- flat days

Detta är särskilt relevant för prop firm analysis.

Session Performance

London och New York ska analyseras separat.

Exempel:

London

- setups
- trades
- expectancy
- PF
- max DD
- win rate

New York

samma.

Det gör det möjligt att upptäcka om endast en session faktiskt driver strategins edge.

Long vs Short

Long och short ska analyseras separat.

Exempel:

Long expectancy = +0.34R

Short expectancy = +0.09R

Asymmetri är viktig information.

Den ska inte automatiskt ändra strategy rules.

Setup Grade Analysis

A+, A, B och C ska alltid kunna analyseras individuellt.

Det är en av de första analyserna vi kommer vilja göra eftersom Canonical v1.0 tillåter samtliga grades.

SMT Analysis

SMT ska kunna jämföras.

Exempel:

iFVG + CISD + SMT

mot:

iFVG + CISD utan SMT

Detta gör det möjligt att mäta om SMT faktiskt tillför edge eller främst fungerar som visuell confirmation.

Unknown SMT

SMT = UNKNOWN

ska analyseras separat från:

SMT = FALSE

Annars kan data availability skapa falska slutsatser.

Entry Confirmation Analysis

Performance ska kunna jämföras mellan:

- iFVG only
- CISD only
- iFVG + CISD
- iFVG + CISD + SMT

Detta ligger nära den canonical grade-modellen.

Liquidity Type

Atlas ska kunna analysera manipulation mot olika typer av liquidity.

Exempel:

- previous day high/low
- previous session
- previous 4H
- equal highs/lows
- intermediate liquidity

- FVG

Detta kan avslöja att vissa contexttyper är mer värdefulla.

FVG Timeframe

5m och 15m FVG ska analyseras separat.

Det ska även gå att analysera kombinationer där båda finns.

Attempt Performance

Attempt 1, 2 och 3 ska mätas var för sig.

Exempel:

Attempt 1

+0.32R expectancy

Attempt 2

+0.18R

Attempt 3

-0.11R

Ett sådant resultat skulle skapa en tydlig research hypothesis.

Det skulle inte automatiskt ändra Canonical v1.0.

Re-entry Contribution

Systemet ska kunna beräkna:

Hur stor del av total performance kommer från re-entrys?

Det ska även kunna jämföra strategin counterfactually med endast första attempt.

Break-Even Performance

Break-even-regeln ska analyseras separat.

Systemet ska kunna mäta:

- BE frequency
- loss reduction
- missed eventual winners
- net impact på expectancy
- net impact på drawdown

Target Analysis

Canonical target är första relevanta liquidity target som erbjuder minst 2R.

Analytics ska kunna mäta:

- planned R

- realized R
- MFE after exit
- distance to next liquidity

Detta hjälper framtida target research.

R:R Distribution

Alla trades behöver inte ha exakt samma initial R:R.

Systemet ska kunna visa fördelningen.

Exempel:

- 2.0–2.49R
- 2.5–2.99R
- 3.0R+

Det kan senare visa om högre theoretical R:R faktiskt leder till bättre realized expectancy.

MFE Analysis

MFE ska användas för att förstå hur långt vinnande och förlorande trades rör sig positivt.

Det kan exempelvis avslöja:

Många BE trades når +1.5R innan de återvänder.

Det är värdefull researchdata.

MAE Analysis

MAE ska användas för att förstå adverse movement.

Det kan exempelvis visa:

Vinnande Grade A-setups går sällan mer än -0.35R innan expansion.

Sådana findings ska valideras innan de får påverka SL eller entry.

Time in Trade

Analytics ska mäta:

- average duration
- median duration
- duration per outcome
- duration per session

Detta hjälper till att förstå strategi- och executionbehavior.

Time to MFE

Systemet kan senare mäta hur lång tid det tar innan MFE uppstår.

Detta kan ge bättre förståelse för trade progression.

Entry Time Analysis

Performance ska kunna analyseras inom sessionen.

Exempel:

- 02:00–03:00
- 03:00–04:00
- 04:00–05:00

och:

- 10:00–11:00
- 11:00–12:00

Detta är research, inte canonical filter.

Day-of-Week Analysis

När sample size är tillräcklig kan performance delas upp efter veckodag.

Atlas ska vara försiktig med att dra slutsatser eftersom segmenteringen snabbt skapar små samples.

Monthly och Seasonal Analysis

Samma princip gäller månad och kvartal.

Seasonality ska ses som exploratory analytics tills robust evidens finns.

Market Regime Analysis

När Market Regime Layer finns ska performance kunna analyseras per regime.

Exempel:

- trending
- ranging
- high volatility
- low volatility
- expansion
- compression

Det kan bli en av Atlas viktigaste framtida researchdimensioner.

Regime Stability

Det räcker inte att en strategy totalt har positiv expectancy.

Vi vill veta om den:

- fungerar i många regimes
- eller är extremt beroende av en särskild miljö

En smal edge kan fortfarande vara användbar.

Men vi måste veta att den är smal.

Volatility Analysis

Volatility state kan jämföras mot:

- outcome
- slippage
- stop distance
- trade duration

Detta kan hjälpa både strategy research och execution analysis.

Execution Quality Metrics

Prestationsanalysen ska ha ett separat executionområde.

Minst:

- average slippage
- median slippage
- worst slippage
- commissions
- average latency
- order rejection rate
- missed executions
- modification failures

Strategy vs Execution Performance

Systemet ska kunna estimeras:

Theoretical Strategy Result

och:

Actual Execution Result

Skillnaden är:

Execution Drag

Detta är centralt för att förstå var förbättringar ska göras.

Approval Latency

När human approval används ska systemet mäta hur mycket latency det tillför.

Det kan analyseras mot:

- fill degradation
- R:R degradation
- missed trades

Data Quality Segmentation

Trades från perioder med degraded data ska kunna analyseras separat.

De ska inte utan vidare blandas med healthy execution.

Technical Incident Impact

Analytics ska kunna visa hur technical incidents påverkade performance.

Exempel:

- missed winners
- unexpected losses
- increased slippage
- failed exits

Det gör teknisk stabilitet mätbar ekonomiskt.

Prop Firm Analytics

När Prop Firm Engine används ska systemet även mäta:

- maximum-loss headroom
- daily-loss usage
- breach count
- near-breach events
- consistency state
- challenge pass/fail
- days to target

Challenge Pass Rate

Genom simulationer kan Omnira uppskatta hur ofta strategin klarar ett specifikt program under definierade antaganden.

Detta ska alltid kopplas till exakt PropFirmProfile version.

Strategy Profitability vs Challenge Suitability

En lönsam strategi är inte automatiskt optimal för en prop firm.

Exempel:

En strategi kan ha stark long-term expectancy men:

- djupa korta drawdowns
- många losing streaks
- hög intraday variance

Det kan göra den olämplig för vissa drawdownmodeller.

Sharpe Ratio

Sharpe kan användas som kompletterande riskjusterat mått där datastrukturen gör det meningsfullt.

Det jämför i grunden excess return mot total variation.

För vår intraday strategy ska Sharpe inte behandlas som ett universellt huvudmått.

Resultatet påverkas av:

- vald tidsperiod
- return aggregation
- risk-free-rate-antagande
- icke-normal return distribution

Det ska därför användas tillsammans med mer direkt tradingstatistik.

Sortino Ratio

Sortino liknar Sharpe men fokuserar på downside variation.

Det kan vara mer intuitivt relevant för trading eftersom positiv variation inte behandlas som samma typ av risk.

Även Sortino ska ses som kompletterande.

Risk-Adjusted Metrics är inte Edge Proof

En hög Sharpe eller Sortino i ett litet backtest är inte bevis på robust edge.

Sample size och out-of-sample-resultat är fortfarande kritiska.

Calmar-Liknande Analysis

Systemet kan senare använda relationen mellan avkastning och maximum drawdown som ytterligare riskjusterad analys.

Exakt metric implementation ska standardiseras om den används.

Variance och Standard Deviation

Distributionen av trade returns ska kunna analyseras.

Hög variance betyder att resultatet är mer ojämnt.

Detta kan vara relevant för risk och challenge survival.

Distribution of R

Vi vill se hela outcome-distributionen.

Exempel:

- -1R
- BE
- +2R
- +2.5R
- +3R+

En histogramliknande analys kan visa om strategin har:

- många små outcomes
- få stora vinnare
- stark asymmetri

Skew

Return distribution kan vara positivt eller negativt skev.

Detta är relevant eftersom två strategier med samma expectancy kan ha mycket olika riskprofil.

Tail Risk

Systemet ska leta efter extrema negativa outcomes.

Canonical SL reducerar normal risk men actual fills kan ändå påverkas av exempelvis:

- gaps
- extreme volatility
- broker issues

Tail observations ska granskas separat.

Confidence Intervals

När sample size tillåter ska systemet kunna uppskatta osäkerhetsintervall kring viktiga statistics.

Exempel:

Estimated expectancy = $+0.28R$

men med ett intervall som visar att den verkliga långsiktiga expectancy är osäker.

Det är mer ärligt än att behandla sample average som exakt sanning.

Statistical Significance

Om Atlas hittar ett samband ska den inte endast fråga:

Är skillnaden positiv?

utan också:

Kan skillnaden rimligen vara slump?

Exakt statistisk metod beror på metric och datastruktur.

Systemet ska inte använda p-värden mekaniskt som enda besluts Kriterium.

Practical Significance

En skillnad kan vara statistiskt detekterbar men ekonomiskt ointressant.

Exempel:

En regel förbättrar expectancy med:

$+0.01R$

men:

- minskar trades kraftigt
- gör strategy mer komplex
- ökar parameter sensitivity

Det kanske inte är en meningsfull förbättring.

Economic Significance

Atlas ska därför också bedöma:

- net expectancy gain
- drawdown change
- execution impact
- opportunity cost
- complexity cost

Sample Size Tiers

Systemet bör senare definiera evidensnivåer.

Exempel konceptuellt:

- INSUFFICIENT
- PRELIMINARY
- MODERATE
- STRONG

Exakta thresholds ska inte låsas förrän vi analyserat strategy frequency och statistikmodellen.

Small Sample Warning

När sample är litet ska Atlas automatiskt uttrycka försiktighet.

Exempel:

A+ visar $+0.61R$ expectancy, men resultatet bygger endast på 18 trades och ska betraktas som preliminärt.

Segment Explosion

Om vi delar datasetet efter:

- session
- direction
- grade
- liquidity
- regime
- weekday

kan vi snabbt skapa hundratals små grupper.

Analytics ska aktivt motverka falska slutsatser från sådan segmentering.

Multiple Hypothesis Testing

Atlas Learning Layer kommer över tid kunna testa många idéer.

Ju fler hypoteser som testas, desto större risk att något ser bra ut av slump.

Research Registry ska därför registrera:

- number of hypotheses
- test history

- successful and failed findings

Negativa resultat ska bevaras.

Stability Over Time

Edge ska analyseras över tid.

Exempel:

2024

+0.34R

2025

+0.29R

2026

+0.31R

ser mer stabilt ut än:

2024

+1.10R

2025

-0.42R

2026

+0.18R

även om total average kanske är positiv.

Rolling Metrics

Systemet ska kunna beräkna rolling performance.

Exempel:

- rolling 20 trades
- rolling 50 trades
- rolling 100 trades

Det gör strategy degradation lättare att upptäcka.

Rolling Expectancy

Rolling expectancy kan visa om edge gradvis försvagas.

Det är viktigt för framtida live monitoring.

Rolling Drawdown

Även drawdown behavior kan följas över tid.

Strategy Drift

Strategy drift innebär att faktisk performance förändras från den historiskt validerade profilen.

Exempel:

- expectancy sjunker
- slippage ökar
- setup distribution förändras
- regime distribution förändras

Atlas ska kunna upptäcka sådan drift.

Performance Degradation Alert

När performance avviker kraftigt från förväntad range kan systemet skapa:

```
PERFORMANCE_REVIEW_REQUIRED
```

Det ska inte automatiskt innebära att strategin stängs om ingen separat policy säger det.

Safety Trigger vs Research Alert

Det är viktigt att skilja:

Hard Safety Trigger

från:

Analytics Alert

Analytics får säga:

Performance har försämrats.

Risk Engine bestämmer om trading måste stoppas enligt explicit safety policy.

Out-of-Sample Performance

Out-of-sample-statistik ska visas separat.

Det är en av de starkaste signalerna på om research findings generaliserar.

Backtest vs Forward

Analytics ska jämföra:

- backtest
- forward
- demo
- live

Om results divergerar ska Atlas försöka identifiera orsaken.

Backtest-to-Live Decay

Det är normalt att actual performance är något sämre än idealiserat backtest.

Systemet ska mäta denna skillnad.

Exempel:

Backtest +0.34R

Forward +0.27R

Live +0.21R

Denna decay kan bli en viktig forecasting input.

Benchmarking Candidate Versions

Candidate Strategy v1.1 ska jämföras mot canonical v1.0.

Det ska ske med samma relevanta:

- dataset
- costs
- risk profile
- execution assumptions

Jämförelsen ska vara så rättvis som möjligt.

Candidate Scorecard

En candidate ska inte bedömas på total profit ensam.

Scorecard kan innehålla:

- expectancy
- PF
- max DD
- losing streak
- stability
- OOS performance
- forward performance
- execution sensitivity
- sample size

Robustness

En strategyförändring ska betraktas starkare om förbättringen syns:

- över flera perioder
- i OOS
- i forward
- över flera närliggande parameter-värden

än om den endast fungerar i ett specifikt historiskt segment.

Sensitivity Analysis

Systemet ska kunna stressa viktiga antaganden.

Exempel:

- högre slippage
- högre commissions
- delayed entry
- small data errors
- different parameter values

Edge som överlever stress är mer robust.

Monte Carlo

När trade sample är tillräckligt stort kan Monte Carlo och resampling användas för att uppskatta möjliga future paths.

Det kan ge distributioner för:

- drawdown
- final result
- losing streak
- challenge failure

Detta är betydligt mer informativt än en enda historical equity curve.

Risk of Ruin

På längre sikt kan systemet uppskatta risk-of-ruin-liknande metrics under givna riskmodeller.

För prop firms kan motsvarande fråga vara:

Hur stor simulerad sannolikhet finns att maximum loss brechas innan target nås?

Detta kan bli mycket värdefullt.

Probability är modellberoende

Sådana sannolikheter ska aldrig presenteras som säker framtid.

De beror på:

- historisk distribution
- stationarity-antaganden
- sample size
- simulation method

Atlas ska alltid förklara detta.

Atlas Performance Analyst

Atlas ska kunna fungera som en analytiker ovanpå dessa metrics.

Exempel:

Canonical v1.0 är fortsatt positiv totalt, men rolling 50-trade expectancy har sjunkit från +0.29R till +0.08R. Förändringen sammanfaller främst med Grade C och tredje attempts. A/A+ är fortfarande positiva. Sample är ännu för litet för en canonical ändring men motiverar en research hypothesis.

Detta är den typ av analys vi vill ha.

Atlas ska inte jaga siffror

AI:n ska inte optimera mot en enda metric såsom:

- maximum profit
- highest Sharpe
- highest win rate

Målet är robust riskjusterad edge.

Multi-Metric Evaluation

En strategyförändring ska därför bedömas på flera dimensioner samtidigt.

Exempel:

Candidate A

Högre profit men högre drawdown.

Candidate B

Lite lägre profit men lägre drawdown och bättre stability.

Vilken som är bäst beror på systemmål och prop constraints.

Findings

Analytics kan skapa strukturerade findings.

Exempel:

FIND-0042

Observation: Attempt 3 visar negativ expectancy.

Sample: 287 third-attempt observations.

Strategy: v1.0

Result:

-0.14R expectancy.

Status: Research finding.

Hypothesis Generation

Atlas kan skapa:

HYP-0042

Testa candidate där max attempts per thesis = 2.

Det är ett experiment.

Canonical v1.0 ändras inte.

Findings måste kunna motbevisas

Varje finding ska kunna gå vidare till:

- independent dataset
- OOS
- forward test

Om resultatet inte replikerar ska hypotesen kunna avslås.

Research Memory

Både bekräftade och avslagna findings ska bevaras.

Det hindrar Atlas från att testa samma dåliga idé om och om igen utan att minnas tidigare resultat.

Self-Improvement Scorecard

Atlas Trading Learning & Improvement Layer ska utvärdera sina egna rekommendationer.

Vi ska kunna mäta:

Hur många candidateförslag blev faktiskt bättre OOS?

Om Atlas ständigt producerar överfit candidates är Learning Layer själv dåligt kalibrerat.

Prediction Calibration

Om Atlas senare ger probabilistiska eller confidence-baserade predictions ska de kunna kalibreras mot verkligt outcome.

Confidence ska inte få vara dekorativt.

Performance Dashboard

Omnira Trading ska senare kunna ha en central analyticsvy.

Exempel:

Strategy: Omnira Liquidity Manipulation v1.0

Environment: Forward Demo

Trades: 524

Expectancy: +0.26R

Profit Factor: 1.48

Win Rate: 43.7%

Average Win: +2.21R

Average Loss: -0.98R

Max Drawdown: -8.6R

Longest Losing Streak: 7

Status: VALIDATION RUNNING

Segment Dashboard

Användaren ska kunna välja exempelvis:

New York → A+ → Long → Attempt 1

och se relevant performance.

Systemet ska samtidigt visa sample size tydligt.

Equity Curve

Analytics UI ska kunna visa:

- gross curve
- net curve
- drawdown curve

Kurvor ska kunna filtreras efter version och environment.

R Curve

En R-baserad cumulative curve ska också finnas.

Det gör strategy behavior lättare att jämföra mellan olika account sizes.

Prop Firm Dashboard

Prop Mode ska kunna visa:

- current headroom
- simulated challenge survival
- pass/fail history
- average days to target
- common breach reasons

Atlas Daily Brief

Atlas ska senare kunna sammanfatta aktuell trading performance.

Exempel:

De senaste 50 trades har expectancy +0.19R, lägre än långtidsbaslinjen +0.27R. Försämringen kommer huvudsakligen från Grade C. Execution metrics är stabila och slippage ligger inom normal range.

Weekly Performance Review

En veckoreview kan innehålla:

- setups
- trades
- R
- expectancy
- execution quality
- risk usage

- prop state
- anomalies
- research findings

Monthly Strategy Review

Månadsreview ska fokusera mer på:

- rolling statistics
- regime shifts
- segment stability
- candidate hypotheses
- forward vs historical performance

Promotion Gates

Performance metrics ska senare användas för att definiera konkreta promotion gates.

Exempel:

Backtest → Forward

kräver historisk edge och robustness.

Forward → Demo Auto

kräver både strategy och technical performance.

Demo Auto → Controlled Live

kräver ännu striktare criteria.

Exakta thresholds låses senare.

Ingen godtycklig Performance Gate

Vi ska inte välja thresholds enbart därför att de låter bra.

Exempel:

Win Rate > 60%

är meningslöst om strategin inte är designad för sådan win rate.

Gates ska utgå från strategy behavior och riskkrav.

Performance Stop Policy

När live väl finns kan en separat policy senare definiera när kraftig degradation kräver:

- review
- risk reduction
- automation suspension

Detta ska vara explicit policy.

Atlas får inte själv ändra risk live bara för att en metric försämrats.

Datakvalitet före slutsats

Ingen analyticsmodell är bättre än datan den använder.

Innan Atlas producerar en stark slutsats ska den kunna bedöma:

- completeness
- quality
- sample size
- version consistency

Inga fabricerade metrics

Om data saknas ska Atlas säga:

Cannot calculate

inte gissa.

Det gäller särskilt:

- fees
- slippage
- MFE
- MAE
- historical sample

Analytics Reproducibility

Varje större rapport ska kunna reproduceras.

Den ska referera till:

- query/filter
- dataset
- strategy version
- date range
- analytics version

Analytics Versioning

Om en metric senare beräknas annorlunda ska analytics logic versionshanteras.

Historiska rapporter ska kunna förstås enligt den metod som användes då.

Source of Truth

Raw Journal och Performance Records är source of truth.

Atlas-generated prose är tolkning.

Om Atlas summerar fel ska originalmetrics fortfarande finnas kvar.

Den centrala prestationsprincipen

Omnira ska aldrig fråga endast:

Tjänade strategin pengar?

Den ska fråga:

Tjänade den pengar på ett robust, reproducerbart och riskmässigt acceptabelt sätt som sannolikt representerar edge snarare än slump?

Det är en mycket högre standard.

Det är också den standard som krävs innan systemet får större autonomi.

Kapitelstatus

Kapitel: 14 – Prestationsanalys

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Primärt performanceformat: R + net dollar

Core metrics: Expectancy, Profit Factor, Drawdown, Win/Loss/BE, MFE/MAE, streaks

Risk-adjusted metrics: Sharpe/Sortino som kompletterande analys där relevant

Segmentation: Session, grade, direction, attempt, liquidity, FVG, SMT, regime

Sample size: Ska alltid exponeras

Out-of-sample: Central evidens

Rolling performance: Planerad

Candidate comparison: Versionsstyrd

Self-improvement: Analytics skapar findings och hypotheses

Automatisk canonical ändring: Förbjuden

Prestationsanalysen ska omvandla tradinghistorik till mätbar kunskap utan att förväxla historisk korrelation med bevisad framtida edge.

KAPITEL 15

Felmoder och failure scenarios

Omnira Trading System ska byggas utifrån antagandet att saker förr eller senare kommer att gå fel. Inte kanske.

Förr eller senare.

Det kan vara:

- felaktig marknadsdata
- avbrott i nätverk
- providern som tappar anslutningen
- fel konto aktivt
- stale account state
- duplicate execution requests
- felaktig symbol mapping
- broker rejection
- stop loss som inte aktiveras
- news-data som saknas
- Prop Firm Rules Engine som inte kan fastställa aktuell state
- Atlas som tolkar något fel
- runner crash
- Omnira outage
- restart mitt i en öppen position

Målet med failure handling är därför inte att skapa ett system där inget kan gå fel.

Målet är:

När något går fel ska systemet falla på ett kontrollerat, synligt och så säkert sätt som möjligt.

Den grundläggande säkerhetsprincipen är:

UNKNOWN ≠ SAFE

Om systemet inte vet om det är säkert att fortsätta ska ny trading stoppas.

Failure handling är en del av arkitekturen

Failure scenarios får inte hanteras som något vi lägger till efter att tradingbotten fungerar.

De ska byggas in från början.

Varje viktig komponent ska kunna svara på:

- Vad händer om jag inte får data?
- Vad händer om datan motsäger tidigare state?
- Vad händer om nästa komponent inte svarar?
- Vad händer om samma request kommer två gånger?

- Vad händer om processen dör efter halva operationen?
- Hur återställer vi korrekt state?

Fail Closed

För exekveringskritiska beslut ska standardprincipen vara:

FAIL CLOSED

Exempel:

Om Risk Engine inte kan svara:

UNKNOWN

ska det inte tolkas som:

ALLOW

Det ska tolkas som:

NO NEW TRADE

Samma princip gäller bland annat:

- Prop Firm Rules
- market data health
- account state
- execution state
- account identity
- kill switch state

Failure Severity

Alla problem är inte lika allvarliga.

Systemet bör kunna klassificera incidents exempelvis som:

INFO

Ingen direkt tradingpåverkan.

WARNING

Degraded state men ingen omedelbar kapitalrisk.

BLOCKING

Ny execution förbjuden.

CRITICAL

Aktiv position eller account safety kan vara hotad.

EMERGENCY

Omedelbar skyddsåtgärd kan krävas enligt explicit policy.

Analysis Failure vs Execution Failure

Systemet ska skilja mellan problem som påverkar analys och problem som påverkar faktisk trading.

Exempel:

Atlas AI är offline.

Strategy Engine kan fortfarande fungera.

Detta kan vara:

```
AI_ANALYSIS_UNAVAILABLE
```

Det behöver inte automatiskt betyda att broker-native SL på en redan öppen position påverkas.

Däremot:

```
ACCOUNT_STATE_UNKNOWN
```

är execution-critical.

Market Data Stale

Scenario:

Realtime market data slutar uppdateras.

Systemet kanske fortfarande har senaste candle.

Men den är gammal.

Expected behavior:

```
MARKET_DATA_STALE  
→
```

ingen ny executable Strategy Signal.

Atlas Market View ska visa att data är stale.

Systemet får inte fortsätta som om priset fortfarande befinner sig på senaste observerade nivå.

Missing Candles

Scenario:

1m-serien innehåller:

10:01

10:02

10:04

men saknar:

10:03

Expected:

INCOMPLETE_SERIES

Strategy Engine ska inte fortsätta entry logic på antagandet att datan är komplett.

Duplicate Market Data

Scenario:

Samma 1m candle levereras två gånger.

Without protection kan systemet skapa dubbla events.

Market Data Layer ska deduplicera.

Samma candle ska inte kunna skapa:

- två manipulation events
- två Strategy Signals
- två Trade Proposals

Out-of-Order Data

Scenario:

10:04

anländer före:

10:03

Systemet ska inte processa dem blint i arrival order.

Det ska antingen:

- reorder inom definierad buffer
- eller markera serien invalid

beroende på implementation.

Provider Disagreement

Scenario:

Primary provider och fallback provider rapporterar olika priser eller candles.

Systemet får inte automatiskt välja det värde som bäst passar setupen.

Expected:

DATA_SOURCE_CONFLICT

och vid execution-critical discrepancy:

NO NEW TRADE

Provider Switch Mid-Setup

Scenario:

En setup startar på provider A.

Provider A faller bort.

Provider B finns tillgänglig.

Systemet ska inte tyst fortsätta setupen som om datan vore identisk.

Ett source transition event ska skapas.

Om consistency inte kan verifieras ska setupen blockeras eller invalidated enligt policy.

Clock Drift

Scenario:

Windows-runners systemklocka är fel.

Det kan påverka:

- session windows
- proposal expiry
- news exits
- latency
- daily resets

Clock drift ska därför monitoreras.

För stor drift ska bli blockerande.

Timezone Failure

Scenario:

Systemet tolkar New York som permanent UTC-4 trots daylight saving.

Det kan öppna strategy window en timme fel.

Detta är inte ett kosmetiskt fel.

Det är ett strategy integrity failure.

Time conversion ska därför testas runt DST transitions.

News Data Missing

Scenario:

Economic calendar provider är unavailable.

Systemet kan inte avgöra om high-impact USD event finns.

Expected:

```
NEWS_STATE_UNKNOWN
```

I execution-enabled mode:

NO NEW TRADE**News Data Stale**

Att kalendern laddades för flera dagar sedan är inte samma sak som att den är aktuell.

News-data ska ha freshness state.

Stale calendar i execution mode ska blockera nya entries där news-regeln kräver korrekt information.

News Time Changed

Ekonomiska events kan i vissa fall uppdateras.

Systemet måste därför kunna hantera att scheduled timestamp förändras.

Aktuell policy ska alltid använda senaste verifierade event state.

Historisk audit ska fortfarande visa vilken kalenderstate som användes tidigare.

Atlas Hallucination

Scenario:

Strukturerad data säger:

SMT = UNKNOWN

men Atlas säger:

SMT är bekräftad.

Expected behavior:

- canonical structured state ändras inte
- AI discrepancy loggas
- UI ska prioritera structured source of truth
- incident kan markeras som AI quality issue

AI-text får aldrig skriva över Strategy Engine state.

Atlas Unavailable

Scenario:

AI-tjänsten är offline.

Systemet ska tydligt skilja:

AI_UNAVAILABLE

från:

TRADING_SYSTEM_UNAVAILABLE

Beroende på operation mode kan deterministic analysis eventuellt fortsätta.

Men ingen komponent får låtsas att AI Analysis genomförts.

Strategy Engine Crash

Scenario:

Strategy Engine kraschar mitt i en session.

Ny Strategy Signal ska stoppas.

När motorn startar om ska den återuppbygga state från:

- market data
- persisted setup state
- events

innan den får fortsätta.

Strategy State Corruption

Scenario:

Systemet kan inte avgöra om setupen står i:

```
WAIT_FOR_CONFIRMATION
```

eller:

```
SIGNAL_CREATED
```

Expected:

```
STRATEGY_STATE_UNKNOWN
```

Ingen ny execution från denna thesis.

State ska reconcileras eller setupen invalidated på ett kontrollerat sätt.

Duplicate Strategy Signal

Scenario:

En bug producerar samma Strategy Signal två gånger.

Signal identity ska göra dessa detekterbara.

Duplicate signal får inte skapa flera exekverbara proposals.

Risk Engine Unavailable

Scenario:

Strategy PASS.

Risk Engine svarar inte.

Final:

```
NO TRADE
```

Risk är obligatorisk gate.

Risk State Stale

Scenario:

Risk Engine har account snapshot från flera minuter sedan.

Under tiden kan en annan position ha öppnats eller P/L förändrats.

Risk evaluation får inte göras på arbiträrt gammal state.

Critical account state behöver definierad freshness.

Daily Loss State Mismatch

Scenario:

Omnira tror:

realized daily loss = \$150

men providerns history visar:

\$300

Expected:

```
RISK_RECONCILIATION_FAILURE
```

Ny execution blockeras.

Daily Stop Breach

Canonical intern regel:

\$450 realized daily loss

När gränsen nås:

- inga nya trades
- eventuell öppen position stängs direkt enligt riskpolicyn
- state låses fram till daily reset
- incident och risk event loggas

Daily reset får inte kunna kringgås genom runner restart.

Attempt Counter Lost

Scenario:

Process restart gör att systemet glömmer att två attempts redan tagits på samma 4H-thesis.

Attempt state ska därför vara persistent.

Efter restart ska:

```
attempts_used = 2
```

fortfarande gälla.

Prop Firm Engine Unavailable

Scenario:

Internal Risk = PASS.

Prop Engine svarar inte.

Final:

NO TRADE

Prop firm-compliance är obligatorisk på account där sådan profil är aktiv.

Prop Rule State Unknown

Scenario:

Systemet vet inte aktuell trailing drawdown floor.

Expected:

PROP_STATE_UNKNOWN

Ny execution blockeras.

Det är bättre att missa en trade än att riskera challenge breach på okänd state.

Outdated Prop Rules

Scenario:

Systemets ruleset kan vara gammalt och provider har ändrat programmet.

Om profilens freshness-policy kräver review ska state kunna bli:

RULESET_REVIEW_REQUIRED

Live automation ska inte fortsätta blint med potentiellt utdaterade hard rules.

Prop Firm Breach

Scenario:

En regel har redan brutits.

Systemet ska:

- markera account PROP_BLOCKED
- stoppa nya trades
- logga breach
- visa exakt vilken regel som bröts

Systemet får inte försöka trade sig ur ett compliance breach.

Wrong Account Active

Scenario:

Runnern förväntar sig ett demo-account.

Providern har ett live-account aktivt.

Expected:

ACCOUNT_MISMATCH

och:

NO EXECUTION

Detta är ett av de viktigaste safety scenarios.

Wrong Environment

Scenario:

Execution Intent säger:

DEMO

men runnern är konfigurerad mot:

LIVE

Expected:

ENVIRONMENT_MISMATCH

Request ska nekas.

Wrong Symbol Mapping

Scenario:

Canonical:

MNQ

mappas av misstag till fel futureskontrakt eller annat symbolnamn.

Runner ska verifiera:

- symbol
- contract metadata
- instrument mapping
- allowlist

innan execution.

Osäker mapping:

NO TRADE

Wrong Quantity

Scenario:

Risk Engine godkänner:

1 MNQ

men execution request innehåller:

2 MNQ

Runnern ska jämföra quantity mot approved intent.

Mismatch:

EXECUTION_INTEGRITY_FAILURE

Ingen order.

Position Size Rounding Error

Om broker volume step kräver avrundning ska systemet aldrig avrunda upp om det ökar risk över godkänd nivå.

Kan quantity inte representeras säkert:

DENY

Proposal Expired

Scenario:

Trade Proposal godkändes men execution sker för sent.

Expected:

PROPOSAL_EXPIRED

Ingen order skickas.

Ny evaluation krävs.

Price Moved Before Execution

Scenario:

Proposal skapades med giltigt 2.4R.

När ordern ska skickas återstår bara 1.7R.

Pre-execution revalidation ska stoppa traden.

Risk Changed Before Execution

Scenario:

En annan position har just stängts med loss och daily risk headroom har minskat.

Pre-execution riskkontroll ska få sista aktuella state.

Tidigare Risk PASS är inte ett permanent tillstånd.

Prop State Changed Before Execution

Samma princip gäller Prop Firm Engine.

Aktuell headroom ska verifieras direkt före execution.

Duplicate Execution Request

Scenario:

Nätverket orsakar retry.

Samma Execution Intent kommer två gånger.

Expected:

exakt en broker-order

Runner ska svara:

ALREADY_PROCESSED

för duplicate idempotency key.

Runner Crash Before Submission

Scenario:

Runner tar emot intent men kraschar innan broker request skickas.

Efter restart ska reconciliation visa:

- ingen order
- intent ej exekverad

Systemet kan därefter besluta om proposal fortfarande är giltig eller expired.

Ingen automatisk gissning.

Runner Crash After Submission

Detta är betydligt farligare.

Scenario:

Runner skickar broker request.

Providern tar emot den.

Runner kraschar innan local confirmation sparas.

Efter restart vet Omnira inte om ordern exekverades.

Expected:

UNKNOWN_EXECUTION_STATE
→ reconciliation mot providern

innan någon retry får ske.

Network Lost Before Broker Response

Samma princip.

Systemet får aldrig anta:

no response = no order

Det måste kontrollera actual broker state.

Broker Rejection

Scenario:

Ordern nekas.

Exempel:

- invalid volume
- invalid stops
- market closed
- insufficient resources

Systemet ska:

- registrera raw response
- skapa strukturerad reason
- inte anta att position öppnats
- reconcila state där relevant

Partial Fill

Scenario:

Order quantity fylls endast delvis.

Position state ska baseras på actual fills.

Risk och SL/TP måste utvärderas mot den verkliga positionen.

Systemet får inte låtsas att full quantity exekverats.

Unexpected Fill Price

Scenario:

Slippage blir större än planerat.

Efter actual fill måste systemet beräkna verklig:

- risk
- R:R
- exposure

Om fill skapar säkerhetsproblem ska explicit post-fill protection policy användas.

Det får inte improviseras av AI.

Stop Loss Missing

Ett av systemets mest kritiska scenarios.

Scenario:

Position öppnas.

Förväntad broker-native SL finns inte.

Expected:

CRITICAL_EXECUTION_INCIDENT

Systemet ska:

- upptäcka mismatch
- försöka applicera skydd enligt explicit recovery policy
- blockera nya trades
- varna omedelbart
- eskalera om skydd inte kan verifieras

En position får aldrig visas som skyddad när broker state säger motsatsen.

Wrong Stop Loss

Samma princip gäller om SL finns men ligger på fel nivå.

EXPECTED_SL != BROKER_SL

ska behandlas som execution discrepancy.

Take Profit Missing

TP är mindre kritisk än SL ur kapitalkontrollperspektiv men fortfarande ett executionfel när plan kräver broker-native TP

Mismatch ska loggas och hanteras.

Break-Even Modification Failure

Scenario:

Canonical BE-trigger inträffar.

Omnira skickar modification.

Broker nekar.

Systemet ska visa:

BE_NOT_CONFIRMED

Inte:

POSITION_AT_BE

Actual broker state vinner.

News Exit Failure

Scenario:

T-15m inträffar.

Systemet försöker stänga.

Broker request misslyckas.

Detta är ett critical management incident.

Systemet ska:

- retrya endast enligt säker policy
- monitorera actual position
- eskalera
- journalföra all action

Time Exit Failure

Samma princip för New York 4h exit.

Provider Client Closed

Scenario:

Windows lever.

Runner lever.

Men providern är stängt.

Expected:

```
PROVIDER_DISCONNECTED
```

Ny execution stoppas.

Runner ska inte betraktas som healthy execution host.

Provider Connected but Broker Offline

Terminalprocessen kan vara igång utan fungerande broker connection.

Systemet ska skilja dessa states.

Broker Market Closed

Strategy Engine ska normalt redan förstå relevant session, men broker kan ändå neka trading.

Broker state är sista praktiska execution boundary.

Ingen blind retry.

Windows Sleep

Scenario:

Windows-riggen går i sleep mitt under trading window.

Detta är ett deployment configuration failure.

Riggens health checks ska upptäcka olämplig power state.

Produktionsmiljön ska ha sleep avstängt.

Windows Restart

Efter OS-restart ska autostart kunna starta:

- runner
- providern där konfigurationen tillåter

Men trading får inte börja förrän startup reconciliation är klar.

Disk Full

Scenario:

Runner kan inte skriva journal eller local state.

Detta kan påverka audit och idempotency.

Disk capacity ska monitoreras.

Kritisk diskbrist kan behöva blockera ny execution.

Database Unavailable

Scenario:

Omnira kan inte lagra Trade Proposal eller audit events.

Ett system som inte kan skapa tillförlitlig audit ska inte fortsätta live execution som vanligt.

Expected behavior ska vara fail closed för nya trades när critical persistence saknas.

Event Bus Failure

Om realtime events inte når UI är detta inte automatiskt samma sak som att backend state saknas.

Systemet ska skilja:

UI_UPDATE_FAILURE

från:

TRADING_STATE_FAILURE

Men om downstream safety components faktiskt inte får events krävs blockering.

UI Crash

UI är inte source of truth.

Om browsern kraschar ska:

- broker-native protection finnas kvar
- backend state finnas kvar
- runner fortsätta enligt policy

Men användaren kan förlora visibility.

Det ska tydligt återställas när UI reconnectar.

UI Shows Stale State

UI ska visa freshness.

En gammal chart eller riskstatus ska inte se identisk ut med live state.

User Double Clicks Approve

Scenario:

Användaren trycker approval två gånger.

Approval/idempotency-logik ska göra att detta inte kan skapa två executions.

Manual External Trade

Scenario:

Användaren öppnar MNQ manuellt i providern.

Omnira expected:

0 positions

Provider observed:

1 position

Expected:

- detect manual/unknown origin
- account risk uppdateras
- ny Omnira execution blockeras enligt position-limit/reconciliation policy

Systemet ska inte ignorera positionen för att den skapades utanför Omnira.

Manual Modification

Scenario:

Användaren flyttar SL direkt i providern.

Omnira ska upptäcka att actual broker state har förändrats.

Detta ska journalföras som:

```
MANUAL_EXTERNAL_MODIFICATION
```

Systemet får inte fortsätta visa gammal SL som aktuell.

Manual Close

Samma sak om användaren stänger positionen manuellt.

Omnira ska reconcila och journalföra origin för exit.

Unknown Position

Om origin inte kan fastställas:

```
UNKNOWN_POSITION
```

Ny trading blockeras tills state hanterats.

Orphan Order

Broker visar pending/active order som Omnira inte känner igen.

Samma princip:

```
RECONCILIATION_REQUIRED
```

Kill Switch Activation

När kill switch aktiveras ska systemet tydligt veta scope.

Exempel:

```
GLOBAL  
ACCOUNT  
STRATEGY  
INSTRUMENT  
RUNNER
```

Ny relevant execution ska stoppas omedelbart.

Kill Switch State Unknown

Om systemet inte kan läsa aktuell kill-switch state får det inte anta:

```
OFF
```

Unknown ska blockera execution.

Kill Switch Does Not Equal Emergency Close

Vanlig kill switch stoppar nya trades.

Den ska inte automatiskt stänga alla öppna positions om inte explicit emergency policy säger det.

Detta minskar risken för destruktiva automationseffekter.

Emergency Close Failure

Om emergency close används men broker inte bekräftar ska systemet fortsätta betrakta positionen som öppen.

Det ska aldrig visa:

```
CLOSED
```

förrän broker state verifierat det.

Omnira Backend Outage

Scenario:

Omniras centrala backend går ner.

Runnern ska inte börja fatta egna strategybeslut.

Expected:

- inga nya trades
- öppna broker-native protections kvar
- local runner state bevaras
- reconciliation efter återkomst

Runner Isolated from Omnira

Scenario:

Runner har kontakt med providern men inte Omnira.

Runnern ska som standard:

inte skapa nya trades

Detta är en viktig authority boundary.

Full Internet Outage

Om Windows-riggen tappar internet:

- broker connection kan falla
- Omnira connection faller
- active management kan påverkas

Broker-native SL/TP är därför centrala.

När internet återkommer krävs reconciliation.

Power Failure

Scenario:

Windows-datorn stängs av helt.

Local software kan inte göra något.

Detta är ytterligare skäl till:

- broker-native protection
- framtida VPS
- startup reconciliation

Operational uptime ska mätas.

VPS Failure

Att senare flytta till VPS eliminerar inte failure.

Samma recoverymodell ska fungera även där.

Infrastructure host får bytas utan att safety assumptions förändras.

Broker Outage

Broker kan vara unavailable även när internet fungerar.

Om broker inte kan ta emot orders:

- inga nya trades
- position state kan bli osäkert
- systemet ska monitorera reconnect
- reconciliation krävs

Exchange Halt

Marknaden kan stoppas eller hamna i ovanliga trading conditions.

Strategy Engine får inte anta normal execution.

Market/broker state ska kunna markera abnormal conditions.

Extreme Slippage

Canonical risk gäller planned risk.

I extrema marknader kan actual loss bli större.

Systemet ska därför mäta:

risk overrun

Exempel:

Planned loss = -1R

Actual loss = -1.34R

Detta är viktigt för tail-risk-analys.

Gap Through Stop

Broker-native SL garanterar inte alltid exakt stop price.

Gap risk är residual market risk.

Det ska dokumenteras, inte låtsas bort.

Commission Changes

Om broker ändrar fees kan actual expectancy påverkas.

Execution/analytics ska upptäcka förändrad cost profile.

Detta är inte nödvändigtvis ett safety failure men kan skapa:

PERFORMANCE_REVIEW_REQUIRED

Contract Rollover Error

Scenario:

Omnira fortsätter handla gammalt futureskontrakt.

Systemet ska använda explicit contract mapping och rollover checks.

Mismatch ska blockera execution.

Liquidity Detection Bug

Scenario:

Strategy Engine identifierar liquidity fel på grund av kodbugg.

Golden tests och regression suite ska försöka upptäcka detta före deployment.

Om detection logic ändras utan strategy/detection version update ska CI kunna falla.

iFVG/CISD Detection Error

Eftersom dessa patterns är entrykritiska måste deras exakta detection rules implementeras deterministiskt.

En visuell eller AI-baserad improvisation får inte vara production source of truth.

Fel i denna logik är strategy integrity incidents.

SMT Comparison Failure

Scenario:

NQ-data är live men ES-feed är stale.

Systemet får inte ge:

SMT = FALSE

utan:

SMT = UNKNOWN

A+ får inte delas ut på saknad comparison data.

Detection Version Mismatch

Scenario:

Live Strategy Engine använder detection v1.1 men backtest-resultaten byggdes med v1.0 utan att detta framgår.

Detta är ett reproducibility failure.

Detection versions ska vara explicit bundna till StrategyVersion/test runs.

Configuration Drift

Scenario:

Två runners eller environments använder olika risk config trots att båda säger v1.0.

Config ska vara immutable/versioned så att samma version betyder samma regler.

Secret Exposure

Scenario:

Providercredentials råkar loggas.

Detta är ett security incident.

Secrets ska:

- inte skrivas i logs
- inte skickas till AI
- inte exponeras i frontend
- inte committas i Git

Incident ska behandlas separat från trading failure.

Unauthorized Execution Request

Scenario:

En falsk eller obehörig request når runnern.

Runnern ska verifiera:

- authentication
- authorization
- allowed environment
- account
- intent identity

Unauthorized request:

REJECT

och security incident.

Replay Attack

En gammal legitim execution request försöker skickas igen.

Idempotency + expiry ska förhindra att den exekveras.

Corrupted Execution Intent

Scenario:

Payload är ofullständig eller har ogiltig signatur/integritet.

Runnern ska neka hela requesten.

Ingen partial execution.

Version Incompatibility

Scenario:

Omnira backend skickar en execution intent-version som runnern inte förstår.

Expected:

UNSUPPORTED_INTENT_VERSION

Ingen trade.

Deployment During Open Position

Koddeploy medan position är öppen är ett särskilt riskområde.

Production deployment policy ska senare definiera om och hur detta tillåts.

Systemet ska åtminstone kunna:

- bevara position state
- verifiera runner compatibility
- reconcile efter restart

Migration Failure

Vid databas- eller schemaändringar får trade-critical data inte förloras.

Migrationer ska kunna verifieras och rollbackas där möjligt.

Journal Failure

Om systemet exekverar trade men inte kan journalföra den korrekt är detta ett audit incident.

Execution state ska rekonstrueras från broker history.

Analytics Failure

Om analytics pipeline är trasig betyder det inte automatiskt att Risk Engine är trasig.

Dessa concerns ska hållas separata.

Systemet ska kunna trade säkert utan att performance dashboard fungerar, om operation policy tillåter det.

Learning Layer Failure

Atlas Trading Learning & Improvement Layer är aldrig i hard execution path.

Om den går ner ska canonical strategy inte förändras.

Detta är en av fördelarna med separationen.

Bad Self-Improvement Hypothesis

Scenario:

Atlas föreslår en strategiändring som ser fantastisk ut på historiken men är överfit.

Det får ingen productioneffekt förrän candidate passerat:

- backtest
- OOS
- forward test
- review

Governance är failure protection mot dåliga förbättringar.

Unauthorized Self-Modification

Scenario:

En AI-komponent försöker skriva om canonical Strategy/Risk/Prop rules direkt.

Arkitekturen ska inte ge den sådan behörighet.

Detta ska betraktas som policy/security violation.

Performance Collapse

Scenario:

Live expectancy faller kraftigt.

Analytics ska kunna skapa:

```
PERFORMANCE_REVIEW_REQUIRED
```

En separat performance stop policy kan senare avgöra om automation ska pausas.

Atlas får inte improvisera förändringar.

Strategy Edge Disappears

Det är möjligt att strategin helt enkelt slutar fungera.

Systemets uppgift är inte att försvara strategin emotionellt.

Om:

- OOS
- forward
- live

visar bestående negativ expectancy ska systemet kunna rekommendera suspension och research.

Canonical status är inte samma sak som evig lönsamhet.

Correlated Failures

Flera failures kan ske samtidigt.

Exempel:

- high-impact news
- extreme volatility
- market data degradation
- broker latency

Systemet ska inte anta att failures alltid sker isolerat.

Stress testing ska inkludera kombinationer.

Failure Cascades

Ett litet fel kan skapa större problem.

Exempel:

stale market data

- fel Strategy Signal
- fel Trade Proposal
- fel execution

Data Quality Gate ska stoppa kedjan tidigt.

Systemarkitekturen ska försöka fånga problem så nära källan som möjligt.

Blast Radius

Varje komponent ska ha minimal authority.

Detta begränsar hur mycket skada ett fel kan göra.

Exempel:

Strategy Engine kan skapa signal.

Den kan inte skapa broker-order.

Atlas kan skapa analys.

Den kan inte ändra risk.

Execution Runner kan skicka godkänd order.

Den kan inte skapa strategy thesis.

Graceful Degradation

Vissa failures kan tillåta reducerad funktion.

Exempel:

AI unavailable:

Strategy visualisering kan fortfarande fungera.

Analytics unavailable:

Riskkontroll kan fortfarande fungera.

Men execution-critical dependencies ska inte degraderas på ett sätt som gör safety osäker.

Degraded Mode

Systemet kan senare ha explicit:

DEGRADED

state.

Det ska vara tydligt vilka funktioner som fortfarande är tillåtna.

Exempel:

Analysis available

Execution blocked

Det är bättre än ett binärt "online/offline".

Recovery är inte samma som Restart

Att starta om en process betyder inte att problemet är löst.

Efter failure krävs ofta:

- state verification
- broker sync
- data sync
- reconciliation

innan component blir READY.

Recovery State Machine

Ett generellt mönster är:

```
FAILURE_DETECTED  
→  
BLOCK  
→
```

RECONNECT / RESTART

```
→  
RESYNC  
→  
RECONCILE  
→  
VERIFY  
→  
READY
```

Inte:

```
FAILURE  
→  
RESTART  
→  
TRADE
```

Reconciliation som central recoverymekanism

Broker state är avgörande för verklig exposure.

Efter execution-relaterade fel ska systemet läsa actual:

- account
- orders
- positions
- deals

innan det återgår till normal drift.

Incident Lifecycle

Varje viktig incident ska kunna gå genom:

```
OPEN
→
INVESTIGATING
→
CONTAINED
→
RESOLVED
→
POSTMORTEM_COMPLETE
```

Detta gör technical quality mätbar.

Incident ID

Exempel:

INC-2026-0041

Incidenten ska kunna länkas till:

- runner
- trade
- execution
- account
- relevant logs

Root Cause

Postmortem ska försöka skilja:

- symptom
- root cause

Exempel:

Symptom:

duplicate proposal

Root cause:

event consumer processed same event twice without deduplication

Åtgärd ska adressera root cause.

Corrective Action

Incident review ska dokumentera:

- immediate fix
- permanent fix
- tests added
- monitoring added
- version released

No Blame Data

Incidenter ska ses som systemdata.

Målet är inte att dölja problem.

Målet är att göra samma problem mindre sannolikt nästa gång.

Failure Injection

Systemet ska aktivt utsättas för fel före live.

Exempel:

- disconnect network
- kill runner process
- close providern
- duplicate execution request
- corrupt test payload
- create manual position
- remove news provider
- delay market data
- simulate broker rejection
- simulate failed SL modification

Detta är en del av validation.

Chaos Testing

När systemet mognar kan kontrollerad chaos testing användas i demo/staging.

Målet är att bevisa att systemet inte bara fungerar under perfekta förhållanden.

Golden Failure Tests

Precis som vi har golden trades ska vi ha golden failure scenarios.

Exempel:

```
Wrong account → execution must be blocked  
Duplicate intent → exactly one order  
News state unknown → no new trade  
Unknown position → reconciliation required
```

Dessa ska ligga i regression suite.

Failure Regression

När ett incident har inträffat bör ett automatiserat test läggas till där möjligt.

Princip:

Ett känt failure ska helst bli ett permanent test.

Safety Metrics

Systemet ska kunna mäta:

- duplicate orders
- unknown positions
- reconciliation failures
- account mismatches
- SL verification failures
- stale-data blocks
- broker rejections
- technical incidents
- uptime

Zero-Tolerance Metrics

Vissa metrics ska ha mål:

Duplicate unintended orders = 0

Wrong-account executions = 0

Unprotected positions caused by Omnira = 0

Preventable prop breaches = 0

Detta är viktigare än hög win rate i tidiga deploymentfaser.

MTTR

Systemet kan senare mäta:

Mean Time To Recovery

för technical incidents.

Det hjälper till att förbättra operational maturity.

Availability

Runner uptime och broker connection uptime ska mätas.

Men hög uptime får aldrig prioriteras över fail-closed safety.

Det är bättre att vara säkert offline än osäkert "online".

Human Alerting

Kritiska incidents ska kunna ge användaren tydlig notification.

Exempel:

CRITICAL: MNQ-position är öppen men broker-native SL kan inte verifieras.

Alert ska innehålla:

- vad som hänt
- account
- position
- system action
- vad som kräver mänsklig attention

Alert Fatigue

Alla mindre warnings ska inte skickas som emergency alerts.

Severity och notification policy ska minska risken att användaren börjar ignorera varningar.

Atlas under Incident

Atlas kan hjälpa till att:

- sammanfatta incident
- förklara state
- visa relevant timeline
- föreslå felsökningssteg

Atlas får inte hitta på att incident är löst.

RESOLVED

ska komma från verifierad systemstate.

Incident Timeline

Atlas ska kunna visa:

10:24:09 Order filled

10:24:10 SL verification failed

10:24:10 Account execution blocked

10:24:11 Recovery modification attempted

10:24:12 Broker confirmed SL

10:24:13 Position protected

Det gör händelsen begriplig.

Audit

Alla safety-relevanta failures ska kunna granskas i efterhand.

Historiken ska visa:

- trigger
- state
- system reaction
- broker reality
- recovery
- final outcome

Failure Data och Self-Improvement

Learning Layer ska kunna analysera technical failures.

Exempel:

74 % av execution incidents inträffar efter network reconnect.

Det kan skapa engineering hypothesis.

Self-improvement gäller alltså inte bara strategin.

Systemet kan även förbättra:

- execution
- operations
- monitoring
- data quality

Technical Candidate Improvement

Exempel:

HYP-OPS-014

Inför längre reconciliation lock efter providern reconnect.

Detta ska testas precis som andra systemförändringar.

Safety First

Om en förbättring ökar execution speed men gör failure handling mindre robust ska den normalt inte accepteras.

I tidiga faser är prioriteringen:

- säkerhet
- korrekthet
- reproducerbarhet
- stabilitet
- hastighet

Residual Risk Register

Alla risks kan inte elimineras.

Systemet ska därför ha ett register över kända residual risks.

Exempel:

- gap through SL
- broker outage
- internet outage före active news exit
- extreme slippage
- exchange halt

För varje risk ska vi kunna dokumentera:

- likelihood
- potential impact
- mitigations
- remaining exposure

Accepted Risk

Vissa residual risks kommer senare behöva accepteras.

Det ska ske explicit.

Inte genom att vi glömt dem.

Live Readiness

Live ska inte godkännas förrän de viktigaste failure scenarios har testats på demo.

Det gäller särskilt:

- duplicate execution
- wrong account
- runner crash
- providerklient restart
- network loss
- unknown position
- SL failure
- news failure
- reconciliation
- kill switch

Autonomy ökar kraven

När systemet senare går från manual approval till automation ökar kraven på failure handling.

Människan är då inte längre med i varje beslut.

Det betyder att systemets:

- monitoring
- audit
- kill switches
- recovery
- alerts

måste vara ännu starkare.

Failure Principle

Den viktigaste principen i detta kapitel är:

Ett trading-system är inte robust för att det fungerar när allt går rätt. Det är robust när det beter sig förutsägbart när något går fel.

Omnira Trading ska därför designas så att:

- osäker state är synlig
- osäker state inte tolkas som säker
- nya trades stoppas tidigt vid kritiska problem
- broker reality verifieras efter execution
- recovery alltid följs av reconciliation

- incidents bevaras som lärdom

Det är denna disciplin som gör framtida autonomi möjlig.

Kapitelstatus

Kapitel: 15 – Felmoder och failure scenarios

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Primary safety behavior: FAIL CLOSED

Unknown critical state: NO NEW TRADE

Recovery: Reconnect + Resync + Reconcile + Verify

Duplicate execution protection: Obligatorisk

Wrong account protection: Obligatorisk

Broker SL verification: Säkerhetskritisk

Incident journal: Obligatorisk

Failure injection: Obligatorisk före live

Residual risk register: Ska etableras

Live deployment: Ej tillåten innan critical failure gates är verifierade

Omnira ska inte byggas på förhoppningen att fel inte inträffar.

Det ska byggas för att överleva dem.

KAPITEL 16

Säkerhet och kill switches

Säkerhet i Omnira Trading System handlar inte endast om att skydda lösenord eller API-nycklar.

Det handlar om att begränsa vad varje komponent får göra, hur ett beslut får förvandlas till en order och hur systemet stoppas när något inte längre är säkert.

Ett trading-system som kan exekvera orders måste betraktas som ett privilegierat system.

Därför ska säkerhetsarkitekturen bygga på:

- least privilege
- explicit authorization
- environment separation
- account verification
- allowlists
- immutable audit
- fail closed
- kill switches
- recovery och reconciliation

Den centrala principen är:

Ingen komponent ska få större behörighet än den behöver för sin uppgift.

Säkerhet som authority architecture

Omnira Trading ska inte bygga säkerheten kring en enda stor:

`AUTHORIZED = true`

Istället ska authority vara uppdelad.

Exempel:

Strategy Engine får:

- läsa market data
- skapa Strategy Signal

Atlas får:

- läsa trading state
- skapa analys
- skapa explanations
- föreslå hypotheses

Risk Engine får:

- bedöma risk
- neka trade

Prop Firm Rules Engine får:

- bedöma compliance
- neka trade

Execution Gateway får:

- verifiera execution authority
- skapa Execution Intent

Execution Runner får:

- läsa providern
- exekvera endast godkända intents

Ingen komponent ska ensam kunna gå från idé till broker-order.

Least Privilege

Varje service ska endast ha den behörighet den behöver.

Exempel:

Analytics behöver inte kunna skicka orders.

Atlas Learning Layer behöver inte ha providern credentials.

Frontend behöver inte ha broker access.

Strategy Engine behöver inte ha order submission capability.

Execution Runner behöver inte kunna ändra strategy rules.

Detta minskar blast radius om en komponent innehåller:

- bug
- kompromettering
- felkonfiguration

Separation of Duties

Systemet ska använda flera kontrollpunkter.

Exempel:

Strategy Engine

säger:

VALID SETUP

Risk Engine säger:

ALLOW

Prop Firm Engine säger:

ALLOW

Approval Policy säger:

AUTHORIZED

Execution Gateway säger:

VALID INTENT

Runner verifierar:

VALID ACCOUNT + VALID ENVIRONMENT

Först därefter får broker request skapas.

Atlas får inte ha broker credentials

Atlas ska aldrig behöva känna till:

- providern login password
- broker API secrets
- execution secrets
- signing keys

AI-systemet ska få strukturerade tradingobjekt.

Inte credentials.

Secrets ska hållas utanför prompts

Credentials får inte skickas i:

- AI prompts
- logs
- frontend state
- analytics payloads
- screenshots

Atlas behöver exempelvis veta:

```
account = PROP_ACCOUNT_01
```

inte dess faktiska login password.

Secret Storage

Secrets ska lagras i en dedikerad secret storage-mekanism.

Exakt teknik beslutas vid implementation.

Kraven är:

- encrypted storage
- access control
- environment separation
- rotation capability
- audit
- inga secrets i Git

Git och Credentials

Inga credentials får committas.

Det gäller:

- .env
- config files
- example scripts
- test fixtures
- screenshots
- logs

Repository scanning ska senare kunna leta efter möjliga secrets.

Demo och Live isoleras

Demo och live ska behandlas som separata security domains.

De ska ha separata:

- accounts
- credential references
- environment configuration
- execution permissions
- audit context

En demo deployment får inte kunna falla tillbaka till live credentials.

Default Environment

Default ska vara:

```
NON_LIVE
```

Ny installation eller ny runner ska alltså inte få live execution capability automatiskt.

Live Enablement

Live execution ska kräva explicit activation.

Denna activation ska vara separat från att koden råkar innehålla en Execution Adapter.

Det ska inte gå:

deploy code

```
→
```

live trading accidentally enabled

Environment Verification

Execution Runner ska verifiera environment mot account.

Exempel:

Intent:

```
DEMO
```

Account:

```
LIVE
```

Resultat:

```
ENVIRONMENT_MISMATCH  
→  
DENY
```

Environment Lock

När en runner är provisionerad för en viss environment bör dess permissions begränsa vilka accounts den kan använda.

Exempel:

Runner DEMO-01

får endast använda demo accounts.

Detta är starkare än att förlita sig på en runtime flag.

Account Allowlist

Varje runner ska ha en explicit allowlist.

Exempel:

```
allowed_accounts = [ACCOUNT_01]
```

Om providern visar:

```
ACCOUNT_02
```

Resultat:

```
ACCOUNT_NOT_ALLOWED
```

Ingen execution.

Account Identity Verification

Account verification ska ske:

- vid startup
- efter reconnect
- före execution
- när account state förändras oväntat

Systemet ska aldrig anta att samma providern-terminal alltid har samma account aktivt.

Instrument Allowlist

Execution ska också kunna begränsas till vissa instrument.

I första fasen kan allowlist exempelvis vara:

MNQ

Det innebär att ett felaktigt execution intent för:

BTCUSD

eller:

ES

inte kan exekveras även om brokern erbjuder instrumentet.

Contract Allowlist

För futures räcker inte alltid canonical instrument.

Systemet ska kunna verifiera rätt:

- exchange
- contract
- expiry

Det skyddar mot fel rollover mapping.

Strategy Allowlist

En live runner kan också begränsas till specifika StrategyVersion IDs.

Exempel:

Omnira Liquidity Manipulation v1.0

En candidate:

v1.1-candidate

ska inte kunna exekveras i live environment.

Candidate är inte Production

Candidate versions ska tekniskt behandlas som icke-produktionsgodkända.

De får användas i:

- backtest
- research
- shadow
- forward test

men inte live execution innan canonical promotion.

Risk Profile Binding

Varje live account ska ha explicit aktiv RiskProfile version.

Execution får inte använda:

latest risk profile

dynamiskt.

Den ska använda:

approved risk profile version X

Prop Profile Binding

Samma princip gäller PropFirmProfile.

Live-account ska referera till en specifik versionsstyrd profil.

Signed Execution Intents

Execution Intent bör autentiseras så att runnern kan verifiera:

- att requesten kommer från rätt Omnira-miljö
- att innehållet inte ändrats
- att requesten är aktuell

Det kan göras genom kryptografisk signering eller motsvarande autentiserad mekanism.

Exakt implementation beslutas senare.

Intent Integrity

Runnern ska kunna verifiera kritiska fields såsom:

- execution ID
- account
- instrument
- quantity
- direction
- SL
- TP
- expiry
- environment

Om payload förändrats efter authorization ska requesten nekas.

Intent Expiry

Även en korrekt signerad request ska kunna vara för gammal.

valid signature

är inte samma sak som:

valid trade

Expiry måste verifieras separat.

Replay Protection

En tidigare korrekt Execution Intent får inte kunna spelas upp senare.

Protection ska använda:

- unique ID
- idempotency
- expiry
- processed state

Authentication

Runner och Omnira ska endast kommunicera via autentiserade channels.

En lokal process eller extern klient ska inte kunna skicka trading requests utan korrekt identity.

Authorization

Authentication besvarar:

Vem är du?

Authorization besvarar:

Vad får du göra?

Båda behövs.

En autentiserad Analytics-service får fortfarande inte ha rätt att skicka orders.

Transport Security

Kommunikation mellan Omnira och runner ska vara krypterad.

Detta gäller särskilt om runnerna senare körs:

- på annan dator
- över internet
- på VPS

Local Network är inte automatiskt Trusted

Även om runnerna initialt står i samma hemnätverk ska systemet inte förlita sig på:

Den som är på LAN får handla.

Execution requests ska fortfarande autentiseras.

Network Exposure

Runners attack surface ska hållas så liten som möjligt.

Den ska inte exponera:

- onödiga ports
- debug interfaces
- admin endpoints

mot internet.

VPN

VPN kan användas om broker/prop policy tillåter det.

Men VPN är inte i sig en säkerhetsmodell.

Application-level authentication och authorization krävs fortfarande.

VPS

Samma princip gäller framtida VPS.

En VPS kan öka uptime men skapar också en internetansluten host som måste skyddas.

Operating System Hardening

Windows-runnerg ska senare ha en deployment baseline.

Exempel:

- uppdaterat OS
- minimerade onödiga services
- firewall
- restricted user account
- disk encryption där lämpligt
- automatic security updates enligt kontrollerad policy

Dedicated Trading Host

Den initiala executiondatorn bör så långt det är praktiskt användas som dedikerad tradinghost när execution aktiveras.

Ju fler okontrollerade program som körs på samma host, desto större risk för:

- crashes
- malware
- resource exhaustion
- accidental interference

Administrator Privileges

Runnerg ska inte köras med full administrator authority om detta inte krävs.

Least privilege gäller även på operativsystemsnivå.

Provider Credentials

Providercredentials ska endast finnas där de faktiskt behövs.

Idealt är det runner/terminalmiljön.

Omnira frontend, Strategy Engine och Learning Layer behöver inte ha tillgång till dem.

Credential Rotation

Systemet ska stödja rotation av credentials utan att strategy eller journalhistorik påverkas.

Accounts refereras internt genom stable account IDs, inte credentials.

Credential Revocation

Om credentials misstänks vara komprometterade ska de kunna revokeras och ersättas.

Execution ska blockeras tills account access åter verifierats.

Kill Switch Architecture

Kill switch är en explicit säkerhetsmekanism som blockerar execution.

Omnira ska stödja flera scopes:

- GLOBAL
- ACCOUNT
- STRATEGY
- INSTRUMENT
- RUNNER

Detta gör att ett problem kan isoleras utan att alltid slå ut hela systemet.

Global Kill Switch

Global kill betyder:

ingen ny execution någonstans i Omnira Trading

Den används vid systemövergripande problem.

Exempel:

- critical security incident
- corrupted risk state
- major platform incident

Account Kill Switch

Account kill blockerar endast ett specifikt account.

Exempel:

Ett prop account behöver utredas.

Övriga demo environments kan fortfarande fungera.

Strategy Kill Switch

Strategy kill blockerar en specifik StrategyVersion.

Exempel:

Ett allvarligt detection bug upptäcks i Strategy v1.0.

Andra framtida strategier behöver inte automatiskt blockeras.

Instrument Kill Switch

Instrument kill kan stoppa exempelvis:

MNQ

utan att påverka andra framtida markets.

Det kan vara relevant vid:

- contract mapping issue
- abnormal market
- corrupted feed

Runner Kill Switch

Runner kill blockerar execution genom en specifik execution host.

Exempel:

Windows-host har instabil providern connection.

Kill Switch Evaluation

Varje execution ska kontrollera alla relevanta scopes.

Exempel:

Global:

OFF

Account:

OFF

Strategy:

OFF

Instrument:

ON

Resultat:

DENY

Kill Switch är Hard Gate

Atlas får inte resonera sig förbi kill switch.

A+ setup

eller:

high AI confidence

spelar ingen roll.

Aktiv relevant kill switch:

NO NEW TRADE**Persistent Kill Switch**

Kill switch state ska överleva:

- service restart
- runner restart
- UI reconnect

En restart får inte automatiskt återaktivera trading.

Secure Default after Failure

Efter vissa critical failures kan systemet automatiskt sätta ett scope till:

BLOCKED

eller motsvarande kill state.

Exempel:

- wrong account
- reconciliation failure
- unprotected position
- unknown execution state

Återaktivering ska kräva verifierad recovery.

Kill Switch Audit

Varje ändring ska journalföras.

Exempel:

- who/what activated
- timestamp
- scope
- reason
- previous state
- new state

Human Kill Switch

Användaren ska ha en mycket tydlig möjlighet att stoppa trading.

Den ska vara snabb att nå i Omnira Trading UI.

UI Confirmation

För att aktivera kill switch bör minimalt friktion användas.

Att stoppa trading ska vara enkelt.

Att återaktivera trading kan däremot kräva mer verifiering.

Detta är en avsiktlig asymmetri.

Stop Easy, Restart Harder

Princip:

Det ska vara lätt att göra systemet säkrare och svårare att göra det farligare.

Därför kan:

KILL

vara ett enkelt explicit action.

Men:

RE-ENABLE LIVE AUTOMATION

bör kräva health checks och eventuellt mänsklig approval.

Kill Switch vs Emergency Close

Kill switch och emergency close är två olika funktioner.

Kill switch:

STOP NEW TRADES

Emergency Close:

CLOSE EXISTING EXPOSURE

De ska inte blandas.

Varför de separeras

Om användaren upptäcker en bugg i nya entries men har en korrekt skyddad position öppen kan det vara säkrare att låta den följa sin plan.

Ett generellt kill-kommando ska därför inte automatiskt göra en market close om det inte uttryckligen valts.

Emergency Close

Systemet ska stödja en separat privilegierad:

EMERGENCY CLOSE

Den ska användas när öppen exposure behöver avvecklas snabbt enligt definierad policy eller human action.

Emergency Close Authority

Eftersom emergency close påverkar riktiga positions är det en privilegierad action.

Den ska kräva:

- authenticated user/system authority
- explicit account/position scope

- audit

Emergency Close All

En framtida:

CLOSE ALL

funktion kan vara värdefull men är mycket kraftfull.

Den ska:

- vara tydligt märkt
- visa environment
- visa affected positions
- journalföras

Automated Emergency Protection

Vissa safety conditions kan senare få explicit automation.

Exempel:

Canonical riskbeslutet säger att när intern realized daily loss når \$450 ska nya trades blockeras och risk state sättas till BLOCKED / DAILY_STOP.

Den interna dagsregeln tvångsstänger däremot **inte** en redan öppen position. Eftersom mätaren endast räknar realiserade förluster kan gränsen inte brytas av en position som fortfarande är öppen. En öppen position hanteras vidare genom sina egna exitregler: teknisk SL, TP, canonical break-even, London window-close break-even 05:00, news exit, New York time exit samt Prop Firm Rules.

Automatisk stängning av en öppen position kan däremot bli aktuell för prop-relaterade gränser som använder equity, floating eller trailing. Sådan policy hör till Prop Firm Rules Engine, inte till Omniras interna dagsregel.

Detta är inte Atlas discretion.

Det är fördefinierad Risk Policy.

Prop Emergency Protection

Samma sak kan senare definieras för imminent hard prop breach.

Men sådan policy måste dokumenteras och testas innan live.

Stop Loss som säkerhetsbarriär

Broker-native stop loss är en central riskkontroll.

Det är inte samma sak som kill switch.

SL skyddar den enskilda positionen även om Omnira eller runnern går offline.

SL Verification

Efter entry ska systemet verifiera:

expected broker-native SL

mot:

actual broker SL

Om detta inte matchar ska det behandlas som critical safety incident.

No Unprotected Position Assumption

Systemet får aldrig anta att SL finns bara för att den skickades i requesten.

Broker state är source of truth.

Take Profit

TP är inte samma säkerhetsnivå som SL men ska också verifieras om planen kräver broker-native TP

Active Management Residual Risk

Break-even, news exit och time exit kräver aktiv runtime.

Detta innebär residual risk om:

- Omnira går ner
- runner går ner
- network går ner

Denna risk ska dokumenteras.

Independent Broker Protection

Ju mer protection som kan ligga broker-native utan att förstöra strategy behavior, desto bättre resilience får systemet.

Fail-Closed Execution

Execution Gateway ska neka request om någon kritisk dependency är:

- unavailable
- stale
- unknown
- inconsistent

Exempel:

Risk Decision missing

→ DENY

Prop Decision missing

→ DENY

Account unknown

→ DENY

Kill state unknown

```
→ DENY
```

No Default Allow

Systemet får inte implementera mönstret:

try:

```
verify_risk()
```

except:

```
continue_trade()
```

Safety exceptions ska inte omvandlas till permission.

Explicit ALLOW

Execution får ske först när systemet har explicit:

```
ALLOW
```

från samtliga obligatoriska gates.

Avsaknad av DENY är inte samma sak som ALLOW.

Approval Security

Human approval ska vara bundet till exakt proposalversion.

Om proposal ändras efter approval ska tidigare approval bli invalid.

What You See Is What Gets Executed

Approval UI ska visa de parametrar som faktiskt blir signerade/godkända.

Exempel:

- account
- instrument
- side
- quantity
- SL
- TP
- risk

Det får inte visas 1 MNQ men exekveras 2 MNQ.

Proposal Mutation

Efter approval ska critical execution fields vara immutable.

Behöver de förändras:

```
→ ny proposal  
→ ny approval
```

Automation Policy Security

Demo/Live automation ska använda explicit policy version.

Exempel:

AUTO_POLICY-v1

Policyn ska beskriva:

- environment
- accounts
- strategies
- allowed risk
- execution windows

No Self-Escalation

Atlas får aldrig ändra:

Live Manual

till:

Live Automated

på eget initiativ.

Autonomy level är governance state.

Autonomy Capability

Operation mode ska vara explicit.

Exempel:

```
ANALYSIS_ONLY  
READ_ONLY  
DEMO_MANUAL  
DEMO_AUTO  
LIVE_MANUAL  
CONTROLLED_LIVE_AUTO
```

En component får endast använda capabilities som mode tillåter.

Mode Downgrade

Systemet ska kunna sänka autonomy automatiskt vid problem.

Exempel:

```
DEMO_AUTO  
→  
READ_ONLY
```

vid critical execution incident.

Att gå tillbaka upp ska däremot kräva ny verification.

One-Way Safety Bias

Detta är en generell säkerhetsprincip:

Automatiskt:

MORE AUTONOMY → LESS AUTONOMY

kan vara tillåtet.

Automatiskt:

LESS AUTONOMY → MORE AUTONOMY

ska inte vara tillåtet.

Audit Logging

Alla security-sensitive actions ska skapa audit event.

Exempel:

- live execution enabled
- kill switch changed
- credential rotated
- account binding changed
- automation policy activated
- risk profile activated
- prop profile activated
- emergency close used

Append-Only Audit

Audit history ska i största möjliga mån vara append-only.

Historiska events ska inte kunna raderas för att dölja misstag.

Actor Identity

Audit event ska veta vem eller vad som gjorde action.

Exempel:

- USER
- RISK_ENGINE
- PROP_ENGINE
- SYSTEM_RECOVERY
- EXECUTION_RUNNER

Atlas är inte Audit Authority

Atlas kan sammanfatta audit logs.

AI:n får inte skriva om eller radera dem.

Security Events

Security events ska hållas separata från vanliga strategy events.

Exempel:

```
AUTH_FAILURE
UNAUTHORIZED_EXECUTION_ATTEMPT
SECRET_EXPOSURE
INVALID_INTENT_SIGNATURE
```

Failed Authentication

Repeated auth failures mot runnern ska monitoreras.

Systemet ska kunna skapa security incident.

Rate Limiting

Privilegierade endpoints bör skyddas mot onormalt många requests.

Detta gäller särskilt execution endpoints.

Execution Volume Guard

Utöver trading Risk Engine kan systemet ha tekniska sanity checks.

Exempel:

Om Strategy v1.0 normalt endast får 1 öppen position och max 3 attempts per thesis men runnern plötsligt får hundratals intents:

Detta ska behandlas som systemfel eller attack, inte som normal trading.

Circuit Breakers

Systemet kan använda technical circuit breakers.

Exempel:

N execution failures within X time

```
→
```

runner execution block.

Exakta thresholds definieras senare.

Duplicate Order Circuit Breaker

Om duplicate anomaly någonsin observeras ska live automation kunna stoppas omedelbart.

Duplicate unintended execution är zero-tolerance.

Wrong Account Circuit Breaker

Account mismatch ska alltid blockera execution.

Ingen retry tills account är verifierat.

Data Integrity Security

Security handlar också om att skydda beslutsdata från otillåtna förändringar.

Strategy config, Risk Profile och Prop Profile ska versionshanteras och endast ändras genom kontrollerad process.

Canonical Rule Integrity

Canonical rule-dokument och aktiva machine-readable configs ska ha tydlig relation.

Systemet ska kunna verifiera:

active strategy config = approved canonical version

Configuration Signing / Hashing

På sikt kan kritiska config artifacts ha checksum/hash eller signing för att upptäcka oavsiktliga förändringar.

No Silent Configuration Drift

Om live-runnern använder annan config än backend förväntar sig ska execution blockeras.

Version Handshake

Omnira och runner bör kunna utbyta versionsinformation.

Exempel:

- runner version
- intent schema
- active capabilities

Incompatible versions:

NO EXECUTION

Deployment Security

Production deployment ska skilja mellan:

- code deploy
- config activation
- live authorization

Ett nytt code build ska inte automatiskt ändra trading authority.

Deployment Sign-Off

Större säkerhetskritiska deployment changes ska senare ha verifieringschecklist.

Exempel:

- tests pass
- demo verified
- migration verified
- runner compatible
- rollback available

Rollback

Om en ny version har fel ska systemet kunna återgå till tidigare verifierad version där tekniskt möjligt. Rollback får inte förstöra broker reconciliation.

Open Position During Deployment

Deployment med öppen live position ska betraktas som särskilt riskfyllt.

Default policy bör vara att undvika onödiga deployments under active exposure.

Exakta regler definieras senare.

Database Access

Services ska inte alla ha full write-access till hela tradingdatabasen.

Access bör begränsas efter responsibility när arkitekturen implementeras.

Frontend Access

Frontend ska aldrig direkt kunna skriva:

- broker order
- RiskDecision
- PropDecision

Frontend requests ska gå genom backend authority layers.

CSRF / Session Security

Human approval måste skyddas som en känslig action.

En webbsida ska inte kunna trigga approval utan giltig användarsession och rätt security controls.

Exakt webbsäkerhetsimplementation definieras när UI/backend byggs.

Human Factors

Säkerhet handlar också om att minska mänskliga misstag.

Live UI ska därför mycket tydligt visa:

LIVE

och:

- account
- environment
- risk

- quantity

innan approval.

Demo/Live Visual Separation

Demo och live ska inte se så identiska ut att de lätt kan förväxlas.

UI ska använda tydlig miljöindikering.

Confirmation for Dangerous Actions

Farliga actions såsom:

- enable live automation
- emergency close all
- change live risk profile

ska kräva tydlig confirmation.

No Confirmation Fatigue

Normala tradingflöden ska samtidigt inte bombardera användaren med meningslösa dialogs.

Extra confirmation ska reserveras för verkligt privilegierade actions.

Live Risk Profile Change

Riskregler ska inte ändras mitt i aktiv trading utan kontrollerad process.

En ny RiskProfile version ska:

- skapas
- granskas
- aktiveras explicit

Prop Rule Changes

Samma gäller PropFirmProfile.

Om firman ändrar regler ska systemet blockera eller kräva review enligt freshness-policy innan ny version används.

Canonical Strategy Changes

Atlas Learning Layer får inte skriva direkt till active StrategyVersion.

Förslag:

candidate

```
→ test
→ approval
→ canonical
```

Self-Improvement Security Boundary

Learning Layer får ha write access till:

- findings
- hypotheses
- candidate definitions

men inte till:

- active live strategy
- active risk profile
- active prop rules
- execution permissions

Detta är ett viktigt architectural permission boundary.

Prompt Injection och AI Inputs

Om Atlas senare analyserar externa textkällor såsom:

- news
- broker documentation
- prop firm pages

ska dessa betraktas som data, inte systeminstruktioner.

Extern text ska inte kunna instruera Atlas att:

- kringgå risk
- ändra live config
- exekvera trade

AI Output Validation

AI-genererade outputs som påverkar strukturerade systemfält ska valideras mot schemas och authority policy.

Fri text får inte bli broker request.

No Tool Authority by Language Alone

En mening såsom:

Take the trade now.

ska inte vara tillräcklig execution authorization om den kommer från en AI-analysis-output.

Authority ska representeras genom separata systemobjekt.

Security Monitoring

Omnira ska senare monitorera bland annat:

- auth failures
- invalid execution intents
- unexpected account changes
- runner changes
- kill switch events
- config changes

- duplicate attempts

Alerting

Kritiska säkerhetshändelser ska notifiera användaren.

Exempel:

Execution blocked: active providern account does not match approved live account.

Security Incident Response

Ett security incident ska kunna följa:

```
DETECTED
→
CONTAINED
→
INVESTIGATED
→
RECOVERED
→
POSTMORTEM
```

Vid osäkerhet ska execution-capability reduceras.

Compromised Runner

Om runnern misstänks komprometterad ska:

- runner kill aktiveras
- credentials kunna roteras
- account state verifieras från trusted channel
- ny deployment provisioneras vid behov

Compromised Credentials

Vid misstänkt credential exposure ska trading inte fortsätta som vanligt.

Credential rotation och account review krävs.

Security Backup

Kritiska configs och auditdata ska backas upp.

Secrets kräver separat säker backup-policy.

Restore

Efter restore ska systemet inte automatiskt anta att säkerhetsstate är aktuell.

Exempel:

En backup kan innehålla gammal kill-switch state eller gammal PropProfile.

Restore måste följas av reconciliation och config verification.

Disaster Recovery

Systemet ska senare ha dokumenterad plan för:

- backend loss
- runner loss
- database recovery
- new execution host
- credential recovery

Målet är säker återställning, inte snabbast möjliga tradingåterstart.

Security Testing

Före live ska systemet testas mot exempelvis:

- unauthorized request
- replayed intent
- expired intent
- modified payload
- wrong account
- wrong environment
- disallowed instrument
- duplicate request
- killed service
- stale kill-switch state

Negative Security Tests

Det räcker inte att testa att rätt request fungerar.

Vi måste också bevisa att fel request inte fungerar.

Penetration och Dependency Review

När systemet närmar sig riktig live automation bör det genomgå separat säkerhetsreview.

Det kan inkludera:

- dependency vulnerabilities
- network exposure
- auth configuration
- secret handling
- privilege review

Third-Party Risk

Systemet är beroende av komponenter såsom:

- broker
- providern
- data providers
- VPS

- AI provider
- news provider

Dessa har egen risk.

Omnira ska minimera hur mycket authority ett fel i extern tjänst får.

AI Provider Outage

AI outage ska inte ge execution authority till något annat system av misstag.

Det ska endast skapa den fallback som definierats för operation mode.

Market Data Provider Compromise

Om data ser orimlig ut eller integrity checks faller ska execution stoppas.

Risk Engine kan inte skydda mot en strategi som matas med helt fel prisdata om datakvaliteten inte kontrolleras först.

Supply Chain Security

Dependencies och packages som används i runner/backend ska hållas kontrollerade.

Security updates ska hanteras genom normal engineeringprocess.

Audit Export

Vid behov ska säkerhets- och tradingaudit kunna exporteras för granskning.

Det ska hjälpa vid:

- debugging
- prop firm incident
- external security review

Zero-Trust Principle

Omnira Trading ska i praktiken följa en förenklad zero-trust-princip:

Verifiera varje privilegierad handling utifrån aktuell identity, state och policy.

Att något var korrekt igår betyder inte automatiskt att det är korrekt idag.

Security och Performance

Säkerhetskontroller får skapa viss latency.

För den första versionen är det acceptabelt.

Vi optimerar inte bort:

- account verification
- risk revalidation
- signature checks

för att spara några millisekunder utan evidens att det behövs.

Safety vs Speed

Prioritet:

Safety
→ Correctness
→ Auditability
→ Reliability
→ Performance

Execution speed optimeras senare inom dessa gränser.

Kill Switch Dashboard

Atlas Market View ska tydligt visa:

Global Kill: OFF

Account Kill: OFF

Strategy Kill: OFF

Instrument Kill: OFF

Runner Kill: OFF

Execution State: READY

Om någon ändras:

EXECUTION BLOCKED

ska vara omedelbart synligt.

Live Safety Header

I live mode bör Trading UI alltid visa en kompakt safety header:

- LIVE
- account
- runner
- providern
- Risk Engine
- Prop Firm
- kill switch
- reconciliation

Detta ger snabb mänsklig situationsbild.

Atlas och säkerhet

Atlas ska kunna förklara säkerhetsstate.

Exempel:

Trading är blockerad eftersom execution-runnern återanslöt till providern och ännu inte har klarat reconciliation.

Det är en lämplig AI-roll.

Atlas ska inte kunna säga:

Det verkar nog okej, vi kör ändå.

Safety Governance

Säkerhetskritiska policyförändringar ska versionshanteras.

Exempel:

- autonomy policy
- emergency close policy
- risk profile
- execution policy

Det ska gå att se vilken policy en live trade använde.

Security as Code

Där det är praktiskt ska säkerhetsregler vara:

- explicit
- testbara
- maskinläsbara

inte bara finnas som text i boken.

Boken definierar avsikten.

Kod och konfiguration implementerar den.

Tester bevisar beteendet.

Golden Safety Tests

Systemet ska ha automatiserade säkerhetsfall såsom:

```
Wrong account  
→ DENY
```

```
Wrong environment  
→ DENY
```

```
Kill active  
→ DENY
```

```
Expired intent  
→ DENY
```

```
Replay  
→ DENY
```

Unknown Risk state
→ DENY

Unknown Prop state
→ DENY

Disallowed symbol
→ DENY

Duplicate intent
→ exakt en execution

Safety Sign-Off

Före Controlled Live ska safety test suite vara en explicit gate.

Ett känt critical failing test innebär:

NO LIVE

Säkerhetens relation till autonomi

Autonomi är inte att ta bort säkerhetskontroller.

Autonomi betyder att fler normala beslut kan ske utan mänskligt klick medan samma eller starkare controls ligger kvar.

Ju högre autonomi:

desto högre krav på:

- authentication
- monitoring
- audit
- recovery
- fail closed
- kill switches

Den centrala säkerhetsprincipen

Omnira Trading ska byggas utifrån frågan:

Om denna komponent går fel eller komprometteras, hur mycket kan den faktiskt göra?

Det bästa svaret är inte:

Förhoppningsvis går den aldrig fel.

Det bästa svaret är:

Dess behörighet är begränsad, nästa lager verifierar den, och systemet kan stoppas innan felet växer.

Det är grunden för säker autonom trading.

Kapitelstatus

Kapitel: 16 – Säkerhet och kill switches

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Security model: Least privilege + separation of duties

Default environment: Icke-live

Live authority: Explicit

Account allowlist: Obligatorisk

Instrument allowlist: Obligatorisk

Strategy version binding: Obligatorisk

Execution intent integrity: Obligatorisk

Replay/idempotency protection: Obligatorisk

Kill switch scopes: Global, Account, Strategy, Instrument, Runner

Emergency Close: Separat från Kill Switch

Secrets in AI/frontend/Git: Förbjudet

Self-improvement direct production write: Förbjudet

Unknown critical security state: Fail closed

Live safety testing: Obligatoriskt före aktivering

Omnira Trading ska göra det lätt att stoppa trading och svårt att oavsiktligt starta eller förändra privilegierad live execution.

KAPITEL 17

Demofas

Demofasen är den första fas där Omnira Trading System får exekvera riktiga orders genom providern, men utan att verkligt kapital står på spel.

Detta är en avgörande övergång.

Fram till denna punkt har systemet främst bevisat att det kan:

- förstå strategy rules
- läsa market data
- skapa signals
- analysera risk
- simulera trades
- hantera historik
- genomföra backtest
- genomföra forward test utan verklig kapitalrisk

Demofasen ska bevisa att hela kedjan fungerar tillsammans i faktisk drift.

Den centrala principen är:

Demo är inte en lekplats. Demo är en verifieringsmiljö för samma system som senare kan få live authority.

Demofasen i systemets progression

Den rekommenderade ordningen är:

Analysis Only

→

Shadow Mode

→

Demo Manual Approval

→

Demo Automation

Först därefter får Controlled Live övervägas.

Analysis Only

Analysis Only är den första realtime-fasen.

Systemet får:

- läsa market data

- identifiera setups
- skapa Strategy Signals
- köra AI Analysis
- köra Risk Engine
- köra Prop Firm Rules Engine
- skapa hypotetiska Trade Proposals
- journalföra hela processen

Men:

ingen order får skickas.

Målet med Analysis Only

Målet är att verifiera att Strategy Engine i realtime gör samma sak som vi förväntar oss från dokumentation och backtest.

Vi ska kunna kontrollera:

- rätt 4H-open
- rätt session
- rätt liquidity
- rätt FVG
- rätt manipulation
- rätt 1m confirmation
- rätt setup grade
- rätt entry
- rätt SL
- rätt TP
- rätt R:R
- rätt news policy
- rätt re-entry state

Analysis Only Acceptance Criteria

Fasen ska inte betraktas som godkänd förrän systemet konsekvent kan:

- använda korrekt timezone
- identifiera rätt trading windows
- hantera realtime candles
- undvika duplicate signals
- journalföra state transitions
- visa samma state i Atlas Market View
- hantera stale/missing data korrekt
- undvika look-ahead
- producera reproducerbara signals

Shadow Mode

Shadow Mode är nästa steg.

Systemet beter sig som om det skulle exekvera, men skickar fortfarande ingen order.

Det ska skapa:

- planned entry
- planned quantity
- planned SL
- planned TP
- simulated fill
- simulated result

Shadow Mode gör det möjligt att testa hela pre-execution pipeline utan broker risk.

Shadow Mode ska vara realistiskt

Shadow Mode ska inte anta perfekt execution.

Det bör kunna använda:

- realtime bid/ask
- current price
- simulated latency
- estimated slippage
- commissions
- fees

Det gör jämförelsen mot senare demo fills mer relevant.

Shadow Mode Acceptance Criteria

Shadow Mode ska bland annat verifiera:

- correct proposal generation
- correct RiskDecision
- correct PropDecision
- correct quantity
- correct expiry
- correct simulated entry
- correct simulated position management
- correct break-even
- correct news exit
- correct time exit
- correct journal

Manual Verification mot Chart

Under Analysis och Shadow Mode ska systemets interpretation regelbundet jämföras med mänsklig chart review.

Detta gäller särskilt:

- liquidity
- swing structure
- FVG
- manipulation
- iFVG
- CISD
- SMT

Syftet är inte att låta människan ersätta Strategy Engine.

Syftet är att verifiera att implementationen matchar Canonical Specification.

Detection Rules måste vara deterministiska

Innan Demofasen når faktisk execution måste entrykritiska patterns ha exakta machine-readable detection rules.

Det gäller särskilt:

- iFVG
- CISD

Strategy Engine får inte vara beroende av att Atlas visuellt gissar dessa patterns.

Samma detection logic måste användas i:

- backtest
- replay
- forward
- demo
- live

Demo Manual Approval

När Shadow Mode fungerar stabilt aktiveras:

DEMO MANUAL

Detta är första gången systemet får skicka en riktig broker order genom providern.

Men varje trade kräver explicit mänskligt approval.

Varför Manual Approval först

Human approval ger en extra kontrollpunkt medan execution-infrastrukturen fortfarande valideras.

Det gör det möjligt att inspektera:

- account
- instrument
- direction
- quantity
- risk
- entry

- SL
- TP
- R:R
- Strategy status
- Risk status
- Prop status

innan ordern skickas.

Demo Account Only

Demo Manual får endast använda explicit godkänt demo-account.

Runnern ska verifiera:

```
environment = DEMO
```

och:

```
account = approved demo account
```

före varje execution.

Live Credentials ska inte behövas

Under Demofasen ska systemet kunna fungera utan live execution credentials.

Detta minskar risken att fel environment används av misstag.

Demo Manual Execution Flow

Den fulla kedjan är:

```
Realtime Market Data
→ Strategy Engine
→ AI Analysis
→ Risk Engine
→ Prop Firm Rules Engine
→ Trade Proposal
→ Human Approval
→ Execution Gateway
→ Execution Intent
→ Execution Provider Adapter
→ Futures Execution Provider (demo)
→ Broker Demo Environment
→ Journal
```

Trade Proposal Review

Före approval ska användaren kunna se minst:

- environment
- account
- instrument

- direction
- setup grade
- strategy version
- planned entry
- SL
- TP
- R:R
- quantity
- risk \$
- risk %
- daily risk state
- Prop Firm state
- expiry

Approval Integrity

Approval ska vara bundet till exakt proposal.

Om critical values förändras efter approval:

- price
- quantity
- SL
- TP
- R:R

ska tidigare approval inte längre vara giltigt.

Pre-Execution Revalidation

Efter approval men före order submission ska systemet kontrollera allt igen.

Minst:

- proposal still valid
- current price
- R:R
- risk
- account
- open position
- attempt count
- news
- prop state
- kill switches
- runner state
- providern state

Demo Order

När alla gates passerar får systemet skicka order till providerns demomiljö.

Ordern ska därefter verifieras mot actual broker state.

Fill Verification

Efter order submission ska systemet kontrollera:

- actual fill
- actual quantity
- actual position
- actual SL
- actual TP

Det räcker inte att providern returnerade ett positivt request response.

Broker-Native SL

En demoposition ska inte räknas som korrekt etablerad förrän systemet verifierat att stop loss verkligen finns hos broker.

Om SL saknas:

CRITICAL DEMO INCIDENT

Detta ska behandlas som om felet hade inträffat live.

Broker-Native TP

TP ska också verifieras där strategy plan kräver det.

Break-Even på Demo

Demofasen ska bevisa hela canonical break-even-flödet.

För long:

- nearest confirmed 1m swing high efter entry identifieras
- pris tar swing high
- BE event triggas
- SL modification skickas
- broker bekräftar SL = entry

Short motsvarande med swing low.

Ingen falsk BE-status

Om providern nekar modification ska UI och journal visa:

```
BREAK_EVEN_FAILED
```

inte:

```
BREAK_EVEN_ACTIVE
```

Stop Loss Testing

Vi ska aktivt samla demoexempel där SL träffas.

Detta verifierar:

- stop placement
- broker execution
- trade close detection
- final P/L
- final R
- attempt counter

Take Profit Testing

På samma sätt ska TP-flödet verifieras.

News Exit Testing

Canonical Strategy v1.0 kräver att öppen position stängs:

T-15m

före relevant high-impact USD-news.

Demo ska användas för att verifiera hela flödet:

- NewsEvent
- countdown
- exit trigger
- close request
- actual broker close
- journal reason = NEWS_EXIT

News Blackout Testing

No new entry:

T-1h → T+4h

ska också verifieras.

Vi ska särskilt testa boundary cases.

Exempel:

T-60m exakt

T+4h exakt

Time Exit Testing

New York-positioner ska stängas senast fyra timmar efter entry enligt Strategy v1.0.

Demo ska verifiera:

TIME_EXIT

hela vägen till actual broker close.

London Position Management

London-positioner ska inte automatiskt få samma 4h time exit.

Demo ska verifiera att session-specifik management fungerar korrekt.

Re-entry på Demo

Re-entrylogiken ska testas med actual demo trades.

Efter loss:

- thesis behålls
- nytt valid setup kan tas
- attempt count ökar

Max:

3 attempts

Efter:

- winner
- break-even

ska inga fler re-entrys tas på samma thesis.

Persistence av Attempt Count

Runner eller Omnira restart får inte återställa attempts.

Detta ska aktivt testas.

One Position Rule

Demo ska verifiera att systemet aldrig öppnar en andra Omnira-position när en position redan är öppen.

Opposite Setup

Om motsatt setup identifieras medan position är öppen ska Strategy v1.0 ignorera den för execution.

Den kan fortfarande journalföras för research.

Daily Risk

Demo ska använda den canonical interna Risk Profile:

- \$150 max risk/trade
- \$450 realized daily loss
- max 1 open position
- max 3 attempts per thesis

Daily Reset

Daily risk reset ska verifieras vid:

00:00 America/New_York

Det ska testas även runt daylight saving transitions.

Daily Stop Test

Demo ska aktivt testa att:

realized daily loss $\geq$ \$450

leder till:

- block new trades
- close existing position enligt canonical risk policy
- persistent daily stop
- correct UI state
- correct reset nästa tradingdag

Minimum Quantity Denial

Vi behöver ett test där technical SL gör att minsta möjliga contract size överskrider riskbudget.

Expected:

```
RISK_DENY
```

Runner får aldrig flytta technical SL för att skapa en tillåten trade.

Demo Prop Firm Profile

Även om demoaccount inte är en riktig challenge kan ett simulerat PropFirmProfile appliceras.

Detta verifierar compliance engine samtidigt med execution.

Prop Firm Denial Test

Vi ska aktivt skapa ett test där:

Risk PASS

men:

Prop DENY

Expected:

```
ingen order
```

Unknown Prop State

Om prop engine inte kan avgöra state:

```
ingen demo order
```

Detta verifierar fail-closed-beteende.

Duplicate Execution Test

Ett av Demofasens viktigaste test är att samma Execution Intent skickas flera gånger.

Expected:

exakt en providern-order

Approval Double-Click Test

Samma gäller om användaren dubbelklickar approval.

Det ska fortfarande bli:

en trade

Runner Crash Before Order

Vi ska testa:

- intent mottaget
- runner crash före submission
- restart
- reconciliation

Expected:

ingen duplicate trade.

Runner Crash After Order

Vi ska också testa den svårare varianten:

- order skickas
- runner crash innan confirmation sparas

Efter restart:

- read providern positions/orders/deals
- reconcile
- hitta actual trade
- inte skicka om

Provider Restart

Provideranslutningen ska medvetet startas om under demo.

Runnern ska:

- upptäcka disconnect
- blockera execution
- reconnecta
- resynca
- reconcila
- återgå till ready

Windows Restart

Hela Windows-host ska kunna startas om.

Efter startup:

- runner start
- providern start
- account verification
- position sync
- reconciliation

Ingen trade före READY.

Network Disconnect

Nätverket ska medvetet brytas under:

- ingen position
- öppen position

Vi ska verifiera skillnaden mellan:

- new execution
- existing broker-native protection
- active management outage

Open Position under Outage

Om en position har verifierad broker-native SL och TP ska dessa skydd kvarstå när Omnira går offline.

Demo ska användas för att kontrollera detta praktiskt.

Active Management Outage

Break-even, news exit och time exit kan kräva aktiv runtime.

Demo ska hjälpa oss förstå residual risk om systemet är unavailable precis när sådan action krävs.

Manual providern Trade

Vi ska manuellt öppna en position direkt i demo-providern.

Omnira ska upptäcka:

```
MANUAL_EXTERNAL
```

eller:

```
UNKNOWN_POSITION
```

Ny Omnira execution ska blockeras enligt aktuell policy.

Manual providern Modification

Vi ska manuellt ändra SL eller TP i terminalen.

Omnira ska upptäcka discrepancy och uppdatera actual broker state.

Manual providern Close

Om positionen stängs manuellt ska journalen visa att exit inte kom från normal strategy management.

Wrong Account Test

Runnern ska medvetet möta fel demoaccount.

Expected:

ACCOUNT_MISMATCH

NO EXECUTION

Detta är obligatoriskt före nästa fas.

Wrong Environment Test

Ett testintent för fel environment ska nekas.

Wrong Symbol Test

Execution Intent med instrument utanför allowlist ska nekas.

Wrong Quantity Test

Ett modifierat intent med annan quantity än RiskDecision ska nekas.

Expired Proposal Test

En gammal proposal ska aldrig kunna exekveras.

Kill Switch Testing

Varje kill switch-scope ska testas på demo:

- GLOBAL
- ACCOUNT
- STRATEGY
- INSTRUMENT
- RUNNER

Aktiv relevant kill:

NO NEW EXECUTION

Persistent Kill Switch

Vi ska starta om systemet medan kill switch är aktiv.

Efter restart ska den fortfarande vara aktiv.

Emergency Close Testing

Separat emergency close ska testas på demo.

Vi ska verifiera:

- explicit action
- correct account
- correct position
- close result
- audit
- reconciliation

Kill Switch ska inte automatiskt Emergency Close

Vi ska också verifiera att vanlig kill switch inte stänger befintlig position om sådan policy inte explicit begär det.

Data Failure Testing

Under demo ska vi simulera:

- stale data
- missing bars
- ES SMT data missing
- news data missing

Systemet ska reagera enligt canonical fail-closed-policy.

AI Failure Testing

Atlas ska kunna stängas av.

Vi ska verifiera att:

- canonical Strategy state förblir korrekt
- AI-unavailable state visas
- systemet följer operation-mode-policy
- inga hallucinerade AI-resultat används som execution permission

AI Contradiction Test

Test:

Structured state:

SMT = UNKNOWN

Mockad AI analysis:

SMT confirmed

Expected:

canonical state förblir UNKNOWN.

Execution får aldrig baseras på AI-felaktigheten.

Journal Completeness

Varje demo trade ska kunna rekonstrueras.

Målet är:

- signal
- decisions
- approval
- execution
- fills
- management
- exit
- final metrics

utan luckor.

Audit Completeness

Privilegierade actions ska också finnas.

Exempel:

- demo execution enabled
- kill switch changes
- emergency close
- account binding

Technical Incident Threshold

Demo ska inte betraktas som godkänd om critical execution incidents fortfarande inträffar återkommande.

Exempel på blockerande problem:

- duplicate unintended order
- wrong account execution
- incorrect quantity
- missing SL
- unreconciled position
- silent data corruption

Incident Rate

Det räcker inte att fixa ett problem en gång.

Systemet ska visa stabilitet över tillräcklig driftstid.

Demo Manual Performance

Vi ska mäta både:

Strategy performance

och:

Operational performance

under Demo Manual.

Strategy metrics:

- expectancy
- R
- PF
- drawdown

Operational metrics:

- latency
- slippage
- execution failures
- uptime
- reconciliation failures

Human Approval Latency

Demo Manual är rätt fas för att mäta hur användarens approval påverkar entry.

Om delay regelbundet gör att:

- R:R faller under 2
- trades missas
- slippage blir stor

ska detta dokumenteras.

Det kan bli argument för Demo Automation.

Manual Approval ska inte kringgå Rules

Användaren ska inte kunna välja:

Execute Anyway

på ett Risk DENY eller Prop DENY.

Approval gäller endast en proposal som redan passerat hard gates.

Demo Automation

När Demo Manual är stabil kan:

DEMO AUTO

aktiveras.

Nu tas människan bort från per-trade approval.

Detta är första gången Omnira genomför hela tradingkedjan självständigt.

Vad Demo Automation inte förändrar

Följande finns kvar:

- Strategy Engine

- Data Quality Gate
- AI policy
- Risk Engine
- Prop Firm Engine
- proposal
- pre-execution revalidation
- Execution Gateway
- idempotency
- runner checks
- kill switches

Endast human approval ersätts av en explicit automation policy.

Automation Policy

Demo Automation ska ha en versionsstyrd policy.

Den kan exempelvis begränsa:

- accounts
- environment
- strategy
- instruments
- risk profile
- sessions

Ingen automatisk Live-promotion

Demo Automation får aldrig själv ändra:

DEMO

till:

LIVE

även efter stark performance.

Live är en separat human-governed promotion.

Demo Automation Acceptance Criteria

Innan Demo Automation kan betraktas som godkänd ska systemet visa:

- zero unintended duplicate orders
- zero wrong-account orders
- zero wrong-environment orders
- correct quantity
- reliable broker-native SL
- reliable TP
- correct BE

- correct news exits
- correct time exits
- correct daily stop
- correct attempt handling
- stable reconciliation
- stable restart recovery
- working kill switches
- complete journal
- complete audit

Performance är också relevant

Teknisk perfektion räcker inte om strategin inte uppvisar tillräcklig edge.

Demo Automation måste därför också producera en relevant performance sample.

Men Profit ensam räcker inte

En lönsam demo med technical incidents är inte live-ready.

Exempel:

+20R

men:

2 duplicate orders

är:

FAIL

ur safety-perspektiv.

Safety dominerar Promotion

En enda kritisk kategori såsom wrong-account execution kan blockera promotion oavsett P/L.

Minimum Demo Duration

Exakt minimum ska definieras senare utifrån:

- strategy frequency
- trade count
- market regime coverage
- operational uptime

Demofasen får inte avslutas efter några bra dagar.

Sample Size

Demo-resultat ska alltid visa antal:

- setups
- signals

- executed trades

Regime Coverage

Demo bör få möta flera typer av marknadsförhållanden.

Om alla trades råkar ske under en enda lugn period är systemet mindre bevisat.

Session Coverage

Både:

- London
- New York

ska valideras.

Grade Coverage

A+, A, B och C ska följas.

Det krävs inte nödvändigtvis stort sample i varje grade före första tekniska sign-off, men performance coverage ska tydligt dokumenteras.

Prop Simulation Coverage

Om första riktiga målet är prop firm ska relevant PropFirmProfile simuleras under en betydande del av demofasen.

Demo vs Backtest

Atlas ska jämföra:

- backtest expectancy
- forward/shadow expectancy
- demo expectancy

och försöka förstå skillnader.

Demo vs Shadow Fills

Shadow fill model ska jämföras med actual demo fills.

Detta hjälper oss förbättra framtida backtest realism.

Slippage Calibration

Actual demo slippage ska sparas per:

- session
- volatility
- entry type

Det kan senare förbättra cost model.

Demo Commission Calibration

Om demo environment simulerar fees tillräckligt realistiskt ska dessa jämföras mot backtest assumptions.

Technical Performance Dashboard

Omnira ska senare kunna visa exempelvis:

Mode: DEMO AUTO

Runner Uptime: 99.9%

Orders: 124

Duplicate Orders: 0

Account Mismatches: 0

SL Verification Failures: 0

Reconciliation Failures: 0

Average Execution Latency: X

Average Slippage: Y

Strategy Dashboard

Parallellt:

Trades: 124

Expectancy: +0.24R

PF: 1.43

Max DD: -7.1R

Win Rate: 42%

BE Rate: 15%

Incident Dashboard

Alla incidents ska vara synliga.

Exempel:

Critical: 0

Blocking: 1

Warnings: 7

Det gör go/no-go review enklare.

Demo Promotion Review

När vi anser Demofasen färdig ska en formell review genomföras.

Den ska granska:

Strategy

Har v1.0 fortfarande edge?

Risk

Följs alla canonical limits?

Prop

Fungerar regelmotorn?

Execution

Är orderflödet korrekt?

Reliability

Klarar systemet restarts och outages?

Security

Fungerar auth, allowlists och kill switches?

Audit

Kan varje trade rekonstrueras?

PASS

Om samtliga definierade gates uppfylls:

DEMO PHASE = PASS

Det betyder:

Systemet är kandidat för Controlled Live evaluation.

Det betyder inte:

Full autonom live trading är godkänd.

FAIL

Om kritiska problem återstår:

DEMO PHASE = FAIL

Problemen ska fixas.

Relevant testsektion ska upprepas.

INCONCLUSIVE

Om exempelvis sample size eller market coverage är för liten:

INCONCLUSIVE

Testet fortsätter.

No Schedule Pressure

Demofasen ska inte godkännas för att vi blivit otåliga.

Marknaden bestämmer hur snabbt relevant sample samlas.

Dokumenterade Gates

Promotion ska baseras på:

- test IDs
- logs
- metrics
- incidents
- performance
- regression suite

inte endast känslan:

Det verkar fungera.

Demo Release Candidate

När Demo Auto är stabil kan systemet få en release candidate-version.

Exempel:

Omnira Trading Runtime – Demo RC1

RC-versionen ska köras oförändrad under slutlig demo validation.

Freeze före Sign-Off

Under slutlig sign-off bör kritiska rules och runtime code vara låsta.

Om code ändras materiellt måste relevant verification köras om.

Golden Demo Run

En slutlig verifieringsperiod kan behandlas som:

GOLDEN DEMO RUN

Den ska använda exakt den configuration som senare föreslås gå till Controlled Live.

Self-Improvement under Demo

Atlas Learning Layer får fortsätta:

- analysera
- skapa findings
- skapa hypotheses
- skapa candidates

Men den aktiva demo strategy versionen får inte förändras mitt i en valideringsrun.

Candidate Testing parallellt

Det går att fortsätta forska på candidate v1.1 parallellt.

Men production candidate för live ska fortfarande kunna vara Canonical v1.0 tills en ny version separat bevisats bättre.

Ingen Optimization-by-Demo

Vi ska inte ändra strategy efter varje demoförlust.

Det skulle förstöra testintegriteten.

Demofasen som generalrepetition

Demofasen ska behandlas som generalrepetition för live.

Skillnaden är kapitalet.

Arkitektur, controls, journal och execution ska däremot vara så nära den framtida livekedjan som möjligt.

Vad demo inte kan bevisa

Demo kan inte fullständigt bevisa:

- real-money broker behavior
- live prop firm enforcement
- verklig slippage
- psychological pressure
- payout behavior
- alla live-specific restrictions

Därför behövs Controlled Live efteråt.

Vad demo ska bevisa

Demo ska däremot ge mycket stark evidens för att:

Omnira kan köra hela tradingprocessen autonomt utan att bryta sina egna regler under normal och testad abnormal drift.

Säkerhetsprincip före Live

Före första riktiga kapitalrisken ska vi kunna säga:

Vi har medvetet försökt få systemet att göra fel på demo och verifierat att de kritiska safety gates stoppar det.

Det är en mycket högre standard än att bara konstatera att botten kan placera en order.

Kapitelstatus

Kapitel: 17 – Demofas

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Progression: Analysis Only → Shadow → Demo Manual → Demo Auto

Demo account: Obligatoriskt före live

Human approval: Första executionfas

Demo automation: Första fullständiga autonoma executionfas

Failure injection: Obligatoriskt

Daily risk verification: Obligatoriskt

Prop simulation: Planerad

Kill switch verification: Obligatoriskt

Restart/reconciliation: Obligatoriskt

Critical execution incidents vid sign-off: Ska vara noll i definierade zero-tolerance-kategorier

Demo PASS: Krävs före Controlled Live

Demofasen ska behandlas som den sista stora tekniska säkerhetsprövningen innan Omnira Trading System får möjlighet att påverka verkligt kapital.

KAPITEL 18

Live-deployment

Live-deployment är den punkt där Omnira Trading System för första gången får påverka verkligt kapital.

Detta är inte bara nästa tekniska steg.

Det är en förändring av systemets risknivå.

Från och med denna fas kan:

- implementation bugs
- brokerproblem
- dataproblem
- konfigurationsfel
- felaktig riskberäkning
- mänskliga misstag

få verklig ekonomisk konsekvens.

Därför ska live-deployment ske gradvis och under strikt kontroll.

Den centrala principen är:

Live är en ny valideringsfas, inte slutmålet.

Live börjar inte med full automation

Den första livefasen ska vara:

LIVE MANUAL APPROVAL

Inte:

FULLY AUTONOMOUS LIVE

Det innebär att Strategy Engine, Risk Engine, Prop Firm Engine och Execution Layer fungerar som i demo, men varje trade kräver explicit mänskligt godkännande.

Progression från Demo till Live

Rekommenderad progression:

Demo Auto – PASS

→

Live Read Only

→

Live Manual Approval

→

Controlled Live Automation

→

Gradvis uppskalning

Varje steg ska ha separat sign-off.

Live Read Only

Innan systemet får skicka första liveordern ska det först ansluta till det riktiga kontot i:

READ_ONLY

Vi ska verifiera:

- account identity
- broker/server
- account balance
- equity
- margin
- symbols
- positions
- orders
- deal history
- relevant market data
- prop profile mapping

Ingen liveorder ska skickas i denna fas.

Varför Live Read Only behövs

Demo och live kan skilja sig.

Exempel:

- symbols
- contract naming
- account permissions
- broker server
- tick values
- market data
- fees
- execution settings

Live Read Only gör att dessa skillnader upptäcks innan kapital riskeras.

Live Account Verification

Det första livekontot ska bindas explicit till:

- TradingAccount
- broker
- environment = LIVE
- RiskProfile
- PropFirmProfile där relevant
- approved instruments
- approved strategy versions
- approved runner

Ingen implicit mapping är tillåten.

Live Account Allowlist

Runnern ska endast få exekvera mot exakt godkända live accounts.

Om ett annat account är aktivt:

```
ACCOUNT_MISMATCH
```

```
→
```

```
NO EXECUTION
```

Live Instrument Allowlist

För den första livefasen ska execution surface hållas liten.

Om den faktiska execution-strategin använder MNQ kan allowlist initialt begränsas till:

```
MNQ
```

Det minskar blast radius.

Strategy Version Lock

Live ska endast använda en explicit godkänd canonical strategy version.

Exempel:

Omnira Liquidity Manipulation – Canonical v1.0

Candidate-versioner får inte kunna exekveras live.

Detection Version Lock

Alla strategy-critical detection rules måste också vara versionsbundna.

Det gäller särskilt:

- liquidity detection
- FVG
- swing detection
- iFVG

- CISD
- SMT

Live ska använda exakt samma detectionversion som den version som passerat validation.

Risk Profile Lock

Det livekonto som aktiveras ska ha en specifik:

RiskProfileVersion

Det får inte använda:

latest

som dynamisk pointer.

En profiländring ska kräva separat activation.

Prop Firm Profile Lock

Om account är prop firm-baserat ska det också bindas till en verifierad:

PropFirmProfileVersion

Profilen ska baseras på aktuell dokumentation för det faktiska programmet.

Prop Rule Verification före första live-trade

Innan ett prop account aktiveras ska användaren manuellt verifiera:

- provider
- program
- account size
- drawdownmodell
- daily loss-regel
- reset policy
- position limits
- news rules
- holding rules
- relevant network/VPN/VPS-regler

Datum för verifiering ska sparas.

Ingen Rule Guessing

Om en prop firm-regel är oklar:

UNKNOWN

ska inte tolkas som:

ALLOWED

Regeln måste verifieras innan live automation.

Live Deployment Sign-Off

Innan live capability aktiveras ska en formell sign-off genomföras.

Den ska minst kontrollera:

Strategy

- canonical version
- backtest PASS
- forward evidence
- demo evidence

Risk

- active RiskProfile
- daily stop
- position sizing
- max attempts
- kill switches

Prop Firm

- active ruleset
- rule source
- verified date

Execution

- runner healthy
- providern verified
- account verified
- symbol mapping verified
- idempotency verified
- reconciliation verified

Security

- credentials protected
- environment separation
- allowlists
- auth
- audit

Live Release Candidate

Den kod och configuration som ska användas live bör först behandlas som en release candidate.

Exempel:

Omnira Trading Runtime – Live RC1

RC1 ska motsvara den version som slutligen kördes i Golden Demo Run.

No Last-Minute Changes

Efter Golden Demo Run bör inga materiella strategy-, risk- eller executionförändringar göras utan ny relevant verification.

Annars är det inte samma system som testades.

Deployment Freeze Window

Inför första liveperioden bör critical configuration frysas.

Det innebär att vi inte samtidigt:

- byter strategy
- ändrar risk
- byter market provider
- byter runner
- ändrar providern integration

om det inte är absolut nödvändigt.

För många samtidiga förändringar gör root-cause analysis svårare.

Initial Live Risk

Den första livefasen ska använda mycket liten kapitalrisk.

Målet är inte att maximera profit.

Målet är att bevisa att:

Systemets demoresultat och tekniska beteende överförs till verklig execution.

Risk ska därför hållas på den lägsta praktiskt användbara nivån som fortfarande låter strategin exekveras korrekt.

Technical SL ska inte ändras

Även när live-risken reduceras ska position size anpassas.

Technical SL ska fortfarande bestämmas av strategy structure.

Om minsta kontrakt ger för hög risk:

NO TRADE

Live Manual Approval

Den första liveexecutionen ska kräva explicit human approval.

Användaren ska se:

- LIVE
- account
- instrument
- direction

- quantity
- planned entry
- SL
- TP
- initial R:R
- risk \$
- risk %
- daily risk
- prop headroom
- strategy version
- setup grade

Live UI måste vara omisskännligt

LIVE-mode ska vara visuellt mycket tydligt.

Det ska inte vara möjligt att enkelt tro att användaren befinner sig i demo.

First Live Trade

Den första live-traden ska behandlas som ett systemtest.

Vi vill verifiera:

- proposal
- approval
- execution intent
- runner receipt
- order submission
- actual fill
- SL
- TP
- journal
- position sync

Profit eller loss är sekundärt.

First Live Fill Review

Efter första fill ska systemet direkt jämföra:

Planned Entry

mot:

Actual Fill

och:

Planned Risk

mot:

Actual Risk

Detta visar om demo execution assumptions var realistiska.

Broker-Native SL Verification

Ingen liveposition ska betraktas som korrekt skyddad innan broker-native SL verifierats.

Om SL inte kan verifieras:

CRITICAL LIVE INCIDENT

Initial Live Monitoring

De första live-trades ska övervakas extra noggrant.

Vi ska följa:

- fill latency
- slippage
- commissions
- SL
- TP
- break-even
- trade close
- reconciliation

Detta skapar live execution baseline.

Manual Approval och 1m Strategy

Eftersom strategy entry sker på 1m kan manual approval försämra execution.

Detta ska mätas.

Om live manual approval regelbundet leder till:

- missed entries
- R:R under 2
- hög slippage

ska systemet inte kringgå regeln.

Det ska istället skapa evidens för varför Controlled Live Automation senare kan behövas.

No Forced Entry after Approval Delay

Om priset har rört sig för långt när user approve:ar ska Pre-Execution Revalidation stoppa traden.

Live FOMO får aldrig överstyra strategy rules.

Live Risk Engine

Risk Engine ska använda riktig account state.

Canonical baseline:

- \$150 max risk/trade
- \$450 realized daily loss
- max 1 open position
- max 3 attempts per thesis

Den faktiska Controlled Live Risk Profile kan senare sättas lägre under första livevalidationen genom explicit versionerad profil.

Lägre Live Risk är tillåtet

Canonical strategy regler behöver inte ändras för att vi använder lägre live risk.

RiskProfile är separat från StrategyVersion.

Detta är viktigt.

Ingen automatisk Risk Increase

Om första tio trades går bra får Atlas inte höja risk.

Riskökning kräver en separat scaling gate.

Daily Stop i Live

Daily stop måste vara aktiv från första live-traden.

När canonical aktiva RiskProfile gräns nås:

- inga nya trades
- öppen position hanteras enligt riskpolicy
- status låses
- audit event skapas

Kill Switch före första trade

Före första liveexecutionen ska användaren själv ha testat var:

Global Kill

och relevant:

Account Kill

finns och fungerar.

Man ska inte behöva lära sig detta under en incident.

Human Emergency Procedure

En enkel första incidentprocedur ska finnas:

- Stoppa nya trades.
- Kontrollera faktisk providern-position.
- Kontrollera broker-native SL.
- Använd Emergency Close endast när det verkligen behövs.

- Reconcile systemet.
- Dokumentera incidenten.

Live providern Reconciliation

Broker state är source of truth för actual exposure.

Live-systemet ska kontinuerligt jämföra:

- account
- orders
- positions
- deals

mot Omniras expected state.

Reconciliation Mismatch

Vid mismatch:

LIVE EXECUTION BLOCKED

tills problemet är löst.

Manual External Position

Under early live bör manuella positions helst undvikas på samma account.

Om de ändå uppstår måste Omnira upptäcka dem och inkludera exposure i risk.

One System per Account

Under första livevalidationen är det säkrast om samma konto inte används samtidigt av flera tradingbots.

Det reducerar ambiguity.

Account State Freshness

Risk och Prop checks måste baseras på färsk live account data.

En gammal account snapshot får inte användas för ny execution.

Prop Firm Headroom

För prop account ska UI visa aktuell headroom.

Exempel:

Daily Loss Headroom

Maximum Loss Headroom

Position Limit

Rule Status

Detta ska vara synligt före approval.

Prop Firm Safety Buffer

När relevant kan en separat operational buffer användas ovanför firmans absoluta limit.

Exakt värde ska:

- definieras
- testas
- versioneras

inte improviseras under live.

Live News Data

News source måste vara healthy.

Om relevant news state är:

UNKNOWN

ska:

NO NEW LIVE TRADE

Live News Exit

T-15m news exit ska först ha testats på demo.

I live ska trigger och execution loggas noggrant.

News Exit Outage

Om systemet förlorar connection nära T-15m finns residual risk.

Detta är en av anledningarna till att liveövervakning och framtida VPS kan vara viktiga.

Live Break-Even

BE modification ska verifieras mot broker state på samma sätt som i demo.

Det ska inte räcka att systemet skickade requesten.

Live Time Exit

New York max fyra timmar ska följas och verifieras.

Live Slippage

Actual live slippage ska analyseras separat från demo.

Demo slippage kan vara mer idealiserad än live.

Live Fees

Actual commissions och fees ska användas i performance analytics.

Actual Risk Overrun

Om actual fill gör att trade riskerar mer än planerat ska överrun mätas.

Exempel:

Planned = 1.00R

Actual = 1.07R

Detta ska ingå i execution quality.

Critical Risk Overrun

Om slippage skapar en betydande breach av godkänd risk ska detta bli Risk/Execution Incident.

Exakt policy definieras innan liveautomation.

Live Manual Phase Duration

Live Manual ska inte avslutas efter ett fåtal trades bara för att de fungerade.

Det krävs relevant:

- trade sample
- execution sample
- market regime coverage
- incident-free drift

Exakta promotion thresholds definieras i scaling/gate-kapitlen.

Strategy Performance i Live

Vi ska jämföra:

- Backtest
- Forward
- Demo
- Live

Det viktiga är om live beter sig rimligt nära den tidigare evidence-profilen.

Technical Performance i Live

Parallellt mäts:

- runner uptime
- providern availability
- slippage
- order rejection
- reconciliation incidents

- SL/TP incidents
- latency

Live Performance Decay

En viss försämring från backtest/demo kan vara normal.

Men systemet ska mäta den.

Exempel:

Backtest expectancy = +0.30R

Demo = +0.25R

Live = +0.17R

Då behöver vi förstå varför.

Controlled Live Automation

Först när Live Manual har tillräcklig teknisk och tradingmässig evidens får Controlled Live Automation övervägas.

Det innebär att human approval per trade tas bort.

Controlled betyder begränsad

Controlled Live Automation ska inte betyda:

Atlas får göra vad den vill.

Automation ska vara begränsad till explicit:

- account
- strategy
- strategy version
- instrument
- risk profile
- prop profile
- session
- execution policy

Automation Policy

Ett exempel kan vara:

CONTROLLED_LIVE_AUTO-v1

som endast tillåter:

- specific account
- MNQ
- Omnira Liquidity Manipulation v1.0
- specific RiskProfile
- specific PropFirmProfile
- canonical sessions

No Cross-Account Autonomy

Att ett account godkänts för automation innebär inte att andra accounts automatiskt får samma permission.

Autonomy är account-scoped.

No Strategy Autonomy Inheritance

När framtida Strategy v2 skapas får den inte automatiskt ärva liveautomation från v1.

Ny strategy kräver egen validation.

Automation Kill Switch

Controlled Live Auto ska alltid kunna stoppas omedelbart.

En user action ska kunna sänka mode till exempelvis:

```
LIVE_MANUAL
```

eller:

```
READ_ONLY
```

Automatic Safety Downgrade

Critical events ska kunna sänka autonomy.

Exempel:

```
CONTROLLED_LIVE_AUTO  
→  
READ_ONLY
```

vid:

- reconciliation failure
- account mismatch
- repeated execution incident
- unknown prop state

Ingen automatisk Re-enable

Efter safety downgrade ska systemet inte själv återgå till full automation bara för att connection återkommer.

Verifiering krävs.

Live Incident Policy

En zero-tolerance-incident ska kunna stoppa automation.

Exempel:

- unintended duplicate order
- wrong account

- wrong quantity
- unverified SL
- unknown position efter execution

Incident-free Window

Promotion och fortsatt automation ska kräva stabil drift.

Det räcker inte att ett problem fixats fem minuter tidigare.

Rollback

Om en ny live runtime-version orsakar problem ska systemet kunna:

- stoppa execution
- återgå till verifierad version där möjligt
- reconcila
- verifiera account

Rollback är inte klar förrän broker state är korrekt.

Live Deployment under Open Position

Undvik onödiga deployment changes med öppen position.

Om en kritisk fix måste deployas krävs särskild procedure och position awareness.

Monitoring

Live operation ska ha en tydlig health dashboard.

Minst:

- Omnira backend
- market data
- news data
- Risk Engine
- Prop Engine
- runner
- providern
- broker connection
- reconciliation
- kill switches

Alerts

Kritiska live alerts ska prioriteras.

Exempel:

CRITICAL

Live MNQ-position saknar verifierad broker-native SL.

Detta ska inte drunkna bland vanliga tradingnotiser.

Live Daily Review

I början ska varje live tradingdag kunna reviewas.

Atlas kan sammanfatta:

- setups
- proposals
- approvals
- trades
- risk
- execution
- incidents
- discrepancies

First 10 / 25 / 50 Trade Reviews

Early live kan ha särskilda review checkpoints.

Exempel:

- efter 10 trades
- efter 25
- efter 50

Dessa siffror är reviewpunkter, inte ännu färdiga statistical promotion gates.

Vid varje checkpoint granskas både:

- strategy behavior
- technical behavior

Learning Layer i Live

Atlas Trading Learning & Improvement Layer får analysera live data från första dagen.

Det får skapa:

- findings
- hypotheses
- candidate versions

Men får fortfarande inte ändra live canonical rules.

Live Findings

Exempel:

Actual live slippage är systematiskt högre mellan 10:00–10:05 än i demo.

Det kan skapa en execution research hypothesis.

No Live Experimentation

Production account ska inte användas för okontrollerade experiment.

Candidate strategy/risk/execution changes ska testas utanför live först.

A/B Testing

Om framtida A/B testing används får den inte introduceras utan egen governance och riskmodell.
Det är inte del av initial live deployment.

Live Data som högsta evidensnivå

Live performance är viktig evidens eftersom den inkluderar:

- verkliga fills
- verkliga fees
- verklig infrastructure
- verkliga prop rules

Men även live sample kan vara litet eller regime-specific.

Det är inte automatiskt sanningen om framtiden.

Psychological Separation

Systemet ska minska risken att kortsiktiga live outcomes leder till emotionella regeländringar.

Exempel:

Fem losses i rad ska inte leda till:

Höj risk för att vinna tillbaka.

Canonical governance finns just för detta.

Manual Override

Människan ska kunna:

- stoppa trading
- sänka autonomy
- emergency close

Men ska inte ha en enkel knapp:

IGNORE RISK DENY

Safety-regler måste behålla sin integritet även live.

Prop Challenge Discipline

Om account är challenge-baserat ska systemet inte jaga target.

En nästan klar challenge ska fortfarande följa:

- Strategy
- Risk
- Prop rules

Funded Account Discipline

När en challenge passerats är funded inte ett läge där riskregler kan lättas automatiskt.

Det nya accountet får sin egen verifierade profile och deployment gate.

Payout State

När relevant ska payout-cycle och funded-specific rules hanteras av Prop Firm Engine.

Trading Strategy ska inte ändras för att en payout närmar sig om ingen versionsstyrd policy säger det.

Live Security

Live runnerna ska vara hårdare skyddad än demo.

Minst:

- least privilege
- secrets
- auth
- allowlist
- environment lock
- network security
- auditing

Dedicated Execution Host

När riktig kapitalrisk finns blir det ännu viktigare att execution-host är stabil och kontrollerad.

Initialt kan Windows-riggen användas.

Men systemet ska kontinuerligt mäta:

- uptime
- outages
- latency

för att avgöra när VPS blir motiverat.

VPS Migration Gate

Flytt till VPS ska betraktas som deploymentförändring.

Det kräver:

- environment verification
- broker/prop policy check
- providern verification
- runner verification
- read-only test
- demo eller controlled verification där relevant

No Architecture Rewrite for VPS

Strategy, Risk, Prop, Journal och Atlas ska inte behöva ändras.

Endast execution host/deployment layer ska flyttas.

Disaster Recovery

Live-deployment ska ha en plan för att återställa:

- runner
- providern
- account binding
- configurations
- database
- audit

Efter restore:

READ ONLY

tills reconciliation passerat.

Backup är inte Live State

En backup kan vara gammal.

Broker state måste alltid läsas på nytt efter restore.

Live Deployment Checklist

Inför varje ny live activation ska checklistan minst omfatta:

- correct code release
- correct strategy version
- correct RiskProfile
- correct PropFirmProfile
- correct account
- correct environment
- correct symbol mapping
- correct runner
- healthy market data
- healthy news
- reconciliation PASS
- kill switches OFF men verifierade
- audit active
- no unresolved critical incident

Go / No-Go

Slutbeslutet är:

GO

eller:

NO-GO

Om någon safety-critical gate är osäker:

NO-GO

Live Does Not Mean Done

Efter live start fortsätter utvecklingen.

Systemet går från:

Build

till:

Operate + Learn + Improve

Det innebär:

- monitoring
- analytics
- incident review
- research
- controlled evolution

Controlled Evolution

Live strategy ska förändras långsamt och genom versioner.

Exempel:

Canonical v1.0

```
→ data
→ candidate v1.1
→ backtest
→ OOS
→ forward
→ review
→ canonical v1.1
→ separat live deployment
```

Live Roll Forward

En ny canonical version ska behandlas som en ny deployment candidate.

Den är inte säker bara för att föregående version var det.

Scaling är separat

Att systemet är live-ready betyder inte att risk ska maximeras.

Riskuppskalning är nästa separata problem.

Det behandlas i kapitlet:

Kriterier för uppskalning

Live Deployment Principle

Den viktigaste principen är:

Första live-traden ska vara början på en ny testfas, inte slutet på utvecklingen.

Systemet ska först bevisa:

- correct real execution
- correct real risk
- correct real prop compliance
- stable infrastructure
- consistent strategy behavior

med minimal praktisk risk.

Först när detta håller över tid får större autonomi och större kapital övervägas.

Kapitelstatus

Kapitel: 18 – Live-deployment

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Första live mode: READ ONLY

Första execution mode: LIVE MANUAL APPROVAL

Full live automation: Inte tillåten initialt

Controlled Live Automation: Kräver separat gate

Initial kapitalrisk: Ska hållas mycket låg

Canonical StrategyVersion: Explicit låst

RiskProfile: Explicit låst

PropFirmProfile: Explicit låst där relevant

Broker-native SL verification: Obligatorisk

Kill switch: Obligatorisk och verifierad före live

Reconciliation: Obligatorisk

Automatic autonomy increase: Förbjuden

Live self-modification: Förbjuden

Live data: Används för fortsatt validation och learning

Live-deployment ska genomföras som en kontrollerad och reversibel övergång från bevisad demo-drift till minimal verklig kapitalrisk.

KAPITEL 19

Kriterier för uppskalning

Uppskalning i Omnira Trading System betyder att systemet får hantera större ekonomisk exponering eller högre grad av autonomi.

Det kan exempelvis innebära:

- högre risk per trade
- större account size
- fler accounts
- fler prop firm-konton
- fler tillåtna instruments
- fler StrategyVersions
- övergång från Live Manual till Controlled Live Automation

Uppskalning får aldrig ske enbart därför att systemet har haft en bra vecka eller några stora vinnare.

Den centrala principen är:

Risk och autonomi ska öka först efter dokumenterad evidens, inte efter kortsiktig framgång.

Två typer av uppskalning

Omnira ska skilja mellan två huvudsakliga dimensioner.

Kapitaluppskalning

Det innebär att systemet får riskera mer kapital.

Exempel:

- större risk per trade
- större funded account
- fler konton
- större sammanlagd exposure

Autonomiuppskalning

Det innebär att systemet får fatta och genomföra fler beslut utan mänsklig approval.

Exempel:

LIVE MANUAL

→

CONTROLLED LIVE AUTO

Dessa två dimensioner ska inte automatiskt följa varandra.

Högre autonomi betyder inte högre risk

Det ska vara möjligt att köra:

CONTROLLED LIVE AUTO

med mycket låg risk.

Det ska också vara möjligt att köra:

LIVE MANUAL

på ett större account.

Autonomy och capital risk är separata controls.

Uppskalning är en promotion

Varje scale-up ska behandlas som en promotion mellan definierade system states.

Exempel:

```
RISK_TIER_1  
→  
RISK_TIER_2
```

eller:

```
LIVE_MANUAL  
→  
CONTROLLED_LIVE_AUTO
```

Promotion kräver PASS på definierade gates.

Ingen implicit scaling

Systemet får aldrig använda logik såsom:

Kontot är upp 10 %, därför dubblar vi risken.

Riskökning kräver explicit ny RiskProfile version.

RiskProfile-versionering

Exempel:

LIVE-RISK-v1

```
max_risk_per_trade = $50
```

senare:

LIVE-RISK-v2

```
max_risk_per_trade = $75
```

eller annan framtida nivå.

Varje förändring ska:

- dokumenteras

- motiveras
- granskas
- aktiveras explicit

Canonical Risk är inte samma som Initial Live Risk

Den interna strategy/risk-baslinjen kan vara:

\$150 per trade

men första live deployment kan använda lägre risk.

Detta är inte en StrategyVersion-förändring.

Det är en RiskProfile-fråga.

Varför börja mindre

Den första liveperioden har osäkerheter som demo inte fullt ut kan simulera.

Exempel:

- live slippage
- fees
- broker behavior
- prop firm enforcement
- nätverk
- execution timing
- verkliga account states

Mindre initial risk begränsar kostnaden om någon antagelse visar sig vara fel.

Första scaling-gaten

Innan risk får ökas från den första live-nivån ska systemet minst ha bevisat:

- stabil live execution
- korrekt risk
- korrekt SL/TP
- korrekt reconciliation
- inga zero-tolerance incidents
- relevant trade sample
- acceptabel performance
- acceptabel drawdown
- stabilitet över tid

Profit ensam är inte en gate

Exempel:

Systemet är:

+15R

men har haft:

- en duplicate order
- två SL-verification failures
- en wrong-account incident

Resultat:

NO SCALE

Safety dominerar profit.

Zero-Tolerance Incidents

Vissa incidenter ska blockera uppskalning oavsett P/L.

Exempel:

- wrong account execution
- unintended duplicate order
- wrong quantity
- trade i fel environment
- execution utan giltigt Risk PASS
- execution utan giltigt Prop PASS
- unprotected live position orsakad av systemet
- bypassad kill switch

Incident-Free Window

Efter en critical incident ska systemet behöva bevisa stabil drift igen.

Ett problem som fixades igår är inte automatiskt tillräckligt för scale-up idag.

Exakt incident-free window definieras senare.

Performance Gate

Performance ska bedömas med flera metrics.

Minst:

- trade count
- expectancy
- Profit Factor
- maximum drawdown
- losing streak
- win/loss/BE distribution
- stability över tid
- execution costs

Expectancy

Risk får inte ökas om actual live expectancy är tydligt negativ.

Ett positivt account-resultat kan ändå vara slump om sample är litet.

Sample Size

Varje scalingbeslut ska visa antal live trades.

Exempel:

+8R efter 7 trades

är mycket svag evidens.

Det ska inte behandlas som robust edge.

Minimum Sample är Strategy-Specific

Det finns ingen universell siffra som automatiskt gör en strategi statistiskt bevisad.

Trade frequency, return distribution och market regime spelar roll.

Därför ska vi definiera scaling thresholds först när vi har faktisk Strategy v1.0-data.

Time Requirement

Sample size ensam räcker inte heller.

Exempel:

200 trades under samma mycket speciella marknadsperiod kan ge mindre robust evidens än ett mer varierat sample över längre tid.

Därför ska både:

- antal trades
- elapsed time

vägas in.

Regime Coverage

Scale-up ska helst bygga på performance över flera marknadsförhållanden.

Exempel:

- trend
- range
- high volatility
- low volatility

Systemet behöver inte vara lika bra i alla regimes.

Men vi ska förstå dess svagheter.

Session Coverage

Eftersom Strategy v1.0 använder:

- London
- New York

ska båda sessionerna ingå i scaling review.

Om nästan all live-data kommer från en session ska detta markeras.

Grade Coverage

A+, A, B och C ska analyseras.

Om exempelvis C grade visar negativ live expectancy ska detta vara känt innan risk ökas.

Det behöver inte automatiskt förändra Canonical v1.0.

Det kan däremot blockera scale-up eller skapa candidate research.

Attempt Coverage

Attempt 1, 2 och 3 ska analyseras separat.

Detta är viktigt eftersom senare attempts kan påverka drawdown oproportionerligt.

Long och Short

Båda riktningarna ska granskas.

Om live edge endast finns på long ska vi inte låtsas att hela strategy är lika bevisad i båda riktningar.

Backtest-to-Live Consistency

Scale-up kräver inte identiska resultat mellan backtest och live.

Men behavior ska vara rimligt kompatibelt.

Exempel:

Backtest:

+0.30R expectancy

Demo:

+0.25R

Live:

+0.21R

kan vara logiskt.

Men:

Backtest:

+0.30R

Live:

-0.24R

kräver utredning innan scaling.

Forward-to-Live Consistency

Samma jämförelse ska göras mellan forward/demo och live.

Execution Quality Gate

Scale-up ska endast ske om execution är stabil.

Metrics kan inkludera:

- average slippage
- worst slippage
- order rejection
- latency
- missed executions
- modification failures

Slippage Stability

Om större quantity påverkar fills kan tidigare execution data inte automatiskt extrapoleras.

Detta blir viktigare när systemet skalar upp i kontrakt.

Capacity

Strategin kan ha en praktisk execution capacity.

För MNQ är små positioner enkla.

Vid större size eller flera accounts kan:

- liquidity
- slippage
- fill behavior

förändras.

Scale-up ska därför ske stegvis.

One Variable at a Time

När möjligt bör vi inte samtidigt:

- dubbla risk
- byta VPS
- byta strategy
- byta broker
- aktivera automation

Det gör det svårt att förstå vad som orsakade en förändring.

Controlled Scaling Steps

Scale-up bör ske i små steg.

Exempel konceptuellt:

Tier 1

→ validation

Tier 2

→ validation

Tier 3

→ validation

inte:

tiny

→

maximum

Risk Tier

Varje Risk Tier ska ha explicit definition.

Exempel:

- risk per trade
- account scope
- max exposure
- environment
- effective date

Scaling Hold Period

Efter riskökning ska den nya nivån köras tillräckligt länge för att verifiera att behavior fortfarande är stabilt.

Under denna tid ska ingen ytterligare riskökning ske.

Scale-Up behöver nytt baslinjedataset

När risk eller execution size förändras blir den nya nivån en egen operational baseline.

Det gäller särskilt om quantity ökar.

Scale-Down

Uppskalning måste alltid vara reversibel.

Systemet ska kunna:

- sänka risk
- sänka autonomy
- blockera konto
- återgå till tidigare RiskProfile

Automatic Scale-Down

Det kan senare vara tillåtet för safety policy att automatiskt sänka risk/autonomy vid definierade conditions.

Exempel:

- technical incidents
- performance degradation
- prop headroom stress

Men en sådan policy måste vara explicit definierad och testad.

Ingen automatisk Scale-Up

Detta är en viktig asymmetri.

Automatiskt:

HIGHER → LOWER

kan senare tillåtas.

Automatiskt:

LOWER → HIGHER

ska inte ske utan governance.

Drawdown Gate

Scale-up ska ta hänsyn till actual drawdown.

Exempel:

En strategy kan ha god expectancy men mycket större live drawdown än historiskt.

Det kan vara tecken på:

- regime shift
- execution degradation
- underestimated variance

Risk ska då inte höjas.

Drawdown Relative to Historical Distribution

Vi ska inte endast fråga:

Är drawdown under en godtycklig procent?

Vi ska fråga:

Ligger current drawdown inom vad Strategy v1.0 historiskt har visat som rimligt?

Drawdown Stress Testing

Innan risk ökas ska systemet kunna simulera hur den nya risken påverkar tidigare drawdowns.

Exempel:

Historisk max DD:

-10R

Vid:

\$150/R

motsvarar detta:

-\$1,500

Om account/prop limits inte klarar motsvarande stress finns ingen scale rationale.

Monte Carlo före Riskökning

När sample size är tillräckligt kan Monte Carlo-liknande simulation användas för att uppskatta distributionen av:

- drawdown
- losing streak
- challenge failure

under föreslagen nya risknivå.

Risk of Ruin / Breach

Vi ska särskilt uppskatta risken att:

- internal limits
- prop maximum loss
- trailing drawdown

nås under realistiska trade sequences.

Prop Firm Headroom

Prop firm-scaling måste ta hänsyn till reglerna för varje account.

Större account betyder inte automatiskt proportionellt större säker risk.

Prop Firm Account Scaling

När en challenge är passerad kan nästa scalingsteg vara:

- större account
- ytterligare account
- nytt funded program

Varje nytt account ska ha egen deployment och ruleset verification.

Flera Prop Accounts

Om Omnira senare hanterar flera prop accounts uppstår portfolio-level risk.

Exempel:

Samma MNQ trade kan exekveras på fem accounts samtidigt.

Det innebär större sammanlagd ekonomisk exposure även om varje konto följer sin egen limit.

Global Exposure Layer

Vid multi-account behöver Risk Engine därför senare stödja global exposure.

Exempel:

Total live risk across accounts

Detta blir ett nytt scalingkrav.

Copy Trading

Om samma Trade Proposal används på flera accounts ska execution fortfarande vara account-specifik.

Varje konto kan ha:

- olika headroom
- olika quantity
- olika prop rules

Ett account kan PASS medan ett annat DENY.

No All-or-Nothing Multi-Account Assumption

Om fem accounts är kopplade och två inte får ta traden ska de två avstå.

Systemet får inte kringgå regler för att hålla accounts synkroniserade.

Correlated Risk

Flera accounts som tar samma MNQ trade representerar i praktiken korrelerad risk.

Det ska behandlas som sådan.

Multiple Strategies

När framtida Strategy #2 introduceras ska scale-up även ta hänsyn till korrelation mellan strategies.

Två strategies kan se diversifierade ut men ändå förlora samtidigt i samma market regime.

Portfolio Risk

Det framtida Risk Engine-lagret ska därför kunna hantera:

- account risk
- strategy risk
- instrument risk
- portfolio risk

Men initial Strategy #1 kan börja enklare.

Autonomy Scaling

Separat från kapitalrisk ska systemet kunna gå:

LIVE MANUAL

→

CONTROLLED LIVE AUTO

Detta kräver sin egen promotion gate.

Controlled Live Automation Gate

Innan human approval per trade tas bort ska systemet minst ha visat:

- stabil live Strategy behavior
- stabil risk

- stabil Prop Engine
- stabil execution
- inga zero-tolerance incidents
- komplett audit
- fungerande alerts
- fungerande kill switches
- fungerande auto-downgrade
- relevant live sample

Manual Approval Performance

Om manual approval fungerar bra finns inget krav att ta bort den.

Automation ska ge tydlig nytta.

Exempel:

- minskad latency
- färre missed trades
- mer konsekvent execution

utan att safety försämrats.

Automation Benefit Analysis

Vi ska kunna jämföra:

Live Manual

mot:

simulated/observed automatic execution

och fråga:

- hur mycket latency sparas?
- hur mycket R:R förbättras?
- hur många trades fångas?
- ökar technical risk?

Automation är inte en belöning

Controlled Live Auto ska inte aktiveras som en medalj för bra performance.

Det ska aktiveras när det är den mest rationella executionmodellen.

Automation Scope

Första Controlled Live Auto bör ha smal scope.

Exempel:

- ett account
- ett instrument

- en strategy
- en RiskProfile
- ett begränsat operation mode

No Immediate Multi-Account Auto

När första account klarar automation ska vi inte automatiskt ge alla framtida accounts samma permission.

Autonomy Rollback

Vid problem ska systemet snabbt kunna gå:

CONTROLLED LIVE AUTO

→

LIVE MANUAL

eller:

READ_ONLY

Performance Degradation Gate

Om rolling performance tydligt försämras ska scale-up pausas.

Beroende på framtida policy kan automation också pausas.

Statistical Thresholds

Exakta numeric thresholds för scaling ska inte hittas på innan systemet har verklig data.

Exempel på framtida thresholds kan beröra:

- minimum trade count
- minimum forward/live duration
- positive expectancy
- max tolerated drawdown
- incident-free window

Men dessa ska kalibreras mot Strategy v1.0.

Varför vi inte låser siffrorna nu

Att idag säga:

Risk får öka efter exakt 100 trades.

skulle ge en falsk precision.

Om strategin endast tar några trades per vecka eller har hög variance kan 100 vara otillräckligt.

Om den producerar mycket fler oberoende observations kan andra kriterier vara mer relevanta.

Evidence Scorecard

Varje framtida scale proposal bör ha ett scorecard.

Exempel:

Strategy

- expectancy
- PF
- drawdown
- sample
- regime stability

Execution

- slippage
- latency
- incidents
- uptime

Risk

- daily stop behavior
- headroom
- risk overruns

Prop

- breach simulations
- actual compliance

Safety

- zero-tolerance incidents
- reconciliation
- kill switch

Scale Proposal

Riskökning ska behandlas som ett objekt.

Exempel:

SCALE-PROP-001

Det ska innehålla:

- current tier
- proposed tier
- reason
- evidence
- expected effect

- stress test
- approval

Atlas Roll i Scaling

Atlas ska kunna förbereda analysen.

Exempel:

De senaste 184 live trades har expectancy $+0.24R$, inga critical execution incidents har inträffat under 73 dagar och observed max drawdown ligger inom historical simulation range. Candidate scale from Risk Tier 1 to Tier 2 kan därför granskas.

Atlas får föreslå.

Atlas får inte aktivera.

Atlas ska även argumentera emot

Atlas ska aktivt söka motbevis.

Exempel:

Total performance är positiv men 61 % av vinsten kommer från två extrema trades och New York expectancy är negativ. Jag rekommenderar inte scaling ännu.

Detta är en viktig del av beslutsstödet.

Self-Improvement och Scaling

Learning Layer får hitta förbättringar parallellt.

Men risk ska inte höjas samtidigt som en oprövad candidate strategy introduceras.

Vi ska helst veta vad vi skalar.

Stable Canonical Period

Scale-up bör baseras på en stabil canonical version.

Om strategy rules precis har ändrats behöver den nya versionen egen evidens.

Version Reset

Ny major strategy version kan innebära att vissa scaling assumptions måste omvärderas.

Autonomy och risk permission ska inte automatiskt ärvas.

Risk Scaling och Edge Decay

Även en historiskt stark strategy kan tappa edge.

Därför ska scaling aldrig bli permanent i betydelsen:

När vi nått \$X risk går vi aldrig ner.

Riskenivå ska kunna omprövas.

Periodic Scale Review

Även utan incident ska systemet regelbundet reviewa om nuvarande risk fortfarande är motiverad.

Scaling och Withdrawal/Payout

Prop firm payout får inte automatiskt skapa högre risk.

Ekonomisk framgång ska inte förändra RiskProfile utan separat beslut.

Scaling och Profit Cushion

Ett större profit cushion kan förändra prop firm headroom.

Det kan göra högre risk tekniskt möjlig.

Men det är inte samma sak som att högre risk är statistiskt motiverad.

No House Money Effect

Systemet ska undvika resonemanget:

Vi handlar bara med vinsten nu.

Kapital som ligger på account är fortfarande kapital som riskeras.

Scaling och Psychology

En systematisk tradingmotor ska minska psykologisk risk.

Det innebär bland annat att inte:

- öka risk efter vinnare av eufori
- öka risk efter förlust för recovery
- minska strategy discipline nära challenge target

Scaling efter Losing Streak

Risk ska inte automatiskt höjas för att vinna tillbaka.

En längre losing streak kan snarare trigga:

REVIEW

Scaling efter Winning Streak

Samma sak åt andra hållet.

Winning streak är inte automatiskt bevis på förbättrad edge.

Scaling och Technical Capacity

När quantity blir större ska vi kontrollera om systemets execution fortfarande fungerar lika bra.

Exempel:

- fills
- slippage
- partial fills

En ny quantity tier kan därför kräva egen demo/live validation.

Scale Test

Innan full riskökning kan ett litet intermediate tier användas.

Det fungerar som controlled experiment.

Multi-Account Rollout

När flera accounts senare används bör rollout ske ett account i taget eller i små grupper.

Det minskar blast radius.

Global Kill Switch blir ännu viktigare

När systemet skalar över flera accounts måste användaren snabbt kunna stoppa all ny execution.

Portfolio Dashboard

Omnira ska senare kunna visa:

- total live exposure
- total risk
- accounts
- prop headroom
- strategies
- correlated exposure

Detta blir viktigt vid större scale.

Scale-Down Triggers

Framtida explicit policy kan inkludera triggers som:

- critical incident
- abnormal drawdown
- persistent negative rolling expectancy
- prop rule changes
- execution degradation

Dessa kan resultera i:

- risk reduction
- automation suspension
- read-only

Scale Freeze

Vid osäkerhet kan systemet sätta:

```
SCALING_FROZEN
```

Trading kan eventuellt fortsätta på nuvarande nivå.

Men ingen ytterligare riskökning får ske.

Freeze är inte Kill

Detta skiljer sig från kill switch.

SCALING_FROZEN

betyder:

Nuvarande godkända risk fortsätter, men ingen promotion till högre risk får ske.

Governance Review

Varje större scale-up bör avslutas med explicit:

APPROVE

eller:

REJECT

eller:

DEFER

DEFER

DEFER betyder exempelvis:

Resultaten är lovande men sample size är ännu för liten.

Det är ett fullt legitimt beslut.

Documentation

Varje scale-up ska dokumenteras i:

- audit
- RiskProfile
- performance report
- change history

Rollback Criteria

När scale-up godkänns ska vi också definiera vad som skulle få oss att backa.

Detta förhindrar att vi bara tänker på uppsidan.

Scaling History

Systemet ska kunna visa:

Tier 1

startdatum

```
→ result
→ promotion
```

Tier 2

startdatum

```
→ result
```

och så vidare.

Atlas Market View

I UI ska aktuell risk/autonomy tier vara tydlig.

Exempel:

Live Mode: Controlled Auto

Risk Tier: 2

Risk per Trade: \$X

Scale Status: VALIDATING

Next Tier: LOCKED

No Hidden Scaling

Användaren ska aldrig upptäcka i efterhand att systemet ökat risk.

Alla ändringar ska vara explicita och synliga.

Scale Governance och Self-Improvement

Atlas kan säga:

Data stödjer att vi testar högre risk.

Men Risk Scaling är inte en learning output som automatiskt går till production.

Det följer samma princip som strategy evolution:

```
Observe
→ Measure
→ Propose
→ Test
→ Review
→ Approve
```

Långsiktigt mål

Målet är inte maximalt riskutnyttjande.

Målet är att hitta en nivå där:

- edge är bevisad
- drawdown är acceptabel
- prop constraints respekteras

- infrastructure är stabil
- risk of failure är rimlig
- systemet går att övervaka

Skalning och företagsekonomi

När systemet senare hanterar verklig kapitalallokering ska beslut också kunna ta hänsyn till:

- prop challenge fees
- payouts
- account replacement cost
- infrastructure cost
- expected return
- failure probability

Det gör trading-systemet till ett kapitalallokeringssystem, inte bara en strategy bot.

Scale Efficiency

Atlas ska kunna mäta om högre risk faktiskt ger proportionell förbättring i net outcome.

Om större execution size skapar betydligt högre slippage kan scaling efficiency sjunka.

Optimal är inte Maximum

Den högsta möjliga risknivån behöver inte vara den bästa.

Systemets mål är robust långsiktig expectancy med kontrollerad downside.

Scaling Principle

Den viktigaste principen är:

Risk ska förtjänas genom evidens.

Varje högre risknivå innebär att samma systemfel eller losing streak kostar mer.

Därför ska högre risk kräva starkare bevis än lägre risk.

Kapitelstatus

Kapitel: 19 – Kriterier för uppskalning

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Scaling dimensions: Kapital + autonomi

Automatic risk increase: Förbjuden

Automatic autonomy increase: Förbjuden

Scale-down: Ska vara möjlig

Safety incidents: Kan blockera scaling oavsett profit

Performance gates: Multi-metric

Sample size: Obligatoriskt men exakta thresholds definieras senare

Regime/time coverage: Krävs

RiskProfile versioning: Obligatoriskt

Multi-account risk: Framtida portfolio layer

Atlas: Får rekommendera scaling, inte aktivera den

Rollback: Ska definieras vid varje större scale-up

Omnira Trading ska aldrig skala för att systemet känner sig framgångsrikt.

Det ska skala först när systemet kan visa varför högre risk är rationell.

KAPITEL 20

Lärdomar och förändringshistorik

Omnira Trading System ska inte bara komma ihåg vilka trades som togs.

Det ska också komma ihåg varför systemet ser ut som det gör.

Varje större regel, arkitekturförändring, riskbeslut, incident och strategy revision ska kunna spåras över tid.

Detta är viktigt eftersom ett trading-system som utvecklas under flera år annars lätt tappar bort:

- varför en regel infördes
- varför en regel togs bort
- vilka problem som ledde till en arkitekturförändring
- vilka candidate strategies som redan testats
- vilka hypoteser som visade sig fel
- vilka incidenter som redan inträffat
- vilka lessons learned som blev permanenta systemprinciper

Den centrala principen är:

Omnira ska inte bara versionshantera vad som ändrades. Systemet ska också bevara varför det ändrades.

Förändringshistorik som permanent del av systemet

Change History ska vara en förstaklassad del av Omnira Trading.

Den ska kunna dokumentera förändringar i:

- Strategy
- Risk
- Prop Firm Rules
- Execution
- Market Data
- AI Analysis
- Security
- Infrastructure
- Analytics
- Learning Layer
- UI
- Operational policy

På så sätt blir utvecklingshistoriken en del av själva systemets kunskap.

Versioner ska ha mening

En version ska inte bara vara ett nummer.

Exempel:

Strategy v1.0

ska representera ett specifikt regelverk.

Om strategy senare ändras till:

v1.1

ska det vara möjligt att förstå:

- vad som ändrades
- varför det ändrades
- vilken evidens som låg bakom
- vilka tester som genomfördes
- vem som godkände
- när den aktiverades

Semantic Versioning som princip

En förenklad versioneringsmodell kan användas.

Exempel:

v1.0

för första canonical productionversion.

v1.1

för en kompatibel men materiell strategiändring.

v2.0

för större förändring av strategy logic eller architecture.

Exakta regler kan senare standardiseras.

Candidate Versions

Förbättringar ska först finnas som candidate.

Exempel:

v1.1-candidate-01

Den är inte canonical.

Den representerar en hypotes under test.

Candidate Lifecycle

En candidate ska kunna gå genom:

PROPOSED
→
BACKTESTING
→
OOS_TESTING
→
FORWARD_TESTING
→
REVIEW
→
APPROVED

eller:

REJECTED

Först efter approval får den bli canonical.

Rejected Candidates ska bevaras

En candidate som inte fungerar ska inte raderas.

Exempel:

v1.1-candidate-03

visar sämre out-of-sample-resultat.

Status:

REJECTED

Detta är fortfarande värdefull kunskap.

Varför negativa resultat är viktiga

Om rejected research försvinner kan Atlas senare föreslå exakt samma idé igen.

Genom att spara den kan systemet istället veta:

Den här hypotesen testades redan mot dataset X och misslyckades.

Det minskar onödigt arbete och selection bias.

Research Registry

Alla större hypotheses ska kunna lagras i ett Research Registry.

Exempel:

HYP-0042

Hypothesis: Blockera Grade C under New York.

Reason: Preliminary negative expectancy.

Tested against: Strategy v1.0 dataset.

Result: No robust improvement OOS.

Status: REJECTED.

Finding → Hypothesis

Atlas Learning Layer ska skilja mellan:

Finding

och:

Hypothesis

Exempel:

Finding:

Attempt 3 visar lägre expectancy.

Hypothesis:

Max attempts bör kanske sänkas från 3 till 2.

Finding beskriver data.

Hypothesis föreslår en möjlig förklaring eller förändring.

Hypothesis är inte Rule

Det är viktigt att systemet aldrig omvandlar:

interesting observation

direkt till:

production rule

Det är exakt detta governance-lagret ska förhindra.

Change Proposal

En förändring som är mogen för riktig evaluation ska få ett formellt Change Proposal.

Exempel:

CHANGE-STRAT-0012

Det kan innehålla:

- affected component
- current version
- proposed version
- reason
- supporting findings
- expected benefit
- known risks
- validation plan

Change Classification

Förändringar kan klassificeras efter typ.

Exempel:

Strategy Change

Ändrar tradingregler.

Risk Change

Ändrar kapital- eller exposure policy.

Prop Change

Ändrar firm-specific compliance logic.

Execution Change

Ändrar order/management behavior.

Infrastructure Change

Ändrar runner, VPS, provider eller deployment.

Security Change

Ändrar permissions, auth eller safety.

Material vs Non-Material Change

Inte alla kodförändringar kräver ny strategy version.

Exempel:

UI text correction:

icke-material.

Men:

ändrad iFVG detection:

material.

Systemet ska kunna skilja mellan:

behavior-preserving change

och:

behavior-changing change

Strategy Behavior Change

Om en kodändring kan förändra:

- vilka setups som identifieras
- entry
- SL
- TP
- grade

- re-entry
- news handling

ska den behandlas som strategy behavior change.

Risk Behavior Change

Om en förändring påverkar:

- allowed quantity
- daily stop
- risk calculation
- exposure

krävs ny RiskProfile eller Risk Engine version där relevant.

Execution Behavior Change

Ändringar i:

- order type
- slippage policy
- retry
- fill handling
- SL/TP submission

ska dokumenteras och verifieras separat.

Reason for Change

Varje materiell förändring ska ha en tydlig anledning.

Exempel:

Attempt 3 produced persistent negative expectancy across backtest, OOS and forward testing.

Det är starkare än:

Vi ville testa något nytt.

Evidence Reference

Change Proposal ska kunna länka till:

- Finding IDs
- BacktestRuns
- ForwardTestRuns
- live metrics
- incidents
- external rule changes

No Evidence Change

Ibland kan en förändring ändå behövas utan performance evidens.

Exempel:

- security vulnerability
- broker API change
- prop firm rule change

Då ska reason anges som exempelvis:

```
SECURITY_REQUIRED
```

eller:

```
COMPLIANCE_REQUIRED
```

Change Approval

Materiella productionändringar kräver explicit approval.

Approval ska kunna dokumentera:

- approver
- timestamp
- version
- evidence reviewed

Atlas får föreslå

Atlas får skapa:

- Finding
- Hypothesis
- Candidate
- Change Proposal draft

Atlas får inte ge sig själv final approval.

Canonical Promotion

När en candidate passerat samtliga gates ska den kunna promoveras.

Exempel:

v1.1-candidate-04

```
→
```

Canonical v1.1

Promotion ska vara ett explicit event.

Promotion Record

Promotion ska dokumentera:

- old canonical version
- new canonical version
- tests

- approval
- activation date
- rollback plan

Canonical betyder Active Truth

Canonical betyder:

Detta är den officiellt godkända definitionen som systemet ska implementera i relevant production scope.

Det betyder inte:

Denna version är perfekt för alltid.

Deprecation

När ny canonical version aktiveras ska gammal version kunna markeras:

DEPRECATED

men historiken ska bevaras.

Historical Trades behåller sin Version

En trade från v1.0 ska alltid fortsätta vara:

StrategyVersion = v1.0

även efter att v1.1 blivit active.

Annars förstörs analytics integrity.

Rollback

Om en ny canonical version orsakar problem ska systemet kunna återgå till tidigare verifierad version.

Rollback ska dokumenteras som ett eget event.

Rollback är inte Delete

v1.1 försvinner inte bara för att systemet går tillbaka till v1.0.

Den kan markeras:

ROLLED_BACK

med reason.

Rollback Reason

Exempel:

Forward-to-live divergence exceeded expected range.

eller:

Execution bug introduced in v1.1.

Detta blir värdefull learning.

Incident Learning

Technical incidents ska kunna skapa lessons learned.

Exempel:

Incident:

SL verification failed after reconnect

Lesson:

Reconciliation must verify broker-native protective orders before returning runner to READY.

Detta kan sedan skapa:

- code fix
- regression test
- architecture rule

Incident → System Principle

Vissa incidents kan bli permanenta designprinciper.

Exempel:

Ett duplicate order incident kan leda till principen:

No execution without persistent idempotency state.

Detta är hur systemet mognar.

Postmortem

Större incidents ska få ett postmortem.

Det ska minst innehålla:

- what happened
- impact
- timeline
- root cause
- contributing factors
- recovery
- corrective actions
- prevention

No Blame

Postmortem ska fokusera på systemförbättring.

Målet är inte att hitta någon att skylla på.

Det viktiga är:

Hur gör vi detta svårare att upprepa?

Lessons Learned Registry

Systemet ska kunna ha ett separat register över permanenta lessons.

Exempel:

LESSON-0017

Broker response is not sufficient proof that SL is active; actual position state must be verified.

Severity

Lessons kan klassificeras:

- INFO
- OPERATIONAL
- IMPORTANT
- SAFETY_CRITICAL

Safety-Critical Lessons

En safety-critical lesson ska kunna skapa obligatoriska:

- tests
- policies
- controls

External Changes

Systemet måste också kunna dokumentera förändringar som kommer utifrån.

Exempel:

- prop firm ändrar drawdown rule
- broker ändrar symbol mapping
- providern API ändras
- news provider ändrar schema

Dessa kan kräva ny internal version.

Prop Firm Rule Change History

Exempel:

PropProfile v3

ersätts av:

PropProfile v4

på grund av ny official provider rule.

Historiska account decisions ska fortsätta referera till v3 där det var den aktiva versionen.

Rule Effective Date

Extern regeländring ska ha:

- discovered_at

- effective_from
- verified_at
- source

Unknown Historical Rules

Om vi inte säkert vet vilken external rule som gällde historiskt ska systemet markera osäkerheten.

Det ska inte skriva om historiken med dagens regel.

Market Data Provider Changes

Byter vi provider ska detta registreras.

Det kan påverka:

- candles
- FVG
- swing detection
- backtest comparisons

Därför är provider change inte alltid trivial.

Data Migration History

Om historiska datasets korrigeras eller migreras ska gamla testresultat fortfarande veta vilken data de använde.

Detection Logic History

Exakta detectionalgoritmer ska versionshanteras.

Exempel:

SWING-DETECT-v1

FVG-DETECT-v1

IFVG-DETECT-v1

CISD-DETECT-v1

SMT-DETECT-v1

Detta är särskilt viktigt för reproducibility.

Pattern Definition Changes

Om vi senare förbättrar definitionen av CISD ska detta inte göras tyst.

Det kan förändra strategy behavior.

Ny detection/strategy candidate krävs.

AI Layer History

AI-lagret ska också ha förändringshistorik.

Exempel:

- model updated
- analysis prompt updated
- market regime classifier updated
- explanation policy updated

Det gör det möjligt att utvärdera om Atlas blivit bättre.

AI Model Change är inte automatiskt Strategy Change

Om Atlas endast förklarar bättre men Strategy Engine är oförändrad behöver strategy version inte ändras.

Men AI analysis version ska ändras.

AI Decision-Support Change

Om AI senare får en formell roll i någon gate måste en modell/policyändring behandlas betydligt striktare.

Learning Layer Evolution

Self-improvement-systemet självt ska också utvecklas.

Exempel:

Learning Engine v1

kan senare ersättas av:

v2

med bättre statistical validation.

Vi ska kunna mäta om den nya versionen producerar bättre candidates.

Learn from Learning

Atlas ska alltså även kunna lära sig:

Vilka typer av hypotheses tenderar att hålla OOS?

Det är meta-learning.

Failed Hypothesis Rate

Learning Layer ska mäta hur stor andel candidates som:

- failar backtest
- failar OOS
- failar forward
- blir canonical

Om nästan allt blir canonical är governance sannolikt för svag.

Research Discipline

Att många hypotheses blir rejected är normalt.

Det är ett tecken på att testsystemet faktiskt filtrerar.

Change Frequency

Production strategy ska inte förändras så ofta att systemet aldrig hinner samla stabil evidens.

För hög change frequency skapar:

- version fragmentation
- små samples
- svår analytics
- hög overfitting risk

Stable Evaluation Windows

När en canonical version går in i forward/live validation bör den få tillräckligt stabil evaluationperiod.

Vi ska inte ändra den efter varje förlust.

Emergency Changes

Ibland krävs snabb production change.

Exempel:

critical security problem.

Emergency change ska ändå dokumenteras.

Efteråt krävs full review och test där relevant.

Emergency Change Status

Exempel:

```
EMERGENCY_PATCH
```

kan användas tills normal canonical process slutförts.

Hotfix vs Strategy Change

En bugfix som återställer systemet till dokumenterad canonical behavior behöver inte nödvändigtvis vara ny strategyversion.

Men fixen måste dokumenteras.

Example

Canonical säger:

Minimum R:R = 2.0

Kodbugg tillåter:

1.8

Att fixa detta till 2.0 är en implementation correction.

Det är inte en ny strategyregel.

Implementation Bug Registry

Det ska finnas historik över buggar som påverkade eller kunde påverka trading behavior.

Backfill

Om ett implementationfel hittas ska vi kunna analysera tidigare data och fråga:

Vilka historical decisions påverkades?

Detta kan kräva corrected analytics.

Correction får inte skriva om Raw History

Original systemevent bevaras.

En ny interpretation kan läggas ovanpå.

Historical Reclassification

Exempel:

En setup journalfördes som Grade B på grund av bug.

Efter fix kan research markera:

corrected grade = A

Original grade ska fortfarande finnas i audit.

Change Log

Varje release bör ha en tydlig changelog.

Exempel:

v1.1

Changed

- max attempts reduced 3 → 2

Reason

- negative expectancy attempt 3 across OOS + forward

Evidence

- FIND-0042
- BT-00124
- FT-00018

Approved

- date
- actor

Human-Readable + Machine-Readable

Change history ska finnas både som:

- strukturerade records
- mänskligt läsbar sammanfattning

Atlas kan generera den läsbara versionen från strukturerade data.

Boken som Canonical Context

Denna bok ska fungera som den högre nivån av arkitektur- och governancekunskap.

Den ska senare kunna uppdateras när systemet når nya canonical milestones.

Men den ska inte ersätta machine-readable versionsobjekt.

Book Versioning

Själva boken bör också versionshanteras.

Exempel:

Omnira Trading System – Canonical Book v1.0

senare:

v1.1

när större godkända förändringar införts.

Bokförändring efter Systemförändring

Systemförändring ska först vara:

- beslutad
- testad
- godkänd

därefter uppdateras canonical bok där det behövs.

Boken ska inte göra oprövade hypotheses till officiell arkitektur.

Change History Appendix

En framtida canonical bok kan innehålla ett appendix med:

- version
- date
- major changes
- reason

Det ger snabb historisk överblick.

Atlas Knowledge Graph

På längre sikt kan change history kopplas till Omniras Intelligence Graph.

Exempel:

Strategy v1.1

linked to:

- Finding
- Backtest
- Forward Test
- Incident
- Approval
- Deployment

Det gör systemets kunskap navigerbar.

Why Graph Relations Matter

Då kan Atlas svara:

Varför ändrade vi max attempts till 2?

och följa:

Rule

- Change
- Finding
- Evidence
- Approval

istället för att gissa från gamla chattexter.

Decision Record

Större arkitekturbeslut bör kunna sparas som:

Architecture Decision Records

Exempel:

ADR-TRADING-007

Decision: Execution Runner ska vara separerad från Strategy Engine.

Reason: Least privilege, providern isolation, VPS migration, lower blast radius.

Status: Accepted.

Superseded Decisions

När ett beslut senare ändras ska gammalt ADR kunna markeras:

SUPERSEDED BY ADR-X

inte raderas.

Why Matters

Detta gör att framtida utvecklare eller Atlas kan förstå:

Det här var inte en slumpmässig kodstruktur.

Det fanns en säkerhetsmässig anledning.

Decision Debt

Om vi tillfälligt gör en kompromiss ska detta kunna registreras.

Exempel:

Initial market data provider används som single source under MVP. Multi-source validation postponed.

Detta blir:

KNOWN LIMITATION

istället för bortglömd teknisk skuld.

Known Limitations Registry

Systemet ska ha en lista över sådant vi vet ännu inte är perfekt.

Exempel:

- active management outage risk
- no redundant execution runner yet
- initial news provider dependency
- limited regime classifier

Detta gör roadmap mer verklighetsbaserad.

Technical Debt

Teknisk skuld ska kunna prioriteras efter:

- safety impact
- operational impact
- performance impact
- developer cost

Safety debt ska prioriteras högst.

Unresolved Questions

Systemet ska också kunna bevara öppna designfrågor.

Exempel:

- exact scale thresholds
- AI fallback policy
- future VPS redundancy

Open Questions ska inte behandlas som beslut.

Status Taxonomy

En bra knowledge model ska kunna skilja:

```
CANONICAL
CANDIDATE
HYPOTHESIS
FINDING
REJECTED
DEPRECATED
OPEN_QUESTION
KNOWN_LIMITATION
```

Detta är centralt för att Atlas inte ska blanda ihop kunskapsnivåer.

Confidence in Knowledge

Research findings kan också ha:

- sample size
- confidence
- evidence quality

Canonical rules behöver däremot inte betyda att de är statistiskt perfekta.

Canonical betyder att det är den aktiva beslutade policyn.

Strategy Evolution

Den långsiktiga evolutionen ska kunna se ut så här:

```
Canonical v1.0
→ Observation
→ Learning
→ Finding
→ Hypothesis
→ v1.1 Candidate
→ Backtest
→ OOS
→ Forward
→ Review
→ Canonical v1.1
→ Live Deployment
→ Observation
```

och därefter fortsätter cykeln.

Evolution utan Chaos

Detta gör att systemet kan utvecklas kontinuerligt utan att production blir ett ständigt experiment.

Self-Improvement Constitutional Rule

Den viktigaste regeln för systemets framtida lärande är:

Atlas får utveckla kunskap snabbare än systemet får utveckla production authority.

Learning kan ske kontinuerligt.

Productionförändring ska vara långsammare och mer kontrollerad.

Why This Matters

Ett autonomt system som kan förändra sig själv snabbare än vi kan validera förändringarna är svårt att kontrollera.

Därför separeras:

Learning Speed

från:

Deployment Speed

Historical Baselines

Canonical versions ska fungera som historiska baselines.

Atlas ska alltid kunna jämföra:

v1.1

mot:

v1.0

och fråga:

Blev systemet faktiskt bättre?

Improvement Must Be Measured

En ny version är inte bättre bara för att den är ny.

Den ska utvärderas mot föregående baseline.

Regression Detection

Om ny version försämrar:

- execution
- drawdown
- stability
- compliance

ska detta kunna upptäckas.

No Sunk Cost

Om en candidate krävt mycket utvecklingsarbete men data visar att den är sämre ska den kunna rejected.

Utvecklingskostnad är inte bevis på edge.

Strategy Retirement

En strategy kan på längre sikt få status:

RETIRED

om edge försvinner eller en bättre strategy ersätter den.

Historiken ska ändå bevaras.

Re-Activation

En retired strategy ska inte återaktiveras utan ny validation mot aktuell market environment.

Prop Program Retirement

Samma princip kan gälla prop profiles om ett program stängs eller ändras.

Knowledge Retention

Även gamla strategy failures är värdefulla.

De kan hjälpa framtida Atlas förstå vilka idéer som inte fungerat.

Atlas Questions History Should Answer

Systemet ska på sikt kunna svara:

Varför använder vi 2R minimum?

Varför har vi max tre attempts?

Varför stängs positioner T-15 före news?

Varför kör vi Read Only innan execution?

När infördes denna RiskProfile?

Vilket test ledde till Strategy v1.2?

Har vi testat att ta bort SMT tidigare?

Om svaren finns strukturerat behöver vi inte lita på mänskligt minne.

Human Review

Atlas ska hjälpa till att hitta historiken.

Men större ändringar ska fortfarande kunna reviewas av användaren.

Change Summary

Vid ny release ska Atlas kunna skapa en enkel sammanfattning:

v1.2 ändrar endast re-entry behavior. Inga ändringar görs i entry, SL, TP eller news policy.

Förändringen baseras på 742 historical observations, OOS PASS och 81 forward trades.

Det gör release review effektiv.

No Hidden Changes

Ingen materiell strategy-, risk- eller executionförändring ska gömmas inne i:

refactor

eller:

cleanup

Den ska identifieras explicit.

Code Review and Tests

Material changes ska kunna verifieras genom:

- diff
- automated tests
- golden cases
- regression suite

Documentation och code måste stämma överens.

Documentation Drift

Om code och canonical specification skiljer sig ska detta betraktas som:

DOCUMENTATION / IMPLEMENTATION DRIFT

Det måste lösas.

Machine-Readable Canonical Config

På längre sikt ska centrala canonical regler finnas som versionsstyrd config som Strategy Engine använder direkt där detta är möjligt.

Det reducerar risken att bok och implementation glider isär.

Human-Readable Canonical Specification

Boken och strategy specification förklarar:

- intention
- rationale
- definitions
- architecture

Machine config implementerar reglerna.

De kompletterar varandra.

Atlas Governance Review

Atlas kan periodiskt sammanfatta:

- active canonical versions
- open candidates
- unresolved hypotheses
- recent incidents
- pending rule reviews
- known limitations

Detta blir en trading governance dashboard.

Quarterly / Periodic Review

På längre sikt kan större systemreview genomföras periodiskt.

Den ska inte automatiskt ändra någonting.

Den ska identifiera:

- stale configs
- unresolved debt
- declining strategy
- new research
- security issues

No Change is also a Decision

Om review visar att v1.0 fortfarande fungerar väl kan beslutet vara:

NO CHANGE

Detta ska kunna dokumenteras.

Kontinuerlig förbättring betyder inte kontinuerlig förändring.

Stable Systems are Valuable

I ett system med verkligt kapital kan stabilitet vara mer värdefullt än konstant experimentation.

Timeline

Omnira ska kunna visa en kronologisk trading-system timeline.

Exempel:

2026-08

Strategy v1.0 canonical

2026-10

Backtest baseline completed

2026-12

Demo RC1

2027-01

First Controlled Live

2027-05

v1.1 candidate rejected

2027-08

v1.2 promoted

Datumen ovan är endast exempel på hur en framtida timeline kan se ut, inte en planerad tidslinje.

System Memory

Change History blir tillsammans med:

- Journal
- Analytics
- Research Registry
- Incident Registry

systemets långsiktiga tradingminne.

Från Data till Wisdom

Flödet är:

```
Market Data
→ Events
→ Journal
→ Analytics
→ Findings
→ Lessons
→ Canonical Knowledge
```

Det är denna kedja som gör att Omnira blir bättre över tid.

Den långsiktiga visionen

På lång sikt ska en ny utvecklare eller framtida Atlas kunna öppna Omnira och förstå:

- hur systemet fungerar idag
- hur det fungerade tidigare
- varför det förändrades
- vilka idéer som testades
- vilka incidenter som format architecture
- vilken evidens som stödjer aktuella rules

utan att behöva rekonstruera flera års historia från gamla chattar och lösa dokument.

Systemet ska kunna förklara sig självt

Målet är att Atlas ska kunna svara:

Varför ser Trading System ut så här?

med ett svar som bygger på faktisk versionerad historik.

Inte på spekulaton.

Bokens roll efter v1.0

När denna bok är färdig fungerar den som grundläggande systemkonstitution.

Den definierar:

- vision

- strategy
- risk
- architecture
- data
- AI
- execution
- providern
- testing
- prop firm
- journal
- analytics
- failure handling
- security
- deployment
- scaling
- evolution

Framtida canonical förändringar ska kunna uppdatera boken kontrollerat.

Vad boken inte är

Boken är inte:

- bevis på att strategy är profitable
- tillstånd att gå live
- ersättning för tests
- ersättning för Risk Engine
- ett automatiskt mandate för Atlas

Den är systemets definierade grund.

Evidensen byggs därefter genom implementation och validation.

Från bok till system

Efter boken går projektet vidare från:

Design

till:

Implementation

Men implementation ska fortfarande ske phase-gated.

Vi bygger inte allt på en gång.

Canonical Implementation Order

Den övergripande ordningen är:

Fas 0

Specifications och architecture

→

Fas 1

Trading Core

→

Fas 2

Futures Connectivity (Read Only)

→

Fas 3

Strategy Engine

→

Fas 4

AI Analysis

→

Fas 5

Risk Engine

→

Fas 6

Manual Approval

→

Fas 7

Demo Automation

→

Fas 8

Backtest + Forward Validation

→

Fas 9

Prop Firm Mode

→

Fas 10

Controlled Live

Varje fas kräver verification innan nästa.

Viktiga kvarvarande Specifications

Även med boken färdig finns detaljer som måste lösas innan relevant implementation.

Särskilt viktiga är:

- exakt deterministic iFVG detection
- exakt deterministic CISD detection
- equal-high/equal-low tolerance
- SMT correspondence rules där ytterligare precision krävs
- första market-data provider
- första news-data provider
- första faktiska PropFirmProfile
- exact promotion thresholds
- exact live safety policies

Dessa ska inte gissas inne i koden.

Pattern Detection Specification

Entrykritiska visuella begrepp behöver ett eget machine-readable specificationarbete innan Strategy Engine implementeras.

Det gäller framför allt:

iFVG

och:

CISD

Detta är en kvarvarande implementation gate.

Architecture Canonicalization

De separata Fas 0-dokumenterna ska senare genomgå en contradiction review.

Vi ska då kontrollera att:

- Strategy

- Architecture
- Data Model
- Risk
- Prop
- Learning
- Security

inte motsäger varandra.

Först därefter ska architecturepaketet låsas som canonical.

Trading Book Canonical Review

Även boken ska få en slutlig review.

Den ska kontrollera:

- chapter consistency
- terminology
- rule consistency
- version references
- unresolved questions
- accidental contradictions

Canonical v1.0 Book Release

När review passerat kan hela boken paketeras som:

Omnira Trading System – Från strategi till autonom exekvering – Canonical v1.0

Den blir då den övergripande dokumentationsbasen inför implementation.

Evolution efter Canonical v1.0

Canonical v1.0 är startpunkten för systemets faktiska evidenshistorik.

Därefter kommer:

- code
- tests
- data
- failures
- improvements

att avgöra hur framtida versioner ser ut.

Den sista principen

Omnira Trading ska aldrig bli ett system som förändras utan minne.

Det ska vara ett system som:

- Observerar
 - Dokumenterar
 - Mäter
 - Lär
 - Testar
 - Förändrar kontrollerat
 - Kommer ihåg varför

Det är skillnaden mellan en bot som bara exekverar trades och ett trading-system som kan utvecklas ansvarsfullt över många år.

Kapitelstatus

Kapitel: 20 – Lärdomar och förändringshistorik

Bok: Omnira Trading System – Från strategi till autonom exekvering

Status: Baseline dokumenterad

Change history: Obligatorisk

Rejected candidates: Bevaras

Research Registry: Planerad

Incident lessons: Bevaras

Canonical promotion: Explicit och versionsstyrd

Rollback: Bevarar historik

Atlas self-improvement: Tillåtet

Atlas self-modification i production: Förbjudet

Boken: Grund för framtida Canonical v1.0

Nästa huvudsteg: Slutreview av boken och Fas 0-specifikationerna innan implementation

Omnira Trading System ska kunna bli bättre utan att förlora förståelsen för hur, när eller varför det förändrades.